The gcodepreview PythonSCAD library*

Author: William F. Adams
willadams at aol dot com

2026/03/07

Abstract

The gcodepreview library allows using PythonSCAD (Python in OpenSCAD) to move a tool in lines and arcs and output DXF and G-code files so as to work as a CAD/CAM program for CNC.

Contents

1 readme.md **4**

2 Usage and Templates **8**
2.1 gcpdxf.py . . . 9
2.2 gcpcutdxf.py . . . 12
2.3 gcodepreviewtemplate.py . . . 14
2.4 gcodepreviewtemplate.scad . . . 19
2.5 gpcthreedp.py . . . 23
2.6 gcodepreviewtemplate.txt . . . 24

3 gcodepreview **25**
3.1 Tools for 3D Previewing G-code . . . 25
3.2 Module Naming Convention . . . 27
3.3 Implementation files and gcodepreview class . . . 30
3.4 Tools and Shapes and Changes . . . 38
3.5 Shapes and tool movement . . . 53
3.6 Difference of Stock, Rapids, and Toolpaths . . . 84
3.7 Output files . . . 85
3.8 Cutting shapes and expansion . . . 98
3.9 (Reading) G-code Files . . . 121
3.10 Parts, Assemblies, Cut-Lists, and Views . . . 124

4 Notes **126**
4.1 Other Resources . . . 126
4.2 Future . . . 127

Index **131**
Routines . . . 132
Variables . . . 133

*This file (gcodepreview) has version number v0.9313, last revised 2026/03/07.

Contents

1 readme.md **4**

2 Usage and Templates **8**
2.1 gcpdxf.py . . . 9
2.2 gcpcutdxf.py . . . 12
2.3 gcodepreviewtemplate.py . . . 14
2.4 gcodepreviewtemplate.scad . . . 19
2.5 gpcthreedp.py . . . 23
2.6 gcodepreviewtemplate.txt . . . 24

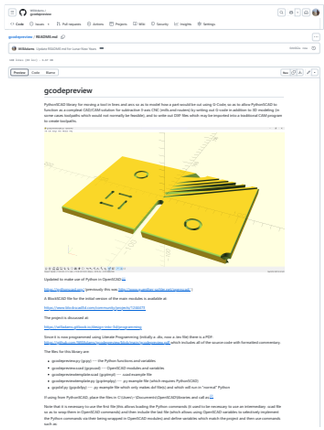
3 gcodepreview **25**
3.1 Tools for 3D Previewing G-code . . . 25
3.1.1 Cutviewer . . . 25
3.1.1.1 Stock size and placement . . . 25
3.1.1.2 Tool Shapes . . . 26
3.1.1.2.1 Tool/Mill (Square, radiused, ball-nose, and tapered-ball) . . . 26
3.1.1.2.2 Corner Rounding, (roundover) . . . 27
3.1.1.2.3 V shaped tooling (and variations) . . . 27
3.2 Module Naming Convention . . . 27
3.2.1 Parameters and Default Values . . . 30
3.3 Implementation files and gcodepreview class . . . 30
3.3.1 init . . . 31
3.3.2 Position and Variables . . . 33
3.3.3 Initial Modules . . . 34
3.3.3.1 setupstock . . . 34
3.3.3.2 setupcuttingarea . . . 36
3.3.3.3 debug . . . 37
3.3.4 Adjustments and Additions . . . 37
3.4 Tools and Shapes and Changes . . . 38
3.4.1 Numbering for Tools . . . 38
3.4.1.1 toolchange . . . 42
3.4.1.1.1 settoolparameters . . . 42
3.4.1.1.2 toolchange . . . 43
3.4.1.1.3 Square (including O-flute) . . . 43
3.4.1.1.4 Ball-nose . . . 45
3.4.1.1.5 V . . . 46
3.4.1.1.6 Keyhole . . . 48
3.4.1.1.7 Bowl . . . 48
3.4.1.1.8 Tapered ball nose . . . 49
3.4.1.1.9 Roundover (cove tooling) . . . 50
3.4.1.1.10 Dovetails . . . 51
3.4.1.1.11 closing G-code . . . 52
3.4.2 Laser support . . . 52
3.5 Shapes and tool movement . . . 53
3.5.1 Tooling for Undercutting Toolpaths . . . 53
3.5.2 Generalized commands and cuts . . . 53
3.5.3 Movement and color . . . 54
3.5.3.1 toolmovement . . . 55
3.5.3.1.1 Square (including O-flute) . . . 56
3.5.3.1.2 Ball nose . . . 56
3.5.3.1.3 bowl . . . 56
3.5.3.1.4 V . . . 57
3.5.3.1.5 Keyhole . . . 57
3.5.3.1.6 Tapered ball nose . . . 58
3.5.3.1.7 Dovetails . . . 58
3.5.3.2 Concave toolshapes . . . 58
3.5.3.2.1 Roundover tooling . . . 58
3.5.3.3 shaftmovement . . . 59
3.5.3.4 tool outlines . . . 59
3.5.3.4.1 defineshaft . . . 59
3.5.3.4.2 Square (including O-flute) . . . 60
3.5.3.4.3 Ball-nose . . . 60
3.5.3.4.4 V tool outline . . . 61
3.5.3.4.5 Keyhole outline . . . 61
3.5.3.4.6 Bowl outline . . . 61
3.5.3.4.7 Tapered ball nose . . . 62
3.5.3.4.8 Roundover (cove tooling) . . . 62
3.5.3.5 rapid and cut (lines) . . . 62
3.5.3.6 Arcs . . . 65
3.5.4 tooldiameter . . . 70

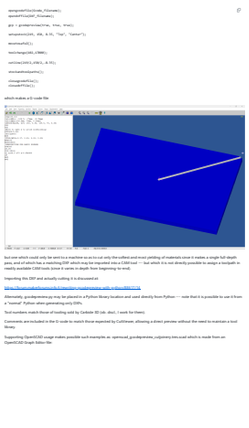
3.5.5 Feeds and Speeds . . . 72
3.5.6 3D Printing . . . 72
3.5.6.1 fullcontrolgcode . . . 73
3.5.6.2 Previewing/verifying G-code for 3D printers . . . 77
3.5.6.3 Time and Firmware for 3D printers . . . 77
3.5.6.4 Sample 3D printing file . . . 77
3.5.6.5 Initialize . . . 80
3.5.6.6 extrude . . . 83
3.5.6.7 Shutdown . . . 83
3.5.6.8 fullcontrolgcode commands . . . 84
3.6 Difference of Stock, Rapids, and Toolpaths . . . 84
3.7 Output files . . . 85
3.7.1 Python and OpenSCAD File Handling . . . 85
3.7.2 DXF Overview . . . 86
3.7.2.1 Writing a preamble to DXF files . . . 88
3.7.2.1.1 DXF Lines and Arcs . . . 88
3.7.3 G-code Overview . . . 97
3.7.3.1 Closings . . . 98
3.8 Cutting shapes and expansion . . . 98
3.8.1 Building blocks . . . 98
3.8.1.1 List of shapes . . . 99
3.8.1.1.1 circles . . . 100
3.8.1.1.2 rectangles . . . 101
3.8.1.1.3 Keyhole toolpath and undercut tooling . . . 104
3.8.1.2 Joinery . . . 111
3.8.1.2.1 Dovetail joinery and tooling . . . 111
3.8.1.2.2 Full-blind box joints . . . 113
3.8.1.3 Fonts . . . 121
3.9 (Reading) G-code Files . . . 121
3.10 Parts, Assemblies, Cut-Lists, and Views . . . 124

4 Notes **126**
4.1 Other Resources . . . 126
4.1.1 Mathematics . . . 126
4.1.2 Coding Style . . . 126
4.1.3 Coding References . . . 126
4.1.4 Documentation Style . . . 126
4.2 Future . . . 127
4.2.1 Images . . . 127
4.2.2 Bézier curves in 2 dimensions . . . 127
4.2.3 Bézier curves in 3 dimensions . . . 128

Index **131**
Routines . . . 132
Variables . . . 133

1 readme.md







```
1 rdme # gcodepreview
2 rdme
3 rdme OpenPythonSCAD library for moving a tool in lines and arcs so as to
       model how a part would be cut or extruded using G-Code, so as
      to allow use as a compleat CAD/CAM solution for subtractive or
      additive 3-axis CNC (4th-axis support may come in a future
      version) by writing out G-code in addition to 3D modeling (in
      certain cases toolpaths which would not normally be feasible in
      typical tools), and to write out DXF files which may be imported
       into a traditional CAM program to create toolpaths.
4 rdme
5 rdme ![OpenSCAD gcodepreview Unit Tests](https://raw.githubusercontent.
      com/WillAdams/gcodepreview/main/gcodepreviewtemplate.png?raw=
      true)
6 rdme
7 rdme Uses [Python in OpenSCAD](https://pythonscad.org/)[^pythonscad]
8 rdme
9 rdme [^pythonscad]: Previously this was http://www.guenther-sohler.net/
      openscad/
10 rdme
11 rdme A BlockSCAD file for the initial version of the
12 rdme main modules is available at:
13 rdme
14 rdme https://www.blockscad3d.com/community/projects/1244473
15 rdme
16 rdme The project is discussed at:
17 rdme
18 rdme https://willadams.gitbook.io/design-into-3d/programming
19 rdme
20 rdme Since it is now programmed using Literate Programming (initially a
       .dtx, now a .tex file) there is a PDF: https://github.com/
      WillAdams/gcodepreview/blob/main/gcodepreview.pdf which includes
       all of the source code with commentary.
21 rdme
22 rdme The files for this library are:
23 rdme
24 rdme  - gcodepreview.py (gcpy) --- the Python class/functions and
           variables
25 rdme  - gcodepreview.scad (gcpscad) --- OpenSCAD modules and parameters
26 rdme
27 rdme And there several sample/template files which may be used as the
       starting point for a given project:
28 rdme
29 rdme  - gcodepreviewtemplate.txt (gcptmpl) --- .txt file collecting
           various commands with brief comments which may be used as a
           quick reference or copy-pasting from
30 rdme  - gcodepreviewtemplate.py (gcptmplpy) --- .py example file
31 rdme  - gcodepreviewtemplate.scad (gcptmplscad) --- .scad example file
32 rdme  - gcpdxf.py (gcpdxfpy) --- .py example file which only makes dxf
           file(s) and which will run in "normal" Python in addition to
           PythonSCAD
33 rdme  - gcpgc.py (gcpgc) --- .py example which loads a G-code file and
           generates a 3D preview showing how the G-code will cut
34 rdme  - gcpthreedp.py --- Template for 3D printing using Full Control G-
           code https://fullcontrolgcode.com/
35 rdme
36 rdme Note that additional templates are in: https://github.com/WillAdams
      /gcodepreview/tree/main/templates
```

```
37 rdme
38 rdme If using from PythonSCAD, place the files in C:\Users\\\~\Documents
       \OpenSCAD\libraries or, load them from Github using the command:
39 rdme
40 rdme     nimport("https://raw.githubusercontent.com/WillAdams/
           gcodepreview/refs/heads/main/gcodepreview.py")
41 rdme
42 rdme If using gcodepreview.scad call as:
43 rdme
44 rdme     use <gcodepreview.py>
45 rdme     include <gcodepreview.scad>
46 rdme
47 rdme Note that it is necessary to use the first file (this allows
       loading the Python commands and then include the last file (
       which allows using OpenSCAD variables to selectively implement
       the Python commands via their being wrapped in OpenSCAD modules)
        and define variables which match the project and then use
       commands such as:
48 rdme
49 rdme     opengcodefile(Gcode_filename);
50 rdme     opendxffile(DXF_filename);
51 rdme
52 rdme     gcp = gcodepreview("cut", true, true);
53 rdme
54 rdme     setupstock(219, 150, 8.35, "Top", "Center");
55 rdme
56 rdme     movetosafeZ();
57 rdme
58 rdme     toolchange(102, 17000);
59 rdme
60 rdme     cutline(219/2, 150/2, -8.35);
61 rdme
62 rdme     stockandtoolpaths();
63 rdme
64 rdme     closegcodefile();
65 rdme     closedxffile();
66 rdme
67 rdme which makes a G-code file:
68 rdme
69 rdme ![OpenSCAD template G-code file](https://raw.githubusercontent.com/
       WillAdams/gcodepreview/main/gcodepreview_template.png?raw=true)
70 rdme
71 rdme but one which could only be sent to a machine so as to cut only the
        softest and most yielding of materials since it makes a single
       full-depth pass, and which has a matching DXF which may be
       imported into a CAM tool --- but which it is not directly
       possible to assign a toolpath in readily available CAM tools (
       since it varies in depth from beginning-to-end which is not
       included in the DXF since few tools make use of that information
       ).
72 rdme
73 rdme Importing this DXF and actually cutting it is discussed at:
74 rdme
75 rdme https://forum.makerforums.info/t/rewriting-gcodepreview-with-python
       /88617/14
76 rdme
77 rdme Alternately, gcodepreview.py may be placed in a Python library
       location and used directly from Python to generate DXFs as shown
        in gcpdxf.py (generating a 3D preview requires OpenPythonSCAD
       and generating G-code without a preview is not supported).
78 rdme
79 rdme In the current version, tool numbers may match those of tooling
       sold by Carbide 3D (ob. discl., I work for them) and other
       vendors, or, a vendor-neutral system may be worked up and used
       as desired.
80 rdme
81 rdme Comments are included in the G-code to match those expected by
       CutViewer, allowing a direct preview without the need to
       maintain a tool library (for such tooling as that program
       supports).
82 rdme
83 rdme Supporting OpenSCAD usage makes possible such examples as:
       openscad_gcodepreview_cutjoinery.tres.scad which is made from an
        OpenSCAD Graph Editor file:
84 rdme
85 rdme ![OpenSCAD Graph Editor Cut Joinery File](https://raw.
       githubusercontent.com/WillAdams/gcodepreview/main/
       OSGE_cutjoinery.png?raw=true)
```

```
86 rdme
87 rdme Written as a [Literate Program](http://literateprogramming.com/) in
        [lualatex](https://www.luatex.org/) which is a version of
        Donald E. Knuth's [TeX typesetting program](https://tug.org/
        whatis.html) extended by the [Lua programming language](https://
        www.lua.org/) using a custom package, [literati.sty](https://
        github.com/WillAdams/gcodepreview/blob/main/literati.sty)
        developed with a bit of help on [tex.stackexchange.com](https://
        tex.stackexchange.com/questions/722886/how-to-write-out-multiple
        -text-files-from-multiple-instances-of-latex-environmen) rather
        than using a more typical IDE because of the need for keeping
        multiple files in synch and so as to have a single point of
        control (the .tex source file used to generate the .pdf and all
        other files for this project).
88 rdme
89 rdme | Version       | Notes         |
90 rdme | ------------- | ------------- |
91 rdme | 0.1           | Version  supports setting up stock, origin, rapid
          positioning, making cuts, and writing out matching G-code, and
          creating a DXF with polylines.                                 |
92 rdme |               |  - separate dxf files are written out for each
          tool where tool is ball/square/V and small/large (10/31/23)

          |
93 rdme |               |  - re-writing as a Literate Program using the
          LaTeX package docmfp (begun 4/12/24)

          |
94 rdme |               |  - support for additional tooling shapes such as
          dovetail and keyhole tools

          |
95 rdme | 0.2           | Adds support for arcs, specialty toolpaths such
          as Keyhole which may be used for dovetail as well as keyhole
          cutters
                                                                         |
96 rdme | 0.3           | Support for curves along the 3rd dimension,
          roundover tooling

          |
97 rdme | 0.4           | Rewrite using literati documentclass, suppression
          of SVG code, dxfrectangle

          |
98 rdme | 0.5           | More shapes, consolidate rectangles, arcs, and
          circles in gcodepreview.scad

          |
99 rdme | 0.6           | Notes on modules, change file for setupstock

          |
100 rdme | 0.61         | Validate all code so that it runs without errors
          from sample (NEW: Note that this version is archived as
          gcodepreview-openscad_0_6.tex and the matching PDF is available
          as well|
101 rdme | 0.7          | Re-write completely in Python

          |
102 rdme | 0.8          | Re-re-write completely in Python and OpenSCAD,
          iteratively testing

          |
103 rdme | 0.801        | Add support for bowl bits with flat bottom

          |
104 rdme | 0.802        | Add support for tapered ball-nose and  V tools
          with flat bottom

          |
105 rdme | 0.803        | Implement initial color support and joinery
          modules (dovetail and full blind box joint modules)

          |
106 rdme | 0.9          | Re-write to use Python lists for 3D shapes for
          toolpaths and rapids.

          |
107 rdme | 0.91         | Finish converting to native OpenPythonSCAD
```

```
          trigonometric functions.

          |
108 rdme | 0.92          | Remove multiple DXFs and unimplemented features,
          add hooks for 3D printing.

          |
109 rdme | 0.93          | Initial support for 3D printing.

          |
110 rdme | 0.931         | Update support for OpenSCAD modules.

          |
111 rdme | 0.9312        | Update DXF file-handling for Carbide Create 839.

          |
112 rdme | 0.9313        | Update Full_Blind_Box_Joint to use layers, add
          LWPOLYLINE where appropriate

          |
113 rdme
114 rdme To do:
115 rdme
116 rdme  - implement OpenSCAD commands for 3D printing
117 rdme  - implement 3D printing commands beyond straight-line extrude
118 rdme  - add toolpath for cutting countersinks using ball-nose tool from
            inside working out
119 rdme  - create additional template and sample files
120 rdme  - fully implement/verify describing/saving/loading tools using
            CutViewer comments
121 rdme  - support for additional tooling shapes (lollipop cutters)
122 rdme  - threadmilling
123 rdme  - create font using lines and arcs with parameters for overshoot
            and width/spacing
124 rdme
125 rdme Possible future improvements:
126 rdme
127 rdme  - implement skin()
128 rdme  - support for 4th-axis
129 rdme  - support for post-processors
130 rdme  - support for two-sided machining (import an STL or other file to
            use for stock, or possibly preserve the state after one cut and
             then rotate the cut stock/part)
131 rdme  - create a single line font for use where text is wanted
132 rdme  - Support for METAPOST and Bézier curves (latter required for
            fonts if not to be limited to lines and arcs) and surfaces
133 rdme
134 rdme Note for G-code generation that it is up to the user to implement
          Depth per Pass so as to not take a single full-depth pass as
          noted above. Working from a DXF of course allows one to off-load
            such considerations to a specialized CAM tool.
135 rdme
136 rdme Issues/Research:
137 rdme
138 rdme  - determine why one quadrant of arc command doesn't work in
            OpenSCAD
139 rdme  - clock-wise arcs
140 rdme  - verify OpenSCAD wrapper and add any missing commands for Python
141 rdme  - verify support for shaft on tooling
142 rdme
143 rdme Deprecated features:
144 rdme
145 rdme  - exporting SVGs --- coordinate system differences between
            OpenSCAD/DXFs and SVGs would require managing the inversion of
            the coordinate system (using METAPOST, which shares the same
            orientation and which can write out SVGs may be used for future
             versions)
146 rdme  - using linear/rotate_extrude --- 2D geometry is rotated to match
            the arc of the movement, which is appropriate to a 5-axis
            machine, but not workable for a 3-axis. Adding an option to
            support the use of such commands for horizontal movement is
            within the realm of possibility.
147 rdme  - multiple DXF files
148 rdme  - RapCAD support
```

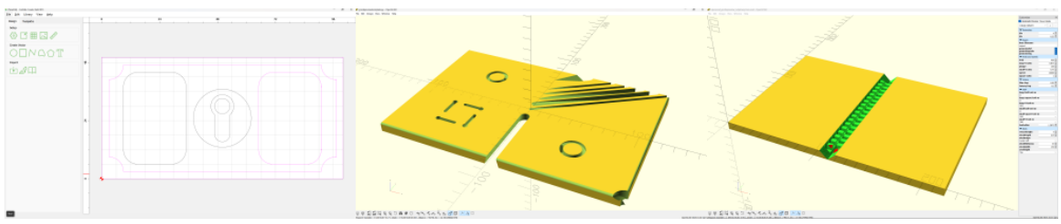
2 Usage and Templates

The gcodepreview library allows the modeling of 2D geometry and 3D shapes using Python or by calling Python from within Open(Python)SCAD, enabling the creation of 2D DXFs, G-code (which cuts a 2D or 3D part), or 3D models as a preview of how the file will cut. These abilities may be accessed in "plain" Python (to make DXFs), or Python or OpenSCAD in PythonSCAD (to make DXFs, and/or G-code with 3D modeling) for a preview. Providing them in a programmatic context allows making parts or design elements of parts (*e.g.*, joinery) which would be tedious or difficult (or verging on impossible) to draw by hand in a traditional CAD or vector drawing application. A further consideration is that this is "Design for Manufacture" taken to its ultimate extreme, and that a part so designed is inherently manufacturable (so long as the dimensions and radii allows for reasonable tool (and toolpath) geometries).

Written as a Literate Program[1] in lualatex[2] which is a version of Donald E. Knuth's TeX typesetting program[3] extended by the Lua programming language[4] using a custom package, `literati.sty`[5] developed with a bit of help on `tex.stackexchange.com`[6] rather than using a more typical IDE because of the need for keeping multiple files in synch and so as to have a single point of control (the `.tex` source file used to generate the `.pdf` and all other files for this project).

The various commands are shown all together in templates so as to provide examples of usage, and to ensure that the various files are used/included as necessary, all variables are set up with the correct names (note that the sparse template in `readme.md` eschews variables), and that if enabled, files are opened before being written to, and that each is closed at the end in the correct order. Note that while the template files seem overly verbose, they specifically incorporate variables for each supported tool shape, possibly in two different sizes, and a feed rate parameter or ratio for each, which may be used (by setting a tool #) or ignored (by leaving the variable for a given tool at zero (0)).

It should be that the `readme.md` at the project page which serves as an overview, and this section (which serves as a collection of templates and a tutorial) are all the documentation which most users will need (and arguably is still too much). The balance of the document after this section shows all the code and implementation details, and will where appropriate show examples of usage which will be collected in a plain text template file which is concatenated to provide a usable example of each command with (brief) commentary (potentially serving as a how-to guide as well as documenting the code in a minimalistic fashion) as well as Indices (which serve as a front-end for reference).



Some comments on the templates:

- minimal — each is intended as a framework for a minimal working example (MWE) — it should be possible to comment out unused/unneeded portions and so arrive at code which tests any aspect of this project and which may be used as a starting point for a new part/project
- compleat — a quite wide variety of tools are listed (and probably more will be added in the future), but pre-defining them and having these "hooks" seems the easiest mechanism to handle the requirements of subtractive machining.
- shortcuts — as the various examples show, while in real life it is necessary to make many passes with a tool, an expedient efficiency is to forgo the `loop` operation and just use a `hull()` operation and avoid the requirement of implementing Depth per Pass (but note that this will lose the previewing of scalloped tool marks in places where they might appear otherwise)

One fundamental aspect of this tool is the question of *Layers of Abstraction* (as put forward by Dr. Donald Knuth as the crux of computer science) and *Problem Decomposition* (Prof. John Ousterhout's answer to that question). To a great degree, the basic implementation of this tool will use G-code as a reference implementation, simultaneously using the abstraction from the mechanical task of machining which it affords as a decomposed version of that task, and creating what is in essence, both a front-end, and a tool, and an API for working with G-code programmatically. This then requires an architecture which allows 3D modeling (OpenSCAD), and writing out files (Python).

Further features will be added to the templates as they are created, and the main image updated to reflect the capabilities of the system.

[1] http://literateprogramming.com/
[2] https://www.luatex.org/
[3] https://tug.org/whatis.html
[4] https://www.lua.org/
[5] https://github.com/WillAdams/gcodepreview/blob/main/literati.sty
[6] https://tex.stackexchange.com/questions/722886/how-to-write-out-multiple-text-files-from-multiple-instances-of-

2.1 gcpdxf.py

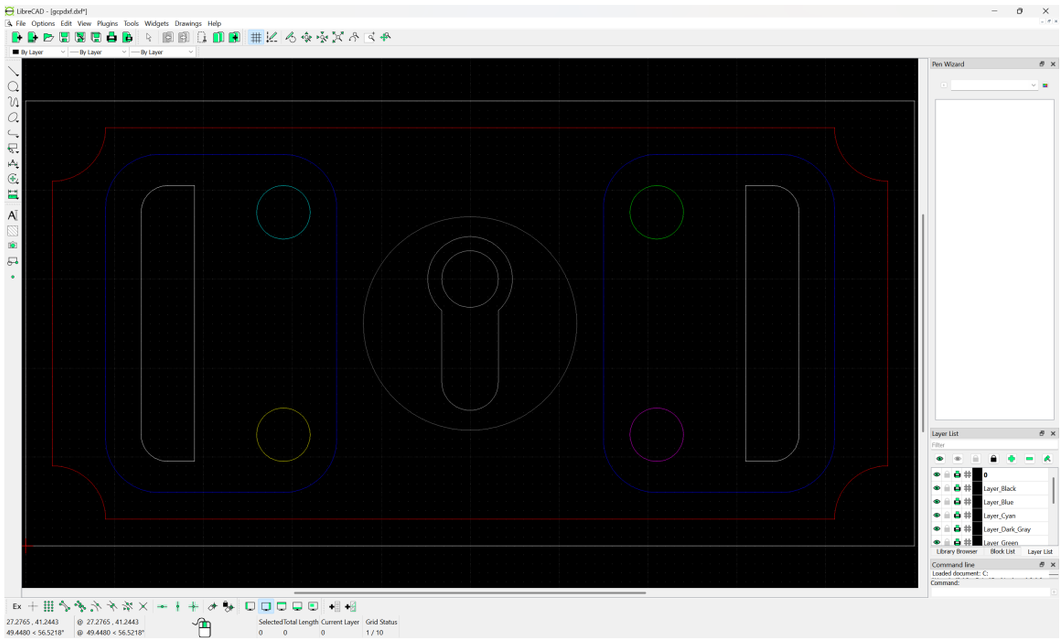
The most basic usage, with the fewest dependencies is to use "plain" Python to create `dxf` files. Note that this example includes an optional command `nimport(<URL>)` which if enabled/uncommented (and the following line commented out), will allow one to use OpenPythonSCAD to import the library from Github, sidestepping the need to download and install the library into an installation of OpenPythonSCAD locally. Usage in "normal" Python will require manually installing the `gcodepreview.py` file where Python can find it. A further consideration is where the file will be placed if the full path is not enumerated, the Desktop is the default destination for Microsoft Windows.

```
1 gcpdxfpy from openscad import *
2 gcpdxfpy     # nimport("https://raw.githubusercontent.com/WillAdams/
                  gcodepreview/refs/heads/main/gcodepreview.py")
3 gcpdxfpy from gcodepreview import *
4 gcpdxfpy
5 gcpdxfpy gcp = gcodepreview("no_preview", # "cut" or "print"
6 gcpdxfpy                     False, # generategcode
7 gcpdxfpy                     True   # generatedxf
8 gcpdxfpy                     )
9 gcpdxfpy
10 gcpdxfpy # [Stock] */
11 gcpdxfpy stockXwidth = 100
12 gcpdxfpy # [Stock] */
13 gcpdxfpy stockYheight = 50
14 gcpdxfpy
15 gcpdxfpy # [Export] */
16 gcpdxfpy Base_filename = "gcpdxf"
17 gcpdxfpy
18 gcpdxfpy
19 gcpdxfpy # [CAM] */
20 gcpdxfpy large_square_tool_num = 102
21 gcpdxfpy # [CAM] */
22 gcpdxfpy small_square_tool_num = 0
23 gcpdxfpy # [CAM] */
24 gcpdxfpy large_ball_tool_num = 0
25 gcpdxfpy # [CAM] */
26 gcpdxfpy small_ball_tool_num = 0
27 gcpdxfpy # [CAM] */
28 gcpdxfpy large_V_tool_num = 0
29 gcpdxfpy # [CAM] */
30 gcpdxfpy small_V_tool_num = 0
31 gcpdxfpy # [CAM] */
32 gcpdxfpy DT_tool_num = 374
33 gcpdxfpy # [CAM] */
34 gcpdxfpy KH_tool_num = 0
35 gcpdxfpy # [CAM] */
36 gcpdxfpy Roundover_tool_num = 0
37 gcpdxfpy # [CAM] */
38 gcpdxfpy MISC_tool_num = 0
39 gcpdxfpy
40 gcpdxfpy # [Design] */
41 gcpdxfpy inset = 3
42 gcpdxfpy # [Design] */
43 gcpdxfpy radius = 6
44 gcpdxfpy # [Design] */
45 gcpdxfpy cornerstyle = "Fillet"  # "Chamfer", "Flipped Fillet"
46 gcpdxfpy
47 gcpdxfpy gcp.opendxffile(Base_filename)
48 gcpdxfpy
49 gcpdxfpy gcp.dxfrectangle(0, 0, stockXwidth, stockYheight)
50 gcpdxfpy
51 gcpdxfpy gcp.setdxfcolor("Red")
52 gcpdxfpy gcp.setdxflayer("Red")
53 gcpdxfpy
54 gcpdxfpy gcp.dxfarc(inset, inset, radius,  0, 90)
55 gcpdxfpy gcp.dxfarc(stockXwidth - inset, inset, radius, 90, 180)
56 gcpdxfpy gcp.dxfarc(stockXwidth - inset, stockYheight - inset, radius, 180,
             270)
57 gcpdxfpy gcp.dxfarc(inset, stockYheight - inset, radius, 270, 360)
58 gcpdxfpy
59 gcpdxfpy gcp.dxfline(inset, inset + radius, inset, stockYheight - (inset +
             radius))
60 gcpdxfpy gcp.dxfline(inset + radius, inset, stockXwidth - (inset + radius),
             inset)
61 gcpdxfpy gcp.dxfline(stockXwidth - inset, inset + radius, stockXwidth -
             inset, stockYheight - (inset + radius))
62 gcpdxfpy gcp.dxfline(inset + radius, stockYheight-inset, stockXwidth - (
```

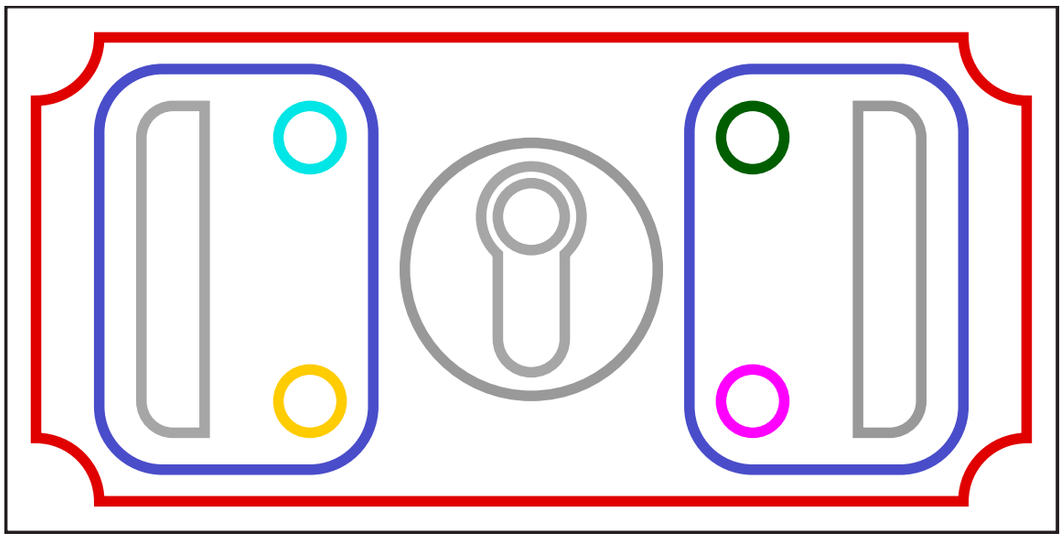
```
            inset + radius), stockYheight - inset)
gcpdxfpy
gcpdxfpy gcp.setdxfcolor("Blue")
gcpdxfpy gcp.setdxflayer("Blue")
gcpdxfpy
gcpdxfpy gcp.dxfrectangle(radius +inset, radius, stockXwidth/2 - (radius *
            4), stockYheight - (radius * 2), cornerstyle, radius)
gcpdxfpy gcp.dxfrectangle(stockXwidth/2 + (radius * 2) + inset, radius,
            stockXwidth/2 - (radius * 4), stockYheight - (radius * 2),
            cornerstyle, radius)
gcpdxfpy
gcpdxfpy gcp.setdxfcolor("Black")
gcpdxfpy gcp.setdxflayer("DEFAULT")
gcpdxfpy
gcpdxfpy gcp.beginpolyline(stockXwidth*0.75+radius*1.5,stockYheight/4-radius
            /2)
gcpdxfpy gcp.addvertex(stockXwidth*0.75+radius,stockYheight/4-radius/2)
gcpdxfpy gcp.addvertex(stockXwidth*0.75+radius,stockYheight*0.75+radius/2)
gcpdxfpy gcp.addvertex(stockXwidth*0.75+radius*1.5,stockYheight*0.75+radius
            /2)
gcpdxfpy gcp.closepolyline()
gcpdxfpy
gcpdxfpy gcp.dxfarc(stockXwidth*0.75+radius*1.5, stockYheight*0.75, radius
            /2,  0, 90)
gcpdxfpy
gcpdxfpy gcp.beginpolyline(stockXwidth*0.75+radius*2,stockYheight*0.75)
gcpdxfpy gcp.addvertex(stockXwidth*0.75+radius*2,stockYheight/4)
gcpdxfpy gcp.closepolyline()
gcpdxfpy
gcpdxfpy gcp.dxfarc(stockXwidth*0.75+radius*1.5, stockYheight/4, radius/2,
            270, 360)
gcpdxfpy
gcpdxfpy gcp.setdxfcolor("Light␣Gray")
gcpdxfpy gcp.setdxflayer("Light␣Gray")
gcpdxfpy
gcpdxfpy gcp.beginpolyline(stockXwidth*0.25-radius*1.5,stockYheight/4-radius
            /2)
gcpdxfpy gcp.addvertex(stockXwidth*0.25-radius,stockYheight/4-radius/2)
gcpdxfpy gcp.addvertex(stockXwidth*0.25-radius,stockYheight*0.75+radius/2)
gcpdxfpy gcp.addvertex(stockXwidth*0.25-radius*1.5,stockYheight*0.75+radius
            /2)
gcpdxfpy gcp.closepolyline()
gcpdxfpy
gcpdxfpy gcp.dxfarc(stockXwidth*0.25-radius*1.5, stockYheight*0.75, radius
            /2,  90, 180)
gcpdxfpy
gcpdxfpy gcp.beginpolyline(stockXwidth*0.25-radius*2,stockYheight*0.75)
gcpdxfpy gcp.addvertex(stockXwidth*0.25-radius*2,stockYheight/4)
gcpdxfpy gcp.closepolyline()
gcpdxfpy
gcpdxfpy gcp.dxfarc(stockXwidth*0.25-radius*1.5, stockYheight/4, radius/2,
            180, 270)
gcpdxfpy
gcpdxfpy gcp.setdxfcolor("Yellow")
gcpdxfpy gcp.setdxflayer("Yellow")
gcpdxfpy gcp.dxfcircle(stockXwidth/4+1+radius/2, stockYheight/4, radius/2)
gcpdxfpy
gcpdxfpy gcp.setdxfcolor("Green")
gcpdxfpy gcp.setdxflayer("Green")
gcpdxfpy gcp.dxfcircle(stockXwidth*0.75-(1+radius/2), stockYheight*0.75,
            radius/2)
gcpdxfpy
gcpdxfpy gcp.setdxfcolor("Cyan")
gcpdxfpy gcp.setdxflayer("Cyan")
gcpdxfpy gcp.dxfcircle(stockXwidth/4+1+radius/2, stockYheight*0.75, radius
            /2)
gcpdxfpy
gcpdxfpy gcp.setdxfcolor("Magenta")
gcpdxfpy gcp.setdxflayer("Magenta")
gcpdxfpy gcp.dxfcircle(stockXwidth*0.75-(1+radius/2), stockYheight/4, radius
            /2)
gcpdxfpy
gcpdxfpy gcp.setdxfcolor("Dark␣Gray")
gcpdxfpy gcp.setdxflayer("Dark␣Gray")
gcpdxfpy gcp.dxfcircle(stockXwidth/2, stockYheight/2, radius * 2)
gcpdxfpy
gcpdxfpy gcp.setdxfcolor("Light␣Gray")
gcpdxfpy gcp.setdxflayer("Light␣Gray")
```

```
126 gcpdxfpy
127 gcpdxfpy gcp.toolchange(374)
128 gcpdxfpy
129 gcpdxfpy gcp.dxfKH(stockXwidth/2, stockYheight/5*3, 0, -7, 270, 11.5875)
130 gcpdxfpy
131 gcpdxfpy gcp.closedxffile()
```

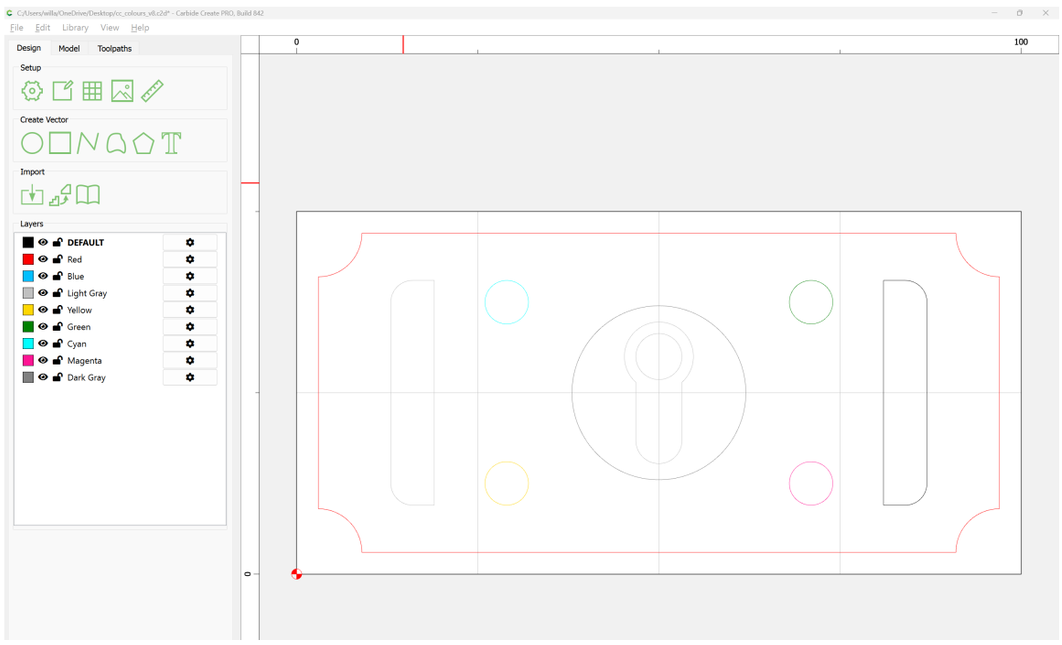
which creates a `.dxf` file which may be imported into any CAD program:



with the appearance (once converted into a `.svg` and then re-saved as a `.pdf`.



and which may be imported into pretty much any CAD or CAM application, e.g., Carbide Create:



As shown/implied by the above code, the following commands/shapes are implemented:

- `dxfrectangle` (specify lower-left corner location for the origin and width (X)/height(Y))
  `dxfrectangleround` (specified as "Fillet" and radius for the round option)
  `dxfrectanglechamfer` (specified as "Chamfer" and radius for the round option)
  `dxfrectangleflippedfillet` (specified as "Flipped Fillet" and radius for the option)
- `dxfcircle` (specifying their center and radius)
- `dxfline` (specifying begin/end points)
- `dxfarc` (specifying arc center, radius, and beginning/ending angles)
- `dxfKH` (specifying origin, depth, angle, distance)

2.2 gcpcutdxf.py

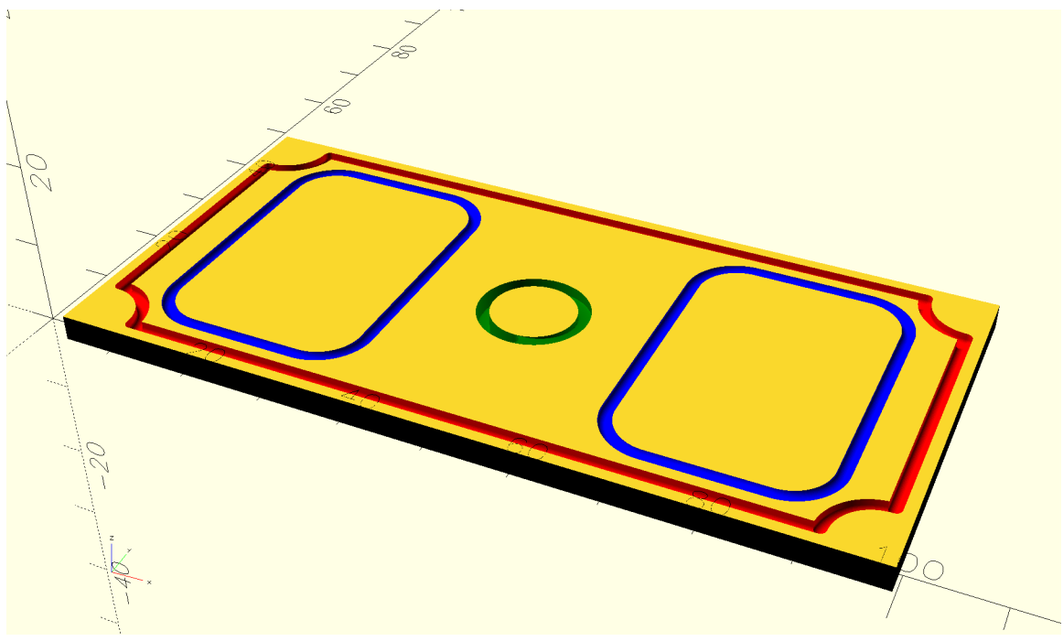
A notable limitation of the above is that there is no interactivity — the `.dxf` file is generated, then must be opened and the result of the run checked (if there is a DXF viewer/editor which will live-reload the file based on it being updated that would be obviated). Reworking the commands for a simplified version of the above design so as to show a 3D model in OpenPythonSCAD is a straight-forward task:

```
1 gcpcutdxfpy from openscad import *
2 gcpcutdxfpy # nimport("https://raw.githubusercontent.com/WillAdams/gcodepreview
      /refs/heads/main/gcodepreview.py")
3 gcpcutdxfpy from gcodepreview import *
4 gcpcutdxfpy
5 gcpcutdxfpy fa = 2
6 gcpcutdxfpy fs = 0.125
7 gcpcutdxfpy
8 gcpcutdxfpy gcp = gcodepreview("cut", # "print" or "no_preview"
9 gcpcutdxfpy                       False, # generategcode
10 gcpcutdxfpy                      True   # generatedxf
11 gcpcutdxfpy                      )
12 gcpcutdxfpy
13 gcpcutdxfpy # [Stock] */
14 gcpcutdxfpy stockXwidth = 100
15 gcpcutdxfpy # [Stock] */
16 gcpcutdxfpy stockYheight = 50
17 gcpcutdxfpy # [Stock] */
18 gcpcutdxfpy stockZthickness = 3.175
19 gcpcutdxfpy # [Stock] */
20 gcpcutdxfpy zeroheight = "Top"  # [Top, Bottom]
21 gcpcutdxfpy # [Stock] */
22 gcpcutdxfpy stockzero = "Lower-Left"  # [Lower-Left, Center-Left, Top-Left,
      Center]
23 gcpcutdxfpy # [Stock] */
24 gcpcutdxfpy retractheight = 3.175
25 gcpcutdxfpy
26 gcpcutdxfpy # [Export] */
```

```
gcpcutdxfpy Base_filename = "gcpdxf"
gcpcutdxfpy
gcpcutdxfpy
gcpcutdxfpy # [CAM] */
gcpcutdxfpy large_square_tool_num = 112
gcpcutdxfpy # [CAM] */
gcpcutdxfpy small_square_tool_num = 0
gcpcutdxfpy # [CAM] */
gcpcutdxfpy large_ball_tool_num = 111
gcpcutdxfpy # [CAM] */
gcpcutdxfpy small_ball_tool_num = 0
gcpcutdxfpy # [CAM] */
gcpcutdxfpy large_V_tool_num = 0
gcpcutdxfpy # [CAM] */
gcpcutdxfpy small_V_tool_num = 0
gcpcutdxfpy # [CAM] */
gcpcutdxfpy DT_tool_num = 374
gcpcutdxfpy # [CAM] */
gcpcutdxfpy KH_tool_num = 0
gcpcutdxfpy # [CAM] */
gcpcutdxfpy Roundover_tool_num = 0
gcpcutdxfpy # [CAM] */
gcpcutdxfpy MISC_tool_num = 0
gcpcutdxfpy
gcpcutdxfpy # [Design] */
gcpcutdxfpy inset = 3
gcpcutdxfpy # [Design] */
gcpcutdxfpy radius = 6
gcpcutdxfpy # [Design] */
gcpcutdxfpy cornerstyle = "Fillet"  # "Chamfer", "Flipped Fillet"
gcpcutdxfpy
gcpcutdxfpy gcp.opendxffile(Base_filename)
gcpcutdxfpy
gcpcutdxfpy gcp.setdxfcolor("Black")
gcpcutdxfpy gcp.setdxflayer("DEFAULT")
gcpcutdxfpy
gcpcutdxfpy gcp.setupstock(stockXwidth, stockYheight, stockZthickness,
                zeroheight, stockzero, retractheight)
gcpcutdxfpy
gcpcutdxfpy gcp.toolchange(large_square_tool_num)
gcpcutdxfpy
gcpcutdxfpy gcp.cutrectangledxf(0, 0, 0, stockXwidth, stockYheight,
                stockZthickness)
gcpcutdxfpy
gcpcutdxfpy gcp.setdxfcolor("Red")
gcpcutdxfpy gcp.setdxflayer("Red")
gcpcutdxfpy
gcpcutdxfpy gcp.toolchange(large_ball_tool_num)
gcpcutdxfpy
gcpcutdxfpy gcp.rapidZ(retractheight)
gcpcutdxfpy gcp.rapid(inset + radius, inset, 0)
gcpcutdxfpy
gcpcutdxfpy gcp.cutline(inset + radius, inset, -stockZthickness/2)
gcpcutdxfpy
gcpcutdxfpy gcp.cutquarterCCNEdxf(inset, inset + radius, -stockZthickness/2,
                radius)
gcpcutdxfpy
gcpcutdxfpy gcp.cutlinedxf(inset, stockYheight - (inset + radius), -
                stockZthickness/2)
gcpcutdxfpy
gcpcutdxfpy gcp.cutquarterCCSEdxf(inset + radius, stockYheight - inset, -
                stockZthickness/2, radius)
gcpcutdxfpy
gcpcutdxfpy gcp.cutlinedxf(stockXwidth - (inset + radius), stockYheight - inset
                , -stockZthickness/2)
gcpcutdxfpy
gcpcutdxfpy gcp.cutquarterCCSWdxf(stockXwidth - inset, stockYheight - (inset +
                radius), -stockZthickness/2, radius)
gcpcutdxfpy
gcpcutdxfpy gcp.cutlinedxf(stockXwidth - (inset), (inset + radius), -
                stockZthickness/2)
gcpcutdxfpy
gcpcutdxfpy gcp.cutquarterCCNWdxf(stockXwidth - (inset + radius), inset, -
                stockZthickness/2, radius)
gcpcutdxfpy
gcpcutdxfpy gcp.cutlinedxf((inset + radius), inset, -stockZthickness/2)
gcpcutdxfpy
gcpcutdxfpy gcp.setdxfcolor("Blue")
```

```
96 gcpcutdxfpy gcp.setdxflayer("Blue")
97 gcpcutdxfpy
98 gcpcutdxfpy gcp.rapidZ(retractheight)
99 gcpcutdxfpy gcp.rapid(radius + inset + radius, radius, 0)
100 gcpcutdxfpy
101 gcpcutdxfpy gcp.cutrectanglerounddxf(radius +inset, radius, 0, stockXwidth/2 -
        (radius * 4), stockYheight - (radius * 2), -stockZthickness/4,
        radius)
102 gcpcutdxfpy
103 gcpcutdxfpy gcp.rapidZ(retractheight)
104 gcpcutdxfpy gcp.rapid(stockXwidth/2 + (radius * 2) + inset + radius, radius, 0)
105 gcpcutdxfpy
106 gcpcutdxfpy gcp.cutrectanglerounddxf(stockXwidth/2 + (radius * 2) + inset,
        radius, 0, stockXwidth/2 - (radius * 4), stockYheight - (radius
        * 2), -stockZthickness/4, radius)
107 gcpcutdxfpy
108 gcpcutdxfpy gcp.setdxfcolor("Green")
109 gcpcutdxfpy gcp.setdxflayer("Green")
110 gcpcutdxfpy
111 gcpcutdxfpy gcp.rapidZ(retractheight)
112 gcpcutdxfpy gcp.rapid(stockXwidth/2, stockYheight/2 - radius, 0)
113 gcpcutdxfpy
114 gcpcutdxfpy gcp.toolchange(large_square_tool_num)
115 gcpcutdxfpy
116 gcpcutdxfpy gcp.cutquarterCCSEdxf(stockXwidth/2 + radius, stockYheight/2, -
        stockZthickness/4, radius)
117 gcpcutdxfpy gcp.cutquarterCCNEdxf(stockXwidth/2, stockYheight/2 + radius, -
        stockZthickness/2, radius)
118 gcpcutdxfpy gcp.cutquarterCCNWdxf(stockXwidth/2 - radius, stockYheight/2, -
        stockZthickness*0.75, radius)
119 gcpcutdxfpy gcp.cutquarterCCSWdxf(stockXwidth/2, stockYheight/2 - radius, -
        stockZthickness, radius)
120 gcpcutdxfpy
121 gcpcutdxfpy gcp.closedxffile()
122 gcpcutdxfpy
123 gcpcutdxfpy gcp.stockandtoolpaths()
```

which creates the design:



and which allows an interactive usage in working up a design such as for lasercutting, and which incorporates an option to the `rapid(x,y,z)` command which simulates turning a laser off, repositioning, then powering up the laser to resume cutting at the new position.

2.3 gcodepreviewtemplate.py

Note that since the v0.7 re-write, it is possible to directly use the underlying Python code. Using Python to generate 3D previews of how DXFs or G-code will cut requires the use of PythonSCAD.

```
1 gcptmplpy #!/usr/bin/env python
2 gcptmplpy
3 gcptmplpy import sys
4 gcptmplpy
5 gcptmplpy try:
```

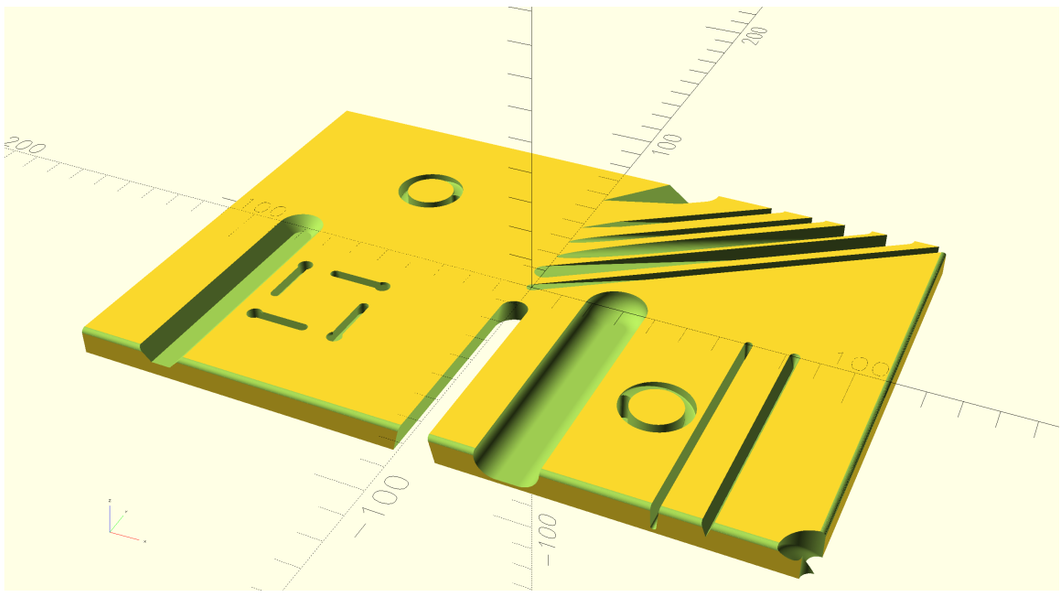
```
gcptmplpy     if 'gcodepreview' in sys.modules:
gcptmplpy         del sys.modules['gcodepreview']
gcptmplpy except AttributeError:
gcptmplpy     pass
gcptmplpy
gcptmplpy from gcodepreview import *
gcptmplpy
gcptmplpy fa = 2
gcptmplpy fs = 0.125
gcptmplpy
gcptmplpy # [Export] */
gcptmplpy Base_filename = "aexport"
gcptmplpy # [Export] */
gcptmplpy generatedxf = True
gcptmplpy # [Export] */
gcptmplpy generategcode = True
gcptmplpy
gcptmplpy # [Stock] */
gcptmplpy stockXwidth = 220
gcptmplpy # [Stock] */
gcptmplpy stockYheight = 150
gcptmplpy # [Stock] */
gcptmplpy stockZthickness = 8.35
gcptmplpy # [Stock] */
gcptmplpy zeroheight = "Top"  # [Top, Bottom]
gcptmplpy # [Stock] */
gcptmplpy stockzero = "Center"  # [Lower-Left, Center-Left, Top-Left, Center]
gcptmplpy # [Stock] */
gcptmplpy retractheight = 9
gcptmplpy
gcptmplpy # [CAM] */
gcptmplpy toolradius = 1.5875
gcptmplpy # [CAM] */
gcptmplpy large_square_tool_num = 201  # [0:0, 112:112, 102:102, 201:201]
gcptmplpy # [CAM] */
gcptmplpy small_square_tool_num = 102  # [0:0, 122:122, 112:112, 102:102]
gcptmplpy # [CAM] */
gcptmplpy large_ball_tool_num = 202  # [0:0, 111:111, 101:101, 202:202]
gcptmplpy # [CAM] */
gcptmplpy small_ball_tool_num = 101  # [0:0, 121:121, 111:111, 101:101]
gcptmplpy # [CAM] */
gcptmplpy large_V_tool_num = 301  # [0:0, 301:301, 690:690]
gcptmplpy # [CAM] */
gcptmplpy small_V_tool_num = 390  # [0:0, 390:390, 301:301]
gcptmplpy # [CAM] */
gcptmplpy DT_tool_num = 814  # [0:0, 814:814, 808079:808079]
gcptmplpy # [CAM] */
gcptmplpy KH_tool_num = 374  # [0:0, 374:374, 375:375, 376:376, 378:378]
gcptmplpy # [CAM] */
gcptmplpy Roundover_tool_num = 56142  # [56142:56142, 56125:56125, 1570:1570]
gcptmplpy # [CAM] */
gcptmplpy MISC_tool_num = 0  # [501:501, 502:502, 45982:45982]
gcptmplpy #501 https://shop.carbide3d.com/collections/cutters/products/501-
              engraving-bit
gcptmplpy #502 https://shop.carbide3d.com/collections/cutters/products/502-
              engraving-bit
gcptmplpy #204 tapered ball nose 0.0625", 0.2500", 1.50", 3.6ř
gcptmplpy #304 tapered ball nose 0.1250", 0.2500", 1.50", 2.4ř
gcptmplpy #648 threadmill_shaft(2.4, 0.75, 18)
gcptmplpy #45982 Carbide Tipped Bowl & Tray 1/4 Radius x 3/4 Dia x 5/8 x 1/4
              Inch Shank
gcptmplpy #13921 https://www.amazon.com/Yonico-Groove-Bottom-Router-Degree/dp
              /B0CPJPTMPP
gcptmplpy
gcptmplpy # [Feeds and Speeds] */
gcptmplpy plunge = 100
gcptmplpy # [Feeds and Speeds] */
gcptmplpy feed = 400
gcptmplpy # [Feeds and Speeds] */
gcptmplpy speed = 16000
gcptmplpy # [Feeds and Speeds] */
gcptmplpy small_square_ratio = 0.75  # [0.25:2]
gcptmplpy # [Feeds and Speeds] */
gcptmplpy large_ball_ratio = 1.0  # [0.25:2]
gcptmplpy # [Feeds and Speeds] */
gcptmplpy small_ball_ratio = 0.75  # [0.25:2]
gcptmplpy # [Feeds and Speeds] */
gcptmplpy large_V_ratio = 0.875  # [0.25:2]
```

```
gcptmplpy # [Feeds and Speeds] */
gcptmplpy small_V_ratio = 0.625  # [0.25:2]
gcptmplpy # [Feeds and Speeds] */
gcptmplpy DT_ratio = 0.75  # [0.25:2]
gcptmplpy # [Feeds and Speeds] */
gcptmplpy KH_ratio = 0.75  # [0.25:2]
gcptmplpy # [Feeds and Speeds] */
gcptmplpy RO_ratio = 0.5  # [0.25:2]
gcptmplpy # [Feeds and Speeds] */
gcptmplpy MISC_ratio = 0.5  # [0.25:2]
gcptmplpy
gcptmplpy # Note that the various ratios are simply declared as a possible
    hook
gcptmplpy # which might be useful and how are handled is left as an exercise
gcptmplpy # for the reader and that they are not applied below.
gcptmplpy # One naive option might be to multiply by the feed rate
gcptmplpy # and divide by speeds.
gcptmplpy
gcptmplpy gcp = gcodepreview("cut", # "print" or "no_preview"
gcptmplpy                    generategcode,
gcptmplpy                    generatedxf,
gcptmplpy                    )
gcptmplpy
gcptmplpy gcp.opengcodefile(Base_filename)
gcptmplpy gcp.opendxffile(Base_filename)
gcptmplpy
gcptmplpy gcp.setupstock(stockXwidth, stockYheight, stockZthickness,
    zeroheight, stockzero, retractheight)
gcptmplpy
gcptmplpy gcp.movetosafeZ()
gcptmplpy
gcptmplpy gcp.toolchange(102, 10000 * small_square_ratio)
gcptmplpy
gcptmplpy gcp.rapidZ(0)
gcptmplpy
gcptmplpy gcp.cutlinedxfgc(stockXwidth/2, stockYheight/2, -stockZthickness)
gcptmplpy
gcptmplpy gcp.rapidZ(retractheight)
gcptmplpy gcp.toolchange(201, 10000)
gcptmplpy gcp.rapidXY(0, stockYheight/16)
gcptmplpy gcp.rapidZ(0)
gcptmplpy gcp.cutlinedxfgc(stockXwidth/16*7, stockYheight/2, -stockZthickness
    )
gcptmplpy
gcptmplpy gcp.rapidZ(retractheight)
gcptmplpy gcp.toolchange(202, 10000)
gcptmplpy gcp.rapidXY(0, stockYheight/8)
gcptmplpy gcp.rapidZ(0)
gcptmplpy gcp.cutlinedxfgc(stockXwidth/16*6, stockYheight/2, -stockZthickness
    )
gcptmplpy
gcptmplpy gcp.rapidZ(retractheight)
gcptmplpy gcp.toolchange(101, 10000)
gcptmplpy gcp.rapidXY(0, stockYheight/16*3)
gcptmplpy gcp.rapidZ(0)
gcptmplpy gcp.cutlinedxfgc(stockXwidth/16*5, stockYheight/2, -stockZthickness
    )
gcptmplpy
gcptmplpy gcp.setzpos(retractheight)
gcptmplpy gcp.toolchange(390, 10000)
gcptmplpy gcp.rapidXY(0, stockYheight/16*4)
gcptmplpy gcp.rapidZ(0)
gcptmplpy gcp.cutlinedxfgc(stockXwidth/16*4, stockYheight/2, -stockZthickness
    )
gcptmplpy gcp.rapidZ(retractheight)
gcptmplpy
gcptmplpy gcp.toolchange(301, 10000)
gcptmplpy gcp.rapidXY(0, stockYheight/16*6)
gcptmplpy gcp.rapidZ(0)
gcptmplpy gcp.cutlinedxfgc(stockXwidth/16*2, stockYheight/2, -stockZthickness
    )
gcptmplpy
gcptmplpy rapids = gcp.rapid(gcp.xpos(), gcp.ypos(), retractheight)
gcptmplpy gcp.toolchange(102, 10000)
gcptmplpy
gcptmplpy gcp.rapid(-stockXwidth/4+stockYheight/16, +stockYheight/4, 0)
gcptmplpy
gcptmplpy #gcp.cutarcCC(0, 90, gcp.xpos()-stockYheight/16, gcp.ypos(),
```

```
                stockYheight/16, -stockZthickness/4)
151 gcptmplpy #gcp.cutarcCC(90, 180, gcp.xpos(), gcp.ypos()-stockYheight/16,
                stockYheight/16, -stockZthickness/4)
152 gcptmplpy #gcp.cutarcCC(180, 270, gcp.xpos()+stockYheight/16, gcp.ypos(),
                stockYheight/16, -stockZthickness/4)
153 gcptmplpy #gcp.cutarcCC(270, 360, gcp.xpos(), gcp.ypos()+stockYheight/16,
                stockYheight/16, -stockZthickness/4)
154 gcptmplpy gcp.cutquarterCCNEdxf(gcp.xpos() - stockYheight/8, gcp.ypos() +
                stockYheight/8, -stockZthickness/4, stockYheight/8)
155 gcptmplpy gcp.cutquarterCCNWdxf(gcp.xpos() - stockYheight/8, gcp.ypos() -
                stockYheight/8, -stockZthickness/2, stockYheight/8)
156 gcptmplpy gcp.cutquarterCCSWdxf(gcp.xpos() + stockYheight/8, gcp.ypos() -
                stockYheight/8, -stockZthickness * 0.75, stockYheight/8)
157 gcptmplpy gcp.cutquarterCCSEdxf(gcp.xpos() + stockYheight/8, gcp.ypos() +
                stockYheight/8, -stockZthickness, stockYheight/8)
158 gcptmplpy
159 gcptmplpy gcp.movetosafeZ()
160 gcptmplpy gcp.rapidXY(stockXwidth/4-stockYheight/16, -stockYheight/4)
161 gcptmplpy gcp.rapidZ(0)
162 gcptmplpy
163 gcptmplpy
164 gcptmplpy #gcp.cutarcCW(180, 90, gcp.xpos()+stockYheight/16, gcp.ypos(),
                stockYheight/16, -stockZthickness/4)
165 gcptmplpy #gcp.cutarcCW(90, 0, gcp.xpos(), gcp.ypos()-stockYheight/16,
                stockYheight/16, -stockZthickness/4)
166 gcptmplpy #gcp.cutarcCW(360, 270, gcp.xpos()-stockYheight/16, gcp.ypos(),
                stockYheight/16, -stockZthickness/4)
167 gcptmplpy #gcp.cutarcCW(270, 180, gcp.xpos(), gcp.ypos()+stockYheight/16,
                stockYheight/16, -stockZthickness/4)
168 gcptmplpy
169 gcptmplpy #gcp.movetosafeZ()
170 gcptmplpy #gcp.toolchange(201, 10000)
171 gcptmplpy #gcp.rapidXY(stockXwidth/2, -stockYheight/2)
172 gcptmplpy #gcp.rapidZ(0)
173 gcptmplpy
174 gcptmplpy #gcp.cutlinedxfgc(gcp.xpos(), gcp.ypos(), -stockZthickness)
175 gcptmplpy #test = gcp.cutlinedxfgc(gcp.xpos(), gcp.ypos(), -stockZthickness)
176 gcptmplpy
177 gcptmplpy #gcp.movetosafeZ()
178 gcptmplpy #gcp.rapidXY(stockXwidth/2-6.34, -stockYheight/2)
179 gcptmplpy #gcp.rapidZ(0)
180 gcptmplpy
181 gcptmplpy #gcp.cutarcCW(180, 90, stockXwidth/2, -stockYheight/2, 6.34, -
                stockZthickness)
182 gcptmplpy
183 gcptmplpy
184 gcptmplpy gcp.movetosafeZ()
185 gcptmplpy gcp.toolchange(814, 10000)
186 gcptmplpy gcp.rapidXY(0, -(stockYheight/2+12.7))
187 gcptmplpy gcp.rapidZ(0)
188 gcptmplpy
189 gcptmplpy gcp.cutlinedxfgc(gcp.xpos(), gcp.ypos(), -stockZthickness)
190 gcptmplpy gcp.cutlinedxfgc(gcp.xpos(), -12.7, -stockZthickness)
191 gcptmplpy
192 gcptmplpy gcp.rapidXY(0, -(stockYheight/2+12.7))
193 gcptmplpy gcp.movetosafeZ()
194 gcptmplpy gcp.toolchange(374, 10000)
195 gcptmplpy gcp.rapidXY(stockXwidth/4-stockXwidth/16, -(stockYheight/4+
                stockYheight/16))
196 gcptmplpy gcp.rapidZ(0)
197 gcptmplpy
198 gcptmplpy gcp.rapidZ(retractheight)
199 gcptmplpy gcp.toolchange(374, 10000)
200 gcptmplpy gcp.rapidXY(-stockXwidth/4-stockXwidth/16, -(stockYheight/4+
                stockYheight/16))
201 gcptmplpy gcp.rapidZ(0)
202 gcptmplpy
203 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -stockZthickness/2)
204 gcptmplpy gcp.cutlinedxfgc(gcp.xpos()+stockYheight/9, gcp.ypos(), gcp.zpos())
205 gcptmplpy
206 gcptmplpy gcp.cutline(gcp.xpos()-stockYheight/9, gcp.ypos(), gcp.zpos())
207 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), 0)
208 gcptmplpy
209 gcptmplpy #key = gcp.cutkeyholegcdxf(KH_tool_num, 0, stockZthickness*0.75, "E
                ", stockYheight/9)
210 gcptmplpy #key = gcp.cutKHgcdxf(374, 0, stockZthickness*0.75, 90,
                stockYheight/9)
211 gcptmplpy #toolpaths = toolpaths.union(key)
```

```
212 gcptmplpy
213 gcptmplpy gcp.rapidZ(retractheight)
214 gcptmplpy gcp.rapidXY(-stockXwidth/4+stockXwidth/16, -(stockYheight/4+
                stockYheight/16))
215 gcptmplpy gcp.rapidZ(0)
216 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -stockZthickness/2)
217 gcptmplpy gcp.cutlinedxfgc(gcp.xpos(), gcp.ypos()+stockYheight/9, gcp.zpos())
218 gcptmplpy
219 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos()-stockYheight/9, gcp.zpos())
220 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), 0)
221 gcptmplpy
222 gcptmplpy gcp.rapidZ(retractheight)
223 gcptmplpy gcp.rapidXY(-stockXwidth/4+stockXwidth/16, -(stockYheight/4-
                stockYheight/8))
224 gcptmplpy gcp.rapidZ(0)
225 gcptmplpy
226 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -stockZthickness/2)
227 gcptmplpy gcp.cutlinedxfgc(gcp.xpos()-stockYheight/9, gcp.ypos(), gcp.zpos())
228 gcptmplpy
229 gcptmplpy gcp.cutline(gcp.xpos()+stockYheight/9, gcp.ypos(), gcp.zpos())
230 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), 0)
231 gcptmplpy
232 gcptmplpy gcp.rapidZ(retractheight)
233 gcptmplpy gcp.rapidXY(-stockXwidth/4-stockXwidth/16, -(stockYheight/4-
                stockYheight/8))
234 gcptmplpy gcp.rapidZ(0)
235 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -stockZthickness/2)
236 gcptmplpy gcp.cutlinedxfgc(gcp.xpos(), gcp.ypos()-stockYheight/9, gcp.zpos())
237 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos()+stockYheight/9, gcp.zpos())
238 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), 0)
239 gcptmplpy
240 gcptmplpy gcp.rapidZ(retractheight)
241 gcptmplpy gcp.toolchange(56142, 10000)
242 gcptmplpy gcp.rapidXY(-stockXwidth/2, -(stockYheight/2+0.508/2))
243 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -1.531)
244 gcptmplpy gcp.cutlinedxfgc(stockXwidth/2+0.508/2, -(stockYheight/2+0.508/2),
                -1.531)
245 gcptmplpy
246 gcptmplpy gcp.rapidZ(retractheight)
247 gcptmplpy
248 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -1.531)
249 gcptmplpy gcp.cutlinedxfgc(stockXwidth/2+0.508/2, (stockYheight/2+0.508/2),
                -1.531)
250 gcptmplpy
251 gcptmplpy gcp.rapidZ(retractheight)
252 gcptmplpy gcp.toolchange(45982, 10000)
253 gcptmplpy gcp.rapidXY(stockXwidth/8, 0)
254 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -(stockZthickness*7/8))
255 gcptmplpy gcp.cutlinedxfgc(gcp.xpos(), -stockYheight/2, -(stockZthickness
                *7/8))
256 gcptmplpy
257 gcptmplpy gcp.rapidZ(retractheight)
258 gcptmplpy gcp.toolchange(204, 10000)
259 gcptmplpy gcp.rapidXY(stockXwidth*0.3125, 0)
260 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -(stockZthickness*7/8))
261 gcptmplpy gcp.cutlinedxfgc(gcp.xpos(), -stockYheight/2, -(stockZthickness
                *7/8))
262 gcptmplpy
263 gcptmplpy gcp.rapidZ(retractheight)
264 gcptmplpy gcp.toolchange(502, 10000)
265 gcptmplpy gcp.rapidXY(stockXwidth*0.375, 0)
266 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -4.24)
267 gcptmplpy gcp.cutlinedxfgc(gcp.xpos(), -stockYheight/2, -4.24)
268 gcptmplpy
269 gcptmplpy gcp.rapidZ(retractheight)
270 gcptmplpy gcp.toolchange(13921, 10000)
271 gcptmplpy gcp.rapidXY(-stockXwidth*0.375, 0)
272 gcptmplpy gcp.cutline(gcp.xpos(), gcp.ypos(), -stockZthickness/2)
273 gcptmplpy gcp.cutlinedxfgc(gcp.xpos(), -stockYheight/2, -stockZthickness/2)
274 gcptmplpy
275 gcptmplpy gcp.rapidZ(retractheight)
276 gcptmplpy
277 gcptmplpy gcp.stockandtoolpaths()
278 gcptmplpy
279 gcptmplpy gcp.closegcodefile()
280 gcptmplpy gcp.closedxffile()
```

Which generates a 3D model which previews in PythonSCAD as:



2.4 gcodepreviewtemplate.scad

Since the project began in OpenSCAD, having an implementation in that language has always been a goal. This is quite straight-forward since the Python code when imported into OpenSCAD may be accessed by quite simple modules which are for the most part, a series of decorators/descriptors which wrap up the Python definitions as OpenSCAD modules. Moreover, such an implementation will facilitate usage by tools intended for this application such as OpenSCAD Graph Editor: https://github.com/derkork/openscad-graph-editor.

```
1 gcptmplscad //!OpenSCAD
2 gcptmplscad
3 gcptmplscad use <gcodepreview.py>
4 gcptmplscad include <gcodepreview.scad>
5 gcptmplscad
6 gcptmplscad $fn = $preview ? 32 : 256;
7 gcptmplscad fn = $preview ? 32 : 256;
8 gcptmplscad
9 gcptmplscad /* [Stock] */
10 gcptmplscad stockXwidth = 220;
11 gcptmplscad /* [Stock] */
12 gcptmplscad stockYheight = 150;
13 gcptmplscad /* [Stock] */
14 gcptmplscad stockZthickness = 8.35;
15 gcptmplscad /* [Stock] */
16 gcptmplscad zeroheight = "Top"; // [Top, Bottom]
17 gcptmplscad /* [Stock] */
18 gcptmplscad stockzero = "Center"; // [Lower-Left, Center-Left, Top-Left, Center
       ]
19 gcptmplscad /* [Stock] */
20 gcptmplscad retractheight = 9;
21 gcptmplscad
22 gcptmplscad /* [Export] */
23 gcptmplscad Base_filename = "export";
24 gcptmplscad /* [Export] */
25 gcptmplscad generatedxf = true;
26 gcptmplscad /* [Export] */
27 gcptmplscad generategcode = true;
28 gcptmplscad
29 gcptmplscad /* [CAM] */
30 gcptmplscad toolradius = 1.5875;
31 gcptmplscad /* [CAM] */
32 gcptmplscad large_square_tool_num = 0; // [0:0, 112:112, 102:102, 201:201]
33 gcptmplscad /* [CAM] */
34 gcptmplscad small_square_tool_num = 102; // [0:0, 122:122, 112:112, 102:102]
35 gcptmplscad /* [CAM] */
36 gcptmplscad large_ball_tool_num = 0; // [0:0, 111:111, 101:101, 202:202]
37 gcptmplscad /* [CAM] */
38 gcptmplscad small_ball_tool_num = 0; // [0:0, 121:121, 111:111, 101:101]
39 gcptmplscad /* [CAM] */
40 gcptmplscad large_V_tool_num = 0; // [0:0, 301:301, 690:690]
41 gcptmplscad /* [CAM] */
42 gcptmplscad small_V_tool_num = 0; // [0:0, 390:390, 301:301]
43 gcptmplscad /* [CAM] */
```

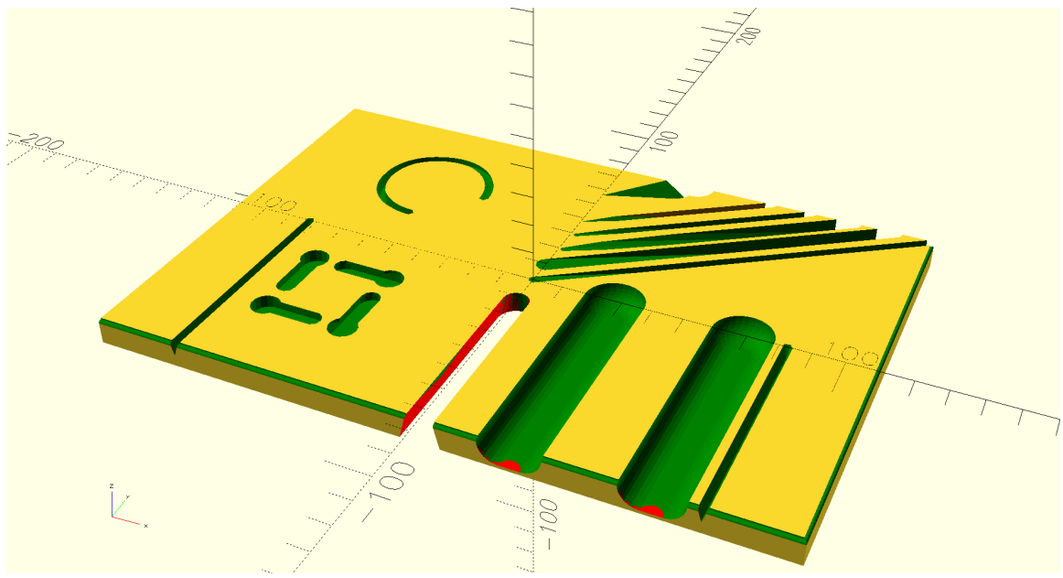
```
gcptmplscad DT_tool_num = 0; // [0:0, 814:814, 808079:808079]
gcptmplscad /* [CAM] */
gcptmplscad KH_tool_num = 0; // [0:0, 374:374, 375:375, 376:376, 378:378]
gcptmplscad /* [CAM] */
gcptmplscad Roundover_tool_num = 0; // [56142:56142, 56125:56125, 1570:1570]
gcptmplscad /* [CAM] */
gcptmplscad MISC_tool_num = 0; // [648:648, 45982:45982]
gcptmplscad //648 threadmill_shaft(2.4, 0.75, 18)
gcptmplscad //45982 Carbide Tipped Bowl & Tray 1/4 Radius x 3/4 Dia x 5/8 x 1/4 Inch Shank
gcptmplscad
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad plunge = 100;
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad feed = 400;
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad speed = 16000;
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad small_square_ratio = 0.75; // [0.25:2]
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad large_ball_ratio = 1.0; // [0.25:2]
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad small_ball_ratio = 0.75; // [0.25:2]
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad large_V_ratio = 0.875; // [0.25:2]
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad small_V_ratio = 0.625; // [0.25:2]
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad DT_ratio = 0.75; // [0.25:2]
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad KH_ratio = 0.75; // [0.25:2]
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad RO_ratio = 0.5; // [0.25:2]
gcptmplscad /* [Feeds and Speeds] */
gcptmplscad MISC_ratio = 0.5; // [0.25:2]
gcptmplscad
gcptmplscad /* [DXF] */
gcptmplscad active_layer = "DEFAULT"; // ["Default", "0"]
gcptmplscad /* [DXF] */
gcptmplscad active_colour = "Black"; // ["Black", "Red", "Green", "Blue", "Cyan", "Magenta", "Dark Gray", "Light Gray", "Yellow", "White"]
gcptmplscad
gcptmplscad thegeneratedxf = generatedxf == true ? 1 : 0;
gcptmplscad thegenerategcode = generategcode == true ? 1 : 0;
gcptmplscad
gcptmplscad gcp = gcodepreview("cut", // or "print" (no preview not suited to OpenSCAD)
gcptmplscad                    thegenerategcode,
gcptmplscad                    thegeneratedxf,
gcptmplscad                    );
gcptmplscad
gcptmplscad opengcodefile(Base_filename);
gcptmplscad opendxffile(Base_filename);
gcptmplscad
gcptmplscad setupstock(stockXwidth, stockYheight, stockZthickness, zeroheight, stockzero);
gcptmplscad
gcptmplscad //echo(gcp);
gcptmplscad //gcpversion();
gcptmplscad
gcptmplscad //c = myfunc(4);
gcptmplscad //echo(c);
gcptmplscad
gcptmplscad //echo(getvv());
gcptmplscad
gcptmplscad cutline(stockXwidth/2, stockYheight/2, -stockZthickness);
gcptmplscad
gcptmplscad rapidZ(retractheight);
gcptmplscad toolchange(201, 10000);
gcptmplscad rapidXY(0, stockYheight/16);
gcptmplscad rapidZ(0);
gcptmplscad cutlinedxfgc(stockXwidth/16*7, stockYheight/2, -stockZthickness);
gcptmplscad
gcptmplscad
gcptmplscad rapidZ(retractheight);
gcptmplscad toolchange(202, 10000);
gcptmplscad rapidXY(0, stockYheight/8);
gcptmplscad rapidZ(0);
```

```
118 gcptmplscad cutlinedxfgc(stockXwidth/16*6, stockYheight/2, -stockZthickness);
119 gcptmplscad
120 gcptmplscad rapidZ(retractheight);
121 gcptmplscad toolchange(101, 10000);
122 gcptmplscad rapidXY(0, stockYheight/16*3);
123 gcptmplscad rapidZ(0);
124 gcptmplscad cutlinedxfgc(stockXwidth/16*5, stockYheight/2, -stockZthickness);
125 gcptmplscad
126 gcptmplscad rapidZ(retractheight);
127 gcptmplscad toolchange(390, 10000);
128 gcptmplscad rapidXY(0, stockYheight/16*4);
129 gcptmplscad rapidZ(0);
130 gcptmplscad
131 gcptmplscad cutlinedxfgc(stockXwidth/16*4, stockYheight/2, -stockZthickness);
132 gcptmplscad rapidZ(retractheight);
133 gcptmplscad
134 gcptmplscad toolchange(301, 10000);
135 gcptmplscad rapidXY(0, stockYheight/16*6);
136 gcptmplscad rapidZ(0);
137 gcptmplscad
138 gcptmplscad cutlinedxfgc(stockXwidth/16*2, stockYheight/2, -stockZthickness);
139 gcptmplscad
140 gcptmplscad
141 gcptmplscad movetosafeZ();
142 gcptmplscad rapid(gcp.xpos(), gcp.ypos(), retractheight);
143 gcptmplscad toolchange(102, 10000);
144 gcptmplscad
145 gcptmplscad //rapidXY(stockXwidth/4+stockYheight/8+stockYheight/16, +
                stockYheight/8);
146 gcptmplscad rapidXY(-stockXwidth/4+stockXwidth/16, (stockYheight/4));//+
                stockYheight/16
147 gcptmplscad rapidZ(0);
148 gcptmplscad
149 gcptmplscad //cutarcCW(360, 270, gcp.xpos()-stockYheight/16, gcp.ypos(),
                stockYheight/16, -stockZthickness);
150 gcptmplscad //gcp.cutarcCW(270, 180, gcp.xpos(), gcp.ypos()+stockYheight/16,
                stockYheight/16))
151 gcptmplscad //cutarcCC(0, 90, gcp.xpos()-stockYheight/16, gcp.ypos(),
                stockYheight/16, -stockZthickness/4);
152 gcptmplscad //cutarcCC(90, 180, gcp.xpos(), gcp.ypos()-stockYheight/16,
                stockYheight/16, -stockZthickness/4);
153 gcptmplscad //cutarcCC(180, 270, gcp.xpos()+stockYheight/16, gcp.ypos(),
                stockYheight/16, -stockZthickness/4);
154 gcptmplscad //cutarcCC(270, 360, gcp.xpos(), gcp.ypos()+stockYheight/16,
                stockYheight/16, -stockZthickness/4);
155 gcptmplscad
156 gcptmplscad movetosafeZ();
157 gcptmplscad //rapidXY(stockXwidth/4+stockYheight/8-stockYheight/16, -
                stockYheight/8);
158 gcptmplscad rapidXY(stockXwidth/4-stockYheight/16, -(stockYheight/4));
159 gcptmplscad rapidZ(0);
160 gcptmplscad
161 gcptmplscad //cutarcCW(180, 90, gcp.xpos()+stockYheight/16, gcp.ypos(),
                stockYheight/16, -stockZthickness/4);
162 gcptmplscad //cutarcCW(90, 0, gcp.xpos(), gcp.ypos()-stockYheight/16,
                stockYheight/16, -stockZthickness/4);
163 gcptmplscad //cutarcCW(360, 270, gcp.xpos()-stockYheight/16, gcp.ypos(),
                stockYheight/16, -stockZthickness/4);
164 gcptmplscad //cutarcCW(270, 180, gcp.xpos(), gcp.ypos()+stockYheight/16,
                stockYheight/16, -stockZthickness/4);
165 gcptmplscad
166 gcptmplscad movetosafeZ();
167 gcptmplscad
168 gcptmplscad rapidXY(-stockXwidth/4 + stockYheight/8, (stockYheight/4));
169 gcptmplscad rapidZ(0);
170 gcptmplscad
171 gcptmplscad cutquarterCCNEdxf(xpos() - stockYheight/8, ypos() + stockYheight/8,
                -stockZthickness/4, stockYheight/8);
172 gcptmplscad cutquarterCCNWdxf(xpos() - stockYheight/8, ypos() - stockYheight/8,
                -stockZthickness/2, stockYheight/8);
173 gcptmplscad cutquarterCCSWdxf(xpos() + stockYheight/8, ypos() - stockYheight/8,
                -stockZthickness * 0.75, stockYheight/8);
174 gcptmplscad //cutquarterCCSEdxf(xpos() + stockYheight/8, ypos() + stockYheight
                /8, -stockZthickness, stockYheight/8);
175 gcptmplscad
176 gcptmplscad movetosafeZ();
177 gcptmplscad toolchange(201, 10000);
178 gcptmplscad rapidXY(stockXwidth /2 -6.34, - stockYheight /2);
```

```
179 gcptmplscad rapidZ(0);
180 gcptmplscad //cutarcCW(180, 90, stockXwidth /2, -stockYheight/2, 6.34, -
                stockZthickness);
181 gcptmplscad
182 gcptmplscad movetosafeZ();
183 gcptmplscad rapidXY(stockXwidth/2, -stockYheight/2);
184 gcptmplscad rapidZ(0);
185 gcptmplscad
186 gcptmplscad //gcp.cutlinedxfgc(gcp.xpos(), gcp.ypos(), -stockZthickness);
187 gcptmplscad
188 gcptmplscad movetosafeZ();
189 gcptmplscad toolchange(814, 10000);
190 gcptmplscad rapidXY(0, -(stockYheight/2+12.7));
191 gcptmplscad rapidZ(0);
192 gcptmplscad
193 gcptmplscad cutlinedxfgc(xpos(), ypos(), -stockZthickness);
194 gcptmplscad cutlinedxfgc(xpos(), -12.7, -stockZthickness);
195 gcptmplscad rapidXY(0, -(stockYheight/2+12.7));
196 gcptmplscad
197 gcptmplscad //rapidXY(stockXwidth/2-6.34, -stockYheight/2);
198 gcptmplscad //rapidZ(0);
199 gcptmplscad
200 gcptmplscad //movetosafeZ();
201 gcptmplscad //toolchange(374, 10000);
202 gcptmplscad //rapidXY(-(stockXwidth/4 - stockXwidth /16), -(stockYheight/4 +
                stockYheight/16))
203 gcptmplscad
204 gcptmplscad //cutline(xpos(), ypos(), (stockZthickness/2) * -1);
205 gcptmplscad //cutlinedxfgc(xpos() + stockYheight /9, ypos(), zpos());
206 gcptmplscad //cutline(xpos() - stockYheight /9, ypos(), zpos());
207 gcptmplscad //cutline(xpos(), ypos(), 0);
208 gcptmplscad
209 gcptmplscad movetosafeZ();
210 gcptmplscad
211 gcptmplscad toolchange(374, 10000);
212 gcptmplscad rapidXY(-stockXwidth/4-stockXwidth/16, -(stockYheight/4+
                stockYheight/16))
213 gcptmplscad //rapidXY(-(stockXwidth/4 - stockXwidth /16), -(stockYheight/4 +
                stockYheight/16))
214 gcptmplscad rapidZ(0);
215 gcptmplscad
216 gcptmplscad cutline(xpos(), ypos(), (stockZthickness/2) * -1);
217 gcptmplscad cutlinedxfgc(xpos() + stockYheight /9, ypos(), zpos());
218 gcptmplscad cutline(xpos() - stockYheight /9, ypos(), zpos());
219 gcptmplscad cutline(xpos(), ypos(), 0);
220 gcptmplscad
221 gcptmplscad rapidZ(retractheight);
222 gcptmplscad rapidXY(-stockXwidth/4+stockXwidth/16, -(stockYheight/4+
                stockYheight/16));
223 gcptmplscad rapidZ(0);
224 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), -stockZthickness/2);
225 gcptmplscad cutlinedxfgc(gcp.xpos(), gcp.ypos()+stockYheight/9, gcp.zpos());
226 gcptmplscad cutline(gcp.xpos(), gcp.ypos()-stockYheight/9, gcp.zpos());
227 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), 0);
228 gcptmplscad
229 gcptmplscad rapidZ(retractheight);
230 gcptmplscad rapidXY(-stockXwidth/4+stockXwidth/16, -(stockYheight/4-
                stockYheight/8));
231 gcptmplscad rapidZ(0);
232 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), -stockZthickness/2);
233 gcptmplscad cutlinedxfgc(gcp.xpos()-stockYheight/9, gcp.ypos(), gcp.zpos());
234 gcptmplscad cutline(gcp.xpos()+stockYheight/9, gcp.ypos(), gcp.zpos());
235 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), 0);
236 gcptmplscad
237 gcptmplscad rapidZ(retractheight);
238 gcptmplscad rapidXY(-stockXwidth/4-stockXwidth/16, -(stockYheight/4-
                stockYheight/8));
239 gcptmplscad rapidZ(0);
240 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), -stockZthickness/2);
241 gcptmplscad cutlinedxfgc(gcp.xpos(), gcp.ypos()-stockYheight/9, gcp.zpos());
242 gcptmplscad cutline(gcp.xpos(), gcp.ypos()+stockYheight/9, gcp.zpos());
243 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), 0);
244 gcptmplscad
245 gcptmplscad rapidZ(retractheight);
246 gcptmplscad toolchange(45982, 10000);
247 gcptmplscad rapidXY(stockXwidth/8, 0);
248 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), -(stockZthickness*7/8));
249 gcptmplscad cutlinedxfgc(gcp.xpos(), -stockYheight/2, -(stockZthickness*7/8));
```

```
250 gcptmplscad
251 gcptmplscad rapidZ(retractheight);
252 gcptmplscad toolchange(204, 10000);
253 gcptmplscad rapidXY(stockXwidth*0.3125, 0);
254 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), -(stockZthickness*7/8));
255 gcptmplscad cutlinedxfgc(gcp.xpos(), -stockYheight/2, -(stockZthickness*7/8));
256 gcptmplscad
257 gcptmplscad rapidZ(retractheight);
258 gcptmplscad toolchange(502, 10000);
259 gcptmplscad rapidXY(stockXwidth*0.375, 0);
260 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), -4.24);
261 gcptmplscad cutlinedxfgc(gcp.xpos(), -stockYheight/2, -4.24);
262 gcptmplscad
263 gcptmplscad rapidZ(retractheight);
264 gcptmplscad toolchange(13921, 10000);
265 gcptmplscad rapidXY(-stockXwidth*0.375, 0);
266 gcptmplscad cutline(gcp.xpos(), gcp.ypos(), -stockZthickness/2);
267 gcptmplscad cutlinedxfgc(gcp.xpos(), -stockYheight/2, -stockZthickness/2);
268 gcptmplscad
269 gcptmplscad rapidZ(retractheight);
270 gcptmplscad gcp.toolchange(56142, 10000);
271 gcptmplscad gcp.rapidXY(-stockXwidth/2, -(stockYheight/2+0.508/2));
272 gcptmplscad cutlineZgcfeed(-1.531, plunge);
273 gcptmplscad //cutline(gcp.xpos(), gcp.ypos(), -1.531);
274 gcptmplscad cutlinedxfgc(stockXwidth/2+0.508/2, -(stockYheight/2+0.508/2),
                -1.531);
275 gcptmplscad
276 gcptmplscad rapidZ(retractheight);
277 gcptmplscad //#gcp.toolchange(56125, 10000)
278 gcptmplscad cutlineZgcfeed(-1.531, plunge);
279 gcptmplscad //toolpaths.append(gcp.cutline(gcp.xpos(), gcp.ypos(), -1.531))
280 gcptmplscad cutlinedxfgc(stockXwidth/2+0.508/2, (stockYheight/2+0.508/2),
                -1.531);
281 gcptmplscad
282 gcptmplscad stockandtoolpaths();
283 gcptmplscad //stockwotoolpaths();
284 gcptmplscad //outputtoolpaths();
285 gcptmplscad
286 gcptmplscad //makecube(3, 2, 1);
287 gcptmplscad
288 gcptmplscad //instantiatecube();
289 gcptmplscad
290 gcptmplscad closegcodefile();
291 gcptmplscad closedxffile();
```

Which generates a 3D model which previews in OpenSCAD as:



2.5 gpcthreedp.py

Setting up 3D printing will require accommodating the requirements of both the printer *and* filament being used. The most straight-forward and expedient way to arrive at this is to leverage a traditional 3D printer slicer which has settings appropriate to the machine and filament being

used which are tuned to the sort of part being made/printing being done, export the G-code, and use that as a template for setting up 3D printing.

Towards that end, a G-code file for a very basic 3D printer was output for printing PLA from an Ordbot Quantum

```
gcpthreedp #gcpthreedp.py --- Template for 3D printing
gcpthreedp #                  Initial version.
gcpthreedp #!/usr/bin/env python
gcpthreedp
gcpthreedp import sys
gcpthreedp
gcpthreedp try:
gcpthreedp     if 'gcodepreview' in sys.modules:
gcpthreedp         del sys.modules['gcodepreview']
gcpthreedp except AttributeError:
gcpthreedp     pass
gcpthreedp
gcpthreedp from gcodepreview import *
gcpthreedp
gcpthreedp fa = 2
gcpthreedp fs = 0.125
gcpthreedp
gcpthreedp # [Export] */
gcpthreedp Base_filename = "aexport"
gcpthreedp # [Export] */
gcpthreedp generatedxf = False
gcpthreedp # [Export] */
gcpthreedp generategcode = True
gcpthreedp # [3D Printing] */
gcpthreedp printer_name = 'prusa_i3' # generic / ultimaker2plus / prusa_i3 /
               ender_3 / cr_10 / bambulab_x1 / toolchanger_T0
gcpthreedp # [3D Printing] */
gcpthreedp nozzlediameter = 0.4
gcpthreedp # [3D Printing] */
gcpthreedp filamentdiameter = 1.75
gcpthreedp # [3D Printing] */
gcpthreedp extrusionwidth = 0.6
gcpthreedp # [3D Printing] */
gcpthreedp layerheight = 0.2
gcpthreedp # [3D Printing] */
gcpthreedp extruder_temperature =200
gcpthreedp # [3D Printing] */
gcpthreedp bed_temperature = 60
gcpthreedp
gcpthreedp gcp = gcodepreview("print", # "cut" or "no_preview"
gcpthreedp                    generategcode,
gcpthreedp                    generatedxf,
gcpthreedp                    )
gcpthreedp
gcpthreedp gcp.initializeforprinting(nozzlediameter,
gcpthreedp                           filamentdiameter,
gcpthreedp                           extrusionwidth,
gcpthreedp                           layerheight,
gcpthreedp                           "absolute",
gcpthreedp                           extruder_temperature,
gcpthreedp                           bed_temperature,
gcpthreedp                           printer_name,
gcpthreedp                           Base_filename)
gcpthreedp
gcpthreedp gcp.extrude(9, 18, layerheight)
gcpthreedp
gcpthreedp gcp.rapid(125, 125, layerheight)
gcpthreedp gcp.extrude(150, 125, layerheight)
gcpthreedp gcp.extrude(150, 150, layerheight)
gcpthreedp gcp.extrude(125, 150, layerheight)
gcpthreedp gcp.extrude(125, 125, layerheight)
gcpthreedp
gcpthreedp gcp.stockandtoolpaths("toolpaths")
gcpthreedp
gcpthreedp gcp.shutdownafterprinting()
```

2.6 gcodepreviewtemplate.txt

Throughout this document, examples of commands will be shown and then collected in `gcodepreviewtemplate.txt` for easy copy-pasting (insert old computer joke about how many original Cobol programs have been written).

```
1 gcptmpl #gcptemplate.txt --- this file will collect example usages of each
2 gcptmpl #                    command with a brief commentary.
```

3 gcodepreview

This library for OpenPythonSCAD works by using Python code to persistently store and access variables which denote the machine position and describe the characteristics of tools, and to write out files while both modeling the motion of a 3-axis CNC machine (note that at least a 4th additional axis may be worked up as a future option and supporting the work-around of two-sided (flip) machining by using an imported file as the Stock or preserving state and affording a second operation seems promising) and if desired, writing out DXF and/or G-code files (as opposed to the normal technique of rendering to a 3D model and writing out an STL or STEP or other model format and using a traditional CAM application). There are multiple modes for this, doing so may require loading up to two files:

- A Python file: gcodepreview.py (`gcpy`) — this has variables in the traditional sense which are used for tracking machine position and so forth. Note that where it is placed/loaded from will depend on whether it is imported into a Python file:
  `import gcodepreview_standalone as gcp`
  or used in an OpenSCAD file:
  `use <gcodepreview.py>`
  with an additional OpenSCAD module which allows accessing it and that there is an option for loading directly from the Github repository implemented in PythonSCAD
- An OpenSCAD file: gcodepreview.scad (`gcpscad`) — which `use`s the Python file and which is `included` allowing it to access OpenSCAD variables for branching

Note that this architecture requires that many OpenSCAD modules are essentially "Dispatchers" (another term is "Descriptors") which pass information from one aspect of the environment to another, but in some instances it is expedient, or even will be necessary to re-write Python definitions in OpenSCAD rather than calling the matching Python function directly.

In earlier versions there were several possible ways to work with the 3D models of the cuts, either directly displaying the returned 3D model when explicitly called for after storing it in a variable or calling it up as a calculation (Python command `ouput(<foo>)` or OpenSCAD returning a model, or calling an appropriate OpenSCAD command), however as-of v0.9 the tool movements are stored as lists of `hull()` operations which must be processed as such and are `difference`d from the stock. The templates set up these options as noted, and ensure that `True == true`.

PYTHON CODING CONSIDERATIONS: Python style may be checked using a tool such as: https://www.codewof.co.nz/style/python3/. Not all conventions will necessarily be adhered to — limiting line length in particular conflicts with the flexibility of Literate Programming. Note that `numpydoc`-style docstrings are added where appropriate to help define the functionality of each defined module in Python. https://numpydoc.readthedocs.io/en/latest/.

3.1 Tools for 3D Previewing G-code

This problem space, showing the result of cutting stock using tooling in 3D has a number of tools addressing it, Camotics (formerly OpenSCAM) is an opensource option. Many tools simply create a wireframe preview such as https://ncviewer.com/.

3.1.1 Cutviewer

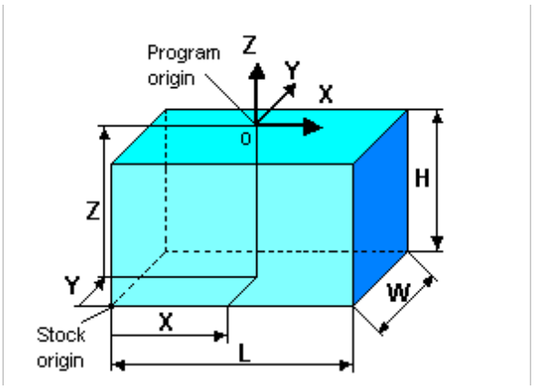
Cutviewer is a notable commercial program which has a unique approach centered on G-code where specially formatted comments fill in the dimensions and other parameters needed for showing the 3D preview.

3.1.1.1 Stock size and placement

Setting the dimensions of the stock, and placing it in 3D space relative to the origin must be done very early in the G-code file.

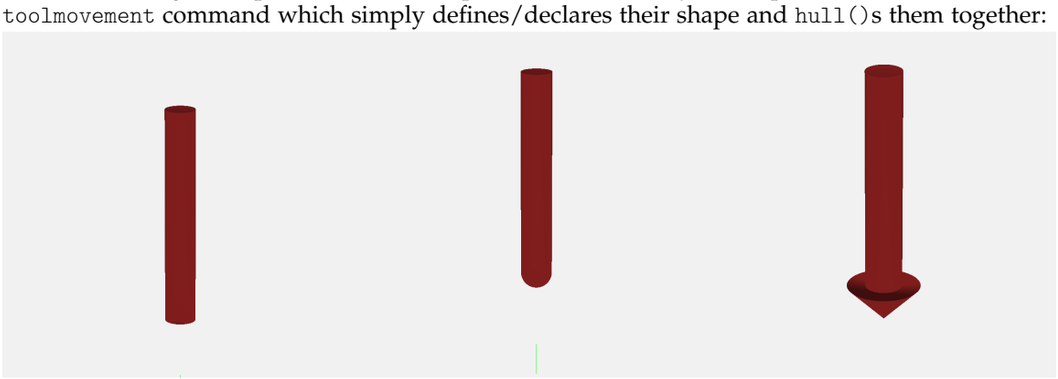
The CutViewer comments are in the form:

```
(STOCK/BLOCK, Length, Width, Height, Origin X, Origin Y, Origin Z)
```



3.1.1.2 Tool Shapes Cutviewer is unable to show tools which undercut, but other tool shapes are represented in a straight-forward and flexible fashion.

Most tooling has quite standard shapes as described by their profile as defined in the `toolmovement` command which simply defines/declares their shape and `hull()`s them together:

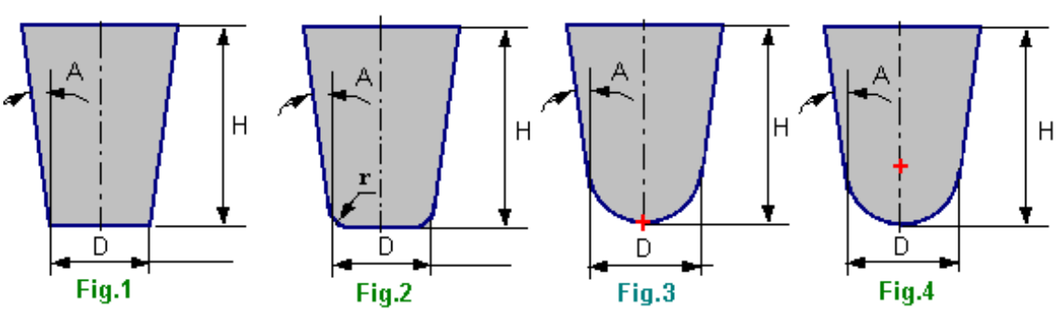


- Square (#201, 102, 211, 213 and 214) — able to cut a flat bottom, perpendicular side and right angle, their simple and easily understood geometry makes them a standard choice
- Ballnose (#202, 101, and 212) — rounded, they are the standard choice for concave and organic shapes
- V tooling (#301, 302, 311 and 312) — pointed at the tip, they are available in a variety of angles and diameters and may be used for decorative V carving, or for chamfering or cutting specific angles

Note that the module for creating movement of the tool will need to handle all of the different tool shapes, generating a list of `hull()` or `rotate_extrude` commands which describe the 3D region which tool movement describes.

3.1.1.2.1 Tool/Mill (Square, radiused, ball-nose, and tapered-ball) The CutViewer values include:

```
TOOL/MILL, Diameter, Corner radius, Height, Taper Angle
```

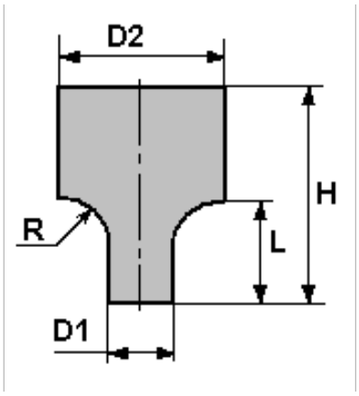


Note that it is possible to use these definitions for a wide variety of tooling, e.g., a Carbide 3D #301 V tool being represented as:

```
(TOOL/MILL,0.10, 0.05, 6.35, 45.00)
```

3.1.1.2.2 Corner Rounding, (roundover) One notable tool option which cannot be supported using the Tool/Mill description is corner rounding/roundover tooling:

`TOOL/CRMILL, Diameter1, Diameter2, Radius, Height, Length`



3.1.1.2.3 V shaped tooling (and variations) Cutviewer has multiple V shaped tooling definitions:

- ;TOOL/CHAMFER, Diameter, Point Angle, Height
- ;TOOL/CHAMFER, Diameter, Point Angle, Height, Chamfer Length (note that this is the definition of a flat-bottomed V tool)
- ;TOOL/DRILL, Diameter, Point Angle, Height
- ;TOOL/CDRILL, D1, A1, L, D2, A2, H

Since such tooling may be represented (albeit with a slight compromise which arguably is a nod to the real world) using the Tool/Mill definition from above, it seems unlikely that such tooling definitions will be supported.

3.2 Module Naming Convention

> *The beginning of wisdom is to call things by their right names.*
> — CONFUCIUS

One of the hard things in computer science, naming modules (and certain variables) requires that the conventions of G-code, the various file types which are written to, and the actions which the system takes are all taken into due consideration so as to arrive at a consistent scheme.

Number will be abbreviated as `num` rather than `no`, and the short form will be used internally for variable names, while the compleat word will be used in commands.

In some instances, `the` will be used as a prefix.

Tool #s where used will be the first argument where possible — this makes it obvious if they are not used — the negative consideration, that it then doesn't allow for a usage where a DEFAULT tool is used is not an issue since the command `currenttoolnumber()` may be used to access that number, and is arguably the preferred mechanism.

In natural languages such as English, there is an order to various parts of speech such as adjectives — since various prefixes and suffixes will be used for module names, having a consistent ordering/usage will help in consistency and make expression clearer. The ordering should be: sequence (if necessary), action, function, parameter, filetype, and where possible a hierarchy of large/general to small/specific should be maintained.

- Both prefix and suffix
  - `dxf` (action (write out to DXF file), filetype)
- Prefixes
  - `generate` (Boolean) — used to identify which types of actions will be done (note that in the interest of brevity the check for this will be deferred until the last possible moment, see below)
  - `write` (action) — used to write to files, will include a check for the matching `generate` command, which being true will cause the write to the file to actually transpire
  - `cut` (action — create tool movement removing volume from 3D object)
  - `extrude` (action) — 3D printing equivalent to cut
  - `rapid` (action) — create tool movement of 3D object so as to show any collision or rubbing

  - `open` (action (file))
  - `close` (action (file))
  - `set` (action/function) — note that the matching `get` is implicit in functions which return variables, e.g., `xpos()`
  - `current`
- Nouns (geometry/shapes)
  - `arc`
  - `line`
  - `rectangle`
  - `circle`
- Suffixes
  - `feed` (parameter)
  - `gcode/gc` (filetype)
  - `pos` — position
  - `tool`
  - `loop`
  - `CC/CW`
  - `number/num` — note that `num` is used internally for variable names, while `number` will be used for module/function names, making it straight-forward to ensure that functions and variables have different names for purposes of scope

Further note that commands which are implicitly for the generation of G-code, such as `toolchange()` will omit `gc` for the sake of conciseness.

In theory, this means that the basic `cut...` and associated commands exist (or potentially exist) in the following forms and have matching versions which may be used when programming in Python or OpenSCAD:

	line			arc		
	cut	dxf	gcode	cut	dxf	gcode
cut	cutline		cutlinegc	cutarc		cutarcgc
dxf	cutlinedxf	dxfline		cutarcdxf	dxfarc	
gcode	cutlinegc		linegc	cutarcgc		arcgc
	cutlinedxfgc			cutarcdxfgc		

Note that certain commands (`dxflinegc`, `dxfarcgc`, `linegc`, `arcgc`) are either redundant or unlikely to be needed, and will most likely not be implemented (it seems contradictory that one would write out a move command to a G-code file without making that cut in the 3D preview). Note that there may be additional versions as required for the convenience of notation or cutting, in particular, a set of `cutarc`<quadrant><direction>`gc` commands was warranted during the initial development of `arc`-related commands.

`generategcode` `generatedxf` `generatecut`

The gcode and dxf columns and the matter of having specific commands which encompass those file types is tied up in having the internal variables `generategcode`, `generatedxf` and implementations, and a strong argument could be made that this should simply be handled by `if...then` structures using those variables. The addition of a `generatecut` variable adds the necessary symmetry. Note that an early option to output a separate file for each tool used has since been deprecated and removed. The need for it was addressed by instead using colour-coded layers. A future update may add support for grouping elements.

A further consideration is that when processing G-code it is typical for a given command to be minimal and only include the axis of motion for the end-position, so for each of the above which is likely to appear in a `.nc/.gcode` file, it will be necessary to have a matching command for the combinatorial possibilities, hence:

cutlineXYZ	cutlineXYZwithfeed
cutlineXY	cutlineXYwithfeed
cutlineXZ	cutlineXZwithfeed
cutlineYZ	cutlineYZwithfeed
cutlineX	cutlineXwithfeed
cutlineY	cutlineYwithfeed
cutlineZ	cutlineZwithfeed

Principles for naming modules (and variables):

- minimize use of underscores (for convenience sake, underscores are not used for index entries)

- identify which aspect of the project structure is being worked with (cut(ting), dxf, gcode, tool, etc.) note the gcodepreview class which will normally be imported as gcp so that module <foo> will be called as gcp.<foo> from Python and by the same <foo> in OpenSCAD

The following commands for various shapes either have been implemented (monospace) or have not yet been implemented, but likely will need to be (regular type):

- rectangle

  cutrectangle

  cutrectangleround

Another consideration is that all commands which write files will check to see if a given filetype is enabled or no, since that check is deferred to the last as noted above for the sake of conciseness.

There are multiple modes for programming PythonSCAD:

- Python — in gcodepreview this allows writing out dxf files and using mutable variables (this is done in current versions of this project)
- OpenSCAD — see: https://openscad.org/documentation.html
- Programming in Python, calling Python from OpenSCAD using dispatchers/descriptors (this is done in current versions of this project)
- Programming in OpenSCAD with variables and calling Python — this requires 3 files and was originally used in the project as written up at: https://github.com/WillAdams/gcodepreview/blob/main/gcodepreview-openscad_0_6.pdf (for further details see below, notably various commented out lines in the source .tex file)
- Programming in OpenSCAD and calling Python where all variables as variables are held in Python classes (this is the technique used up through v0.8)
- Programming in Python and calling OpenSCAD — https://old.reddit.com/r/OpenPythonSCAD/comments/1heczmi/finally_using_scad_modules/

For reference, structurally, when developing OpenSCAD commands which make use of Python variables this was rendered as:

```
The user-facing module is \DescribeRoutine{FOOBAR}

\lstset{firstnumber=\thegcpscad}
\begin{writecode}{a}{gcodepreview.scad}{scad}
module FOOBAR(...) {
    oFOOBAR(...);
}

\end{writecode}
\addtocounter{gcpscad}{4}

which calls the internal OpenSCAD Module \DescribeSubroutine{FOOBAR}{oFOOBAR}

\begin{writecode}{a}{pygcodepreview.scad}{scad}
module oFOOBAR(...) {
    pFOOBAR(...);
 }

\end{writecode}
\addtocounter{pyscad}{4}

which in turn calls the internal Python definitioon \DescribeSubroutine{FOOBAR}{pFOOBAR}

\lstset{firstnumber=\thegcpy}
\begin{writecode}{a}{gcodepreview.py}{python}
def pFOOBAR (...)
    ...

\end{writecode}
\addtocounter{gcpy}{3}
```

Further note that this style of definition might not have been necessary for some later modules since they are in turn calling internal modules which already use this structure.

Lastly note that this style of programming was abandoned in favour of object-oriented dot notation for versions after v0.6 (see below) and that this technique was extended to class nested within another class.

3.2.1 Parameters and Default Values

Ideally, there would be *no* hard-coded values — every value used for calculation will be parameterized, and subject to control/modification. Fortunately, Python affords a feature which specifically addresses this, optional arguments with default values:
https://stackoverflow.com/questions/9539921/how-do-i-define-a-function-with-optional-argumen

In short, rather than hard-code numbers, for example in loops, they will be assigned as default values, and thus afford the user/programmer the option of changing them when the module is called.

3.3 Implementation files and gcodepreview class

Each file will begin with a comment indicating the file type and further notes/comments on usage where appropriate:

```
1 gcpy #!/usr/bin/env python
2 gcpy #if using from PythonSCAD using OpenSCAD code, see gcodepreview.
       scad
3 gcpy
4 gcpy import sys
5 gcpy
6 gcpy # add math functions (sqrt)
7 gcpy import math
8 gcpy
9 gcpy # getting openscad functions into namespace
10 gcpy #https://github.com/gsohler/openscad/issues/39
11 gcpy try:
12 gcpy     from openscad import *
13 gcpy except ModuleNotFoundError as e:
14 gcpy     print("OpenSCAD␣module␣not␣loaded.")
15 gcpy
16 gcpy def pygcpversion():
17 gcpy     thegcpversion = 0.9313
18 gcpy     return thegcpversion
```

The OpenSCAD file must `use` the Python file (note that some test/example code is commented out):

```
1 gcpscad //!OpenSCAD
2 gcpscad
3 gcpscad //gcodepreview version 0.8
4 gcpscad //
5 gcpscad //used via include <gcodepreview.scad>;
6 gcpscad //
7 gcpscad
8 gcpscad use <gcodepreview.py>
9 gcpscad
10 gcpscad module gcpversion(){
11 gcpscad echo(pygcpversion());
12 gcpscad }
13 gcpscad
14 gcpscad //function myfunc(var) = gcp.myfunc(var);
15 gcpscad //
16 gcpscad //function getvv() = gcp.getvv();
17 gcpscad //
18 gcpscad //module makecube(xdim, ydim, zdim){
19 gcpscad //gcp.makecube(xdim, ydim, zdim);
20 gcpscad //}
21 gcpscad //
22 gcpscad //module placecube(){
23 gcpscad //gcp.placecube();
24 gcpscad //}
25 gcpscad //
26 gcpscad //module instantiatecube(){
27 gcpscad //gcp.instantiatecube();
28 gcpscad //}
29 gcpscad //
```

If all functions are to be handled within Python, then they will need to be gathered into a class which contains them and which is initialized so as to define shared variables and initial program state, and then there will need to be objects/commands for each aspect of the program, each of which will utilise needed variables and will contain appropriate functionality. Note that they will be divided between mandatory and optional functions/variables/objects:

- Mandatory
  - gcodepreview (init)

        * generatecut, generatedxf, generategcode
      - stocksetup:
        * stockXwidth, stockYheight, stockZthickness, zeroheight, stockzero, retractheight
      - gcpfiles:
        * basefilename
      - largesquaretool:
        * large_square_tool_num, toolradius, plunge, feed, speed
      - currenttoolnum
        * endmilltype
        * diameter
        * flute
        * shaftdiameter
        * shaftheight
        * shaftlength
        * toolnumber
        * cutcolor
        * rapidcolor
        * shaftcolor
  - Optional
      - smallsquaretool:
        * small_square_tool_num, small_square_ratio
      - largeballtool:
        * large_ball_tool_num, large_ball_ratio
      - largeVtool:
        * large_V_tool_num, large_V_ratio
      - smallballtool:
        * small_ball_tool_num, small_ball_ratio
      - smallVtool:
        * small_V_tool_num, small_V_ratio
      - DTtool:
        * DT_tool_num, DT_ratio
      - KHtool:
        * KH_tool_num, KH_ratio
      - Roundovertool:
        * Roundover_tool_num, RO_ratio
      - misctool:
        * MISC_tool_num, MISC_ratio

gcodepreview init

The class which is defined is `gcodepreview` which begins with the `init` method which allows passing in and defining the variables which will be used by the other methods in this class. Part of this includes handling various definitions for Boolean values.

3.3.1 init

Initialization of the `gcodepreview` object requires handling a number of different cases, two of which are exclusive to each other. It must also take into account the possibility of being called from OpenSCAD

```
20 gcpy class gcodepreview:
21 gcpy
22 gcpy     def __init__(self,
23 gcpy                  cutorprint = "cut", #"cut", "print", "no_preview"
24 gcpy                  generategcode = False,
25 gcpy                  generatedxf = False,
26 gcpy                  gcpfa = 2,
27 gcpy                  gcpfs = 0.125,
28 gcpy                  steps = 10
29 gcpy                  ):
30 gcpy         """
31 gcpy ␣␣␣␣␣␣␣␣Initialize␣gcodepreview␣object.
32 gcpy
33 gcpy ␣␣␣␣␣␣␣␣Parameters
34 gcpy ␣␣␣␣␣␣␣␣----------
```

```
35 gcpy ␣␣␣␣␣␣␣␣cutorprint␣␣␣␣:␣string
36 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Enables␣creation␣of␣3D␣model␣for␣cutting␣or
       ␣printing.
37 gcpy ␣␣␣␣␣␣␣␣generategcode␣:␣boolean
38 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Enables␣writing␣out␣G-code.
39 gcpy ␣␣␣␣␣␣␣␣generatedxf␣␣␣␣:␣boolean
40 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Enables␣writing␣out␣a␣DXF␣file.
41 gcpy
42 gcpy ␣␣␣␣␣␣␣␣Returns
43 gcpy ␣␣␣␣␣␣␣␣-------
44 gcpy ␣␣␣␣␣␣␣␣object
45 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣The␣initialized␣gcodepreview␣object.
46 gcpy ␣␣␣␣␣␣␣␣"""
47 gcpy          if cutorprint == "print":
48 gcpy              self.generatecut = False
49 gcpy              self.generateprint = True
50 gcpy              self.gcodefilext = ".gcode"
51 gcpy          elif cutorprint == "cut":
52 gcpy              self.generatecut = True
53 gcpy              self.generateprint = False
54 gcpy              self.gcodefilext = ".nc"
55 gcpy          else: # no_preview
56 gcpy              self.generatecut = False
57 gcpy              self.generateprint = False
58 gcpy          if generategcode == True:
59 gcpy              self.generategcode = True
60 gcpy          elif generategcode == 1:
61 gcpy              self.generategcode = True
62 gcpy          elif generategcode == 0:
63 gcpy              self.generategcode = False
64 gcpy          else:
65 gcpy              self.generategcode = generategcode
66 gcpy          if generatedxf == True:
67 gcpy              self.generatedxf = True
68 gcpy          elif generatedxf == 1:
69 gcpy              self.generatedxf = True
70 gcpy          elif generatedxf == 0:
71 gcpy              self.generatedxf = False
72 gcpy          else:
73 gcpy              self.generatedxf = generatedxf
74 gcpy # set up 3D previewing parameters
75 gcpy          fa = gcpfa
76 gcpy          fs = gcpfs
77 gcpy          self.steps = steps
78 gcpy # initialize the machine state
79 gcpy          self.mc = "Initialized"
80 gcpy          self.mpx = float(0)
81 gcpy          self.mpy = float(0)
82 gcpy          self.mpz = float(0)
83 gcpy          self.tpz = float(0)
84 gcpy # initialize the toolpath state
85 gcpy          self.retractheight = 5
86 gcpy # initialize the DEFAULT tool
87 gcpy          self.currenttoolnum = 102
88 gcpy          self.endmilltype = "square"
89 gcpy          self.diameter = 3.175
90 gcpy          self.flute = 12.7
91 gcpy          self.shaftdiameter = 3.175
92 gcpy          self.shaftheight = 12.7
93 gcpy          self.shaftlength = 19.5
94 gcpy          self.toolnumber = "100036"
95 gcpy          self.cutcolor = "green"
96 gcpy          self.rapidcolor = "orange"
97 gcpy          self.shaftcolor = "red"
98 gcpy # the command definesquaretool(3.175, 12.7, 20) is used in the
       toolchange command
99 gcpy          self.tooloutline = polygon( points
                   =[[0,0],[3.175,0],[3.175,12.7],[0,12.7]] )
100 gcpy         self.toolprofile = polygon( points
                   =[[0,0],[1.5875,0],[1.5875,12.7],[0,12.7]] )
101 gcpy         self.shaftoutline = polygon( points
                   =[[0,12.7],[3.175,12.7],[3.175,25.4],[0,25.4]] )
102 gcpy         self.shaftprofile = polygon( points
                   =[[0,12.7],[1.5875,12.7],[1.5875,25.4],[0,25.4]] )
103 gcpy         self.currenttoolshape = cylinder(h = self.flute, r = self.
                   shaftdiameter/2)
104 gcpy         sh = cylinder(h = self.flute, r = self.shaftdiameter/2)
105 gcpy         self.currenttoolshaft = sh.translate([0,0,self.flute])
```

```
106 gcpy # debug mode requires a variable to track if it is on or off
107 gcpy         self.debugenable = False
108 gcpy # the variables for holding 3D models must be initialized as empty
          lists so as to ensure that only append or extend commands are
          used with them
109 gcpy         self.rapids = []
110 gcpy         self.toolpaths = []
111 gcpy         print("gcodepreview␣class␣initialized")
112 gcpy
113 gcpy #    def myfunc(self, var):
114 gcpy #        self.vv = var * var
115 gcpy #        return self.vv
116 gcpy #
117 gcpy #    def getvv(self):
118 gcpy #        return self.vv
119 gcpy #
120 gcpy #    def checkint(self):
121 gcpy #        return self.mc
122 gcpy #
123 gcpy #    def makecube(self, xdim, ydim, zdim):
124 gcpy #        self.c=cube([xdim, ydim, zdim])
125 gcpy #
126 gcpy #    def placecube(self):
127 gcpy #        show(self.c)
128 gcpy #
129 gcpy #    def instantiatecube(self):
130 gcpy #        return self.c
```

3.3.2 Position and Variables

In modeling the machine motion and G-code it will be necessary to have the machine track several variables for machine position, the current tool and its parameters, and the current depth in the current toolpath. This will be done using paired functions (which will set and return the matching variable) and a matching variable.

The first such variables are for XYZ position:

mpx
- mpx

mpy
- mpy

mpz
- mpz

Similarly, for some toolpaths it will be necessary to track the depth along the Z-axis as the toolpath is cut out, or the increment which a cut advances — this is done using an internal variable, `tpzinc`.

tpzinc

It will further be necessary to have a variable for the current tool:

currenttoolnum
- currenttoolnum

Note that the `currenttoolnum` variable should always be accessed and used for any specification of a tool, being read in whenever a tool is to be made use of, or a parameter or aspect of the tool needs to be used in a calculation.

In early versions, the implicit union of the 3D model of the tool was available and used where appropriate, but in v0.9, this was changed to using lists for concatenating the `hull`ed shapes of tool movements, so the module, `toolmovement` which given begin/end position returns the appropriate shape(s) as a list.

toolmovement

currenttool

The 3D model of the tool is stored in `currenttool`.

xpos
ypos
zpos

It will be necessary to have Python functions (`xpos`, `ypos`, and `zpos`) which return the current values of the machine position in Cartesian coordinates:

```
132 gcpy     def xpos(self):
133 gcpy         return self.mpx
134 gcpy
135 gcpy     def ypos(self):
136 gcpy         return self.mpy
137 gcpy
138 gcpy     def zpos(self):
139 gcpy         return self.mpz
```

Wrapping these in OpenSCAD functions allows use of this positional information from OpenSCAD:

```
30 gcpscad function xpos() = gcp.xpos();
31 gcpscad
32 gcpscad function ypos() = gcp.ypos();
33 gcpscad
34 gcpscad function zpos() = gcp.zpos();
```

setxpos
setypos
setzpos

and in turn, functions which set the positions: `setxpos`, `setypos`, and `setzpos`.

```
141 gcpy     def setxpos(self, newxpos):
142 gcpy         self.mpx = newxpos
143 gcpy
144 gcpy     def setypos(self, newypos):
145 gcpy         self.mpy = newypos
146 gcpy
147 gcpy     def setzpos(self, newzpos):
148 gcpy         self.mpz = newzpos
```

Using the `set...` routines will afford a single point of control if specific actions are found to be contingent on changes to these positions.

3.3.3 Initial Modules

Initializing the machine state requires zeroing out the three machine position variables:

- mpx
- mpy
- mpz

Rather than a specific command for this, the code will be in-lined where appropriate (note that if machine initialization becomes sufficiently complex to warrant it, then a suitable command will need to be coded). Note that the variables are declared in the `__init__` of the class.

toolmovement
endmilltype
diameter
flute
ra
tip
toolmovement
rapids
toolpaths

The `toolmovement` class requires that the tool be defined in terms of `endmilltype`, `diameter`, `flute` (length), `ra` (radius or angle depending on context), and `tip`, and there is a mechanism which defines an internal tool number as described below. Currently though, the interface calls the `toolchange` routine passing in a manufacturer tool number as an expedient/default/initial option.

There are two variables to record `toolmovement`, `rapids` and `toolpaths`. Initialized as empty lists, toolmovements will be `extend`ed to the lists, then for output, the lists will be expanded and subtracted from the stock separately so that rapids are colour-coded so that if there is an interaction with the stock at rapid speed it will be obvious. A similar method should be implemented for the shafts of tooling.

gcodepreview
setupstock

3.3.3.1 setupstock The first such setup subroutine is `gcodepreview setupstock` which is appropriately enough, to set up the stock, and perform other initializations — initially, the only thing done in Python was to set the value of the persistent (Python) variables (see `initializemachinestate()` above), but the rewritten standalone version handles all necessary actions.

gcp.setupstock

Since part of a class, it will be called as `gcp.setupstock`. It requires that the user set parameters for stock dimensions and so forth, and will create comments in the G-code (if generating that file is enabled) which incorporate the stock dimensions and its position relative to the zero as set relative to the stock.

```
150 gcpy     def setupstock(self, stockXwidth,
151 gcpy                    stockYheight,
152 gcpy                    stockZthickness,
153 gcpy                    zeroheight,
154 gcpy                    stockzero,
155 gcpy                    retractheight):
156 gcpy         """
157 gcpy ␣␣␣␣␣␣␣␣Set␣up␣blank/stock␣for␣material␣and␣position/zero.
158 gcpy
159 gcpy ␣␣␣␣␣␣␣␣Parameters
160 gcpy ␣␣␣␣␣␣␣␣----------
161 gcpy ␣␣␣␣␣␣␣␣stockXwidth␣:␣␣␣float
162 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣X␣extent/dimension
163 gcpy ␣␣␣␣␣␣␣␣stockYheight␣:␣␣float
164 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Y␣extent/dimension
165 gcpy ␣␣␣␣␣␣␣␣stockZthickness␣:␣boolean
166 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Z␣extent/dimension
167 gcpy ␣␣␣␣␣␣␣␣zeroheight␣:␣␣␣␣string
168 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Top␣or␣Bottom,␣determines␣if␣Z␣extent␣will␣
         be␣positive␣or␣negative
169 gcpy ␣␣␣␣␣␣␣␣stockzero␣:␣␣␣␣␣string
170 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Lower-Left,␣Center-Left,␣Top-Left,␣Center,␣
         determines␣XY␣position␣of␣stock
171 gcpy ␣␣␣␣␣␣␣␣retractheight␣:␣float
172 gcpy ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Distance␣which␣tool␣retracts␣above␣surface␣
         of␣stock.
173 gcpy
174 gcpy ␣␣␣␣␣␣␣␣Returns
```

```
175 gcpy ␣␣␣␣␣␣␣␣-------
176 gcpy ␣␣␣␣␣␣␣␣none
177 gcpy ␣␣␣␣␣␣␣␣"""
178 gcpy         self.stockXwidth = stockXwidth
179 gcpy         self.stockYheight = stockYheight
180 gcpy         self.stockZthickness = stockZthickness
181 gcpy         self.zeroheight = zeroheight
182 gcpy         self.stockzero = stockzero
183 gcpy         self.retractheight = retractheight
184 gcpy         self.stock = cube([stockXwidth, stockYheight,
                     stockZthickness])
```

zeroheight
stockzero

A series of `if` statements parse the `zeroheight` (Z-axis) and `stockzero` (X- and Y-axes) parameters so as to place the stock in place and suitable G-code comments are added for CutViewer.

```
186 gcpy         if self.zeroheight == "Top":
187 gcpy             if self.stockzero == "Lower-Left":
188 gcpy                 self.stock = self.stock.translate([0, 0, -self.
                         stockZthickness])
189 gcpy                 if self.generategcode == True:
190 gcpy                     self.writegc("(stockMin:0.00mm,␣0.00mm,␣-", str
                             (self.stockZthickness), "mm)")
191 gcpy                     self.writegc("(stockMax:", str(self.stockXwidth
                             ), "mm,␣", str(stockYheight), "mm,␣0.00mm)")
192 gcpy                     self.writegc("(STOCK/BLOCK,␣", str(self.
                             stockXwidth), ",␣", str(self.stockYheight),
                             ",␣", str(self.stockZthickness), ",␣0.00,␣
                             0.00,␣", str(self.stockZthickness), ")")
193 gcpy             if self.stockzero == "Center-Left":
194 gcpy                 self.stock = self.stock.translate([0, -stockYheight
                          / 2, -stockZthickness])
195 gcpy                 if self.generategcode == True:
196 gcpy                     self.writegc("(stockMin:0.00mm,␣-", str(self.
                             stockYheight/2), "mm,␣-", str(self.
                             stockZthickness), "mm)")
197 gcpy                     self.writegc("(stockMax:", str(self.stockXwidth
                             ), "mm,␣", str(self.stockYheight/2), "mm,␣
                             0.00mm)")
198 gcpy                     self.writegc("(STOCK/BLOCK,␣", str(self.
                             stockXwidth), ",␣", str(self.stockYheight),
                             ",␣", str(self.stockZthickness), ",␣0.00,␣",
                              str(self.stockYheight/2), ",␣", str(self.
                             stockZthickness), ")");
199 gcpy             if self.stockzero == "Top-Left":
200 gcpy                 self.stock = self.stock.translate([0, -self.
                         stockYheight, -self.stockZthickness])
201 gcpy                 if self.generategcode == True:
202 gcpy                     self.writegc("(stockMin:0.00mm,␣-", str(self.
                             stockYheight), "mm,␣-", str(self.
                             stockZthickness), "mm)")
203 gcpy                     self.writegc("(stockMax:", str(self.stockXwidth
                             ), "mm,␣0.00mm,␣0.00mm)")
204 gcpy                     self.writegc("(STOCK/BLOCK,␣", str(self.
                             stockXwidth), ",␣", str(self.stockYheight),
                             ",␣", str(self.stockZthickness), ",␣0.00,␣",
                              str(self.stockYheight), ",␣", str(self.
                             stockZthickness), ")")
205 gcpy             if self.stockzero == "Center":
206 gcpy                 self.stock = self.stock.translate([-self.
                         stockXwidth / 2, -self.stockYheight / 2, -self.
                         stockZthickness])
207 gcpy                 if self.generategcode == True:
208 gcpy                     self.writegc("(stockMin:␣-", str(self.
                             stockXwidth/2), ",␣-", str(self.stockYheight
                             /2), "mm,␣-", str(self.stockZthickness), "mm
                             )")
209 gcpy                     self.writegc("(stockMax:", str(self.stockXwidth
                             /2), "mm,␣", str(self.stockYheight/2), "mm,␣
                             0.00mm)")
210 gcpy                     self.writegc("(STOCK/BLOCK,␣", str(self.
                             stockXwidth), ",␣", str(self.stockYheight),
                             ",␣", str(self.stockZthickness), ",␣", str(
                             self.stockXwidth/2), ",␣", str(self.
                             stockYheight/2), ",␣", str(self.
                             stockZthickness), ")")
211 gcpy         if self.zeroheight == "Bottom":
212 gcpy             if self.stockzero == "Lower-Left":
213 gcpy                  self.stock = self.stock.translate([0, 0, 0])
```

```
214 gcpy                    if self.generategcode == True:
215 gcpy                        self.writegc("(stockMin:0.00mm, 0.00mm, 0.00mm
                                    )")
216 gcpy                        self.writegc("(stockMax:", str(self.
                                    stockXwidth), "mm, ", str(self.stockYheight
                                    ), "mm, ", str(self.stockZthickness), "mm)"
                                    )
217 gcpy                        self.writegc("(STOCK/BLOCK, ", str(self.
                                    stockXwidth), ", ", str(self.stockYheight),
                                     ", ", str(self.stockZthickness), ", 0.00, 
                                    0.00, 0.00)")
218 gcpy                if self.stockzero == "Center-Left":
219 gcpy                    self.stock = self.stock.translate([0, -self.
                                stockYheight / 2, 0])
220 gcpy                    if self.generategcode == True:
221 gcpy                        self.writegc("(stockMin:0.00mm, -", str(self.
                                    stockYheight/2), "mm, 0.00mm)")
222 gcpy                        self.writegc("(stockMax:", str(self.stockXwidth
                                    ), "mm, ", str(self.stockYheight/2), "mm, -"
                                    , str(self.stockZthickness), "mm)")
223 gcpy                        self.writegc("(STOCK/BLOCK, ", str(self.
                                    stockXwidth), ", ", str(self.stockYheight),
                                    ", ", str(self.stockZthickness), ", 0.00, ",
                                     str(self.stockYheight/2), ", 0.00mm)");
224 gcpy                if self.stockzero == "Top-Left":
225 gcpy                    self.stock = self.stock.translate([0, -self.
                                stockYheight, 0])
226 gcpy                    if self.generategcode == True:
227 gcpy                        self.writegc("(stockMin:0.00mm, -", str(self.
                                    stockYheight), "mm, 0.00mm)")
228 gcpy                        self.writegc("(stockMax:", str(self.stockXwidth
                                    ), "mm, 0.00mm, ", str(self.stockZthickness)
                                    , "mm)")
229 gcpy                        self.writegc("(STOCK/BLOCK, ", str(self.
                                    stockXwidth), ", ", str(self.stockYheight),
                                    ", ", str(self.stockZthickness), ", 0.00, ",
                                     str(self.stockYheight), ", 0.00)")
230 gcpy                if self.stockzero == "Center":
231 gcpy                    self.stock = self.stock.translate([-self.
                                stockXwidth / 2, -self.stockYheight / 2, 0])
232 gcpy                    if self.generategcode == True:
233 gcpy                        self.writegc("(stockMin: -", str(self.
                                    stockXwidth/2), ", -", str(self.stockYheight
                                    /2), "mm, 0.00mm)")
234 gcpy                        self.writegc("(stockMax:", str(self.stockXwidth
                                    /2), "mm, ", str(self.stockYheight/2), "mm, 
                                    ", str(self.stockZthickness), "mm)")
235 gcpy                        self.writegc("(STOCK/BLOCK, ", str(self.
                                    stockXwidth), ", ", str(self.stockYheight),
                                    ", ", str(self.stockZthickness), ", ", str(
                                    self.stockXwidth/2), ", ", str(self.
                                    stockYheight/2), ", 0.00)")
236 gcpy            if self.generategcode == True:
237 gcpy                self.writegc("G90");
238 gcpy                self.writegc("G21");
```

Note that while the #102 is declared as a default tool, while it was originally necessary to call a tool change after invoking `setupstock`, in the 2024.09.03 version of PythonSCAD this requirement went away when an update which interfered with persistently setting a variable directly was fixed. The setupstock command is required if working with a 3D project, creating the block of stock which the following toolpath commands will cut away. Note that since Python in OpenPythonSCAD defers output of the 3D model, it is possible to define it once, then set up all the specifics for each possible positioning of the stock in terms of origin.

The OpenSCAD version is simply a descriptor:

```
36 gcpscad module setupstock(stockXwidth, stockYheight, stockZthickness,
               zeroheight, stockzero, retractheight) {
37 gcpscad      gcp.setupstock(stockXwidth, stockYheight, stockZthickness,
                    zeroheight, stockzero, retractheight);
38 gcpscad }
```

3.3.3.2 setupcuttingarea If processing G-code, the parameters passed in are necessarily different, and there is of course, no need to write out G-code.

```
240 gcpy      def setupcuttingarea(self, sizeX, sizeY, sizeZ, extentleft,
```

```
                extentfb, extentd):
241 gcpy #         self.initializemachinestate()
242 gcpy          c=cube([sizeX,sizeY,sizeZ])
243 gcpy          c = c.translate([extentleft,extentfb,extentd])
244 gcpy          self.stock = c
245 gcpy          self.toolpaths = []
246 gcpy          return c
```

(Note that since this command will only be done in the process of inputting a G-code file, there is no need for a matching OpenSCAD module.)

3.3.3.3 debug Rather than endlessly add and then comment out `print()` commands, it is easier to have a variable for this, and a command which wraps the command which checks for that:

```
248 gcpy      def debug(self, *args: any, sep: str = "␣", end: str = "\n", **
                print_kwargs) -> None:
249 gcpy          """
250 gcpy ␣␣␣␣␣␣␣␣Print␣debug␣output␣if␣enabled.
251 gcpy
252 gcpy ␣␣␣␣␣␣␣␣Accepts␣the␣same␣arguments␣as␣built-in␣print␣(except␣file␣
            is␣supported␣via␣print_kwargs).
253 gcpy ␣␣␣␣␣␣␣␣"""
254 gcpy          if not self.debugenable:
255 gcpy              return
256 gcpy          # Build the message and print under a lock to avoid
                    interleaving in multithreaded apps
257 gcpy          self.prefix = "DEBUG:␣"
258 gcpy          msg = self.prefix + sep.join(map(str, args))
259 gcpy          with self._lock:
260 gcpy              print(msg, end=end, **print_kwargs)
```

Note that it will be necessary to manually use commands such as:

```
3 gcptmpl self.debugenable = True
4 gcptmpl
5 gcptmpl testvariable = 1
6 gcptmpl
7 gcptmpl self.outputdebugnote("Current␣value␣of␣testvariable␣is:␣",
            testvariable)
```

3.3.4 Adjustments and Additions

For certain projects and toolpaths it will be helpful to shift the stock, and to add additional pieces to the project.

Shifting the stock is simple:

```
262 gcpy      def shiftstock(self, shiftX, shiftY, shiftZ):
263 gcpy           self.stock = self.stock.translate([shiftX, shiftY, shiftZ
                      ])
```

```
40 gcpscad module shiftstock(shiftX, shiftY, shiftZ) {
41 gcpscad     gcp.shiftstock(shiftX, shiftY, shiftZ);
42 gcpscad }
```

adding stock is similar, but adds the requirement that it include options for shifting the stock:

```
265 gcpy      def addtostock(self, stockXwidth, stockYheight, stockZthickness
                ,
266 gcpy                           shiftX = 0,
267 gcpy                           shiftY = 0,
268 gcpy                           shiftZ = 0):
269 gcpy           addedpart = cube([stockXwidth, stockYheight,
                      stockZthickness])
270 gcpy           addedpart = addedpart.translate([shiftX, shiftY, shiftZ])
271 gcpy           self.stock = self.stock.union(addedpart)
```

the OpenSCAD module is a descriptor as expected:

```
44 gcpscad module addtostock(stockXwidth, stockYheight, stockZthickness,
            shiftX, shiftY, shiftZ) {
45 gcpscad     gcp.addtostock(stockXwidth, stockYheight, stockZthickness,
                shiftX, shiftY, shiftZ);
```

```
46 gcpscad }
```

3.4 Tools and Shapes and Changes

Originally, it was necessary to return a shape so that modules which use a `<variable>.union` command would function as expected even when the 3D model created is stored in a variable.

Due to stack limits in OpenSCAD for the CSG tree, instead, the shapes will be stored in two variables (`rapids`, `toolpaths`) as lists processed/created using a command `toolmovement` which will subsume all tool related functionality. As other routines need access to information about the current tool, appropriate routines will allow its variables and the specifics of the current tool to be queried.

rapids
toolpaths
toolmovement

It will be necessary to describe the tool in four different fashions:

- variables — a full set of variables is required to allow defining a shape and to determine the appropriate fashion in which to treat each tool at need

  tooltype = "mill"

  diameter = first

  cornerradius = second

  height = third

  taperangle

  length

- profile — the profile is a definition of the tool from the centerline to the outer edge which is used when necessary to `rotate_extrude()` the design
- outline — the outline is the entire definition of the tool shape which is used when `rotate_extrude`ing an arc (which will also require a 3D version of the rotated tool profile at each end)
- shape — originally the program used the tool shape and `hull()`ed it from beginning to end of a movement — having the shape pre-made allows it to be `union()`ed at need.

The base/entry functionality has the instance being defined in terms of a basic set of variables (one of which is overloaded to serve multiple purposes, depending on the type of endmill).

Note that it will also be necessary to write out a tool description compatible with the program CutViewer as a G-code comment so that it may be used as a 3D previewer for the G-code for tool changes in G-code. Several forms are available as described below.

3.4.1 Numbering for Tools

Currently, the numbering scheme used is that of the various manufacturers of the tools, or descriptive short-hand numbers created for tools which lack such a designation (with a disclosure that the author is a Carbide 3D employee).

Creating any numbering scheme is like most things in life, a trade-off, balancing length and expressiveness/compleatness against simplicity and usability. The software application Carbide Create (as released by an employer of the main author) has a limit of six digits, which seems a reasonable length from a complexity/simplicity standpoint, but also potentially reasonably expressible.

It will be desirable to track the following characteristics and measurements, apportioned over the digits as follows:

1	2–3	4–5	6
endmill type	radius/angle	cutting diameter(and tip radius for tapered ball nose)	cutting flute length

- 1st digit: endmill type:
  - 0 - manufacturer number
  - 1 - square (incl. “O”-flute)
  - 2 - ball
  - 3 - V
  - 4 - bowl
  - 5 - tapered ball
  - 6 - roundover
  - 7 - thread-cutting
  - 8 - dovetail
  - 9 - other (e.g., user-defined, or unsupported tools, keyhole, lollipop, &c.)

- 2nd and 3rd digits shape radius (ball/roundover) or angle (V), 2nd and 3rd digit together 10–99 indicate measurement in tenth of a millimeter. 2nd digit:
  - 0 - Imperial (00 indicates n/a or square)
  - any other value for both the 2nd and 3rd digits together indicate a metric measurement or an angle in degrees
- 3rd digit (if 2nd is 0 indicating Imperial)
  - 1 - 1/32nd
  - 2 - 1/16
  - 3 - 1/8
  - 4 - 1/4
  - 5 - 5/16
  - 6 - 3/8
  - 7 - 1/2
  - 8 - 3/4
  - 9 - >1″ or other
- 4th and 5th digits cutting diameter as 2nd and 3rd above except 4th digit indicates tip radius for tapered ball nose and such tooling is only represented in Imperial measure:
- 4th digit (tapered ball nose)
  - 1 - 0.01 in (this is the 0.254mm of the #501 and 502)
  - 2 - 0.015625 in (1/64th)
  - 3 - 0.0295
  - 4 - 0.03125 in (1/32nd)
  - 5 - 0.0335
  - 6 - 0.0354
  - 7 - 0.0625 in (1/16th)
  - 8 - 0.125 in (1/8th)
  - 9 - 0.25 in (1/4)
- 6th digit cutting flute length:
  - 0 - other
  - 1 - calculate based on V angle
  - 2 - 1/16
  - 3 - 1/8
  - 4 - 1/4
  - 5 - 5/16
  - 6 - 1/2
  - 7 - 3/4
  - 8 - “long reach” or greater than 3/4″
  - 9 - calculate based on radius
- or 6th digit tip diameter for roundover tooling (added to cutting diameter to arrive at actual cutting diameter — note that these values are the same as for the tip radius of the #501 and 502)
  - 1 - 0.01 in
  - 2 - 0.015625 in (1/64th)
  - 3 - 0.0295
  - 4 - 0.03125 in (1/32nd)
  - 5 - 0.0335
  - 6 - 0.0354
  - 7 - 0.0625 in (1/16th)
  - 8 - 0.125 in (1/8th)
  - 9 - 0.25 in (1/4)

Using this technique to create tool numbers for Carbide 3D tooling we arrive at:

- Square

  #122 == 100012

  #112 == 100024

  #102 == 100036 (also #274 and #326 (Amana 46200-K))

  #201 == 100047 (also #251 and #322 (Amana 46202-K))

  #205 == 100048 (also #324)

  #211 == 100058 (also #213 and #214)

  #213 == 100058 (Rougher, also #213 and #214)

  #214 == 100058 (Compression, also #213 and #214)

  #251 == 100047 (also #201, #278, and #322 (Amana 46202-K))

  #274 == 100036 (also #102 and #326 (Amana 46200-K))

  #278 == 100047

  #282 == 100204

  #322 == 100047 (also #201 and #251)

  #324 == 100048 (also #205 (Amana 46170-K))

  #326 == 100036 (also #102 and #274)

- Ball

  #121 == 201012

  #111 == 202024

  #101 == 203036

  #202 == 204047

  #212 == 205058

  #325 == 204048 (Amana 46376-K)

- V

  #301 == 390074

  #302 == 360071

  #311 == 390121

  #312 == 360121

  #327 == 360098 (Amana RC-1148)

- Tapered Ball Nose

  #501 == 530131

  #502 == 540131

(note that some dimensions were rounded off/approximated)
Extending that to the non-Carbide 3D tooling thus implemented:

- V

  #390 == 390032

- Dovetail

  814 == 814071

  45828 == 808071

- Keyhole Tool

  374 == 906043

  375 == 906053

  376 == 907040

  378 == 907050

- Roundover Tool

  56142 == 602032

  56125 == 603042

  1568 == 603032

  1570

  1572 == 604042

  1574

- Threadmill

  648

- Bowl bit

  45981

  45982

  1370

  1372

Notable limitations:

- No way to indicate number of flutes or flute geometry (an initial version used a first digit of 0 for O-flute, but they are now indicated by 1 (square))
- Lack of precision for metric tooling/limited support for Imperial sizes, notably, the dimensions used are scaled for smaller tooling and are not suited to typically larger scale tooling such as bowl bits
- No way to indicate several fairly common shapes including keyhole, lollipop, and flat-bottomed V/chamfer tools (except of course for using 0... or 9#####)
- coatings are not considered, geometry/size only

A further consideration is that it is not possible to represent tools unambiguously, so that given a tool definition it is possible to derive the manufacturer's tool number, *e.g.*, given a hypothetical command/instruction:

```
self.currenttoolshape = self.toolshapes("square", 6.35, 19.05)
```

it could be viewed as representing any of three different tools (Carbide 3D #201 (upcut), #251 (downcut), and #322 (Amana 46202-K downcut)), it is worth noting that #205E is differentiated due to its longer flute length as-is #324 (Amana 46170-K compression), though the fact of its compression cutting geometry is not recorded. Affording some sort of hinting to the user may be warranted, or a mechanism to allow specifying a given manufacturer tool # as part of setting up a job.

A more likely scheme is that manufacturer tool numbers will continue to be used to identify tooling — it may be that in the future the generated number will be used internally, but the saved manufacturer number will be exported to the G-code file unless something changes.

```
273 gcpy     def currenttoolnumber(self):
274 gcpy         return(self.currenttoolnum)
```

`toolchange` The `toolchange` command will need to set several variables.
Mandatory variables include:

- endmilltype

  O-flute (this may be deprecated and removed)

  square

  ball

  V

  keyhole

  dovetail

  roundover

  tapered ball

- diameter
- flute length

and depending on the tool geometry, several additional variables will be necessary (usually derived from `self.ra`):

- radius
- angle

an optional setting of a `toolnumber` may be useful in the future.

Setting the shaft dimensions would be helpful, and might be used to facilitate keyhole cutters and similar tooling which calculates undercuts.

tool number toolchange

3.4.1.1 toolchange This command accepts a `tool number` and assigns its characteristics as parameters. It then applies the appropriate commands for a `toolchange`. Note that it is expected that this code will be updated as needed when new tooling is introduced as additional modules which require specific tooling are added.

Note that the comments written out in G-code correspond to those used by the G-code previewing tool CutViewer (which is unfortunately, no longer readily available). Similarly, the G-code previewing functionality in this library expects that such comments will be in place so as to model the stock.

A further concern is that early versions often passed the tool into a module using a parameter. That ceased to be necessary in the 2024.09.03 version of PythonSCAD, and all modules should read the tool # from `currenttoolnumber()`.

Note that there are many varieties of tooling and not all will be directly supported, and that at need, additional tool shape support may be added under the variable `misc` (as opposed to the list of explicitly supported variables/shapes/modules).

The original implementation created the model for the tool at the current position, and a duplicate at the end position, wrapping the twain for each end of a given movement in a `hull()` command and then applying a `union`. This approach will not work within Python, so it will be necessary to instead assign and select the tool as part of the `toolmovement` command, collecting all such in a list which is then parsed when the 3D model is generated.

There are two separate commands for handling a tool being changed, the first sets the parameters which describe the tool and may be used to effect the change of a tool either in a G-code file

settoolparameters toolchange

or when making a 3D file, `settoolparameters` (not currently used as noted below) and a second version which processes a toolchange when presented with a tool number, `toolchange` (it may be that the latter will be set up to call the former).

3.4.1.1.1 settoolparameters Not currently used, this command is intended for a future state/need where tools are defined in a vendor-neutral fashion.

```
276 gcpy     def settoolparameters(self, tooltype, first, second, third,
                 fourth, length = 0):
277 gcpy         if tooltype == "mill":
278 gcpy             diameter = first
279 gcpy             cornerradius = second
280 gcpy             height = third
281 gcpy             taperangle = fourth
282 gcpy             if cornerradius == 0:
283 gcpy #M6T122 (TOOL/MILL,0.80, 0.00, 1.59, 0.00)
284 gcpy #M6T112 (TOOL/MILL,1.59, 0.00, 6.35, 0.00)
285 gcpy #M6T102 (TOOL/MILL,3.17, 0.00, 12.70, 0.00)
286 gcpy #M6T201 (TOOL/MILL,6.35, 0.00, 19.05, 0.00)
287 gcpy #M6T205 (TOOL/MILL,6.35, 0.00, 25.40, 0.00)
288 gcpy #M6T251 (TOOL/MILL,6.35, 0.00, 19.05, 0.00)
289 gcpy #M6T322 (TOOL/MILL,6.35, 0.00, 19.05, 0.00)
290 gcpy #M6T324 (TOOL/MILL,6.35, 0.00, 22.22, 0.00)
291 gcpy #M6T326 (TOOL/MILL,3.17, 0.00, 12.70, 0.00)
292 gcpy #M6T602 (TOOL/MILL,25.40, 0.00, 9.91, 0.00)
293 gcpy #M6T603 (TOOL/MILL,25.40, 0.00, 9.91, 0.00)
294 gcpy #M6T274 (TOOL/MILL,3.17, 0.00, 12.70, 0.00)
295 gcpy #M6T278 (TOOL/MILL,6.35, 0.00, 19.05, 0.00)
296 gcpy #M6T282 (TOOL/MILL,2.00, 0.00, 6.35, 0.00)
297 gcpy                 self.endmilltype = "square"
298 gcpy                 self.diameter = diameter
299 gcpy                 self.flute = height
300 gcpy                 self.shaftdiameter = diameter
301 gcpy                 self.shaftheight = height
302 gcpy                 self.shaftlength = height
303 gcpy #
304 gcpy             elif cornerradius > 0 and taperangle == 0:
305 gcpy #M6T121 (TOOL/MILL,0.80, 0.40, 1.59, 0.00)
306 gcpy #M6T111 (TOOL/MILL,1.59, 0.79, 6.35, 0.00)
307 gcpy #M6T101 (TOOL/MILL,3.17, 1.59, 12.70, 0.00)
308 gcpy #M6T202 (TOOL/MILL,6.35, 3.17, 19.05, 0.00)
309 gcpy #M6T325 (TOOL/MILL,6.35, 3.17, 25.40, 0.00)
310 gcpy                 self.endmilltype = "ball"
311 gcpy                 self.diameter = diameter
312 gcpy                 self.flute = height
313 gcpy                 self.shaftdiameter = diameter
314 gcpy                 self.shaftheight = height
315 gcpy                 self.shaftlength = height
316 gcpy #
317 gcpy             elif taperangle > 0:
318 gcpy #M6T301 (TOOL/MILL,0.10, 0.05, 6.35, 45.00)
319 gcpy #M6T302 (TOOL/MILL,0.10, 0.05, 6.35, 30.00)
320 gcpy #M6T327 (TOOL/MILL,0.10, 0.05, 23.39, 30.00)
321 gcpy                 self.endmilltype = "V"
322 gcpy                 self.diameter = Tan(taperangle / 2) * height
```

```
323 gcpy                 self.flute = height
324 gcpy                 self.angle = taperangle
325 gcpy                 self.shaftdiameter = Tan(taperangle / 2) * height
326 gcpy                 self.shaftheight = height
327 gcpy                 self.shaftlength = height
328 gcpy #
329 gcpy         elif tooltype == "chamfer":
330 gcpy             tipdiameter = first
331 gcpy             radius = second
332 gcpy             height = third
333 gcpy             taperangle = fourth
```

toolchange

3.4.1.1.2 toolchange The Python definition for `toolchange` requires the tool number (used to write out the G-code comment description for CutViewer and also expects the speed for the current tool since this is passed into the G-code tool change command as part of the spindle on command. A simple `if–then` structure, the variables necessary for defining the toolshape are (re)defined each time the command is called so that they may be used by the command

toolmovement

`toolmovement` for actually modeling the shapes and the path and the resultant material removal.

```
335 gcpy     def toolchange(self, tool_number, speed = 10000):
336 gcpy         self.currenttoolnum = tool_number
337 gcpy
338 gcpy         if (self.generategcode == True):
339 gcpy             self.writegc("(Toolpath)")
340 gcpy             self.writegc("M05")
```

3.4.1.1.3 Square (including O-flute) The simplest sort of tool, they are defined as a cylinder.

```
342 gcpy         if (tool_number == 102) or (tool_number == 100036): #
                     102/326 == 100036
343 gcpy             self.writegc("(TOOL/MILL,␣3.175,␣0.00,␣0.00,␣0.00)")
344 gcpy             self.endmilltype = "square"
345 gcpy             self.diameter = 3.175
346 gcpy             self.flute = 12.7
347 gcpy             self.shaftdiameter = 3.175
348 gcpy             self.shaftheight = 12.7
349 gcpy             self.shaftlength = 19.5
```

The outline definitions for linear/rotate extrude are the same for this tool as in the default tool definition in `__init__`, but the commands `definesquaretool` and `defineshaft` are used:

```
350 gcpy             self.definesquaretool(self.diameter, self.shaftheight,
                         self.shaftlength)
351 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
352 gcpy             self.toolnumber = 10003
353 gcpy         elif (tool_number == 201) or (tool_number == 100047): #
                     201/251/322 (Amana 46202-K) == 100047
354 gcpy             self.writegc("(TOOL/MILL,␣6.35,␣0.00,␣0.00,␣0.00)")
355 gcpy             self.endmilltype = "square"
356 gcpy             self.diameter = 6.35
357 gcpy             self.flute = 19.05
358 gcpy             self.shaftdiameter = 6.35
359 gcpy             self.shaftheight = 19.05
360 gcpy             self.shaftlength = 20.0
361 gcpy             self.definesquaretool(self.diameter, self.shaftheight,
                         self.shaftlength)
362 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
363 gcpy             self.toolnumber = "100047"
364 gcpy         elif (tool_number == 112) or (tool_number == 100024): #112
                     == 100024
365 gcpy             self.writegc("(TOOL/MILL,␣1.5875,␣0.00,␣0.00,␣0.00)")
366 gcpy             self.endmilltype = "square"
367 gcpy             self.diameter = 1.5875
368 gcpy             self.flute = 6.35
369 gcpy             self.shaftdiameter = 3.175
370 gcpy             self.shaftheight = 6.35
371 gcpy             self.shaftlength = 12.0
372 gcpy             self.definesquaretool(self.diameter, self.shaftheight,
                         self.shaftlength, (self.shaftdiameter - self.
                         diameter)/2)
373 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
```

```
374 gcpy             self.toolnumber = "100024"
375 gcpy         elif (tool_number == 122) or (tool_number == 100012): #122
                     == 100012
376 gcpy             self.writegc("(TOOL/MILL,␣0.79375,␣0.00,␣0.00,␣0.00)")
377 gcpy             self.endmilltype = "square"
378 gcpy             self.diameter = 0.79375
379 gcpy             self.flute = 1.5875
380 gcpy             self.shaftdiameter = 3.175
381 gcpy             self.shaftheight = 1.5875
382 gcpy             self.shaftlength = 12.0
383 gcpy             self.definesquaretool(self.diameter, self.shaftheight,
                         self.shaftlength, (self.shaftdiameter - self.
                         diameter)/2)
384 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
385 gcpy             self.toolnumber = "100012"
386 gcpy         elif (tool_number == 324): #324 (Amana 46170-K) == 100048
387 gcpy             self.writegc("(TOOL/MILL,␣6.35,␣0.00,␣0.00,␣0.00)")
388 gcpy             self.endmilltype = "square"
389 gcpy             self.diameter = 6.35
390 gcpy             self.flute = 22.225
391 gcpy             self.shaftdiameter = 6.35
392 gcpy             self.shaftheight = 22.225
393 gcpy             self.shaftlength = 20.0
394 gcpy             self.definesquaretool(self.diameter, self.shaftheight,
                         self.shaftlength)
395 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
396 gcpy             self.toolnumber = "100048"
397 gcpy         elif (tool_number == 205) or (tool_number == 100048): #205
                     == 100048
398 gcpy             self.writegc("(TOOL/MILL,␣6.35,␣0.00,␣0.00,␣0.00)")
399 gcpy             self.endmilltype = "square"
400 gcpy             self.diameter = 6.35
401 gcpy             self.flute = 25.4
402 gcpy             self.shaftdiameter = 6.35
403 gcpy             self.shaftheight = 25.4
404 gcpy             self.shaftlength = 20.0
405 gcpy             self.definesquaretool(self.diameter, self.shaftheight,
                         self.shaftlength)
406 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
407 gcpy             self.toolnumber = "100048"
408 gcpy         elif (tool_number == 211) or (tool_number == 213) or (
                     tool_number == 214): #211/213/214 == 100058
409 gcpy             self.writegc("(TOOL/MILL,␣8.00,␣0.00,␣0.00,␣0.00)")
410 gcpy             self.endmilltype = "square"
411 gcpy             self.diameter = 8.00
412 gcpy             self.flute = 26.0
413 gcpy             self.shaftdiameter = 8.00
414 gcpy             self.shaftheight = 26.0
415 gcpy             self.shaftlength = 49.0
416 gcpy             self.definesquaretool(self.diameter, self.shaftheight,
                         self.shaftlength)
417 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
418 gcpy             self.toolnumber = "100058"
419 gcpy #
```

The former distinction betwixt Square and O-flute tooling has been removed from the current version.

```
420 gcpy         elif (tool_number == 282) or (tool_number == 100204): #282
                     == 000204
421 gcpy             self.writegc("(TOOL/MILL,␣2.0,␣0.00,␣0.00,␣0.00)")
422 gcpy             self.endmilltype = "O-flute"
423 gcpy             self.diameter = 2.0
424 gcpy             self.flute = 6.35
425 gcpy             self.shaftdiameter = 6.35
426 gcpy             self.shaftheight = 6.35
427 gcpy             self.shaftlength = 12.0
428 gcpy             self.definesquaretool(self.diameter, self.shaftheight,
                         self.shaftlength, (self.shaftdiameter - self.
                         diameter)/2)
429 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
430 gcpy             self.toolnumber = "100204"
431 gcpy         elif (tool_number == 274) or (tool_number == 100036): #274
```

```
                == 000036
432 gcpy            self.writegc("(TOOL/MILL,␣3.175,␣0.00,␣0.00,␣0.00)")
433 gcpy            self.endmilltype = "O-flute"
434 gcpy            self.diameter = 3.175
435 gcpy            self.flute = 12.7
436 gcpy            self.shaftdiameter = 3.175
437 gcpy            self.shaftheight = 12.7
438 gcpy            self.shaftlength = 20.0
439 gcpy            self.definesquaretool(self.diameter, self.shaftheight,
                        self.shaftlength)
440 gcpy            self.defineshaft(self.diameter, self.shaftdiameter,
                        self.flute, 0, self.shaftlength)
441 gcpy            self.toolnumber = "100036"
442 gcpy        elif (tool_number == 278) or (tool_number == 100047): #278
                    == 000047
443 gcpy            self.writegc("(TOOL/MILL,␣6.35,␣0.00,␣0.00,␣0.00)")
444 gcpy            self.endmilltype = "O-flute"
445 gcpy            self.diameter = 6.35
446 gcpy            self.flute = 19.05
447 gcpy            self.shaftdiameter = 3.175
448 gcpy            self.shaftheight = 19.05
449 gcpy            self.shaftlength = 20.0
450 gcpy            self.definesquaretool(self.diameter, self.shaftheight,
                        self.shaftlength)
451 gcpy            self.defineshaft(self.diameter, self.shaftdiameter,
                        self.flute, 0, self.shaftlength)
452 gcpy            self.toolnumber = "100047"
453 gcpy #
```

3.4.1.1.4 Ball-nose The `elif`s continue with ball-nose (note that tapered-ball tooling is covered separately below) which are defined as one would expect by spheres and cylinders. Note that the Cutviewer definition of a the measurement point of a tool being at the center is not yet set up — potentially it opens up greatly simplified toolpath calculations and may be implemented in a future version.

```
454 gcpy        elif (tool_number == 202) or (tool_number == 204047): #202
                    == 204047
455 gcpy            self.writegc("(TOOL/MILL,␣6.35,␣3.175,␣0.00,␣0.00)")
456 gcpy            self.endmilltype = "ball"
457 gcpy            self.diameter = 6.35
458 gcpy            self.flute = 19.05
459 gcpy            self.shaftdiameter = 6.35
460 gcpy            self.shaftheight = 19.05
461 gcpy            self.shaftlength = 20.0
462 gcpy            self.defineballnosetool(self.diameter, self.flute, self
                        .shaftlength)
463 gcpy            self.defineshaft(self.diameter, self.shaftdiameter,
                        self.flute, 0, self.shaftlength)
464 gcpy            self.toolnumber = "204047"
465 gcpy        elif (tool_number == 101) or (tool_number == 203036): #101
                    == 203036
466 gcpy            self.writegc("(TOOL/MILL,␣3.175,␣1.5875,␣0.00,␣0.00)")
467 gcpy            self.endmilltype = "ball"
468 gcpy            self.diameter = 3.175
469 gcpy            self.flute = 12.7
470 gcpy            self.shaftdiameter = 3.175
471 gcpy            self.shaftheight = 12.7
472 gcpy            self.shaftlength = 20.0
473 gcpy            self.defineballnosetool(self.diameter, self.flute, self
                        .shaftlength)
474 gcpy            self.defineshaft(self.diameter, self.shaftdiameter,
                        self.flute, 0, self.shaftlength)
475 gcpy            self.toolnumber = "203036"
476 gcpy        elif (tool_number == 111) or (tool_number == 202024): #111
                    == 202024
477 gcpy            self.writegc("(TOOL/MILL,␣1.5875,␣0.79375,␣0.00,␣0.00)"
                        )
478 gcpy            self.endmilltype = "ball"
479 gcpy            self.diameter = 1.5875
480 gcpy            self.flute = 6.35
481 gcpy            self.shaftdiameter = 3.175
482 gcpy            self.shaftheight = 6.35
483 gcpy            self.shaftlength = 20.0
484 gcpy            self.defineballnosetool(self.diameter, self.flute, self
                        .shaftlength, (self.shaftdiameter - self.diameter)
                        /2)
```

```
485 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
486 gcpy             self.toolnumber = "202024"
487 gcpy         elif (tool_number == 121) or (tool_number == 201012): #121
                     == 201012
488 gcpy             self.writegc("(TOOL/MILL,␣3.175,␣0.79375,␣0.00,␣0.00)")
489 gcpy             self.endmilltype = "ball"
490 gcpy             self.diameter = 0.79375
491 gcpy             self.flute = 1.5875
492 gcpy             self.shaftdiameter = 3.175
493 gcpy             self.shaftheight = 1.5875
494 gcpy             self.shaftlength = 20.0
495 gcpy             self.defineballnosetool(self.diameter, self.flute, self
                         .shaftlength, (self.shaftdiameter - self.diameter)
                         /2)
496 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
497 gcpy             self.toolnumber = "201012"
498 gcpy         elif (tool_number == 325) or (tool_number == 204048): #325
                     (Amana 46376-K) == 204048
499 gcpy             self.writegc("(TOOL/MILL,␣6.35,␣3.175,␣0.00,␣0.00)")
500 gcpy             self.endmilltype = "ball"
501 gcpy             self.diameter = 6.35
502 gcpy             self.flute = 25.4
503 gcpy             self.shaftdiameter = 6.35
504 gcpy             self.shaftheight = 25.4
505 gcpy             self.shaftlength = 20.0
506 gcpy             self.defineballnosetool(self.diameter, self.flute, self
                         .shaftlength, (self.shaftdiameter - self.diameter)
                         /2)
507 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
508 gcpy             self.toolnumber = "204048"
509 gcpy         elif (tool_number == 212) or (tool_number == 205058): #212
                     == 205058
510 gcpy             self.writegc("(TOOL/MILL,␣8.00,␣4.00,␣0.00,␣0.00)")
511 gcpy             self.endmilltype = "ball"
512 gcpy             self.diameter = 8.00
513 gcpy             self.flute = 26.0
514 gcpy             self.shaftdiameter = 8.00
515 gcpy             self.shaftheight = 26.0
516 gcpy             self.shaftlength = 49.0
517 gcpy             self.defineballnosetool(self.diameter, self.flute, self
                         .shaftlength, (self.shaftdiameter - self.diameter)
                         /2)
518 gcpy             self.defineshaft(self.diameter, self.shaftdiameter,
                         self.flute, 0, self.shaftlength)
519 gcpy             self.toolnumber = "204048"
520 gcpy #
```

3.4.1.1.5 V Note that one V tool is described as an Engraver in Carbide Create. While CutViewer has specialty Tool/chamfer and Tool/drill parameters, it is possible to describe a V tool as a Tool/mill (using a very small tip radius).

```
521 gcpy         elif (tool_number == 301) or (tool_number == 390074): #301
                     == 390074
522 gcpy             self.writegc("(TOOL/MILL,␣0.10,␣0.05,␣6.35,␣45.00)")
523 gcpy             self.endmilltype = "V"
524 gcpy             self.diameter = 12.7
525 gcpy             self.flute = 6.35
526 gcpy             self.angle = 90
527 gcpy             self.shaftdiameter = 6.35
528 gcpy             self.shaftheight = 6.35
529 gcpy             self.shaftlength = 20.0
530 gcpy             self.defineVtool(self.diameter, self.flute, self.
                         shaftlength, self.shaftdiameter)
531 gcpy             self.toolnumber = "390074"
532 gcpy         elif (tool_number == 302) or (tool_number == 360071): #302
                     == 360071
533 gcpy             self.writegc("(TOOL/MILL,␣0.10,␣0.05,␣6.35,␣30.00)")
534 gcpy             self.endmilltype = "V"
535 gcpy             self.diameter = 12.7
536 gcpy             self.flute = 11.067
537 gcpy             self.angle = 60
538 gcpy             self.shaftdiameter = 6.35
539 gcpy             self.shaftheight = 11.067
```

```
540 gcpy             self.shaftlength = 20.0
541 gcpy             self.defineVtool(self.diameter, self.flute, self.
                         shaftlength, self.shaftdiameter)
542 gcpy             self.toolnumber = "360071"
543 gcpy         elif (tool_number == 311) or (tool_number == 390121): #311
                     == 390121
544 gcpy             self.writegc("(TOOL/MILL,␣0.10,␣0.05,␣6.00,␣45.00)")
545 gcpy             self.endmilltype = "V"
546 gcpy             self.diameter = 12.0
547 gcpy             self.flute = 6.00
548 gcpy             self.angle = 90
549 gcpy             self.shaftdiameter = 8.00
550 gcpy             self.shaftheight = 8.00
551 gcpy             self.shaftlength = 20.0
552 gcpy             self.defineVtool(self.diameter, self.flute, self.
                         shaftlength, self.shaftdiameter)
553 gcpy             self.toolnumber = "390121"
554 gcpy         elif (tool_number == 312) or (tool_number == 360121): #312
                     == 360121
555 gcpy             self.writegc("(TOOL/MILL,␣0.10,␣0.05,␣6.00,␣30.00)")
556 gcpy             self.endmilltype = "V"
557 gcpy             self.diameter = 12.0
558 gcpy             self.flute = 10.39
559 gcpy             self.angle = 60
560 gcpy             self.shaftdiameter = 8.00
561 gcpy             self.shaftheight = 10.39
562 gcpy             self.shaftlength = 20.0
563 gcpy             self.defineVtool(self.diameter, self.flute, self.
                         shaftlength, self.shaftdiameter)
564 gcpy             self.toolnumber = "360121"
565 gcpy         elif (tool_number == 327) or (tool_number == 360098): #327
                     (Amana RC-1148) == 360098
566 gcpy             self.writegc("(TOOL/MILL,␣0.03,␣0.00,␣13.4874,␣30.00)")
567 gcpy             self.endmilltype = "V"
568 gcpy             self.diameter = 25.4
569 gcpy             self.flute = 22.134
570 gcpy             self.angle = 60
571 gcpy             self.shaftdiameter = 6.35
572 gcpy             self.shaftheight = 22.134
573 gcpy             self.shaftlength = 20.0
574 gcpy             self.defineVtool(self.diameter, self.flute, self.
                         shaftlength, self.shaftdiameter)
575 gcpy             self.toolnumber = "360098"
576 gcpy         elif (tool_number == 323) or (tool_number == 330041): #323
                     == 330041 30 degree V Amana, 45771-K
577 gcpy             self.writegc("(TOOL/MILL,␣0.10,␣0.05,␣11.18,␣15.00)")
578 gcpy             self.endmilltype = "V"
579 gcpy             self.diameter = 6.35
580 gcpy             self.flute = 11.849
581 gcpy             self.angle = 30
582 gcpy             self.shaftdiameter = 6.35
583 gcpy             self.shaftheight = 11.849
584 gcpy             self.shaftlength = 20.0
585 gcpy             self.defineVtool(self.diameter, self.flute, self.
                         shaftlength, self.shaftdiameter)
586 gcpy             self.toolnumber = "330041"
587 gcpy         elif (tool_number == 390) or (tool_number == 390032):
588 gcpy             self.writegc("(TOOL/MILL,0.03,␣0.00,␣1.5875,␣45.00)")
589 gcpy             self.endmilltype = "V"
590 gcpy             self.diameter = 3.175
591 gcpy             self.flute = 1.5875
592 gcpy             self.angle = 90
593 gcpy             self.shaftdiameter = 3.175
594 gcpy             self.shaftheight = 1.5875
595 gcpy             self.shaftlength = 20.0
596 gcpy             self.defineVtool(self.diameter, self.flute, self.
                         shaftlength, self.shaftdiameter)
597 gcpy             self.toolnumber = "390032"
598 gcpy         elif (tool_number == 690) or (tool_number == 290032):
599 gcpy             self.writegc("(TOOL/MILL,0.10,␣0.05,␣12.70,␣45.00)")
600 gcpy             self.endmilltype = "V"
601 gcpy             self.diameter = 1.5875
602 gcpy             self.flute = 0.79375
603 gcpy             self.angle = 90
604 gcpy             self.shaftdiameter = 3.175
605 gcpy             self.shaftheight = 1.5875
606 gcpy             self.shaftlength = 20.0
607 gcpy             self.defineVtool(self.diameter, self.flute, self.
```

```
                    shaftlength, self.shaftdiameter)
608 gcpy             self.toolnumber = "290032"
609 gcpy #
```

3.4.1.1.6 Keyhole

Keyhole tooling will primarily be used with a dedicated toolpath.

```
610 gcpy         elif (tool_number == 374) or (tool_number == 906043): #374
                    == 906043
611 gcpy             self.writegc("(TOOL/MILL,␣9.53,␣0.00,␣3.17,␣0.00)")
612 gcpy             self.endmilltype = "keyhole"
613 gcpy             self.diameter = 9.525
614 gcpy             self.flute = 3.175
615 gcpy             self.radius = 6.35
616 gcpy             self.shaftdiameter = 6.35
617 gcpy             self.shaftheight = 3.175
618 gcpy             self.shaftlength = 20.0
619 gcpy             self.defineKeyholetool(self.diameter, self.flute, self.
                        shaftdiameter, self.shaftheight, self.shaftdiameter,
                         self.shaftlength)
620 gcpy             self.toolnumber = "906043"
621 gcpy         elif (tool_number == 375) or (tool_number == 906053): #375
                    == 906053
622 gcpy             self.writegc("(TOOL/MILL,␣9.53,␣0.00,␣3.17,␣0.00)")
623 gcpy             self.endmilltype = "keyhole"
624 gcpy             self.diameter = 9.525
625 gcpy             self.flute = 3.175
626 gcpy             self.radius = 8
627 gcpy             self.shaftdiameter = 6.35
628 gcpy             self.shaftheight = 3.175
629 gcpy             self.shaftlength = 20.0
630 gcpy             self.defineKeyholetool(self.diameter, self.flute, self.
                        shaftdiameter, self.shaftheight, self.shaftdiameter,
                         self.shaftlength)
631 gcpy             self.toolnumber = "906053"
632 gcpy         elif (tool_number == 376) or (tool_number == 907040): #376
                    == 907040
633 gcpy             self.writegc("(TOOL/MILL,␣12.7,␣0.00,␣4.77,␣0.00)")
634 gcpy             self.endmilltype = "keyhole"
635 gcpy             self.diameter = 12.7
636 gcpy             self.flute = 4.7625
637 gcpy             self.radius = 6.35
638 gcpy             self.shaftdiameter = 6.35
639 gcpy             self.shaftheight = 4.7625
640 gcpy             self.shaftlength = 20.0
641 gcpy             self.defineKeyholetool(self.diameter, self.flute, self.
                        shaftdiameter, self.shaftheight, self.shaftdiameter,
                         self.shaftlength)
642 gcpy             self.toolnumber = "907040"
643 gcpy         elif (tool_number == 378) or (tool_number == 907050): #378
                    == 907050
644 gcpy             self.writegc("(TOOL/MILL,␣12.7,␣0.00,␣4.77,␣0.00)")
645 gcpy             self.endmilltype = "keyhole"
646 gcpy             self.diameter = 12.7
647 gcpy             self.flute = 4.7625
648 gcpy             self.radius = 8
649 gcpy             self.shaftdiameter = 6.35
650 gcpy             self.shaftheight = 4.7625
651 gcpy             self.shaftlength = 20.0
652 gcpy             self.defineKeyholetool(self.diameter, self.flute, self.
                        shaftdiameter, self.shaftheight, self.shaftdiameter,
                         self.shaftlength)
653 gcpy             self.toolnumber = "907050"
654 gcpy #
```

3.4.1.1.7 Bowl

This geometry is also useful for square endmills with a radius.

```
655 gcpy         elif (tool_number == 45981): #45981 == 445981
656 gcpy #Amana Carbide Tipped Bowl & Tray 1/8 Radius x 1/2 Dia x 1/2 x 1/4
        Inch Shank
657 gcpy             self.writegc("(TOOL/MILL,0.03,␣0.00,␣10.00,␣30.00)")
658 gcpy             self.writegc("(TOOL/MILL,␣15.875,␣6.35,␣19.05,␣0.00)")
659 gcpy             self.endmilltype = "bowl"
660 gcpy             self.diameter = 12.7
661 gcpy             self.flute = 12.7
662 gcpy             self.radius = 3.175
```

```
663 gcpy             self.shaftdiameter = 6.35
664 gcpy             self.shaftheight = 12.7
665 gcpy             self.shaftlength = 20.0
666 gcpy             self.definebowltool(self.diameter, self.flute, self.
                        radius, self.shaftdiameter, self.shaftlength)
667 gcpy             self.toolnumber = "445981"
668 gcpy         elif (tool_number == 45982):#0.507/2, 4.509
669 gcpy             self.writegc("(TOOL/MILL,␣15.875,␣6.35,␣19.05,␣0.00)")
670 gcpy             self.endmilltype = "bowl"
671 gcpy             self.diameter = 19.05
672 gcpy             self.flute = 15.875
673 gcpy             self.radius = 6.35
674 gcpy             self.shaftdiameter = 6.35
675 gcpy             self.shaftheight = 15.875
676 gcpy             self.shaftlength = 20.0
677 gcpy             self.definebowltool(self.diameter, self.flute, self.
                        radius, self.shaftdiameter, self.shaftlength)
678 gcpy             self.toolnumber = "445982"
679 gcpy         elif (tool_number == 1370): #1370 == 401370
680 gcpy #Whiteside Bowl & Tray Bit 1/4"SH, 1/8"R, 7/16"CD (5/16" cutting
        flute length)
681 gcpy             self.writegc("(TOOL/MILL,␣11.1125,␣8,␣3.175,␣0.00)")
682 gcpy             self.endmilltype = "bowl"
683 gcpy             self.diameter = 11.1125
684 gcpy             self.flute = 8
685 gcpy             self.radius = 3.175
686 gcpy             self.shaftdiameter = 6.35
687 gcpy             self.shaftheight = 8
688 gcpy             self.shaftlength = 20.0
689 gcpy             self.definebowltool(self.diameter, self.flute, self.
                        radius, self.shaftdiameter, self.shaftlength)
690 gcpy             self.toolnumber = "401370"
691 gcpy         elif (tool_number == 1372): #1372/45982 == 401372
692 gcpy #Whiteside Bowl & Tray Bit 1/4"SH, 1/4"R, 3/4"CD (5/8" cutting
        flute length)
693 gcpy #Amana Carbide Tipped Bowl & Tray 1/4 Radius x 3/4 Dia x 5/8 x 1/4
        Inch Shank
694 gcpy             self.writegc("(TOOL/MILL,␣19.5,␣15.875,␣6.35,␣0.00)")
695 gcpy             self.endmilltype = "bowl"
696 gcpy             self.diameter = 19.5
697 gcpy             self.flute = 15.875
698 gcpy             self.radius = 6.35
699 gcpy             self.shaftdiameter = 6.35
700 gcpy             self.shaftheight = 15.875
701 gcpy             self.shaftlength = 20.0
702 gcpy             self.definebowltool(self.diameter, self.flute, self.
                        radius, self.shaftdiameter, self.shaftlength)
703 gcpy             self.toolnumber = "401372"
704 gcpy #
```

3.4.1.1.8 Tapered ball nose One vendor which provides such tooling is Precise Bits: https://www.precisebits.com/products/carbidebits/taperedcarve250b2f.asp&filter=7, but unfortunately, their tool numbering is ambiguous, the version of each major number (204 and 304) for their 1/4″ shank tooling which is sufficiently popular to also be offered in a ZRN coating could be used. Similarly, the #501 and #502 PCB engravers from Carbide 3D are supported.

Outlines and profiles for these tools are stored in SVG files:

```
501_outline.svg
501_profile.svg
501_shaft_outline.svg
501_shaft_profile.svg
502_outline.svg
502_profile.svg
502_shaft_outline.svg
502_shaft_profile.svg
```

which are then imported into the appropriate variables when a tool is loaded.

```
705 gcpy         elif (tool_number == 501) or (tool_number == 530131): #501
                     == 530131
706 gcpy             self.writegc("(TOOL/MILL,0.03,␣0.00,␣10.00,␣30.00)")
707 gcpy #             self.currenttoolshape = self.toolshapes("tapered ball
        ", 3.175, 5.561, 30, 0.254)
708 gcpy             self.tooloutline = osimport("501_outline.svg")
709 gcpy             self.toolprofile = osimport("501_profile.svg")
710 gcpy             self.endmilltype = "tapered␣ball"
711 gcpy             self.diameter = 3.175
```

```
712 gcpy             self.flute = 5.561
713 gcpy             self.angle = 30
714 gcpy             self.tip = 0.254
715 gcpy             self.shaftdiameter = 3.175
716 gcpy             self.shaftheight = 5.561
717 gcpy             self.shaftlength = 10.0
718 gcpy             self.toolnumber = "530131"
719 gcpy         elif (tool_number == 502) or (tool_number == 540131): #502
                     == 540131
720 gcpy             self.writegc("(TOOL/MILL,0.03,␣0.00,␣10.00,␣20.00)")
721 gcpy #             self.currenttoolshape = self.toolshapes("tapered ball
        ", 3.175, 4.117, 40, 0.254)
722 gcpy             self.endmilltype = "tapered␣ball"
723 gcpy             self.diameter = 3.175
724 gcpy             self.flute = 4.117
725 gcpy             self.angle = 40
726 gcpy             self.tip = 0.254
727 gcpy             self.shaftdiameter = 3.175
728 gcpy             self.shaftheight = 4.117
729 gcpy             self.shaftlength = 10.0
730 gcpy             self.toolnumber = "540131"
731 gcpy #         elif (tool_number == 204):#
732 gcpy #             self.writegc("()")
733 gcpy #             self.currenttoolshape = self.tapered_ball(1.5875,
        6.35, 38.1, 3.6)
734 gcpy #         elif (tool_number == 304):#
735 gcpy #             self.writegc("()")
736 gcpy #             self.currenttoolshape = self.tapered_ball(3.175, 6.35,
         38.1, 2.4)
737 gcpy #
```

3.4.1.1.9 Roundover (cove tooling) Note that the parameters will need to incorporate the tip diameter into the overall diameter.

```
738 gcpy         elif (tool_number == 56125) or (tool_number == 603042):#
                     0.508/2, 1.531 56125 == 603042
739 gcpy             self.writegc("(TOOL/CRMILL,␣0.508,␣6.35,␣3.175,␣7.9375,
                         ␣3.175)")
740 gcpy             self.endmilltype = "roundover"
741 gcpy             self.tipdiameter = 0.508
742 gcpy             self.diameter = 6.35 - self.tipdiameter
743 gcpy             self.flute = 8 - self.tipdiameter
744 gcpy             self.radius = 3.175 - self.tipdiameter/2
745 gcpy             self.shaftdiameter = 6.35
746 gcpy             self.shaftheight = 8
747 gcpy             self.shaftlength = 10.0
748 gcpy             self.defineRoundovertool(self.diameter, self.
                         tipdiameter, self.flute, self.radius, self.
                         shaftdiameter, self.shaftlength)
749 gcpy             self.toolnumber = "603042"
750 gcpy         elif (tool_number == 56142) or (tool_number == 602032):#
                     0.508/2, 2.921 56142 == 602032
751 gcpy             self.writegc("(TOOL/CRMILL,␣0.508,␣3.571875,␣1.5875,␣
                         5.55625,␣1.5875)")
752 gcpy             self.endmilltype = "roundover"
753 gcpy             self.tip = 0.508
754 gcpy             self.diameter = 3.175 - self.tip
755 gcpy             self.flute = 4.7625 - self.tip
756 gcpy             self.radius = 1.5875 - self.tip/2
757 gcpy             self.shaftdiameter = 3.175
758 gcpy             self.shaftheight = 4.7625
759 gcpy             self.shaftlength = 10.0
760 gcpy             self.toolnumber = "602032"
761 gcpy #         elif (tool_number == 312):#1.524/2, 3.175
762 gcpy #             self.writegc("(TOOL/CRMILL, Diameter1, Diameter2,
        Radius, Height, Length)")
763 gcpy #         elif (tool_number == 1568):#0.507/2, 4.509 1568 == 603032
764 gcpy ##FIX             self.writegc("(TOOL/CRMILL, 0.17018, 9.525,
        4.7625, 12.7, 4.7625)")
765 gcpy ##             self.currenttoolshape = self.toolshapes("roundover",
        3.175, 6.35, 3.175, 0.396875)
766 gcpy #             self.endmilltype = "roundover"
767 gcpy #             self.diameter = 3.175
768 gcpy #             self.flute = 6.35
769 gcpy #             self.radius = 3.175
770 gcpy #             self.tip = 0.396875
```

```
771 gcpy #             self.toolnumber = "603032"
772 gcpy ##https://www.amanatool.com/45982-carbide-tipped-bowl-tray-1-4-
        radius-x-3-4-dia-x-5-8-x-1-4-inch-shank.html
773 gcpy #         elif (tool_number == 1570):#0.507/2, 4.509 1570 == 600002
        ?!?
774 gcpy #             self.writegc("(TOOL/CRMILL, 0.17018, 9.525, 4.7625,
        12.7, 4.7625)")
775 gcpy ##             self.currenttoolshape = self.toolshapes("roundover",
        4.7625, 9.525, 4.7625, 0.396875)
776 gcpy #             self.endmilltype = "roundover"
777 gcpy #             self.diameter = 4.7625
778 gcpy #             self.flute = 9.525
779 gcpy #             self.radius = 4.7625
780 gcpy #             self.tip = 0.396875
781 gcpy #             self.toolnumber = "600002"
782 gcpy #         elif (tool_number == 1572): #1572 = 604042
783 gcpy ##FIX             self.writegc("(TOOL/CRMILL, 0.17018, 9.525,
        4.7625, 12.7, 4.7625)")
784 gcpy ##             self.currenttoolshape = self.toolshapes("roundover",
        6.35, 12.7, 6.35, 0.396875)
785 gcpy #             self.endmilltype = "roundover"
786 gcpy #             self.diameter = 6.35
787 gcpy #             self.flute = 12.7
788 gcpy #             self.radius = 6.35
789 gcpy #             self.tip = 0.396875
790 gcpy #             self.toolnumber = "604042"
791 gcpy #         elif (tool_number == 1574): #1574 == 600062
792 gcpy ##FIX             self.writegc("(TOOL/CRMILL, 0.17018, 9.525,
        4.7625, 12.7, 4.7625)")
793 gcpy ##             self.currenttoolshape = self.toolshapes("roundover",
        9.525, 19.5, 9.515, 0.396875)
794 gcpy #             self.endmilltype = "roundover"
795 gcpy #             self.diameter = 9.525
796 gcpy #             self.flute = 19.5
797 gcpy #             self.radius = 9.515
798 gcpy #             self.tip = 0.396875
799 gcpy #             self.toolnumber = "600062"
800 gcpy #
```

3.4.1.1.10 Dovetails Unfortunately, tools which support undercuts such as dovetails are not supported by many CAM tools including Carbide Create and CutViewer (CAMotics will work for such tooling, at least dovetails which may be defined as "stub" endmills with a bottom diameter greater than upper diameter).

```
801 gcpy         elif (tool_number == 814) or (tool_number == 814071): #814
            == 814071
802 gcpy #Item 18J1607, 1/2" 14ř Dovetail Bit, 8mm shank
803 gcpy             self.writegc("(TOOL/MILL,␣12.7,␣6.367,␣12.7,␣0.00)")
804 gcpy         #    dt_bottomdiameter, dt_topdiameter, dt_height, dt_angle
            )
805 gcpy         #    https://www.leevalley.com/en-us/shop/tools/power-tool-
            accessories/router-bits/30172-dovetail-bits?item=18J1607
806 gcpy #             self.currenttoolshape = self.toolshapes("dovetail",
        12.7, 12.7, 14)
807 gcpy             self.endmilltype = "dovetail"
808 gcpy             self.diameter = 12.7
809 gcpy             self.flute = 12.7
810 gcpy             self.angle = 14
811 gcpy             self.toolnumber = "814071"
812 gcpy         elif (tool_number == 808079) or (tool_number == 808071): #
            45828 == 808071
813 gcpy             self.writegc("(TOOL/MILL,␣12.7,␣6.816,␣20.95,␣0.00)")
814 gcpy         #    http://www.amanatool.com/45828-carbide-tipped-dovetail
            -8-deg-x-1-2-dia-x-825-x-1-4-inch-shank.html
815 gcpy #             self.currenttoolshape = self.toolshapes("dovetail",
        12.7, 20.955, 8)
816 gcpy             self.endmilltype = "dovetail"
817 gcpy             self.diameter = 12.7
818 gcpy             self.flute = 20.955
819 gcpy             self.angle = 8
820 gcpy             self.toolnumber = "808071"
821 gcpy #
```

Each tool must be modeled in 3D using OpenSCAD commands, but it will also be necessary to have a consistent structure for managing the various shapes and aspects of shapes.

While tool shapes were initially handled as geometric shapes stored in Python variables, pro-

cessing them as such after the fashion of OpenSCAD required the use of `union()` commands and assigning a small initial object (usually a primitive placed at the origin) so that the union could take place. This has the result of creating a nested union structure in the CSG tree which can quickly become so deeply nested that it exceeds the limits set in PythonSCAD.

As was discussed in the PythonSCAD Google Group (https://groups.google.com/g/pythonscad/c/rtiYa38W8tY), if a list is used instead, then the contents of the list are added all at once at a single level when processed.

An example file which shows this concept:

```
from openscad import *
fn=200

box = cube([40,40,40])

features = []

features.append(cube([36,36,40]) + [2,2,2])
features.append(cylinder(d=20,h=5) + [20,20,-1])
features.append(cylinder(d=3,h=10) ^ [[5,35],[5,35], -1])

part = difference(box, features)

show(part)
```

As per usual, the OpenSCAD command is simply a dispatcher:

```
48 gcpscad module toolchange(tool_number, speed){
49 gcpscad     gcp.toolchange(tool_number, speed);
50 gcpscad }
```

For example:

```
toolchange(small_square_tool_num, speed);
```

(the assumption is that all speed rates in a file will be the same, so as to account for the most frequent use case of a trim router with speed controlled by a dial setting and feed rates/ratios being calculated to provide the correct chipload at that setting.)

3.4.1.1.11 closing G-code With the tools delineated, the module is closed out and the toolchange information written into the G-code as well as the command to start the spindle at the specified speed.

One possible feature for the G-code for tool changes would be to have the various ratios available and then to apply the appropriate one. Directly applying them in the file generated by the user is sufficiently straight-forward that this expedient option seems a needless complexity unless a compelling reason comes up.

```
822 gcpy          self.writegc("M6T", str(tool_number))
823 gcpy #         if (self.endmilltype == "square"):
824 gcpy #             speed = speed *
825 gcpy          self.writegc("M03S", str(speed))
```

3.4.2 Laser support

Two possible options for supporting a laser present themselves: color-coded DXFs or direct G-code support. Note that more recent versions of PythonSCAD have added explicit supportr for outputting G-code for a laser.

An example file using gcodepreview for the latter:
https://lasergrbl.com/test-file-and-samples/depth-of-focus-test/

```
M3 S0
S0
G0X0Y16
S1000
G1X100F1200
S0
M5 S0
M3 S0
S0
G0X0Y12
S1000
G1X100F1000
S0
M5 S0
M3 S0
```

```
S0
G0X0Y8
S1000
G1X100F800
S0
M5 S0
M3 S0
S0
G0X0Y4
S1000
G1X100F600
S0
M5 S0
M3 S0
S0
G0X0Y0
S1000
G1X100F400
S0
M5 S0
```

3.5 Shapes and tool movement

With all the scaffolding in place, it is possible to model the tool and `hull()` between copies of the
`cut...` 3D model of the tool, or a cross-section of it for both `cut...` and `rapid...` operations.
`rapid...` Another possibility is describing tools in terms of outline which will allow using `linear/rotate_extrude` to be used which requires a description of the tools as profiles/outlines, but which matches the `G0/G1` and `G2/G3` G-code commands.

The majority of commands will be more general, focusing on tooling which is generally supported by this library, moving in lines and arcs so as to describe shapes which lend themselves to representation with those tools and which match up with both toolpaths and supported geometry in Carbide Create, and the usage requirements of the typical user.

This structure has the notable advantage that if a tool shape is represented as a list and always handled thus, then representing complex shapes which need to be represented in discrete elements/parts becomes a natural thing to do and the program architecture is simpler since all possible shapes may be handled by the same code/logic with no need to identify different shapes and handle them differently.

Note that it will be preferable to use `extend` if the variable to be added contains a list rather than `append` since the former will flatten out the list and add the individual elements, so that a list remains a list of elements rather than becoming a list of lists and elements, except that there will be at least two elements to each tool model list:

- cutting *tool* shape (note that this may be either a single model, or a list of discrete slices of the tool shape)
- *shaft*

and when a cut is made by hulling each element from the cut begin position to its end position, this will be done using different colors so that the shaft rubbing may be identified on the 3D surface of the preview of the cut.

3.5.1 Tooling for Undercutting Toolpaths

There are several notable candidates for undercutting tooling.

- Keyhole tools — intended to cut slots for retaining hardware used for picture hanging, they may be used to create slots for other purposes Note that it will be necessary to model these thrice, once for the actual keyhole cutting, second for the fluted portion of the shaft, and then the shaft should be modeled for collision https://assetssc.leevalley.com/en-gb/shop/tools/power-tool-accessories/router-bits/30113-keyhole-router-bits
- Dovetail cutters — used for the joinery of the same name, they cut a large area at the bottom which slants up to a narrower region at a defined angle
- Lollipop cutters — normally used for 3D work, as their name suggests they are essentially a (cutting) ball on a narrow stick (the tool shaft), they are mentioned here only for compleatness' sake and are not (at this time) implemented
- Threadmill — used for cutting threads, normally a single form geometry is used on a CNC.

3.5.2 Generalized commands and cuts

The first consideration is a naming convention which will allow a generalized set of associated commands to be defined.

There are three different movements in G-code which will need to be handled. Rapid commands will be used for G0 movements and will not appear in DXFs but will appear in G-code files, while straight line cut (G1) and arc (G2/G3) commands may appear in both G-code and DXF files, depending on the specific command invoked.

3.5.3 Movement and color

toolmovement
shaftmovement

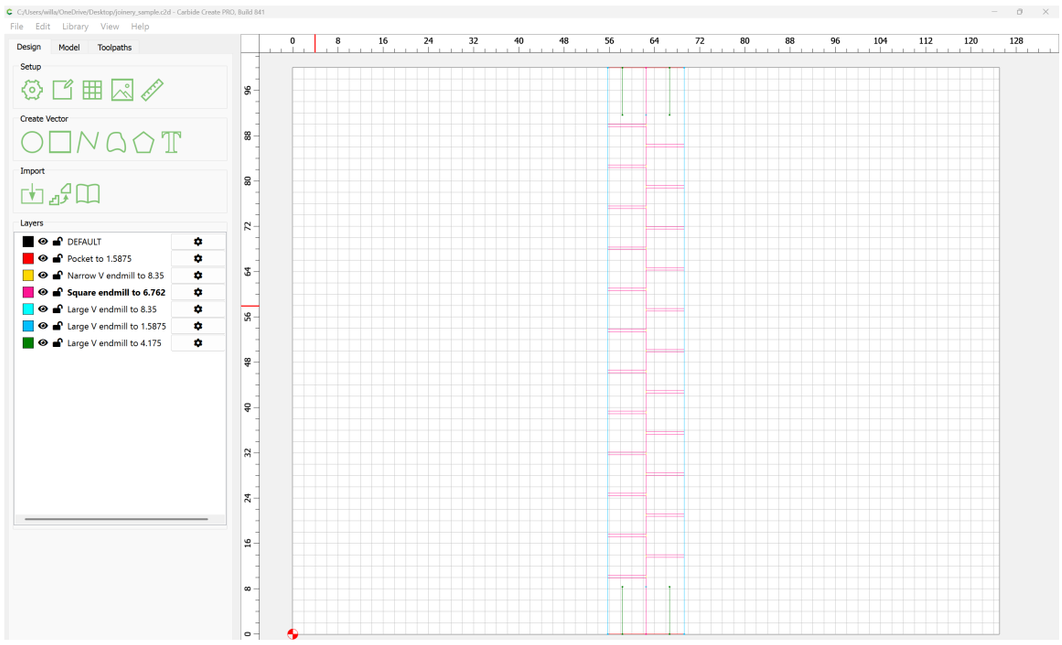
The first command which must be defined is `toolmovement` which is used as the core of the other commands, affording a 3D model of the tool moving in a straight line. A matching `shaftmovement` command will allow modeling collision of the shaft with the stock should it occur. This differentiation raises the matter of color representation. Using a different color for the shape of the endmill when cutting and for rapid movements will similarly allow identifying instances of the tool crashing through stock at rapid speed.

```
827 gcpy      def setcolor(self,
828 gcpy                   cutcolor = "green",
829 gcpy                   rapidcolor = "orange",
830 gcpy                   shaftcolor = "red"):
831 gcpy          self.cutcolor = cutcolor
832 gcpy          self.rapidcolor = rapidcolor
833 gcpy          self.shaftcolor = shaftcolor
```

```
52 gcpscad module setcolor(cutcolor, rapidcolor, shaftcolor){
53 gcpscad     gcp.setcolor(cutcolor, rapidcolor, shaftcolor);
54 gcpscad }
```

The possible colors for OpenSCAD are those of Web colors (https://en.wikipedia.org/wiki/Web_colors), while DXF has its own set of colors based on numbers (see table) and Carbide Create's colour-coding of layers adds another set and applying a Venn diagram and removing problematic extremes we arrive at the third column (Both) as black and white are potentially inconsistent/confusing since at least one CAD program toggles them based on light/dark mode being applied to its interface.

A further consideration is that colors *per se* are not a useful characteristic for the typical usage of cutting a single material (it may be that in the future, supporting this for 3D-printing will be an option). Instead, placing all geometry which is associated with a given tool at a specified depth on an appropriately named layer will facilitate associating said elements with a matching toolpath by using the Carbide Create features for assigning layers when importing a DXF and its ability to set a toolpath to be applied to the elements on a specified layer.



A naming convention of Toolpath type (since the tool associated tool is obvious) or Tool type and size, the word " to " and a specified depth matches how toolpaths are often named and has the potential to be generated automatically. It will be helpful if the colors used are coordinated between the two usages, but that is not strictly necessary (the assignment in Carbide Create will default to Black when a DXF is opened, or will remain as assigned if imported into a `.c2d` file with extant layers of the same name(s)) which if serving as a template will have appropriate toolpaths with matching names and settings.

Most tools are easily implemented with concise 3D descriptions which may be connected with a simple `hull` operation. Note that extending the normal case to a pair of such operations, one for the shaft, the other for the cutting shape will markedly simplify the code, and will make it possible to color-code the shaft which may afford indication of instances of it rubbing against the stock.

Note that the variables `self.rapids` and `self.toolpaths` are used to hold the list of accumulated 3D models of the rapid motions and cuts as elements in lists so that they may be `differenced` from the stock.

Table 1: Colors in OpenSCAD and DXF with Carbide Create swatches

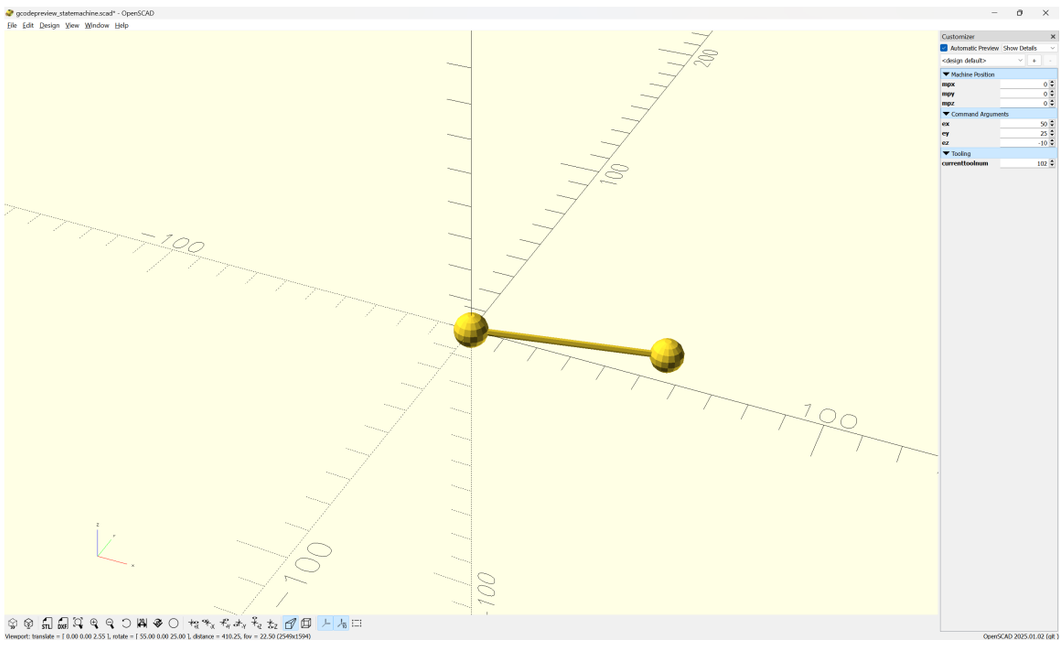
Web Colors (OpenSCAD)	DXF	Both	Carbide Create
Black	"Black" (0)	Black	
Red	"Red" (1)	Red	
Yellow	"Yellow" (2)	Yellow	
Green	"Green" (3)	Green	
Aqua	"Cyan" (4)	Aqua/Cyan	
Blue	"Blue" (5)	Blue	
Fuchsia	"Magenta" (6)	Fuchsia/Magenta	
Gray	"Dark Gray" (8)	(Dark) Gray	
Silver	"Light Gray" (9)	Silver/Light Gray	
Maroon			
Olive			
Lime			
Teal			
Navy			
Purple			

White (7) is omitted from the above list (note that the names are not case-sensitive)

3.5.3.1 toolmovement The `toolmovement` command incorporates the color variables to indicate cutting and differentiate rapid movements and the tool shaft.

When diagramming this, note that there are two possibilities — in the simplest, a movement made from the current position to the end. If we start at the origin, `X0`, `Y0`, `Z0`, then it is simply a straight-line movement (rapid)/cut (possibly a partial cut in the instance of a keyhole or roundover tool), and variables will change or not change value depending on whether the current move is a compleat operation or an incremental portion of a move where the exposed variables will be updated at the end.

The code for showing this graphically is quite straight-forward. A BlockSCAD implementation is available at: https://www.blockscad3d.com/community/projects/1894400, and the OpenSCAD version is only a little more complex (adding code to ensure positioning):




```
835 gcpy     def toolmovement(self, bx, by, bz, ex, ey, ez, step = 0):
836 gcpy         tslist = []
837 gcpy         if step > 0:
838 gcpy             steps = step
839 gcpy         else:
840 gcpy             steps = self.steps
841 gcpy #
```

```
56 gcpscad module toolmovement(bx, by, bz, ex, ey, ez, step){
57 gcpscad     gcp.toolmovement(bx, by, bz, ex, ey, ez, step);
58 gcpscad }
```

For the implementation of tool movement, each different sort of tool will need to be handled separately in an `if | then` structure:

endmill square

3.5.3.1.1 Square (including O-flute)

The `endmill square` is a simple cylinder:

```
842 gcpy         if self.endmilltype == "square":
843 gcpy             ts = cylinder(r1=(self.diameter / 2), r2=(self.diameter
                         / 2), h=self.flute, center = False)
844 gcpy             tslist.append(hull(ts.translate([bx, by, bz]), ts.
                         translate([ex, ey, ez])))
845 gcpy             return tslist
846 gcpy #
847 gcpy #         if self.endmilltype == "O-flute":
848 gcpy #             ts = cylinder(r1=(self.diameter / 2), r2=(self.
                    diameter / 2), h=self.flute, center = False)
849 gcpy #             tslist.append(hull(ts.translate([bx, by, bz]), ts.
                    translate([ex, ey, ez])))
850 gcpy #             return tslist
851 gcpy #
```

ballnose

3.5.3.1.2 Ball nose

The `ballnose` is modeled as a hemisphere joined with a cylinder:

```
852 gcpy         if self.endmilltype == "ball":
853 gcpy             b = sphere(r=(self.diameter / 2))
854 gcpy             s = cylinder(r1=(self.diameter / 2), r2=(self.diameter
                         / 2), h=self.flute, center=False)
855 gcpy             bs = union(b, s)
856 gcpy             bs = bs.translate([0, 0, (self.diameter / 2)])
857 gcpy             tslist.append(hull(bs.translate([bx, by, bz]), bs.
                         translate([ex, ey, ez])))
858 gcpy             return tslist
859 gcpy #
```

3.5.3.1.3 bowl

The bowl tool is modeled as a series of cylinders stacked on top of each other and `hull()`ed together:

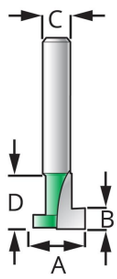
```
860 gcpy         if self.endmilltype == "bowl":
861 gcpy             inner = cylinder(r1 = self.diameter/2 - self.radius, r2
                         = self.diameter/2 - self.radius, h = self.flute)
862 gcpy             outer = cylinder(r1 = self.diameter/2, r2 = self.
                        diameter/2, h = self.flute - self.radius)
863 gcpy             outer = outer.translate([0,0, self.radius])
864 gcpy             slices = hull(outer, inner)
865 gcpy #    slices = cylinder(r1 = 0.0001, r2 = 0.0001, h = 0.0001, center
            =False)
866 gcpy             for i in range(1, 90 - self.steps, self.steps):
867 gcpy                 slice = cylinder(r1 = self.diameter / 2 - self.
                            radius + self.radius * Sin(i), r2 = self.
                            diameter / 2 - self.radius + self.radius * Sin(i
                            +self.steps), h = self.radius/90, center=False)
868 gcpy                 slices = hull(slices, slice.translate([0, 0, self.
                            radius - self.radius * Cos(i+self.steps)]))
869 gcpy             tslist.append(hull(slices.translate([bx, by, bz]),
                        slices.translate([ex, ey, ez])))
870 gcpy             return tslist
871 gcpy #
```

endmill v

3.5.3.1.4 V The `endmill v` is modeled as a cylinder with a zero width base and a second cylinder for the shaft (note that Python's `math` defaults to radians, hence the need to convert from degrees if using it, but fortunately, trigonometric commands have been added to OpenPythonSCAD (`Sin`, `Cos`, `Tan`, `Atan`)):

```
872 gcpy         if self.endmilltype == "V":
873 gcpy             v = cylinder(r1=0, r2=(self.diameter / 2), h=((self.
                        diameter / 2) / Tan((self.angle / 2))), center=False
                        )
874 gcpy #               s = cylinder(r1=(self.diameter / 2), r2=(self.
            diameter / 2), h=self.flute, center=False)
875 gcpy #               sh = s.translate([0, 0, ((self.diameter / 2) / Tan
            ((self.angle / 2)))])
876 gcpy             tslist.append(hull(v.translate([bx, by, bz]), v.
                        translate([ex, ey, ez])))
877 gcpy             return tslist
```

3.5.3.1.5 Keyhole Keyhole toolpaths (see: subsection 3.8.1.1.3 are intended for use with tooling which projects beyond the narrower shaft and so will cut usefully underneath the visible surface. Also described as "undercut" tooling, but see below.





Keyhole Router Bits

#	A	B	C	D
374	3/8″	1/8″	1/4″	3/8″
375	9.525mm	3.175mm	8mm	9.525mm
376	1/2″	3/16″	1/4″	1/2″
378	12.7mm	4.7625mm	8mm	12.7mm

```
879 gcpy         if self.endmilltype == "keyhole":
880 gcpy             kh = cylinder(r1=(self.diameter / 2), r2=(self.diameter
                         / 2), h=self.flute, center=False)
881 gcpy             sh = (cylinder(r1=(self.radius / 2), r2=(self.radius /
                        2), h=self.flute*2, center=False))
882 gcpy             tslist.append(hull(kh.translate([bx, by, bz]), kh.
                        translate([ex, ey, ez])))
883 gcpy             tslist.append(hull(sh.translate([bx, by, bz]), sh.
                        translate([ex, ey, ez])))
884 gcpy             return tslist
```

3.5.3.1.6 Tapered ball nose The tapered ball nose tool is modeled as a sphere at the tip and a pair of cylinders, where one (a cone) describes the taper, while the other represents the shaft.

```
886 gcpy         if self.endmilltype == "tapered␣ball":
887 gcpy             b = sphere(r=(self.tip / 2))
888 gcpy             s = cylinder(r1=(self.tip / 2), r2=(self.diameter / 2),
                         h=self.flute, center=False)
889 gcpy             bshape = union(b, s)
890 gcpy             tslist.append(hull(bshape.translate([bx, by, bz]),
                        bshape.translate([ex, ey, ez])))
891 gcpy             return tslist
```

dovetail **3.5.3.1.7 Dovetails** The dovetail is modeled as a cylinder with the differing bottom and top diameters determining the angle (though dt_angle is still required as a parameter)

```
893 gcpy         if self.endmilltype == "dovetail":
894 gcpy             dt = cylinder(r1=(self.diameter / 2), r2=(self.diameter
                         / 2) - self.flute * Tan(self.angle), h= self.flute,
                         center=False)
895 gcpy             tslist.append(hull(dt.translate([bx, by, bz]), dt.
                        translate([ex, ey, ez])))
896 gcpy             return tslist
897 gcpy         if self.endmilltype == "other":
898 gcpy             tslist = []
899 gcpy #    def dovetail(self, dt_bottomdiameter, dt_topdiameter,
        dt_height, dt_angle):
900 gcpy #        return cylinder(r1=(dt_bottomdiameter / 2), r2=(
        dt_topdiameter / 2), h= dt_height, center=False)
```

3.5.3.2 Concave toolshapes While normal tooling may be represented with a one (or more) hull operation(s) betwixt two 3D toolshapes (or six in the instance of keyhole tools), concave tooling such as roundover/radius tooling require multiple sections or even slices of the tool shape to be modeled separately which are then hulled together. Something of this can be seen in the manual work-around for previewing them: https://community.carbide3d.com/t/using-unsupported-tooling-in-carbide-create-roundover-cove-radius-bits/43723.

Because it is necessary to divide the tooling into vertical slices and call the hull operation for each slice the tool definitions have to be called separately in the cut... modules, or integrated at the lowest level.

3.5.3.2.1 Roundover tooling It is not possible to represent all tools using tool changes as coded above which require using a hull operation between 3D representations of the tools at the beginning and end points. Tooling which cannot be so represented will be implemented separately
roundover below, see paragraph 3.5.3.2 — roundover tooling will need to generate a list of slices of the tool shape hulled together.

```
902 gcpy         if self.endmilltype == "roundover":
903 gcpy             shaft = cylinder(self.steps, self.tip/2, self.tip/2)
904 gcpy             toolpath = hull(shaft.translate([bx, by, bz]), shaft.
                        translate([ex, ey, ez]))
905 gcpy             shaft = cylinder(self.flute, self.diameter/2 + self.tip
                        /2, self.diameter/2 + self.tip/2)
906 gcpy             toolpath = toolpath.union(hull(shaft.translate([bx, by,
                         bz + self.radius]), shaft.translate([ex, ey, ez +
                        self.radius])))
907 gcpy             tslist = [toolpath]
908 gcpy             slice = cylinder(0.0001, 0.0001, 0.0001)
909 gcpy             slices = slice
910 gcpy             for i in range(1, 90 - self.steps, self.steps):
911 gcpy                 dx = self.radius*Cos(i)
912 gcpy                 dxx = self.radius*Cos(i + self.steps)
913 gcpy                 dzz = self.radius*Sin(i)
914 gcpy                 dz = self.radius*Sin(i + self.steps)
915 gcpy                 dh = dz - dzz
916 gcpy                 slice = cylinder(r1 = self.tip/2+self.radius-dx, r2
                             = self.tip/2+self.radius-dxx, h = dh)
917 gcpy                 slices = slices.union(hull(slice.translate([bx, by,
                             bz+dz]), slice.translate([ex, ey, ez+dz])))
918 gcpy                 tslist.append(slices)
919 gcpy             return tslist
```

Note that this routine does *not* alter the machine position variables since it may be called multiple times for a given toolpath, *e.g.,* for arcs. This command will then be called in the definitions for

rapid and cutline which only differ in which variable the 3D model list is unioned with.

shaftmovement A similar routine will be used to handle the shaftmovement.

shaftmovement **3.5.3.3 shaftmovement** The shaftmovement command uses variables defined as part of the tool definition to determine the Z-axis position of the cylinder used to represent the shaft and its diameter and height:

```
921 gcpy      def shaftmovement(self, bx, by, bz, ex, ey, ez):
922 gcpy          tslist = []
923 gcpy          ts = cylinder(r1=(self.shaftdiameter / 2), r2=(self.
                      shaftdiameter / 2), h=self.shaftlength, center = False)
924 gcpy          ts = ts.translate([0, 0, self.shaftheight])
925 gcpy          tslist.append(hull(ts.translate([bx, by, bz]), ts.translate
                      ([ex, ey, ez])))
926 gcpy          return tslist
```

```
60 gcpscad module shaftmovement(bx, by, bz, ex, ey, ez){
61 gcpscad     gcp.shaftmovement(bx, by, bz, ex, ey, ez);
62 gcpscad }
```

3.5.3.4 tool outlines Defining the tools as outlines which may be scaled to different sizes and rotate_extruded requires a series of modules which must define:

- self.tooloutline — the entire outline of the tool used for rotate_extrude when cutting an arc (or a line if linear_extrude is used)
- self.toolprofile — the profile of one half of the tool suited to creating a 3D model using rotate_extrude
- self.shaftoutline
- self.shaftprofile
- self.currenttoolshape
- self.currenttoolshaft

Note that when defining tooling it is expedient to use a mix of the 2D and 3D systems.

The various self.<toolparameters> are defined in toolchange and may be used at need.

An expedient option would seem to be slicing the 3D model and hulling slices from the begin/end positions, but that may result in distortions for certain tool geometries (*e.g.,* keyhole tooling).

There are several possible options for handling outlines and models — a hybrid approach governed by if branches will allow optimization of the resultant CSG commands.

- simple shape and straight move — 3D models of the tool at the begin and end points of the move are hulled
- complex shape and straight move — 3D models of the tool at the begin and end points of the move are connected by a linear_extrude
- any shape and arc move — 3D models of the tool at the begin and end points of the move are connected by a rotate_extrude

Similarly for the tool profiles and outlines and 3D shapes:

- polygon — defining the shape in terms of point positions (note the PythonSCAD has an option for rounding which may be used for some shapes)
- 2D — defining the shape using rectangles or polygons and circles and Boolean operations
- SVG — drawing up the outlines and profiles in a vector drawing tool so that they may be imported as SVG files allows any shape to be imported. Filenames would be mapped to the tool numbering scheme.

3.5.3.4.1 defineshaft A separate command for defining the shaft is expedient, and allows handling the case of the cutting diameter and the shaft diameter being different, and by including both diameters as arguments, allows the transition, if not abrupt, to be modeled. The parameters:

- toolingdiameter
- shaftdiameter
- flute

- transition
- shaft

are obvious except for `shaft` — rather than the O.A.L., this is the expected length of the tool as measured from the specified `flute` and `transition` lengths to the bottom of the collet. In the absence of a specified length, the flute length (assuming no transition) should be a workable approximation.

Frequently, tools will have different diameters for cutting end and shaft — when the former is smaller, the angle typically seems to be 60 degrees — since this should *not* be used for modeling, the expedient solution is to use an easily drawn angle which is obtuse enough to be obvious, so 45 degrees will be used.

```
928 gcpy      def defineshaft(self, toolingdiameter, shaftdiameter, flute,
                  transition, shaft):
929 gcpy          if shaftdiameter == 0:
930 gcpy              self.shaftoutline = polygon(points=[[0, flute], [
                        diameter, flute], [diameter, shaft],[0, shaft]])
931 gcpy              self.shaftprofile = polygon(points=[[0, flute], [
                        diameter/2 ,flute], [diameter/2, shaft], [0, shaft
                        ]])
932 gcpy              sh = cylinder(h = shaft, r = diameter/2)
933 gcpy              self.currenttoolshaft = sh.translate([0,0,flute])
934 gcpy          if shaftdiameter > 0:
935 gcpy              self.shaftoutline = polygon(points=[
936 gcpy                  [shaftdiameter / 2 - toolingdiameter / 2, flute],
937 gcpy                  [0, flute + transition],
938 gcpy                  [0, flute + transition + shaft],
939 gcpy                  [shaftdiameter, flute + transition + shaft],
940 gcpy                  [shaftdiameter, flute + transition],
941 gcpy                  [shaftdiameter / 2 + toolingdiameter / 2, flute],
942 gcpy                  ] )
943 gcpy              self.shaftprofile = polygon( points= [
944 gcpy                  [0, flute],
945 gcpy                  [0, flute + transition + shaft],
946 gcpy                  [shaftdiameter/2, flute + transition + shaft],
947 gcpy                  [shaftdiameter/2, flute + transition],
948 gcpy                  [toolingdiameter/2, flute]
949 gcpy                  ] )
950 gcpy              self.currenttoolshaft = rotate_extrude(self.
                        shaftprofile)
```

```
64 gcpscad module defineshaft(toolingdiameter, shaftdiameter, flute,
               transition, shaft){
65 gcpscad     gcp.defineshaft(toolingdiameter, shaftdiameter, flute,
                   transition, shaft);
66 gcpscad }
```

3.5.3.4.2 Square (including O-flute)

The simplest sort of tooling, which is easily defined using a polygon and cylinder.

```
952 gcpy      def definesquaretool(self, diameter, flute, shaft, offset = 0):
953 gcpy          self.tooloutline = polygon( points=[[0 + offset,0],[
                    diameter + offset,0],[diameter + offset,flute],[0 +
                    offset,flute]] )
954 gcpy          self.toolprofile = polygon( points=[[0,0],[diameter/2,0],[
                    diameter/2,flute],[0,flute]] )
955 gcpy          self.currenttoolshape = cylinder(h = flute, r = diameter/2)
956 gcpy          sh = cylinder(h = flute, r = diameter/2)
```

3.5.3.4.3 Ball-nose

Defined using 2D and 3D primitives which are unioned together, this allows the shape of the tool to be influenced by the variables `fa`/`fs`/`fn`.

```
958 gcpy      def defineballnosetool(self, diameter, flute, shaft, offset =
                  0):
959 gcpy          s = square([diameter,flute - diameter/2])
960 gcpy          sh = s.translate([0 + offset, diameter/2])
961 gcpy          c = circle(d=diameter)
962 gcpy          b = c.translate([diameter/2 + offset, diameter/2])
963 gcpy          self.tooloutline = union(sh, b)
964 gcpy #
965 gcpy          s = square([diameter/2,flute - diameter/2])
```

```
966 gcpy          sh = s.translate([0, diameter/2])
967 gcpy          c = circle(d=diameter)
968 gcpy          b = c.translate([0, diameter/2])
969 gcpy          bn = union(sh, b)
970 gcpy #         bns = bn.translate([0, diameter/2])
971 gcpy          thein = square([diameter/2,flute])
972 gcpy #         theins = thein.translate([diameter/2, 0])
973 gcpy          self.toolprofile = intersection(thein, bn)
974 gcpy #
975 gcpy          self.shaftprofile = polygon( points=[[0,flute],[diameter/2,
                      flute],[diameter/2,shaft],[0,shaft]] )
976 gcpy #
977 gcpy #         b = self.toolprofile
978 gcpy #         bn = b.translate([-diameter/2, 0])
979 gcpy          self.currenttoolshape = rotate_extrude(self.toolprofile)
980 gcpy #
981 gcpy          self.currenttoolshaft = sh.translate([0,0,flute])
```

3.5.3.4.4 V tool outline V shaped tooling often has the V cutting flutes attached to a cylindrical shaft.

```
983 gcpy      def defineVtool(self, diameter, flute, shaft, shaftdiameter =
                  0):
984 gcpy          self.tooloutline = polygon([[diameter/2, 0], [diameter,
                      flute], [0, flute]])
985 gcpy #
986 gcpy
987 gcpy          self.toolprofile = polygon([[0, 0], [diameter/2, flute],
                      [0, flute]])
988 gcpy
989 gcpy #
990 gcpy          if shaftdiameter == 0:
991 gcpy              shaftdiameter = diameter
992 gcpy          self.shaftprofile = polygon([[0, flute], [shaftdiameter/2,
                      flute], [shaftdiameter/2, flute + shaft], [0, flute +
                      shaft]])
993 gcpy
994 gcpy #
995 gcpy          self.currenttoolshape = rotate_extrude(self.toolprofile)
996 gcpy #
997 gcpy          self.currenttoolshaft = rotate_extrude(self.shaftprofile)
```

3.5.3.4.5 Keyhole outline Keyhole outlines will require two cutting surfaces, since it is usual for the shaft to have cutting flutes for clearing the narrow region as part of their functionality.

```
999 gcpy      def defineKeyholetool(self, diameter, flute, narrowdiameter,
                  narrowflute, shaftdiameter, shaftlength):
1000 gcpy
1001 gcpy         self.tooloutline = polygon([[0, 0], [diameter, 0], [
                      diameter, flute], [diameter/2 + narrowdiameter/2, flute
                      ], [diameter/2 + narrowdiameter/2, flute + narrowflute],
                       [diameter/2 - narrowdiameter/2, flute + narrowflute], [
                      diameter/2 - narrowdiameter/2, flute], [0, flute]])
1002 gcpy #
1003 gcpy
1004 gcpy         self.toolprofile = polygon([[0, 0], [diameter/2, 0], [
                      diameter/2, flute], [narrowdiameter/2, flute], [
                      narrowdiameter/2, flute + narrowflute], [0, flute +
                      narrowflute]])
1005 gcpy #
1006 gcpy         self.shaftprofile = polygon([[0, flute + narrowflute], [
                      narrowdiameter/2, flute + narrowflute], [shaftdiameter
                      /2, flute + narrowflute + shaftlength], [0, flute +
                      narrowflute + shaftlength]])
1007 gcpy
1008 gcpy #
1009 gcpy         self.currenttoolshape = rotate_extrude(self.toolprofile)
1010 gcpy #
1011 gcpy         self.currenttoolshaft = rotate_extrude(self.shaftprofile)
```

3.5.3.4.6 Bowl outline Bowl tooling is done using `polygon()` with the third value added so as to cause the rounding of the radius.

```
1013 gcpy      def definebowltool(self, diameter, flute, radius, shaftdiameter
                   , shaftlength):
1014 gcpy #         self.tooloutline =
1015 gcpy #
1016 gcpy          self.toolprofile = polygon([[0,0], [diameter/2, 0, radius],
                        [diameter/2, radius], [diameter/2, flute], [0, flute]])
1017 gcpy #
1018 gcpy          self.shaftprofile = polygon([[0,flute], [shaftdiameter/2,
                       flute], [shaftdiameter/2, flute + shaftlength], [0,
                       flute + shaftlength]])
1019 gcpy #
1020 gcpy          self.currenttoolshape = rotate_extrude(self.toolprofile)
1021 gcpy #
1022 gcpy          self.currenttoolshaft = rotate_extrude(self.shaftprofile)
```

3.5.3.4.7 Tapered ball nose Creating outlines for Tapered ball nose tooling will require that the arc and tangent for the angle and rounding be calculated out if programmed, or instead, they may be drawn.

3.5.3.4.8 Roundover (cove tooling) The `polygon()` command does not afford an option for coves, so it will be necessary to over-draw the geometry, then remove the cove if programming, or, to simply draw the outline.

```
1024 gcpy      def defineRoundovertool(self, diameter, tipdiameter, flute,
                   radius, shaftdiameter, shaftlength):
1025 gcpy #             self.tip = 0.508
1026 gcpy #             self.diameter = 6.35 - self.tip
1027 gcpy #             self.flute = 8 - self.tip
1028 gcpy #             self.radius = 3.175 - self.tip/2
1029 gcpy #             self.shaftdiameter = 6.35
1030 gcpy #             self.shaftheight = 8
1031 gcpy #             self.shaftlength = 10.0
1032 gcpy #          print(diameter)
1033 gcpy #          print(tipdiameter)
1034 gcpy #          print(flute)
1035 gcpy #          print(radius)
1036 gcpy #          print(shaftdiameter)
1037 gcpy #          print(shaftlength)
1038 gcpy #          self.tooloutline =
1039 gcpy #
1040 gcpy          self.toolprofile = polygon([[0,0], [tipdiameter/2, 0], [
                       diameter/2, flute], [0, flute]])
1041 gcpy #
1042 gcpy          self.shaftprofile = polygon([[0,flute], [shaftdiameter/2,
                       flute], [shaftdiameter/2, flute + shaftlength], [0,
                       flute + shaftlength]])
1043 gcpy #
1044 gcpy          self.currenttoolshape = rotate_extrude(self.toolprofile)
1045 gcpy #
1046 gcpy          self.currenttoolshaft = rotate_extrude(self.shaftprofile)
```

rapid **3.5.3.5 rapid and cut (lines)** A matching pair of commands is made for these, and `rapid` is used as the basis for a series of commands which match typical usages of `G0`.

Note the addition of a Laser mode which simulates the tool having been turned off before making a rapid movement — likely further changes will be required.

```
1048 gcpy      def rapid(self, ex, ey, ez, laser = 0):
1049 gcpy #          print(self.rapidcolor)
1050 gcpy          if self.generateprint == True:
1051 gcpy              laser = 1
1052 gcpy          if laser == 0:
1053 gcpy              tm = self.toolmovement(self.xpos(), self.ypos(), self.
                           zpos(), ex, ey, ez)
1054 gcpy              tm = color(tm, self.shaftcolor)
1055 gcpy              ts = self.shaftmovement(self.xpos(), self.ypos(), self.
                           zpos(), ex, ey, ez)
1056 gcpy              ts = color(ts, self.rapidcolor)
1057 gcpy              self.toolpaths.extend([tm, ts])
1058 gcpy          if self.generateprint == True:
1059 gcpy              self.steps.append(self.fgc.Extruder(on=False))
1060 gcpy              self.steps.append(self.fgc.Point(x=ex,y=ey,z=ez))
1061 gcpy              self.steps.append(self.fgc.Extruder(on=True))
1062 gcpy          self.setxpos(ex)
```

```
1063 gcpy          self.setypos(ey)
1064 gcpy          self.setzpos(ez)
1065 gcpy
1066 gcpy      def cutline(self, ex, ey, ez):
1067 gcpy #          print(self.cutcolor)
1068 gcpy #          print(ex, ey, ez)
1069 gcpy          tm = self.toolmovement(self.xpos(), self.ypos(), self.zpos
                       (), ex, ey, ez)
1070 gcpy          if self.cutcolor == "Dark␣Gray":
1071 gcpy              self.cutcolor = "Gray"
1072 gcpy #          print("cutline color =", self.cutcolor)
1073 gcpy          tm = color(tm, self.cutcolor)
1074 gcpy          ts = self.shaftmovement(self.xpos(), self.ypos(), self.zpos
                       (), ex, ey, ez)
1075 gcpy          ts = color(ts, self.rapidcolor)
1076 gcpy          self.setxpos(ex)
1077 gcpy          self.setypos(ey)
1078 gcpy          self.setzpos(ez)
1079 gcpy          if self.generatecut == True:
1080 gcpy              self.toolpaths.extend([tm, ts])
```

It is then possible to add specific `rapid...` commands to match typical usages of G-code. The first command needs to be a move to/from the safe Z height. In G-code this would be:

```
(Move to safe Z to avoid workholding)
G53G0Z-5.000
```

but in the 3D model, since we do not know how tall the Z-axis is, we simply move to safe height and use that as a starting point:

```
1082 gcpy      def movetosafeZ(self):
1083 gcpy          rapid = self.rapid(self.xpos(), self.ypos(), self.
                       retractheight)
1084 gcpy #          if self.generatepaths == True:
1085 gcpy #              rapid = self.rapid(self.xpos(), self.ypos(), self.
       retractheight)
1086 gcpy #              self.rapids = self.rapids.union(rapid)
1087 gcpy #          else:
1088 gcpy #  if (generategcode == true) {
1089 gcpy #  //      writecomment("PREPOSITION FOR RAPID PLUNGE");Z25.650
1090 gcpy #  //G1Z24.663F381.0, "F", str(plunge)
1091 gcpy #          if self.generatepaths == False:
1092 gcpy #              return rapid
1093 gcpy #          else:
1094 gcpy #              return cube([0.001, 0.001, 0.001])
1095 gcpy          return rapid
1096 gcpy
1097 gcpy      def rapidXYZ(self, ex, ey, ez):
1098 gcpy          rapid = self.rapid(ex, ey, ez)
1099 gcpy #          if self.generatepaths == False:
1100 gcpy          return rapid
1101 gcpy
1102 gcpy      def rapidXY(self, ex, ey):
1103 gcpy          rapid = self.rapid(ex, ey, self.zpos())
1104 gcpy #          if self.generatepaths == True:
1105 gcpy #              self.rapids = self.rapids.union(rapid)
1106 gcpy #          else:
1107 gcpy #          if self.generatepaths == False:
1108 gcpy          return rapid
1109 gcpy
1110 gcpy      def rapidXZ(self, ex, ez):
1111 gcpy          rapid = self.rapid(ex, self.ypos(), ez)
1112 gcpy #          if self.generatepaths == False:
1113 gcpy          return rapid
1114 gcpy
1115 gcpy      def rapidYZ(self, ey, ez):
1116 gcpy          rapid = self.rapid(self.xpos(), ey, ez)
1117 gcpy #          if self.generatepaths == False:
1118 gcpy          return rapid
1119 gcpy
1120 gcpy      def rapidX(self, ex):
1121 gcpy          rapid = self.rapid(ex, self.ypos(), self.zpos())
1122 gcpy #          if self.generatepaths == False:
1123 gcpy          return rapid
1124 gcpy
1125 gcpy      def rapidY(self, ey):
1126 gcpy          rapid = self.rapid(self.xpos(), ey, self.zpos())
1127 gcpy #          if self.generatepaths == False:
```

```
1128 gcpy           return rapid
1129 gcpy
1130 gcpy       def rapidZ(self, ez):
1131 gcpy           rapid = [self.rapid(self.xpos(), self.ypos(), ez)]
1132 gcpy #          if self.generatepaths == True:
1133 gcpy #              self.rapids = self.rapids.union(rapid)
1134 gcpy #          else:
1135 gcpy #          if self.generatepaths == False:
1136 gcpy           return rapid
```

Note that rather than re-create the matching OpenSCAD commands as descriptors, due to the issue of redirection and return values and the possibility for errors it is more expedient to simply re-create the matching command (at least for the rapids):

```
68 gcpscad module movetosafeZ(){
69 gcpscad     gcp.rapid(gcp.xpos(), gcp.ypos(), retractheight);
70 gcpscad }
71 gcpscad
72 gcpscad module rapid(ex, ey, ez) {
73 gcpscad     gcp.rapid(ex, ey, ez);
74 gcpscad }
75 gcpscad
76 gcpscad module rapidXY(ex, ey) {
77 gcpscad     gcp.rapid(ex, ey, gcp.zpos());
78 gcpscad }
79 gcpscad
80 gcpscad module rapidXZ(ex, ez) {
81 gcpscad     gcp.rapid(ex, gcp.zpos(), ez);
82 gcpscad }
83 gcpscad
84 gcpscad module rapidZ(ez) {
85 gcpscad     gcp.rapid(gcp.xpos(), gcp.ypos(), ez);
86 gcpscad }
```

Similarly, there is a series of `cutline...` commands as predicted above.

`cut...` `cutline`

The Python commands `cut...` add the `currenttool` to the `toolpath` `hull`ed together at the current position and the end position of the move. For `cutline`, this is a straight-forward connection of the current (beginning) and ending coordinates:

```
1138 gcpy       def moveatfeedrate(self, ex, ey, ez, f):
1139 gcpy           self.writegc("G01␣X", str(ex), "␣Y", str(ey), "␣Z", str(ez)
                        , "␣F", str(f))
1140 gcpy           self.feedrate = f
1141 gcpy           return self.cutline(ex, ey, ez)
1142 gcpy
1143 gcpy       def cutlinedxf(self, ex, ey, ez):
1144 gcpy           self.dxfline(self.xpos(), self.ypos(), ex, ey)
1145 gcpy           self.cutline(ex, ey, ez)
1146 gcpy
1147 gcpy       def cutlinedxfgc(self, ex, ey, ez):
1148 gcpy           self.dxfline(self.xpos(), self.ypos(), ex, ey)
1149 gcpy           self.writegc("G01␣X", str(ex), "␣Y", str(ey), "␣Z", str(ez)
                       )
1150 gcpy           self.cutline(ex, ey, ez)
1151 gcpy
1152 gcpy       def cutvertexdxf(self, ex, ey, ez):
1153 gcpy           self.addvertex(ex, ey, ez)
1154 gcpy           self.cutline(ex, ey, ez)
1155 gcpy
1156 gcpy       def cutpolylinedxf(self, ex, ey, ez):
1157 gcpy           self.addopline(ex, ey, ez)
1158 gcpy           self.cutline(ex, ey, ez)
1159 gcpy
1160 gcpy       def cutvertexdxfgc(self, ex, ey, ez):
1161 gcpy           self.addvertex(ex, ey, ez)
1162 gcpy           self.writegc("G01␣X", str(ex), "␣Y", str(ey), "␣Z", str(ez)
                       )
1163 gcpy           self.cutline(ex, ey, ez)
1164 gcpy
1165 gcpy       def cutlineXYZwithfeed(self, ex, ey, ez, feed):
1166 gcpy           return self.cutline(ex, ey, ez)
1167 gcpy
1168 gcpy       def cutlineXYZ(self, ex, ey, ez):
1169 gcpy           return self.cutline(ex, ey, ez)
1170 gcpy
1171 gcpy       def cutlineXYwithfeed(self, ex, ey, feed):
1172 gcpy           return self.cutline(ex, ey, self.zpos())
```

```
1173 gcpy
1174 gcpy      def cutlineXY(self, ex, ey):
1175 gcpy          return self.cutline(ex, ey, self.zpos())
1176 gcpy
1177 gcpy      def cutlineXZwithfeed(self, ex, ez, feed):
1178 gcpy          return self.cutline(ex, self.ypos(), ez)
1179 gcpy
1180 gcpy      def cutlineXZ(self, ex, ez):
1181 gcpy          return self.cutline(ex, self.ypos(), ez)
1182 gcpy
1183 gcpy      def cutlineXwithfeed(self, ex, feed):
1184 gcpy          return self.cutline(ex, self.ypos(), self.zpos())
1185 gcpy
1186 gcpy      def cutlineX(self, ex):
1187 gcpy          return self.cutline(ex, self.ypos(), self.zpos())
1188 gcpy
1189 gcpy      def cutlineYZ(self, ey, ez):
1190 gcpy          return self.cutline(self.xpos(), ey, ez)
1191 gcpy
1192 gcpy      def cutlineYwithfeed(self, ey, feed):
1193 gcpy          return self.cutline(self.xpos(), ey, self.zpos())
1194 gcpy
1195 gcpy      def cutlineY(self, ey):
1196 gcpy          return self.cutline(self.xpos(), ey, self.zpos())
1197 gcpy
1198 gcpy      def cutlineZgcfeed(self, ez, feed):
1199 gcpy          self.writegc("G01␣Z", str(ez), "F", str(feed))
1200 gcpy          return self.cutline(self.xpos(), self.ypos(), ez)
1201 gcpy
1202 gcpy      def cutlineZwithfeed(self, ez, feed):
1203 gcpy          return self.cutline(self.xpos(), self.ypos(), ez)
1204 gcpy
1205 gcpy      def cutlineZ(self, ez):
1206 gcpy          return self.cutline(self.xpos(), self.ypos(), ez)
```

The matching OpenSCAD command is a descriptor:

```
88 gcpscad module cutline(ex, ey, ez){
89 gcpscad     gcp.cutline(ex, ey, ez);
90 gcpscad }
91 gcpscad
92 gcpscad module cutlinedxfgc(ex, ey, ez){
93 gcpscad     gcp.cutlinedxfgc(ex, ey, ez);
94 gcpscad }
95 gcpscad
96 gcpscad module cutlineZgcfeed(ez, feed){
97 gcpscad     gcp.cutlineZgcfeed(ez, feed);
98 gcpscad }
```

3.5.3.6 Arcs A further consideration here is that G-code and DXF support arcs in addition to the lines already implemented. Implementing arcs wants at least the following options for quadrant and direction:

- cutarcCW — cut a partial arc described in a clock-wise direction
- cutarcCC — counter-clock-wise
- cutarcNWCW — cut the upper-left quadrant of a circle moving clockwise
- cutarcNWCC — upper-left quadrant counter-clockwise
- cutarcNECW
- cutarcNECC
- cutarcSECW
- cutarcSECC
- cutarcNECW
- cutarcNECC
- cutcircleCC — while it won't matter for generating a DXF, when G-code is implemented direction of cut will be a consideration for that
- cutcircleCW
- cutcircleCCdxf

- cutcircleCWdxf

It will be necessary to have two separate representations of arcs — the G-code and DXF may be easily and directly supported with a single command, but representing the matching tool movement in OpenSCAD may be done in two different fashions. Originally, a series of short line movements which approximate the arc cutting in each direction and at changing Z-heights so as to allow for threading and similar operations was implemented, but instead representing the tool as an outline and using `rotate_extrude` to model the movement of the tool's outline representation through the arc movement.

- G-code — G2 (clockwise) and G3 (counter-clockwise) arcs may be specified, and since the endpoint is the positional requirement, it is most likely best to use the offset to the center (I and J), rather than the radius parameter (K) `G2/3 ...`
- DXF — `dxfarc(xcenter, ycenter, radius, anglebegin, endangle, tn)`
- approximation of arc using lines (OpenSCAD) in both clock-wise and counter-clock-wise directions

Cutting the quadrant arcs greatly simplifies the calculation and interface for the modules. A full set of 8 will be necessary, then circles will have a pair of modules (one for each cut direction) made for them.

Parameters which will need to be passed in are:

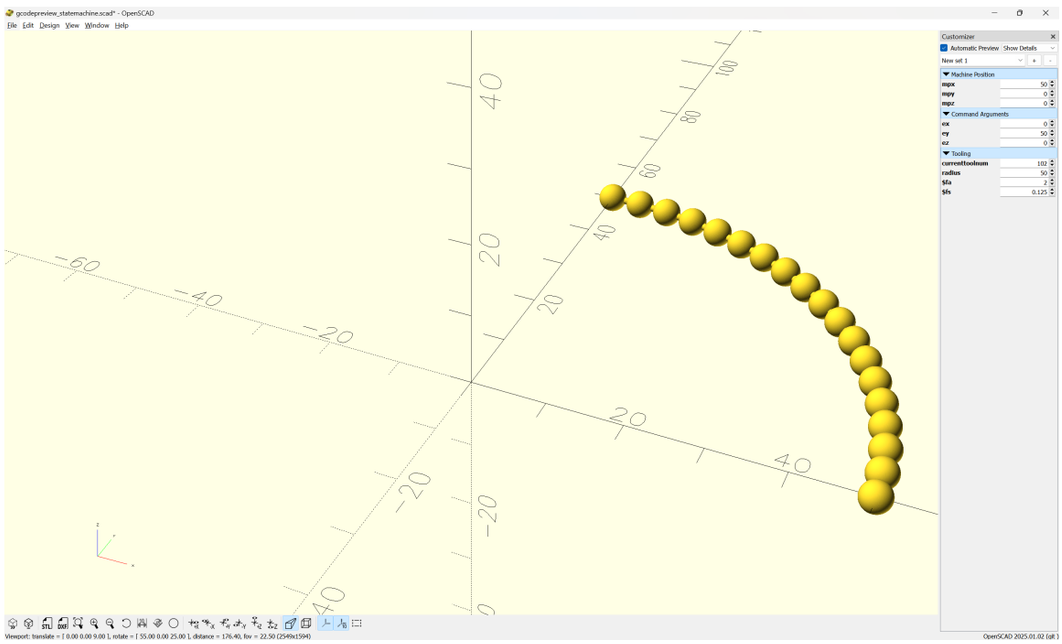
- `ex` — note that the matching origins (`bx`, `by`, `bz`) as well as the (current) toolnumber are accessed using the appropriate commands for machine position
- `ey`
- `ez` — allowing a different Z position will make possible threading and similar helical toolpaths
- `xcenter` — the center position will be specified as an absolute position which will require calculating the offset when it is used for G-code's `IJ`, for which `xctr/yctr` are suggested
- `ycenter`
- `radius` — while this could be calculated, passing it in as a parameter is both convenient and (potentially) could be used as a check on the other parameters
- `tpzreldim` — the relative depth (or increase in height) of the current cutting motion

There are two possibilities for arc movement:

- stepping through the arc and approximating with straight line movements
- using `rotate_extrude` to move an outline of the tool through the specified arc — this has the added complexity of being limited to the range of the arc, requiring that the round profile of the tool be instantiated in 3D at each end

`cutarcCW` `cutarcCC` Stepping through the arc manually is done by iterating through a loop: `cutarcCW` (clockwise) or `cutarcCC` (counterclockwise) to handle the drawing and processing of the `cutline()` toolpaths as short line segments which additionally affords a single point of control for adding additional features such as allowing the depth to vary as one cuts along an arc (the line version is used rather than shape so as to capture the changing machine positions with each step through the loop). Note that the definition matches the `DXF` definition of defining the center position with a matching radius, but it will be necessary to move the tool to the actual origin, and to calculate the end position when writing out a `G2/G3` arc.

This brings to the fore the fact that at its heart, this program is simply graphing math in 3D using tools (as presaged by the book series *Make:Geometry/Trigonometry/Calculus*). This is clear in a depiction of the algorithm for the `cutarcCC/CW` commands, where the `x` value is the cos of the `radius` and the `y` value the sin:



The code for which makes this obvious:

```
/* [Machine Position] */
mpx = 0;
/* [Machine Position] */
mpy = 0;
/* [Machine Position] */
mpz = 0;

/* [Command Arguments] */
ex = 50;
/* [Command Arguments] */
ey = 25;
/* [Command Arguments] */
ez = -10;

/* [Tooling] */
currenttoolnum = 102;

machine_extents();

radius = 50;
$fa = 2;
$fs = 0.125;

plot_arc(radius, 0, 0, 0, radius, 0, 0, 0, radius, 0, 90, 5);

module plot_arc(bx, by, bz, ex, ey, ez, acx, acy, radius, barc, earc, inc){
for (i = [barc : inc : earc-inc]) {
  union(){
    hull()
    {
      translate([acx + cos(i)*radius,
                 acy + sin(i)*radius,
                 0]){
        sphere(r=0.5);
      }
      translate([acx + cos(i+inc)*radius,
                 acy + sin(i+inc)*radius,
                 0]){
        sphere(r=0.5);
      }
    }
      translate([acx + cos(i)*radius,
                 acy + sin(i)*radius,
                 0]){
      sphere(r=2);
    }
      translate([acx + cos(i+inc)*radius,
                 acy + sin(i+inc)*radius,
                 0]){
      sphere(r=2);
    }
  }
```

```
}
}

module machine_extents(){
translate([-200, -200, 20]){
  cube([0.001, 0.001, 0.001], center=true);
}
translate([200, 200, 20]){
  cube([0.001, 0.001, 0.001], center=true);
}
}
```

Note that it is necessary to move to the beginning cutting position before calling, and that it is necessary to pass in the relative change in Z position/depth. (Previous iterations calculated the increment of change outside the loop, but it is more workable to do so inside.)

```
1208 gcpy      def cutarcCC(self, barc, earc, xcenter, ycenter, radius,
                   tpzreldim, stepsizearc=1):
1209 gcpy          tpzinc = tpzreldim / (earc - barc)
1210 gcpy          i = barc
1211 gcpy          while i < earc:
1212 gcpy              self.cutline(xcenter + radius * Cos(i), ycenter +
                         radius * Sin(i), self.zpos()+tpzinc)
1213 gcpy              i += stepsizearc
1214 gcpy #          self.setxpos(xcenter + radius * Cos(earc))
1215 gcpy #          self.setypos(ycenter + radius * Sin(earc))
1216 gcpy
1217 gcpy      def cutarcCW(self, barc, earc, xcenter, ycenter, radius,
                   tpzreldim, stepsizearc=1):
1218 gcpy #          print(str(self.zpos()))
1219 gcpy #          print(str(ez))
1220 gcpy #          print(str(barc - earc))
1221 gcpy #          tpzinc = ez - self.zpos() / (barc - earc)
1222 gcpy #          print(str(tzinc))
1223 gcpy #          global toolpath
1224 gcpy #          print("Entering n toolpath")
1225 gcpy          tpzinc = tpzreldim / (barc - earc)
1226 gcpy #          cts = self.currenttoolshape
1227 gcpy #          toolpath = cts
1228 gcpy #          toolpath = toolpath.translate([self.xpos(), self.ypos(),
          self.zpos()])
1229 gcpy #          toolpath = []
1230 gcpy          i = barc
1231 gcpy          while i > earc:
1232 gcpy              self.cutline(xcenter + radius * Cos(i), ycenter +
                         radius * Sin(i), self.zpos()+tpzinc)
1233 gcpy #              self.setxpos(xcenter + radius * Cos(i))
1234 gcpy #              self.setypos(ycenter + radius * Sin(i))
1235 gcpy #              print(str(self.xpos()), str(self.ypos(), str(self.zpos
          ())))
1236 gcpy #              self.setzpos(self.zpos()+tpzinc)
1237 gcpy              i += abs(stepsizearc) * -1
1238 gcpy #          self.dxfarc(self.currenttoolnumber(), xcenter, ycenter,
          radius, barc, earc)
1239 gcpy #          if self.generatepaths == True:
1240 gcpy #              print("Unioning n toolpath")
1241 gcpy #              self.toolpaths = self.toolpaths.union(toolpath)
1242 gcpy #          else:
1243 gcpy          self.setxpos(xcenter + radius * Cos(earc))
1244 gcpy          self.setypos(ycenter + radius * Sin(earc))
1245 gcpy #          self.toolpaths.extend(toolpath)
1246 gcpy #          if self.generatepaths == False:
1247 gcpy #          return toolpath
1248 gcpy #          else:
1249 gcpy #              return cube([0.01, 0.01, 0.01])
```

Alternately, the command for using `rotate_extrude` is quite straight-forward:

```
1251 gcpy      def extrudearcCC(self, barc, earc, xcenter, ycenter, radius,
                   tpzreldim, stepsizearc=1):
1252 gcpy #          tm = self.toolmovement(self.xpos(), self.ypos(), self.zpos
          (), ex, ey, ez)
1253 gcpy          tm = union(self.toolshape.translate(self.xpos(), self.ypos
                       (), self.zpos()))
1254 gcpy #                     self.toolshape.translate(),
1255 gcpy #                     tooloutline.translate([r-3.175,0,0]).
          rotate_extrude(angle=ang2-ang1).rotz(ang1) + G3_center
```

```
1256 gcpy
1257 gcpy           tm = color(tm, self.cutcolor)
1258 gcpy           ts = self.shaftmovement(self.xpos(), self.ypos(), self.zpos
                        (), ex, ey, ez)
1259 gcpy           ts = color(ts, self.rapidcolor)
1260 gcpy           self.setxpos(ex)
1261 gcpy           self.setypos(ey)
1262 gcpy           self.setzpos(ez)
1263 gcpy           self.toolpaths.extend([tm, ts])
```

Note that it will be necessary to add versions which write out a matching DXF element:

```
1265 gcpy       def cutarcCWdxf(self, barc, earc, xcenter, ycenter, radius,
                    tpzreldim, stepsizearc=1):
1266 gcpy           self.cutarcCW(barc, earc, xcenter, ycenter, radius,
                        tpzreldim, stepsizearc=1)
1267 gcpy           self.dxfarc(self.currenttoolnumber(), xcenter, ycenter,
                        radius, earc, barc)
1268 gcpy #          if self.generatepaths == False:
1269 gcpy #          return toolpath
1270 gcpy #          else:
1271 gcpy #              return cube([0.01, 0.01, 0.01])
1272 gcpy
1273 gcpy       def cutarcCCdxf(self, barc, earc, xcenter, ycenter, radius,
                    tpzreldim, stepsizearc=1):
1274 gcpy           self.cutarcCC(barc, earc, xcenter, ycenter, radius,
                        tpzreldim, stepsizearc=1)
1275 gcpy           self.dxfarc(self.currenttoolnumber(), xcenter, ycenter,
                        radius, barc, earc)
```

Matching OpenSCAD modules are easily made:

```
100 gcpscad module cutarcCC(barc, earc, xcenter, ycenter, radius, tpzreldim){
101 gcpscad     gcp.cutarcCC(barc, earc, xcenter, ycenter, radius, tpzreldim);
102 gcpscad }
103 gcpscad
104 gcpscad module cutarcCW(barc, earc, xcenter, ycenter, radius, tpzreldim){
105 gcpscad     gcp.cutarcCW(barc, earc, xcenter, ycenter, radius, tpzreldim);
106 gcpscad }
```

An alternate interface which matches how G2/G3 arcs are programmed in G-code is a useful option:

```
1277 gcpy       def cutquarterCCNE(self, ex, ey, ez, radius):
1278 gcpy           if self.zpos() == ez:
1279 gcpy               tpzinc = 0
1280 gcpy           else:
1281 gcpy               tpzinc = (ez - self.zpos()) / 90
1282 gcpy #          print("tpzinc ", tpzinc)
1283 gcpy           i = 1
1284 gcpy           while i < 91:
1285 gcpy               self.cutline(ex + radius * Cos(i), ey - radius + radius
                            * Sin(i), self.zpos()+tpzinc)
1286 gcpy               i += 1
1287 gcpy
1288 gcpy       def cutquarterCCNW(self, ex, ey, ez, radius):
1289 gcpy           if self.zpos() == ez:
1290 gcpy               tpzinc = 0
1291 gcpy           else:
1292 gcpy               tpzinc = (ez - self.zpos()) / 90
1293 gcpy #              tpzinc = (self.zpos() + ez) / 90
1294 gcpy           self.debug("tpzinc␣", tpzinc)
1295 gcpy           i = 91
1296 gcpy           while i < 181:
1297 gcpy               self.cutline(ex + radius + radius * Cos(i), ey + radius
                            * Sin(i), self.zpos()+tpzinc)
1298 gcpy               i += 1
1299 gcpy
1300 gcpy       def cutquarterCCSW(self, ex, ey, ez, radius):
1301 gcpy           if self.zpos() == ez:
1302 gcpy               tpzinc = 0
1303 gcpy           else:
1304 gcpy               tpzinc = (ez - self.zpos()) / 90
1305 gcpy #              tpzinc = (self.zpos() + ez) / 90
1306 gcpy #          print("tpzinc ", tpzinc)
1307 gcpy           i = 181
1308 gcpy           while i < 271:
```

```
1309 gcpy             self.cutline(ex + radius * Cos(i), ey + radius + radius
                          * Sin(i), self.zpos()+tpzinc)
1310 gcpy             i += 1
1311 gcpy
1312 gcpy     def cutquarterCCSE(self, ex, ey, ez, radius):
1313 gcpy         if self.zpos() == ez:
1314 gcpy             tpzinc = 0
1315 gcpy         else:
1316 gcpy             tpzinc = (ez - self.zpos()) / 90
1317 gcpy #             tpzinc = (self.zpos() + ez) / 90
1318 gcpy #         print("tpzinc ", tpzinc)
1319 gcpy         i = 271
1320 gcpy         while i < 361:
1321 gcpy             self.cutline(ex - radius + radius * Cos(i), ey + radius
                          * Sin(i), self.zpos()+tpzinc)
1322 gcpy             i += 1
1323 gcpy
1324 gcpy     def cutquarterCCNEdxf(self, ex, ey, ez, radius):
1325 gcpy         self.cutquarterCCNE(ex, ey, ez, radius)
1326 gcpy         self.dxfarc(ex, ey - radius, radius,  0, 90)
1327 gcpy
1328 gcpy     def cutquarterCCNWdxf(self, ex, ey, ez, radius):
1329 gcpy         self.cutquarterCCNW(ex, ey, ez, radius)
1330 gcpy         self.dxfarc(ex + radius, ey, radius, 90, 180)
1331 gcpy
1332 gcpy     def cutquarterCCSWdxf(self, ex, ey, ez, radius):
1333 gcpy         self.cutquarterCCSW(ex, ey, ez, radius)
1334 gcpy         self.dxfarc(ex, ey + radius, radius, 180, 270)
1335 gcpy
1336 gcpy     def cutquarterCCSEdxf(self, ex, ey, ez, radius):
1337 gcpy         self.cutquarterCCSE(ex, ey, ez, radius)
1338 gcpy         self.dxfarc(ex - radius, ey, radius, 270, 360)
```

```
108 gcpscad module cutquarterCCNE(ex, ey, ez, radius){
109 gcpscad      gcp.cutquarterCCNE(ex, ey, ez, radius);
110 gcpscad }
111 gcpscad
112 gcpscad module cutquarterCCNW(ex, ey, ez, radius){
113 gcpscad      gcp.cutquarterCCNW(ex, ey, ez, radius);
114 gcpscad }
115 gcpscad
116 gcpscad module cutquarterCCSW(ex, ey, ez, radius){
117 gcpscad     gcp.cutquarterCCSW(ex, ey, ez, radius);
118 gcpscad }
119 gcpscad
120 gcpscad module cutquarterCCSE(self, ex, ey, ez, radius){
121 gcpscad     gcp.cutquarterCCSE(ex, ey, ez, radius);
122 gcpscad }
123 gcpscad
124 gcpscad module cutquarterCCNEdxf(ex, ey, ez, radius){
125 gcpscad      gcp.cutquarterCCNEdxf(ex, ey, ez, radius);
126 gcpscad }
127 gcpscad
128 gcpscad module cutquarterCCNWdxf(ex, ey, ez, radius){
129 gcpscad      gcp.cutquarterCCNWdxf(ex, ey, ez, radius);
130 gcpscad }
131 gcpscad
132 gcpscad module cutquarterCCSWdxf(ex, ey, ez, radius){
133 gcpscad     gcp.cutquarterCCSWdxf(ex, ey, ez, radius);
134 gcpscad }
135 gcpscad
136 gcpscad module cutquarterCCSEdxf(self, ex, ey, ez, radius){
137 gcpscad     gcp.cutquarterCCSEdxf(ex, ey, ez, radius);
138 gcpscad }
```

3.5.4 tooldiameter

It will also be necessary to be able to provide the diameter of the current tool. Arguably, this would be much easier using an object-oriented programming style/dot notation.

One aspect of tool parameters which will need to be supported is shapes which create different profiles based on how deeply the tool is cutting into the surface of the material at a given point. To accommodate this, it will be necessary to either track the thickness of uncut material at any given point, or, to specify the depth of cut as a parameter.

`tool diameter` The public-facing OpenSCAD code, `tool diameter` simply calls the matching OpenSCAD module which wraps the Python code:

```
140 gcpscad function tool_diameter(td_tool, td_depth) = otool_diameter(td_tool,
        td_depth);
```

`tool diameter` the Python code, `tool diameter` returns appropriate values based on the specified tool number and depth:

```
1340 gcpy     def tool_diameter(self, ptd_tool, ptd_depth):
1341 gcpy # Square 122, 112, 102, 201
1342 gcpy         if ptd_tool == 122:
1343 gcpy             return 0.79375
1344 gcpy         if ptd_tool == 112:
1345 gcpy             return 1.5875
1346 gcpy         if ptd_tool == 102:
1347 gcpy             return 3.175
1348 gcpy         if ptd_tool == 201:
1349 gcpy             return 6.35
1350 gcpy # Ball 121, 111, 101, 202
1351 gcpy         if ptd_tool == 122:
1352 gcpy             if ptd_depth > 0.396875:
1353 gcpy                 return 0.79375
1354 gcpy             else:
1355 gcpy                 return ptd_tool
1356 gcpy         if ptd_tool == 112:
1357 gcpy             if ptd_depth > 0.79375:
1358 gcpy                 return 1.5875
1359 gcpy             else:
1360 gcpy                 return ptd_tool
1361 gcpy         if ptd_tool == 101:
1362 gcpy             if ptd_depth > 1.5875:
1363 gcpy                 return 3.175
1364 gcpy             else:
1365 gcpy                 return ptd_tool
1366 gcpy         if ptd_tool == 202:
1367 gcpy             if ptd_depth > 3.175:
1368 gcpy                 return 6.35
1369 gcpy             else:
1370 gcpy                 return ptd_tool
1371 gcpy # V 301, 302, 390
1372 gcpy         if ptd_tool == 301:
1373 gcpy             return ptd_tool
1374 gcpy         if ptd_tool == 302:
1375 gcpy             return ptd_tool
1376 gcpy         if ptd_tool == 390:
1377 gcpy             return ptd_tool
1378 gcpy # Keyhole
1379 gcpy         if ptd_tool == 374:
1380 gcpy             if ptd_depth < 3.175:
1381 gcpy                 return 9.525
1382 gcpy             else:
1383 gcpy                 return 6.35
1384 gcpy         if ptd_tool == 375:
1385 gcpy             if ptd_depth < 3.175:
1386 gcpy                 return 9.525
1387 gcpy             else:
1388 gcpy                 return 8
1389 gcpy         if ptd_tool == 376:
1390 gcpy             if ptd_depth < 4.7625:
1391 gcpy                 return 12.7
1392 gcpy             else:
1393 gcpy                 return 6.35
1394 gcpy         if ptd_tool == 378:
1395 gcpy             if ptd_depth < 4.7625:
1396 gcpy                 return 12.7
1397 gcpy             else:
1398 gcpy                 return 8
1399 gcpy # Dovetail
1400 gcpy         if ptd_tool == 814:
1401 gcpy             if ptd_depth > 12.7:
1402 gcpy                 return 6.35
1403 gcpy             else:
1404 gcpy                 return ptd_tool
1405 gcpy         if ptd_tool == 808079:
1406 gcpy             if ptd_depth > 20.95:
1407 gcpy                 return 6.816
1408 gcpy             else:
1409 gcpy                 return ptd_tool
1410 gcpy # Bowl Bit
```

```
1411 gcpy #https://www.amanatool.com/45982-carbide-tipped-bowl-tray-1-4-
         radius-x-3-4-dia-x-5-8-x-1-4-inch-shank.html
1412 gcpy         if ptd_tool == 45982:
1413 gcpy             if ptd_depth > 6.35:
1414 gcpy                 return 15.875
1415 gcpy             else:
1416 gcpy                 return ptd_tool
1417 gcpy # Tapered Ball Nose
1418 gcpy         if ptd_tool == 204:
1419 gcpy             if ptd_depth > 6.35:
1420 gcpy                 return ptd_tool
1421 gcpy         if ptd_tool == 304:
1422 gcpy             if ptd_depth > 6.35:
1423 gcpy                 return ptd_tool
1424 gcpy             else:
1425 gcpy                 return ptd_tool
```

tool radius Since it is often necessary to utilise the radius of the tool, an additional command, `tool radius` to return this value is worthwhile:

```
1427 gcpy     def tool_radius(self, ptd_tool, ptd_depth):
1428 gcpy         tr = self.tool_diameter(ptd_tool, ptd_depth)/2
1429 gcpy         return tr
```

(Note that where values are not fully calculated values currently the passed in tool number (`ptd tool`)is returned which will need to be replaced with code which calculates the appropriate values.)

3.5.5 Feeds and Speeds

feed plunge speed There are several possibilities for handling feeds and speeds. Currently, base values for `feed`, `plunge`, and `speed` are used, which may then be adjusted using various `<tooldescriptor>_ratio` values, as an acknowledgement of the likelihood of a trim router being used as a spindle, the assumption is that the `speed` will remain unchanged.

The tools which need to be calculated thus are those in addition to the `large_square` tool:

- `small_square_ratio`
- `small_ball_ratio`
- `large_ball_ratio`
- `small_V_ratio`
- `large_V_ratio`
- `KH_ratio`
- `DT_ratio`

3.5.6 3D Printing

Support for 3D printing requires that there be G-code commands for non-mill/router aspects such as:

- fan(s) on/off
- extruder(s)
- Heater(s)
- temperature(s)
- accelerometers
- load cells
- Filament Sensor(s)
- Filament Cutter(s)
- Display Status
  - Message
  - Build Percentage
  - (Clear) Message
- any additional commands such as “Clean Nozzle”

Moreover, it will be necessary for all values to be adjusted for specific firmware, printer and filament type combinations. Probably the best beginning will be to create a simple file using a tested set of settings in a compatible slicer as a template and to adjust based on the values from such a file.

3.5.6.1 fullcontrolgcode An extant tool for this is: https://fullcontrolgcode.com/ which has a Python implementation at: https://github.com/FullControlXYZ/fullcontrol. It affords a compleat system for programming a 3D printer. The implementation https://py2g.com/ as announced at: https://old.reddit.com/r/FullControl/comments/1mjgta3/i_made_an_online_ide_for_fullcontrol_py2gcom/ affords a straight-forward usage from which the following typical example code is pulled:

```
# see https://py2g.com/customize/grid-bins for a bonus interactive UI to use with this sketch

# ================================================================================
# PARAMETERS
# ================================================================================
layer_height = 0.4
line_width   = 1.2
start_x, start_y = 10, 10
grid_unit    = 25
units_x, units_y, units_z = 4, 8, 0.5
outer_radius = 5
tolerance = 0.05

flow_rate = 1.02 # fill in the gaps

bin_type_outer = True # set True to create a bin container

print_speed = 40 # highest speed you'd want to go
max_flow = 8 # in mm3/s
max_print_speed = max_flow / (layer_height*line_width) # highest speed you can go
print_speed = min(print_speed,max_print_speed)

printer_name = 'generic'
printer_settings = {
    'primer':         'travel',
    'print_speed':    print_speed*60,
    'travel_speed':   20*60,
    'nozzle_temp':    210,
    'bed_temp':       50,
    'fan_percent':    100,
    'extrusion_width':  line_width,
    'extrusion_height': layer_height * flow_rate
}


# ================================================================================
# DERIVED DIMENSIONS
# ================================================================================
len_x = units_x * grid_unit
len_y = units_y * grid_unit
len_z = units_z * grid_unit

lim_left   = start_x  + line_width/2 + tolerance/2
lim_right  = start_x  + len_x  - line_width/2 - tolerance/2
lim_bottom = start_y  + line_width/2 + tolerance/2
lim_top    = start_y  + len_y  - line_width/2 - tolerance/2

# set up outer bin dimensions
if bin_type_outer:
    lim_left   -= line_width + tolerance
    lim_right  += line_width + tolerance
    lim_bottom -= line_width + tolerance
    lim_top    += line_width + tolerance
    outer_radius += line_width + tolerance
    # make outer edge come to the same height as inner bins
    len_z += layer_height*2 + tolerance

ilim_left = lim_left + line_width*2
ilim_right = lim_right - line_width*2
ilim_bottom = lim_bottom + line_width*2
ilim_top = lim_top - line_width*2

outer_left   = lim_left
outer_right  = lim_right
outer_bottom = lim_bottom
outer_top    = lim_top

# ================================================================================
# HELPERS: Roundedrectangle boundaryfinders
# ================================================================================
def find_boundary_x(y, going_right=True):
    if ilim_bottom + outer_radius <= y <= ilim_top - outer_radius:
```

```
        return ilim_right if going_right else ilim_left
    # bottom arc
    if y < ilim_bottom + outer_radius:
        cy = ilim_bottom + outer_radius
        dy = abs(y - cy)
        dx = math.sqrt(max(0, outer_radius**2 - dy**2))
        cx = (ilim_right - outer_radius) if going_right else (ilim_left + outer_radius)
        return cx + ( dx if going_right else -dx )
    # top arc
    if y > ilim_top - outer_radius:
        cy = ilim_top - outer_radius
        dy = abs(y - cy)
        dx = math.sqrt(max(0, outer_radius**2 - dy**2))
        cx = (ilim_right - outer_radius) if going_right else (ilim_left + outer_radius)
        return cx + ( dx if going_right else -dx )
    return ilim_right if going_right else ilim_left

def find_boundary_y(x, going_up=True):
    if ilim_left + outer_radius <= x <= ilim_right - outer_radius:
        return ilim_top if going_up else ilim_bottom
    # left arc
    if x < ilim_left + outer_radius:
        cx = ilim_left + outer_radius
        dx = abs(x - cx)
        dy = math.sqrt(max(0, outer_radius**2 - dx**2))
        cy = (ilim_top - outer_radius) if going_up else (ilim_bottom + outer_radius)
        return cy + ( dy if going_up else -dy )
    # right arc
    if x > ilim_right - outer_radius:
        cx = ilim_right - outer_radius
        dx = abs(x - cx)
        dy = math.sqrt(max(0, outer_radius**2 - dx**2))
        cy = (ilim_top - outer_radius) if going_up else (ilim_bottom + outer_radius)
        return cy + ( dy if going_up else -dy )
    return ilim_top if going_up else ilim_bottom

# ============================================================================
# BUILD STEPS
# ============================================================================
steps    = []
arc_segs = 16
r        = line_width/2


wall_taper = 1.4
if bin_type_outer:
    wall_taper = 0.4

# helper function to draw an outer wall
def add_rounded_rectangle_wall(zh, r, inset = 0):
    rect_left   = outer_left + inset
    rect_right  = outer_right - inset
    rect_bottom = outer_bottom + inset
    rect_top    = outer_top - inset
    corners = [
        fc.Point(x=rect_right - r, y=rect_bottom + r, z=zh),  # br
        fc.Point(x=rect_right - r, y=rect_top    - r, z=zh),  # tr
        fc.Point(x=rect_left  + r, y=rect_top    - r, z=zh),  # tl
        fc.Point(x=rect_left  + r, y=rect_bottom + r, z=zh)   # bl
    ]
    steps.append(fc.Point(x=rect_right - r, y=rect_bottom, z=zh))
    steps.extend(fc.arcXY(corners[0], r, -math.pi/2, +math.pi/2, arc_segs))
    steps.append(fc.Point(x=rect_right, y=rect_top - r, z=zh))
    steps.extend(fc.arcXY(corners[1], r,  0,           math.pi/2, arc_segs))
    steps.append(fc.Point(x=rect_left + r, y=rect_top, z=zh))
    steps.extend(fc.arcXY(corners[2], r,  math.pi/2,   math.pi/2, arc_segs))
    steps.append(fc.Point(x=rect_left, y=rect_bottom + r, z=zh))
    steps.extend(fc.arcXY(corners[3], r,  math.pi,      math.pi/2, arc_segs))

# turn extruder on
steps.append(fc.Extruder(on=True))

# ----------------------------------------------------------------------------
# LAYER 1: HORIZONTAL ZIG-ZAG
# ----------------------------------------------------------------------------
z = layer_height
y = ilim_bottom
dir_h = +1   # +1 = leftright, -1 = rightleft
```

```
# prime at first point
x0 = find_boundary_x(y, going_right=(dir_h>0))
steps.append(fc.Point(x=x0, y=y, z=z))

while True:
    # travel to boundary
    xt = find_boundary_x(y, going_right=(dir_h>0))
    steps.append(fc.Point(x=xt, y=y, z=z))
    current_x = xt

    # next scan-line
    next_y = y + line_width
    if next_y > ilim_top:
        break

    # U-turn semicircle of radius r
    center = fc.Point(x=current_x, y=y + r, z=z)
    if dir_h > 0:
        # right edge: CCW half-circle from bottom to top
        steps.extend(fc.arcXY(center, r, -math.pi/2, +math.pi, arc_segs))
    else:
        # left edge:  CW half-circle from bottom to top
        steps.extend(fc.arcXY(center, r, -math.pi/2, -math.pi, arc_segs))

    y     = next_y
    dir_h = -dir_h

# outline the first layer
weld_offset = (wall_taper+0.5)*line_width
add_rounded_rectangle_wall(z, outer_radius - weld_offset, weld_offset)

# ---------------------------------------------------------------------------
# LAYER 2: VERTICAL ZIG-ZAG
# ---------------------------------------------------------------------------
z += layer_height
x = ilim_left
dir_v = +1   # +1 = bottomtop, -1 = topbottom

# prime at first point
y0 = find_boundary_y(x, going_up=(dir_v>0))
steps.append(fc.Point(x=x, y=y0, z=z))

while True:
    # travel to boundary
    yt = find_boundary_y(x, going_up=(dir_v>0))
    steps.append(fc.Point(x=x, y=yt, z=z))
    current_y = yt

    # next scan-line
    next_x = x + line_width
    if next_x > ilim_right:
        break

    # U-turn semicircle of radius r
    center = fc.Point(x=x + r, y=current_y, z=z)
    if dir_v > 0:
        # top edge: CCW half-circle from left to right
        steps.extend(fc.arcXY(center, r, math.pi, -math.pi, arc_segs))
    else:
        # bottom edge:  CW half-circle from left to right
        steps.extend(fc.arcXY(center, r, math.pi, +math.pi, arc_segs))

    x     = next_x
    dir_v = -dir_v

# ===========================================================================
# WALLS WITH ROUNDED CORNERS (remaining layers)
# ===========================================================================

weld_offset = (wall_taper+1.5)*line_width
add_rounded_rectangle_wall(z, outer_radius - weld_offset, weld_offset)
weld_offset = (wall_taper+0.75)*line_width
add_rounded_rectangle_wall(z, outer_radius - weld_offset, weld_offset)

while z < len_z:
    if wall_taper > 0:
        wall_taper -= layer_height/2
```

```
        wall_taper = max(wall_taper,0)
    add_rounded_rectangle_wall(z, outer_radius, wall_taper*line_width)
    z += layer_height

# repeat final wall and then quick ironing pass to smooth the top
add_rounded_rectangle_wall(z, outer_radius)
add_rounded_rectangle_wall(z, outer_radius)
steps.append(fc.Extruder(on=False))
z += layer_height/10 # lift a bit
add_rounded_rectangle_wall(z, outer_radius)
z += layer_height/10 # lift a bit
add_rounded_rectangle_wall(z, outer_radius)
z += layer_height # lift off
add_rounded_rectangle_wall(z, outer_radius) # maybe unnecessary
steps.append(fc.Point(z=z+20)) # lift after complete
```

A working sample file (from https://github.com/FullControlXYZ/fullcontrol/blob/master/llm_ref.md) is:

```
import fullcontrol as fc

# Define design parameters
layer_height = 0.2

# Create a list of steps
steps = []
steps.append(fc.Point(x=0, y=0, z=0))
steps.append(fc.Point(x=10, y=0, z=0))
steps.append(fc.Point(x=10, y=10, z=0))
steps.append(fc.Point(x=0, y=10, z=0))
steps.append(fc.Point(x=0, y=0, z=layer_height))

# For visualization
fc.transform(steps, 'plot', fc.PlotControls(style='line'))

# For G-code
gcode = fc.transform(steps, 'gcode', fc.GcodeControls(
    printer_name='prusa_i3',
    save_as='my_design',
    initialization_data={
        'print_speed': 1000,
        'nozzle_temp': 210,
        'bed_temp': 60
    }
))
```

As was discussed at: https://old.reddit.com/r/FullControl/comments/1pr0o21/problems_installing_in_new_libraries_folder_in/ running this requires a fairly clean Python installation (if need be, delete and reinstall *everything*), and using code to remove two library folders from the path: https://pastebin.com/LZFeCvVT — the relevant code from that:

```
import sys, os
from openscad import *

def sys_path_site_pkg():
    '''
    Make pip installs from OS level python accessible to PythonScad. Requires matching version (3.12.9)
    '''
    SITE_PKG = rf"C:\Users\{os.getlogin()}\AppData\Local\Programs\Python\Python312\Lib\site-
packages"

    if SITE_PKG not in sys.path:
        sys.path.append(SITE_PKG)

    # Unwind some default folder adds by PythonScad that seem to conflict!!
    # Specifically: ctypes.
    unwinds = set([
        rf"C:\Users\{os.getlogin()}\AppData\Local\Programs\Python\Python312\Lib",
        rf"C:\Users\{os.getlogin()}\AppData\Local\Programs\Python\Python312\DLLs"
    ])

    sys.path = [path for path in sys.path if path not in unwinds]


sys_path_site_pkg()
print('sys.path', sys.path)

import fullcontrol as fc
```

```
# Define design parameters
layer_height = 0.2

# Create a list of steps
steps = []
steps.append(fc.Point(x=0, y=0, z=0))
steps.append(fc.Point(x=10, y=0, z=0))
steps.append(fc.Point(x=10, y=10, z=0))
steps.append(fc.Point(x=0, y=10, z=0))
steps.append(fc.Point(x=0, y=0, z=layer_height))

# For visualization
fc.transform(steps, 'plot', fc.PlotControls(style='line'))

# For G-code
gcode = fc.transform(steps, 'gcode', fc.GcodeControls(
    printer_name='prusa_i3',
    save_as='my_design',
    initialization_data={
        'print_speed': 1000,
        'nozzle_temp': 210,
        'bed_temp': 60
    }
))
```

3.5.6.2 Previewing/verifying G-code for 3D printers

A 3rd-party tool for this is: https://help.prusa3d.com/article/prusaslicer-g-code-viewer_193152

3.5.6.3 Time and Firmware for 3D printers

The various G-code commands are specific to firmware implementations such as https://www.klipper3d.org/G-Codes.html

Where CNC operations normally only are concerned about time in the moment, and pausing until a given time has elapsed, 3D operations, with their control of heating up filament, melting it, and extruding thin ribbons of it require a greater control over time and duration.

3.5.6.4 Sample 3D printing file

```
M106 S0
M106 P2 S0
;TYPE:Custom
;;===== date: 20240520 ====================
;printer_model:Elegoo Centauri Carbon
;initial_filament:PLA
;curr_bed_type:Textured PEI Plate
M400 ; wait for buffer to clear
M220 S100 ;Set the feed speed to 100%
M221 S100 ;Set the flow rate to 100%
M104 S140
M140 S60
G90
G28 ;home
M729 ;Clean Nozzle
M190 S60


;=============turn on fans to prevent PLA jamming=================


    M106 P3 S255
    ;Prevent PLA from jamming


;enable_pressure_advance:false
;This value is called if pressure advance is enabled

M204 S5000 ;Call exterior wall print acceleration


G1 X128.5 Y-1.2 F20000
G1 Z0.3 F900
M73 P1 R0
M109 S210
M83
G92 E0 ;Reset Extruder
G1 F6000
```

```
G1 X-1.2 E10.156 ;Draw the first line
G1 Y98.8 E7.934
M73 P7 R0
G1 X-0.5 Y100 E0.1
M73 P11 R0
G1 Y-0.3 E7.934
G1 X78.5 E6.284
M73 P15 R0
G1 F1680
M73 P18 R0
G1 X98.5 E2
G1 F8400
M73 P21 R0
G1 X118.5 E2
G1 F1680
G1 X138.5 E2
G1 F8400
M73 P24 R0
G1 X158.5 E2
G1 F8400
M73 P25 R0
G1 X178.5 E2
;End PA test.


G3 I-1 J0 Z0.6 F1200.0 ;Move to side a little
M73 P27 R0
G1 F20000
G92 E0 ;Reset Extruder
;LAYER_COUNT:1
;LAYER:0
G90
G21
M83 ; use relative distances for extrusion
; filament start gcode
M106 P3 S200


;LAYER_CHANGE
;Z:0.2
;HEIGHT:0.2
;BEFORE_LAYER_CHANGE
;0.2
G92 E0

G1 E-.8 F1800
;LAYER:1

;_SET_FAN_SPEED_CHANGING_LAYER
SET_VELOCITY_LIMIT ACCEL=500
EXCLUDE_OBJECT_START NAME=Disc_id_0_copy_0
G1 X135.645 Y128.74 F30000
M73 P31 R0
G1 Z.6
G1 Z.2
G1 E.8 F1800
;TYPE:Outer wall
;WIDTH:0.499999
G1 F3000
G3 X128.198 Y121.357 I-7.146 J-.24 E1.19765
M73 P34 R0
G3 X130.232 Y121.573 I.058 J9.145 E.07407
G3 X135.591 Y127.663 I-1.733 J6.927 E.31169
M73 P35 R0
G1 X135.643 Y128.7 E.03754
G1 E-.728 F1800
;WIPE_START
G1 F30000
G1 X135.585 Y129.458 E-.0456
G1 X135.504 Y129.891 E-.0264
;WIPE_END
G1 X132.262 Y122.981 Z.6
M73 P36 R0
G1 X132.077 Y122.586 Z.6
G1 Z.2
M73 P37 R0
G1 E.8 F1800
;TYPE:Bottom surface
```

```
;WIDTH:0.505817
G1 F6300
G1 X133.335 Y123.844 E.06511
G3 X134.64 Y125.803 I-4.602 J4.479 E.08662
G1 X131.189 Y122.353 E.17854
M73 P38 R0
G1 X130.445 Y122.073 E.02909
G1 X130.192 Y122.01 E.00954
G1 X134.995 Y126.813 E.24849
M73 P39 R0
G3 X135.149 Y127.621 I-3.921 J1.166 E.03018
G1 X129.378 Y121.851 E.29858
M73 P40 R0
G2 X128.676 Y121.803 I-.554 J2.949 E.02582
G1 X135.204 Y128.331 E.33779
M73 P41 R0
G3 X135.19 Y128.972 I-3.173 J.251 E.02348
G1 X128.027 Y121.809 E.37065
M73 P42 R0
G2 X127.438 Y121.874 I.029 J2.945 E.02172
M73 P43 R0
G1 X135.124 Y129.56 E.39772
M73 P44 R0
G3 X135.017 Y130.108 I-2.76 J-.255 E.02045
G1 X126.89 Y121.981 E.42051
M73 P45 R0
G1 X126.387 Y122.133 E.01923
G1 X134.868 Y130.614 E.43887
M73 P46 R0
G3 X134.687 Y131.087 I-2.431 J-.66 E.01858
G1 X125.912 Y122.313 E.45404
M73 P47 R0
G2 X125.463 Y122.518 I.79 J2.324 E.01811
M73 P48 R0
G1 X134.481 Y131.536 E.46662
M73 P49 R0
G3 X134.252 Y131.962 I-2.22 J-.918 E.01772
G1 X125.038 Y122.748 E.47677
M73 P50 R0
G2 X124.646 Y123.01 I1.102 J2.07 E.01729
G1 X133.99 Y132.354 E.4835
M73 P52 R0
G3 X133.707 Y132.726 I-1.979 J-1.213 E.01712
G1 X124.273 Y123.292 E.48816
M73 P53 R0
G2 X123.918 Y123.592 I1.305 J1.903 E.01702
G1 X133.406 Y133.079 E.49092
M73 P54 R0
G1 X133.077 Y133.405 E.01694
G1 X123.595 Y123.923 E.49064
M73 P56 R0
G2 X123.291 Y124.274 I1.583 J1.677 E.01701
G1 X132.725 Y133.708 E.48813
M73 P57 R0
G3 X132.354 Y133.992 I-1.59 J-1.689 E.01711
G1 X123.006 Y124.643 E.48373
M73 P58 R0
G1 X122.75 Y125.042 E.01733
M73 P59 R0
G1 X131.959 Y134.251 E.47651
M73 P60 R0
G3 X131.534 Y134.481 I-1.349 J-1.984 E.0177
G1 X122.519 Y125.466 E.46649
M73 P61 R0
G2 X122.31 Y125.912 I2.1 J1.254 E.01805
G1 X131.087 Y134.688 E.45415
M73 P62 R0
G3 X130.615 Y134.871 I-1.138 J-2.244 E.01855
M73 P63 R0
G1 X122.127 Y126.383 E.43917
M73 P64 R0
G1 X121.985 Y126.896 E.01946
G1 X130.105 Y135.016 E.42016
M73 P65 R0
G3 X129.558 Y135.123 I-.806 J-2.651 E.02043
G1 X121.877 Y127.442 E.39747
M73 P66 R0
G2 X121.81 Y128.03 I2.87 J.626 E.02167
```

```
G1 X128.97 Y135.19 E.37051
M73 P68 R0
G3 X128.33 Y135.204 I-.391 J-3.158 E.02348
G1 X121.795 Y128.67 E.33813
M73 P69 R0
G2 X121.851 Y129.38 I3.542 J.078 E.02613
G1 X127.619 Y135.149 E.29847
M73 P70 R0
G3 X126.809 Y134.992 I.366 J-4.085 E.03026
G1 X122.009 Y130.193 E.24836
M73 P71 R0
G1 X122.057 Y130.392 E.00749
G1 X122.28 Y131.031 E.02476
G1 X122.356 Y131.195 E.00663
G1 X125.802 Y134.641 E.17832
M73 P72 R0
G3 X123.807 Y133.3 I2.526 J-5.915 E.0885
G1 X122.586 Y132.079 E.06316
M73 P73 R0
G1 E-.728 F1800
;WIPE_START
G1 F30000
G1 X123.435 Y132.928 E-.072
;WIPE_END
EXCLUDE_OBJECT_END NAME=Disc_id_0_copy_0
M106 S0
M106 P2 S0
;TYPE:Custom
; filament end gcode
;===== date: 20250109 ====================
M400 ; wait for buffer to clear
M140 S0 ;Turn-off bed
M106 S255 ;Cooling nozzle
M83
G92 E0 ; zero the extruder
G2 I1 J0 Z0.7 E-1 F3000 ; lower z a little
M73 P74 R0
G90
G1 Z100 F20000 ; Move print head up
M73 P94 R0
M204 S5000
M400
M83
G1 X202 F20000
M73 P95 R0
M400
G1 Y250 F20000
M73 P97 R0
G1 Y264.5 F1200
M73 P100 R0
M400
G92 E0
M104 S0 ;Turn-off hotend
M140 S0 ;Turn-off bed
M106 S0 ; turn off fan
M106 P2 S0 ; turn off remote part cooling fan
M106 P3 S0 ; turn off chamber cooling fan
M84 ;Disable all steppers
```

3.5.6.5 Initialize Certain commands are only needed for initialization, so may be grouped together in a single command:

```
1431 gcpy      def initializeforprinting(self, nozzlediameter = 0.4,
                   filamentdiameter = 1.75, extrusionwidth = 0.6, layerheight =
                    0.2, extrusiontype = "relative", extruder_temperature =
                   260, bed_temperature = 60, printer_name = "generic",
                   Base_filename = "export"):
1432 gcpy          self.nozzlediameter = nozzlediameter
1433 gcpy          self.filamentdiameter = filamentdiameter
1434 gcpy          self.extrusionwidth = extrusionwidth
1435 gcpy          self.layerheight = layerheight
1436 gcpy          self.extrusiontype = extrusiontype
1437 gcpy          self.extruder_temperature = extruder_temperature
1438 gcpy          self.bed_temperature = bed_temperature
1439 gcpy          self.printer_name = printer_name
1440 gcpy          self.Base_filename= Base_filename
1441 gcpy
```

```
1442 gcpy           self.generategcode == False
1443 gcpy
1444 gcpy           import os
1445 gcpy
1446 gcpy #     def sys_path_site_pkg():
1447 gcpy           '''
1448 gcpy ␣␣␣␣␣␣␣␣Make␣pip␣installs␣from␣OS␣level␣python␣accessible␣to␣
          PythonScad.␣Requires␣matching␣version␣(3.12.9)
1449 gcpy ␣␣␣␣␣␣␣␣'''
1450 gcpy           SITE_PKG = rf"C:\Users\{os.getlogin()}\AppData\Local\
                   Programs\Python\Python312\Lib\site-packages"
1451 gcpy
1452 gcpy           if SITE_PKG not in sys.path:
1453 gcpy               sys.path.append(SITE_PKG)
1454 gcpy
1455 gcpy           # Unwind some default folder adds by PythonScad that seem
                   to conflict!!
1456 gcpy           # Specifically: ctypes.
1457 gcpy           unwinds = set([
1458 gcpy               rf"C:\Users\{os.getlogin()}\AppData\Local\Programs\
                       Python\Python312\Lib",
1459 gcpy               rf"C:\Users\{os.getlogin()}\AppData\Local\Programs\
                       Python\Python312\DLLs"
1460 gcpy           ])
1461 gcpy
1462 gcpy           sys.path = [path for path in sys.path if path not in
                   unwinds]
1463 gcpy
1464 gcpy           import fullcontrol as fc
1465 gcpy
1466 gcpy           self.fgc = fc
1467 gcpy
1468 gcpy           self.steps = []
1469 gcpy
1470 gcpy # initialization/prime procedure
1471 gcpy           self.rapid(10,10,0.3)                                  # G0
                       F8000 X10 Y10 Z0.3
1472 gcpy           self.rapid(self.xpos(),12,0.2)                         # G0
                       F8000 Y12 Z0.2
1473 gcpy           self.extrude(110, self.ypos(),self.zpos(), True)   # G1
                       F1000 X110 E3.326014
1474 gcpy           self.extrude(self.xpos(), 14, self.zpos(), True)   # G1 Y14
                       E0.06652
1475 gcpy           self.extrude(10,self.ypos(), self.zpos(), True)    # G1 X10
                       E3.326014
1476 gcpy           self.extrude(self.xpos(), 16, self.zpos(), True)   # G1 Y16
                       E0.06652
1477 gcpy           self.extrude(self.xpos(), 10, self.zpos(), True)   # G1 Y10
                       E0.199561
1478 gcpy #         self.extrude(20, self.ypos(), self.zpos(), True)   # G1 X20
          E0.332601
1479 gcpy #         self.extrude(self.xpos(), 20,self.zpos(), True)    # G1 Y20
          E0.133041
1480 gcpy           self.rapid(self.xpos(),12,0.2)                         # G0
                       F8000 Y12 Z0.2
1481 gcpy
1482 gcpy           # end position X20, Y20, Z0.2
```

The program https://github.com/FullControlXYZ/fullcontrol/blob/master/models/hex_adapter.ipynb suggests certain variables:

```
# printer/gcode parameters

design_name = 'hex_adapter'
nozzle_temp = 210
bed_temp = 40
print_speed = 1000
fan_percent = 100
printer_name='prusa_i3' # generic / ultimaker2plus / prusa_i3 / ender_3 / cr_10 / bambulab_x1 / toolcha
```

Movement commands add an E position aspect to the command which results in the Extruder advancing to that position so as to extrude a sufficient volume of filament to match the movement and the space which is intended to be filled. Modeling these in 3D without the complexity of managing the entire 3D model and tracking the elevation of the current position relative to the model at a given point in time will require that the user maintain the current layer thickness and ensure that if unsupported, the extruded plastic will be extruded with a fan speed and flow rate which will allow bridging from/to supported areas of the model.

Calculating the volume necessary/the amount extruded will require the nozzle size, the layer height, an estimate for how much the extruded filament will spread out/deform, and the diameter of the filament. Further potential complications include whether the first layer is being extruded (normally this is done at a quite slow speed to facilitate adhesion, which also serves as a chance to catch a problem at an early stage), or if a strand is an inside or outside wall or infill or bridging open space, if it is crossing an already extruded segment(?) and so forth.

```
; --- Start of G-code: Demonstration of Layer and Extrusion Concepts ---
G21 ; Set units to millimeters
G90 ; Use absolute positioning
M82 ; Set extruder to absolute mode
M104 S200 ; Set extruder temperature to 200řC
M140 S60 ; Set bed temperature to 60řC
M190 S60 ; Wait for bed to reach target temp
M109 S200 ; Wait for extruder to reach target temp
G28 ; Home all axes

; --- Initial test extrusion ---
G92 E0 ; Reset extruder position
G1 F100 E5 ; Extrude 5 mm of filament at low speed to prime the nozzle
; Purpose: Ensures clean flow and purges any residual filament

; --- First layer adhesion test ---
G1 Z0.2 ; Move nozzle to first layer height
G1 X10 Y10 F3000 ; Move to starting position
G1 F1800 ; Set slower speed for first layer
G1 E0.8 ; Slight retraction before starting
G1 X100 E10 ; Draw a line along X to test bed adhesion
; Comment: This line helps verify that the first layer sticks properly

; --- Outer wall generation ---
G1 Z0.2 ; Maintain layer height
G1 X100 Y100 E10 ; Move and extrude to start outer square
G1 X10 Y100 E10 ;
G1 X10 Y10 E10 ;
G1 X100 Y10 E10 ;
; Outer walls: Typically printed first to preserve dimensional accuracy

; --- Cornering adjustment ---
G1 F1200 ; Reduce speed at corners
G1 X100 Y100 E0.5 ;
; Comment: Slower cornering helps prevent blobbing and maintains sharp edges

; --- Inner wall generation ---
G1 F1800 ; Resume regular speed
G1 X95 Y95 E8 ;
G1 X15 Y95 E8 ;
G1 X15 Y15 E8 ;
G1 X95 Y15 E8 ;
; Comment: Inner walls follow outer walls to enhance structural strength

; --- Understanding extrusion width ---
; Parameters:
; - Nozzle = 0.4 mm
; - Layer height = 0.2 mm
; - Filament diameter = 1.75 mm

; Flow rate ~ (extrusion_width * layer_height) / ( * (filament_diameter/2)^2)
; Example calculation: (0.4 * 0.2) / ($\pi$ * (0.875)^2)  0.033 mmş/mm

; --- Smooth top layer strategy ---
G1 Z0.4 ; Move to top layer height
G1 X20 Y20 ;
G1 F1500 ;
G1 X90 E3 ; Lay down parallel top layer strokes
G1 X90 Y90 E3 ;
G1 X20 Y90 E3 ;
G1 X20 Y20 E3 ;
G1 F3000 ;
G1 X20 Y20 ;
G1 F1500 ;
G1 X90 E3 ; Repeat for second pass for smoothing
; Tip: Overlapping infill with slightly lower extrusion helps achieve a smooth finish

; --- Wrap up ---
G92 E0 ; Reset extruder
G1 E-2 F1800 ; Retract filament to prevent stringing
M104 S0 ; Turn off hotend
M140 S0 ; Turn off bed
```

```
G28 X0 ; Home X-axis
M84 ; Disable motors
; --- End of G-code demonstration ---
```

3.5.6.6 extrude 3D printing requires control of the extruder, and matching volumetric calculations (or, more accurately, volumetric calculations which then determine the rate of extrusion).

Previewing in 3D/programming for 3D extrusion will likely want previewing not just the extruded shape, but also tracking the volume of material extruded and how it relates to the volume of the object being filled/the intersection of a just-extruded region with previously extruded material, and how large a void is left (presumably those two volumes would match up).

One concern is that G2/G3 support apparently is not common/guaranteed in 3D printer firmwares:

> *available if a gcode_arcs config section is enabled*

https://www.klipper3d.org/G-Codes.html While it is possible to separately control the feed rate of the extrusion, and the length of material extruded:

```
G1 F100 E5 ; Extrude 5 mm of filament at low speed to prime the nozzle
```

The normal usage is to move at a preset Feed rate in terms of motion, and while that movement is being made, extrude a given length of material:

```
; --- First layer adhesion test ---
G1 Z0.2 ; Move nozzle to first layer height
G1 X10 Y10 F3000 ; Move to starting position
G1 F1800 ; Set slower speed for first layer
G1 E0.8 ; Slight retraction before starting
G1 X100 E10 ; Draw a line along X to test bed adhesion
; Comment: This line helps verify that the first layer sticks properly
```

In theory, if one had a layer height equal to the diameter of the filament, and wanted to extrude a circular cross-section of filament, the value for `E` would be equal to the distance traveled.

Apparently, the firmware control is limited so that the extrusion rate cannot be varied relative to the feed rate so that it is not possible to for example, decrease the speed/increase the extrusion rate, resulting in a trapezoidal extrusion.

Given all that, the idealized (normalized?) shape and dimensions of the extrusion would be controlled by:

- layer height (for height along Z)
- extrusion rate (for width in X/Y)

which would be previewed as a rounded cross section, so it should work to create a preview by calculating the volume of material which is being extruded, then determining the volume of a circle of radius layer height/2, subtract that from the extruded volume, then determine what width of rectangle cross section would be necessary at the specified length to make up the difference.

```
1484 gcpy     def extrude(self, ex, ey, ez, extrudeonly = False):
1485 gcpy         if extrudeonly == False:
1486 gcpy             self.steps.append(self.fgc.Point(x=ex, y=ey, z=ez))
1487 gcpy         ew = self.extrusionwidth
1488 gcpy         lh = self.layerheight
1489 gcpy         i = circle(lh/2)
1490 gcpy         j = i.translate([0,lh/2,0])
1491 gcpy         k = intersection(j,square([lh,lh]))
1492 gcpy         l = k.translate([ew/2-lh/2,0,0])
1493 gcpy         m = union(l, square([ew/2-lh/2, lh]))
1494 gcpy         c = rotate_extrude(m)
1495 gcpy         c = c.translate([0,0,-self.layerheight])
1496 gcpy         tslist = hull(c.translate([self.xpos(), self.ypos(),self.
                     zpos()]), c.translate([ex, ey, ez]))
1497 gcpy         self.toolpaths.append(tslist)
1498 gcpy         self.mpx = ex
1499 gcpy         self.mpy = ey
1500 gcpy         self.mpz = ez
```

3.5.6.7 Shutdown Shutting the machine down at the end of a print affords the chance to also write out the G-code using FullControl (as opposed to having a separate command for this)

```
1502 gcpy     def shutdownafterprinting(self, print_speed = 1000):
1503 gcpy         print(self.steps)
1504 gcpy # For G-code
1505 gcpy         gcode = self.fgc.transform(self.steps, 'gcode',
```

```
1506 gcpy                                self.fgc.GcodeControls(printer_name =
                                             self.printer_name,
1507 gcpy                                save_as = self.Base_filename,
1508 gcpy                                initialization_data={
1509 gcpy                                    'print_speed': str(print_speed),
1510 gcpy                                    'nozzle_temp': str(self.
                                                extruder_temperature),
1511 gcpy                                    'bed_temp': str(self.
                                                bed_temperature)
1512 gcpy                                }
1513 gcpy                                ))
```

3.5.6.8 fullcontrolgcode commands At https://github.com/FullControlXYZ/fullcontrol/blob/master/llm_ref.md there are a number of commands beyond the basic `Point` movement implemented above as`extrude()`.

Things which will need to be looked into include:

- printer models for initialization — an if-then structure for the specific implementations may be needed, but if implemented will need to be kept in synch
- rectangle: Requires width and height
  - rectangleXY(start_point, x_size, y_size, cw=False): Generate a 2D XY rectangle, returns a list of FullControl Point objects
  - stadium: Rectangle with semi-circle at each end, requires width and height
- circle: Requires diameter
  - circleXY(centre, radius, start_angle, segments=100, cw=False): Generate a 2D XY circle, returns a list of FullControl Point objects
  - circleXY_3pt(pt1, pt2, pt3, start_angle=None, start_at_first_point=None, segments=100, cw=False): Generate a circle passing through three points, returns a list of FullControl Point objects
- arcXY(centre, radius, start_angle, arc_angle, segments): Generate an arc
- variable_arcXY(centre, start_radius, start_angle, arc_angle, segments, radius_change=0, z_change=0): Generate an arc with variable radius and z-height
- ellipseXY(centre, a, b, start_angle, segments=100, cw=False): Generate a 2D XY ellipse, returns a list of FullControl Point objects
- polygonXY(centre, enclosing_radius, start_angle, sides, cw=False): Generate a 2D XY polygon, returns a list of FullControl Point objects
- Complex Shapes
  - spiralXY(centre, start_radius, end_radius, start_angle, n_turns, segments, cw=False): Generate a 2D XY spiral
  - helixZ(centre, start_radius, end_radius, start_angle, n_turns, pitch_z, segments, cw=False): Generate a helix in the Z direction
- Wave Functions (fullcontrol/geometry/waves.py)
  - squarewaveXY(start_point, direction_vector, amplitude, line_spacing, periods, extra_half_period=False, extra_end_line=False): Generate a square wave
  - squarewaveXYpolar(start_point, direction_polar, amplitude, line_spacing, periods, extra_half_period=False, extra_end_line=False): Generate a square wave using polar coordinates
  - trianglewaveXYpolar(start_point, direction_polar, amplitude, tip_separation, periods, extra_half_period=False): Generate a triangle wave
  - sinewaveXYpolar(start_point, direction_polar, amplitude, period_length, periods, segments_per_period=16, extra_half_period=False, phase_shift=0): Generate a sine wave
  - segmented_line(start_point, end_point, segments): Create a line with multiple segments that can be modified after creation

3.6 Difference of Stock, Rapids, and Toolpaths

At the end of cutting it will be necessary to subtract the accumulated toolpaths and rapids from the stock.

For Python, the initial 3D model is stored in the variable `stock`:

```
1515 gcpy      def stockandtoolpaths(self, option = "stockandtoolpaths"):
1516 gcpy          if option == "stock":
1517 gcpy              show(self.stock)
1518 gcpy          elif option == "toolpaths":
1519 gcpy              show(self.toolpaths)
1520 gcpy          elif option == "rapids":
1521 gcpy              show(self.rapids)
1522 gcpy          else:
1523 gcpy              part = self.stock.difference(self.rapids)
1524 gcpy              part = part.difference(self.toolpaths)
1525 gcpy              show(part)
```

A separate set of commands for showing the outline of the currently selected tool and/or its shaft is useful for checking that a tool outline definition is correctly formed.

```
1527 gcpy      def showtooloutline(self):
1528 gcpy          to = union(self.tooloutline, self.shaftoutline)
1529 gcpy          show(to)
1530 gcpy
1531 gcpy      def showtoolprofile(self):
1532 gcpy          to = union(self.toolprofile, self.shaftprofile)
1533 gcpy          show(to)
1534 gcpy
1535 gcpy      def showtoolshape(self):
1536 gcpy          to = union(self.currenttoolshape, self.currenttoolshaft)
1537 gcpy          show(to)
```

Note that because of the differences in behaviour between OpenPythonSCAD (the `show()` command results in an explicit display of the requested element) and OpenSCAD (there is an implicit mechanism where the 3D element which is returned is displayed), the most expedient mechanism is to have an explicit Python command which returns the 3D model:

```
1539 gcpy      def returnstockandtoolpaths(self):
1540 gcpy          part = self.stock.difference(self.rapids)
1541 gcpy          part = part.difference(self.toolpaths)
1542 gcpy          return part
```

and then make use of that specific command for OpenSCAD:

```
142 gcpscad module stockandtoolpaths(){
143 gcpscad     gcp.returnstockandtoolpaths();
144 gcpscad }
```

forgoing the options of showing toolpaths and/or rapids separately.

3.7 Output files

The `gcodepreview` class will write out DXF and/or G-code files. Note that G-code support is handled in two separate fashions, using Full Control G-code for 3D Printing as described above, and directly managing the file as described below.

3.7.1 Python and OpenSCAD File Handling

The class `gcodepreview` will need additional commands for opening files. The original implemen-
`writeln` tation in RapSCAD used a command `writeln` — fortunately, this command is easily re-created in Python, though it is made as a separate definition for each sort of file which may be opened.

```
1544 gcpy      def writegc(self, *arguments):
1545 gcpy          if self.generategcode == True:
1546 gcpy              line_to_write = ""
1547 gcpy              for element in arguments:
1548 gcpy                  line_to_write += element
1549 gcpy              self.gc.write(line_to_write)
1550 gcpy              self.gc.write("\n")
1551 gcpy
1552 gcpy      def writedxf(self, *arguments):
1553 gcpy  #          global dxfclosed
1554 gcpy          line_to_write = ""
1555 gcpy          for element in arguments:
1556 gcpy              line_to_write += element
1557 gcpy          if self.generatedxf == True:
1558 gcpy              if self.dxfclosed == False:
1559 gcpy                  self.dxf.write(line_to_write)
1560 gcpy                  self.dxf.write("\n")
```

which commands will accept a series of arguments and then write them out to a file object for the appropriate file.

opengcodefile opendxffile

For writing to files it will be necessary to have commands for opening the files: `opengcodefile` and `opendxffile` which will set the associated defaults. There is a separate function for each type of file.

There will need to be matching OpenSCAD modules for the Python functions:

```
146 gcpscad module opendxffile(basefilename){
147 gcpscad     gcp.opendxffile(basefilename);
148 gcpscad }
```

opengcodefile

With matching OpenSCAD commands: `opengcodefile` for OpenSCAD:

```
154 gcpscad module opengcodefile(basefilename, currenttoolnum, toolradius,
        plunge, feed, speed) {
155 gcpscad     gcp.opengcodefile(basefilename, currenttoolnum, toolradius,
            plunge, feed, speed);
156 gcpscad }
```

and Python:

```
1562 gcpy     def opengcodefile(self, basefilename = "export",
1563 gcpy                       currenttoolnum = 102,
1564 gcpy                       toolradius = 3.175,
1565 gcpy                       plunge = 400,
1566 gcpy                       feed = 1600,
1567 gcpy                       speed = 10000
1568 gcpy                       ):
1569 gcpy         self.basefilename = basefilename
1570 gcpy         self.currenttoolnum = currenttoolnum
1571 gcpy         self.toolradius = toolradius
1572 gcpy         self.plunge = plunge
1573 gcpy         self.feed = feed
1574 gcpy         self.speed = speed
1575 gcpy         if self.generategcode == True:
1576 gcpy             self.gcodefilename = basefilename + self.gcodefilext
1577 gcpy             self.gc = open(self.gcodefilename, "w")
1578 gcpy             self.writegc("(Design␣File:␣" + self.basefilename + ")"
                         )
1579 gcpy
1580 gcpy     def opendxffile(self, basefilename = "export"):
1581 gcpy         self.basefilename = basefilename
1582 gcpy         print("Opening␣", self.basefilename)
1583 gcpy #         global generatedxfs
1584 gcpy #         global dxfclosed
1585 gcpy         self.dxfclosed = False
1586 gcpy         self.dxfcolor = "Black"
1587 gcpy         self.dxflayer = "DEFAULT"
1588 gcpy         if self.generatedxf == True:
1589 gcpy             self.generatedxfs = False
1590 gcpy             self.dxffilename = basefilename + ".dxf"
1591 gcpy             self.dxf = open(self.dxffilename, "w")
1592 gcpy             self.dxfpreamble()
```

Future considerations:

- Multiple Preview Modes:
- Fast Preview: Write all movements with both begin and end positions into a list for a specific tool — as this is done, check for a previous movement between those positions and compare depths and tool number — keep only the deepest movement for a given tool.
- Motion Preview: Work up a 3D model of the machine and actually show the stock in relation to it,

3.7.2 DXF Overview

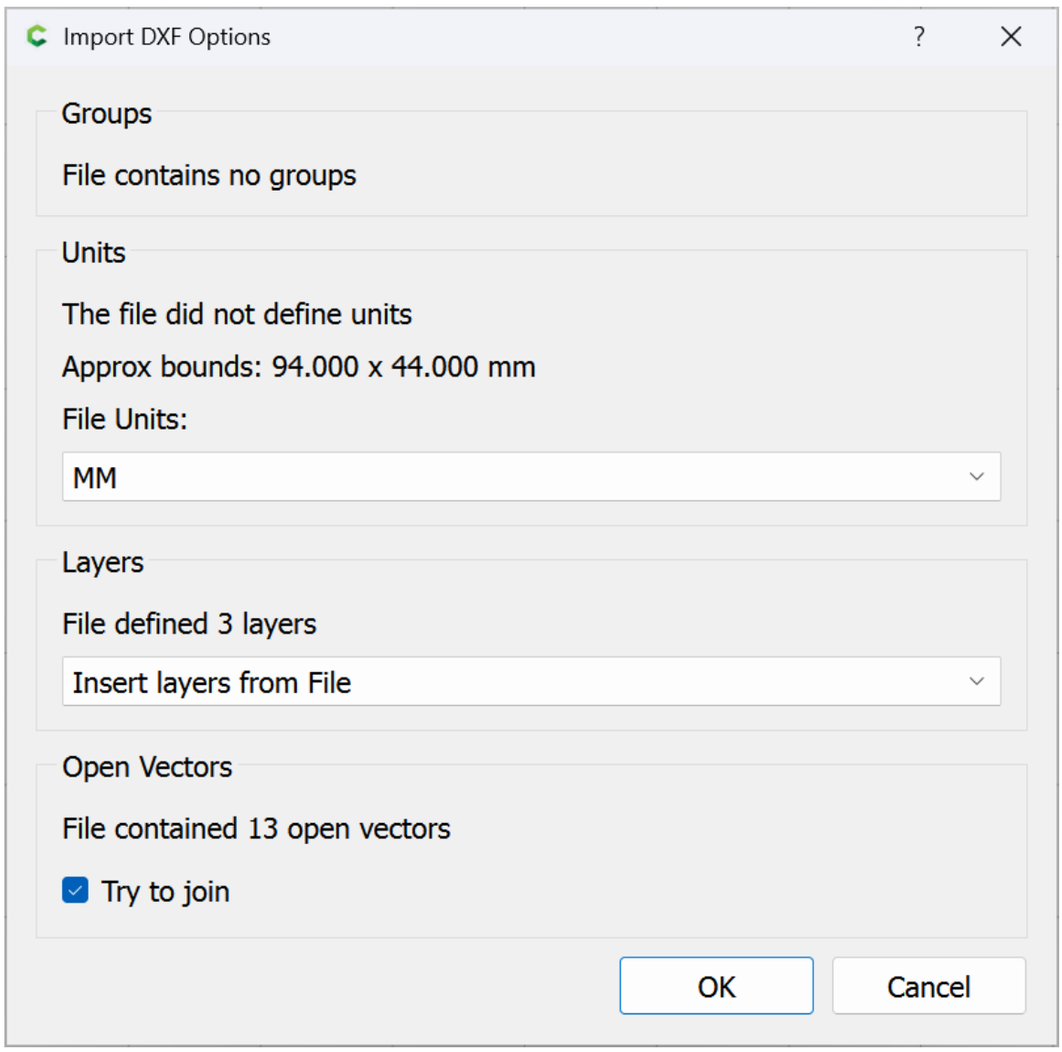
Drawing Exchange Format (DXF) is a file format originally developed by AutoDesk in 1982 for using plain text to describe CAD files. The format specification is available: https://images.autodesk.com/adsk/files/autocad_2012_pdf_dxf-reference_enu.pdf. A different Python project in this space, *ezdxf* also affords extensive documentation: https://ezdxf.readthedocs.io/en/stable/introduction.html. Applications may write out or import it as a mechanism for moving a CAD file from one system to another, or use it as a file format for saving and opening.

Elements in DXFs are represented as lines or arcs. A minimal file showing both as well as setting units to metric and placing each element on a separate layer and assigning a different colour:

```
0
```

```
SECTION
2
HEADER
9
$INSUNITS
70
4
9
$ACADVER
1
AC1014
0
ENDSEC
0
SECTION
2
ENTITIES
0
LINE
8
Layer_Red
62
1
10
0
20
0
30
0.0
11
100
21
0
31
0.0
0
ARC
8
Layer_Blue
62
5
10
3
20
47
40
6
50
270
51
360
0
ENDSEC
0
EOF
```

Gcodepreview will write to DXF files, so a primary consideration is which features of the file format are supported by the applications which the files will be imported into. As of Carbide Create build 839 https://community.carbide3d.com/t/carbide-create-beta-839/101187, it is now possible to import groups and layers, apply/use units, and have a report on open (unjoined) vectors with an option to close them:



With Carbide Create's facility to use Variables for Stock Dimensions https://willadams.gitbook.io/design-into-3d/2d-drawing#a-note-on-dimensions it then becomes possible to realize a fully parametric workflow where toolpaths are defined in terms of Stock Thickness (T) for depth of cut, while the part dimensions are derived from the imported geometry as placed on the associated layer.

An example file which supports units (mm) and a layer (0) and has accurate dimensions is available at: https://github.com/keebio/BDN9-case/blob/master/rev2/bdn9-rev2-mid-enlarged.dxf. Note that layer 0 is required by most DXF implementations, however, Carbide Create does not enforce this requirement.

dxfpreamble

3.7.2.1 Writing a preamble to DXF files

The `dxfpreamble` command writes out a header file which sets units to metric and defines the file version and sets up and ends SECTIONS as necessary.

```
1594 gcpy     def dxfpreamble(self):
1595 gcpy         self.writedxf("0")
1596 gcpy         self.writedxf("SECTION")
1597 gcpy         self.writedxf("2")
1598 gcpy         self.writedxf("HEADER")
1599 gcpy         self.writedxf("9")
1600 gcpy         self.writedxf("$INSUNITS")
1601 gcpy         self.writedxf("70")
1602 gcpy         self.writedxf("4")
1603 gcpy         self.writedxf("9")
1604 gcpy         self.writedxf("$ACADVER")
1605 gcpy         self.writedxf("1")
1606 gcpy         self.writedxf("AC1014")
1607 gcpy         self.writedxf("0")
1608 gcpy         self.writedxf("ENDSEC")
1609 gcpy         self.writedxf("0")
1610 gcpy         self.writedxf("SECTION")
1611 gcpy         self.writedxf("2")
1612 gcpy         self.writedxf("ENTITIES")
```

3.7.2.1.1 DXF Lines and Arcs

There are several elements which may be written to a DXF:

dxfline

- a line `dxfline`

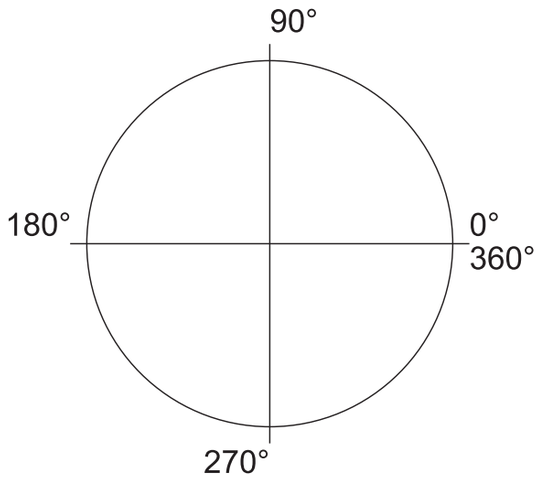
beginopenpolyline addopline finishopenpolyline
- connected lines LWPOLYLINE is one example in DXF beginopenpolyline/addopline/finishopenpolyline. Note that this command may also be used internally in certain commands

beginpolyline addvertex closepolyline
- multiple lines beginpolyline/addvertex/closepolyline (At this time, these commands produce disjoint/discrete line segments, but which should have Z height/depth, and it is expected that the software which imports the file will automatically join as needed)

dxfarc
- arc dxfarc

dxfcircle
- circle — a notable option would be for the arc to close on itself, creating a circle dxfcircle

DXF orders arcs counter-clockwise:



Note that arcs of greater than 90 degrees are not rendered accurately (in certain applications at least), so, for the sake of precision, they should be limited to a swing of 90 degrees or less. Further note that 4 arcs may be stitched together to make a circle:

```
dxfarc(10, 10, 5,   0,  90, small_square_tool_num);
dxfarc(10, 10, 5,  90, 180, small_square_tool_num);
dxfarc(10, 10, 5, 180, 270, small_square_tool_num);
dxfarc(10, 10, 5, 270, 360, small_square_tool_num);
```

The DXF file format supports colors defined by AutoCAD's indexed color system:

Color Code	Color Name
0	Black (or Foreground)
1	Red
2	Yellow
3	Green
4	Cyan
5	Blue
6	Magenta
7	White (or Background)
8	Dark Gray
9	Light Gray

Color codes 10–255 represent additional colors, with hues varying based on RGB values.

dxfcolor dxflayer
Layers and colours are controlled by separate variables: dxfcolor and dxflayer respectively. The latter is initially set to "DEFAULT" and the colour to "Black". Layer names will normally be set to be descriptive (it is possible that this could be done automatically based on toolpath and tool and depth of cut) and to match the name set in the associated Carbide Create template file.

```
1614 gcpy     def setdxfcolor(self, color):
1615 gcpy         self.dxfcolor = color
1616 gcpy         if color == "Cyan":
1617 gcpy             self.cutcolor = "Aqua"
1618 gcpy         elif color == "Magenta":
1619 gcpy             self.cutcolor = "Fuchsia"
1620 gcpy         if color == "Dark Gray":
1621 gcpy #             print("dxfcolor set to ", color)
1622 gcpy             self.cutcolor = "Gray"
1623 gcpy #             print("self.cutcolor = ", self.cutcolor)
1624 gcpy         if color == "Light Gray":
1625 gcpy             self.cutcolor = "Silver"
1626 gcpy         else:
1627 gcpy             self.cutcolor = color
1628 gcpy
```

```
1629 gcpy      def setdxflayer(self, layer):
1630 gcpy          self.dxflayer = layer
1631 gcpy
1632 gcpy      def writedxfcolor(self):
1633 gcpy              self.writedxf("8")
1634 gcpy              self.writedxf(self.dxflayer)
1635 gcpy
1636 gcpy              self.writedxf("62")
1637 gcpy              if (self.dxfcolor == "Black"):
1638 gcpy                  self.writedxf("0")
1639 gcpy              if (self.dxfcolor == "Red"):
1640 gcpy                  self.writedxf("1")
1641 gcpy              if (self.dxfcolor == "Yellow"):
1642 gcpy                  self.writedxf("2")
1643 gcpy              if (self.dxfcolor == "Green"):
1644 gcpy                  self.writedxf("3")
1645 gcpy              if (self.dxfcolor == "Cyan" or self.dxfcolor == "Aqua")
                          :
1646 gcpy                  self.writedxf("4")
1647 gcpy              if (self.dxfcolor == "Blue"):
1648 gcpy                  self.writedxf("5")
1649 gcpy              if (self.dxfcolor == "Magenta" or self.dxfcolor == "
                          Fuchsia"):
1650 gcpy                  self.writedxf("6")
1651 gcpy              if (self.dxfcolor == "White"):
1652 gcpy                  self.writedxf("7")
1653 gcpy              if (self.dxfcolor == "Dark␣Gray" or self.dxfcolor == "
                          Gray"):
1654 gcpy                  self.writedxf("8")
1655 gcpy              if (self.dxfcolor == "Light␣Gray" or self.dxfcolor == "
                          Silver"):
1656 gcpy                  self.writedxf("9")
```

```
158 gcpscad module setdxfcolor(color){
159 gcpscad     gcp.setdxfcolor(color);
160 gcpscad }
161 gcpscad
162 gcpscad module setdxflayer(layer){
163 gcpscad     gcp.setdxflayer(layer);
164 gcpscad }
```

A further refinement would be to connect multiple line segments/arcs into a larger polyline, but since most CAM tools implicitly join elements on import or afford an option to do so (Carbide Create does the latter), that is not necessary.

There are three possible interactions for DXF elements and toolpaths:

- describe the motion of the tool
- define a perimeter of an area which will be cut by a tool
- define a centerpoint for a specialty toolpath such as Drill or Keyhole

and it is possible that multiple such elements could be instantiated for a given toolpath.

When writing out to a DXF file there is a pair of commands, a public facing command which takes in a tool number in addition to the coordinates which then writes out to the main DXF file and then calls an internal command to which repeats the call with the tool number so as to write it out to the matching file.

Note the addition of variables for adding beginning (`zbegin`) and ending (`zend`) variables for setting Z-height — while not convenient to access when using the command directly, it should be workable when using a `polyline` and `addvertex`.

```
1658 gcpy      def dxfline(self, xbegin, ybegin, xend, yend, zbegin = 0.0,
                   zend = 0.0):
1659 gcpy          self.writedxf("0")
1660 gcpy          self.writedxf("LINE")
1661 gcpy #
1662 gcpy          self.writedxfcolor()
1663 gcpy #
1664 gcpy          self.writedxf("10")
1665 gcpy          self.writedxf(str(xbegin))
1666 gcpy          self.writedxf("20")
1667 gcpy          self.writedxf(str(ybegin))
1668 gcpy          self.writedxf("30")
1669 gcpy          self.writedxf(str(zbegin))
1670 gcpy          self.writedxf("11")
1671 gcpy          self.writedxf(str(xend))
```

```
1672 gcpy           self.writedxf("21")
1673 gcpy           self.writedxf(str(yend))
1674 gcpy           self.writedxf("31")
1675 gcpy           self.writedxf(str(zend))
```

In addition to `dxfline` which allows creating a line without consideration of context, there is also a `dxfpolyline` which ideally would create a continuous/joined sequence of line segments — instead, this is used as a shortcut for the `dxfline()` command, but still requires beginning it, adding vertexes, and then when done, ending the sequence — this implementation simplifies the programming and relies on the software which imports the DXF to join lines where necessary.

Note that the input of a single coordinate makes adding support for the Z-axis and accessing it straight-forward.

An example `LWPOLYLINE` DXF with a standard header:

```
0
SECTION
2
HEADER
0
ENDSEC
0
SECTION
2
TABLES
0
ENDSEC
0
SECTION
2
BLOCKS
0
ENDSEC
0
SECTION
2
ENTITIES
```

The actual header:

```
0
LWPOLYLINE
8
0
90
4
70
1
```

Note that `70 | 1` indicates closed, while 0 would be for geometry which does not return to the origin to be closed. The polyline itself is defined as a series of coordinates `10/20/30` for X/Y/Z respectively:

```
10
3.5
20
7.5
30
-1
10
97.3
20
0.0
30
-2
10
100.0
20
50.0
30
-3
10
0.0
20
50.0
30
-4
```

With a standard footer to close the file. Note that `LWPOLYLINE` is an *entity*, so it does not require closing (the next new entity, or the closing of the section which contains all such entities will end the command).

```
0
ENDSEC
0
SECTION
2
OBJECTS
0
ENDSEC
0
EOF
```

Note the inclusion of optional DXF Z-axis coordinates (30).

The point about `70 | 1/0` above and the need for it to appear at the initial instantiation creates an obvious dichotomy, and reason for the provision of two separate commands (or an optional input for open *vs.* closed).

```
1677 gcpy      def beginopenpolyline(self, xbegin, ybegin, zbegin = 0.0, co =
                   0):
1678 gcpy          self.writedxf("0")
1679 gcpy          self.writedxf("LWPOLYLINE")
1680 gcpy          self.writedxfcolor()
1681 gcpy #         self.writedxf("8")
1682 gcpy #         self.writedxf(str(self.dxflayer))
1683 gcpy          self.writedxf("90")
1684 gcpy          self.writedxf("4")
1685 gcpy          self.writedxf("70")
1686 gcpy          self.writedxf(str(co))
1687 gcpy          self.bpx = xbegin
1688 gcpy          self.bpy = ybegin
1689 gcpy          self.bpz = zbegin
1690 gcpy          self.writedxf("10")
1691 gcpy          self.writedxf(str(xbegin))
1692 gcpy          self.writedxf("20")
1693 gcpy          self.writedxf(str(ybegin))
1694 gcpy          self.writedxf("30")
1695 gcpy          self.writedxf(str(zbegin))
```

Note that storing the initial position is not strictly necessary, but is done to maintain parallelism with other polyline commands and that the first node is placed as well.

```
1697 gcpy      def addopline(self, xend, yend, zend = 0.0):
1698 gcpy          self.writedxf("10")
1699 gcpy          self.writedxf(str(xend))
1700 gcpy          self.writedxf("20")
1701 gcpy          self.writedxf(str(yend))
1702 gcpy          self.writedxf("30")
1703 gcpy          self.writedxf(str(zend))
1704 gcpy          self.bpx = xend
1705 gcpy          self.bpy = yend
1706 gcpy          self.bpz = zend
```

An otherwise identical command to close is strictly optional, but is implemented in case it makes coding easier to make clear:

```
1708 gcpy      def finishopenpolyline(self, xend, yend, zend = 0.0):
1709 gcpy          self.writedxf("10")
1710 gcpy          self.writedxf(str(xend))
1711 gcpy          self.writedxf("20")
1712 gcpy          self.writedxf(str(yend))
1713 gcpy          self.writedxf("30")
1714 gcpy          self.writedxf(str(zend))
1715 gcpy          self.bpx = xend
1716 gcpy          self.bpy = yend
1717 gcpy          self.bpz = zend
```

An alternative is to have code which fakes the polyline and just implements `dxfline` for each segment — first, rather than actually beginning the polyline, store the arguments as values:

```
1719 gcpy      def beginpolyline(self, xbegin, ybegin, zbegin = 0.0):
1720 gcpy          self.bpx = xbegin
1721 gcpy          self.bpy = ybegin
```

```
1722 gcpy          self.bpz = zbegin
```

then add as many vertexes as are wanted (note that it will be necessary to add at least one so as to have any output, however, not having one will only result in the initial position values being discarded):

```
1724 gcpy      def addvertex(self, xend, yend, zend = 0.0):
1725 gcpy          self.dxfline(self.bpx, self.bpy, xend, yend, self.bpz, zend
                     )
1726 gcpy          self.bpx = xend
1727 gcpy          self.bpy = yend
1728 gcpy          self.bpz = zend
```

there is no need to end the sequence since the `dxfline()` command is compleat unto itself. While not strictly necessary, having a `closepolyline()` adds symmetry and affords an option for error messages and so forth.

```
1730 gcpy      def closepolyline(self):
1731 gcpy          self.bpx = 0.0
1732 gcpy          self.bpy = 0.0
```

For arcs, there are specific commands for writing out the DXF and G-code files. Note that for the G-code version it will be necessary to calculate the end-position, and to determine if the arc is clockwise or no (G2 vs. G3).

```
1734 gcpy      def dxfarc(self, xcenter, ycenter, radius, anglebegin, endangle
                   ):
1735 gcpy          if (self.generatedxf == True):
1736 gcpy              self.writedxf("0")
1737 gcpy              self.writedxf("ARC")
1738 gcpy #
1739 gcpy              self.writedxfcolor()
1740 gcpy #
1741 gcpy              self.writedxf("10")
1742 gcpy              self.writedxf(str(xcenter))
1743 gcpy              self.writedxf("20")
1744 gcpy              self.writedxf(str(ycenter))
1745 gcpy              self.writedxf("40")
1746 gcpy              self.writedxf(str(radius))
1747 gcpy              self.writedxf("50")
1748 gcpy              self.writedxf(str(anglebegin))
1749 gcpy              self.writedxf("51")
1750 gcpy              self.writedxf(str(endangle))
```

The various textual versions are quite obvious, and due to the requirements of G-code, it is straight-forward to include the G-code in them if it is wanted.

```
1752 gcpy      def cutarcNECCdxf(self, ex, ey, ez, xcenter, ycenter, radius):
1753 gcpy #         global toolpath
1754 gcpy #         toolpath = self.currenttool()
1755 gcpy #         toolpath = toolpath.translate([self.xpos(), self.ypos(),
                   self.zpos()])
1756 gcpy          self.dxfarc(xcenter, ycenter, radius, 0, 90)
1757 gcpy          if (self.zpos == ez):
1758 gcpy              self.settzpos(0)
1759 gcpy          else:
1760 gcpy              self.settzpos((self.zpos()-ez)/90)
1761 gcpy #         self.setxpos(ex)
1762 gcpy #         self.setypos(ey)
1763 gcpy #         self.setzpos(ez)
1764 gcpy #         if self.generatepaths == True:
1765 gcpy #             print("Unioning cutarcNECCdxf toolpath")
1766 gcpy          self.arcloop(1, 90, xcenter, ycenter, radius)
1767 gcpy #             self.toolpaths = self.toolpaths.union(toolpath)
1768 gcpy #         else:
1769 gcpy #             toolpath = self.arcloop(1, 90, xcenter, ycenter,
                   radius)
1770 gcpy #             print("Returning cutarcNECCdxf toolpath")
1771 gcpy          return toolpath
1772 gcpy
1773 gcpy      def cutarcNWCCdxf(self, ex, ey, ez, xcenter, ycenter, radius):
1774 gcpy #         global toolpath
1775 gcpy #         toolpath = self.currenttool()
1776 gcpy #         toolpath = toolpath.translate([self.xpos(), self.ypos(),
                   self.zpos()])
```

```
1777 gcpy          self.dxfarc(xcenter, ycenter, radius, 90, 180)
1778 gcpy          if (self.zpos == ez):
1779 gcpy              self.settzpos(0)
1780 gcpy          else:
1781 gcpy              self.settzpos((self.zpos()-ez)/90)
1782 gcpy #         self.setxpos(ex)
1783 gcpy #         self.setypos(ey)
1784 gcpy #         self.setzpos(ez)
1785 gcpy #         if self.generatepaths == True:
1786 gcpy #             self.arcloop(91, 180, xcenter, ycenter, radius)
1787 gcpy #             self.toolpaths = self.toolpaths.union(toolpath)
1788 gcpy #         else:
1789 gcpy          toolpath = self.arcloop(91, 180, xcenter, ycenter, radius)
1790 gcpy          return toolpath
1791 gcpy
1792 gcpy      def cutarcSWCCdxf(self, ex, ey, ez, xcenter, ycenter, radius):
1793 gcpy #         global toolpath
1794 gcpy #         toolpath = self.currenttool()
1795 gcpy #         toolpath = toolpath.translate([self.xpos(), self.ypos(),
          self.zpos()])
1796 gcpy          self.dxfarc(xcenter, ycenter, radius, 180, 270)
1797 gcpy          if (self.zpos == ez):
1798 gcpy              self.settzpos(0)
1799 gcpy          else:
1800 gcpy              self.settzpos((self.zpos()-ez)/90)
1801 gcpy #         self.setxpos(ex)
1802 gcpy #         self.setypos(ey)
1803 gcpy #         self.setzpos(ez)
1804 gcpy          if self.generatepaths == True:
1805 gcpy              self.arcloop(181, 270, xcenter, ycenter, radius)
1806 gcpy #             self.toolpaths = self.toolpaths.union(toolpath)
1807 gcpy          else:
1808 gcpy              toolpath = self.arcloop(181, 270, xcenter, ycenter,
                      radius)
1809 gcpy              return toolpath
1810 gcpy
1811 gcpy      def cutarcSECCdxf(self, ex, ey, ez, xcenter, ycenter, radius):
1812 gcpy #         global toolpath
1813 gcpy #         toolpath = self.currenttool()
1814 gcpy #         toolpath = toolpath.translate([self.xpos(), self.ypos(),
          self.zpos()])
1815 gcpy          self.dxfarc(xcenter, ycenter, radius, 270, 360)
1816 gcpy          if (self.zpos == ez):
1817 gcpy              self.settzpos(0)
1818 gcpy          else:
1819 gcpy              self.settzpos((self.zpos()-ez)/90)
1820 gcpy #         self.setxpos(ex)
1821 gcpy #         self.setypos(ey)
1822 gcpy #         self.setzpos(ez)
1823 gcpy          if self.generatepaths == True:
1824 gcpy              self.arcloop(271, 360, xcenter, ycenter, radius)
1825 gcpy #             self.toolpaths = self.toolpaths.union(toolpath)
1826 gcpy          else:
1827 gcpy              toolpath = self.arcloop(271, 360, xcenter, ycenter,
                      radius)
1828 gcpy              return toolpath
1829 gcpy
1830 gcpy      def cutarcNECWdxf(self, ex, ey, ez, xcenter, ycenter, radius):
1831 gcpy #         global toolpath
1832 gcpy #         toolpath = self.currenttool()
1833 gcpy #         toolpath = toolpath.translate([self.xpos(), self.ypos(),
          self.zpos()])
1834 gcpy          self.dxfarc(xcenter, ycenter, radius, 0, 90)
1835 gcpy          if (self.zpos == ez):
1836 gcpy              self.settzpos(0)
1837 gcpy          else:
1838 gcpy              self.settzpos((self.zpos()-ez)/90)
1839 gcpy #         self.setxpos(ex)
1840 gcpy #         self.setypos(ey)
1841 gcpy #         self.setzpos(ez)
1842 gcpy          if self.generatepaths == True:
1843 gcpy              self.narcloop(89, 0, xcenter, ycenter, radius)
1844 gcpy #             self.toolpaths = self.toolpaths.union(toolpath)
1845 gcpy          else:
1846 gcpy              toolpath = self.narcloop(89, 0, xcenter, ycenter,
                      radius)
1847 gcpy              return toolpath
1848 gcpy
```

```
1849 gcpy     def cutarcSECWdxf(self, ex, ey, ez, xcenter, ycenter, radius):
1850 gcpy #         global toolpath
1851 gcpy #         toolpath = self.currenttool()
1852 gcpy #         toolpath = toolpath.translate([self.xpos(), self.ypos(),
          self.zpos()])
1853 gcpy         self.dxfarc(xcenter, ycenter, radius, 270, 360)
1854 gcpy         if (self.zpos == ez):
1855 gcpy             self.settzpos(0)
1856 gcpy         else:
1857 gcpy             self.settzpos((self.zpos()-ez)/90)
1858 gcpy #         self.setxpos(ex)
1859 gcpy #         self.setypos(ey)
1860 gcpy #         self.setzpos(ez)
1861 gcpy         if self.generatepaths == True:
1862 gcpy             self.narcloop(359, 270, xcenter, ycenter, radius)
1863 gcpy #             self.toolpaths = self.toolpaths.union(toolpath)
1864 gcpy         else:
1865 gcpy             toolpath = self.narcloop(359, 270, xcenter, ycenter,
                     radius)
1866 gcpy             return toolpath
1867 gcpy
1868 gcpy     def cutarcSWCWdxf(self, ex, ey, ez, xcenter, ycenter, radius):
1869 gcpy #         global toolpath
1870 gcpy #         toolpath = self.currenttool()
1871 gcpy #         toolpath = toolpath.translate([self.xpos(), self.ypos(),
          self.zpos()])
1872 gcpy         self.dxfarc(xcenter, ycenter, radius, 180, 270)
1873 gcpy         if (self.zpos == ez):
1874 gcpy             self.settzpos(0)
1875 gcpy         else:
1876 gcpy             self.settzpos((self.zpos()-ez)/90)
1877 gcpy #         self.setxpos(ex)
1878 gcpy #         self.setypos(ey)
1879 gcpy #         self.setzpos(ez)
1880 gcpy         if self.generatepaths == True:
1881 gcpy             self.narcloop(269, 180, xcenter, ycenter, radius)
1882 gcpy #             self.toolpaths = self.toolpaths.union(toolpath)
1883 gcpy         else:
1884 gcpy             toolpath = self.narcloop(269, 180, xcenter, ycenter,
                     radius)
1885 gcpy             return toolpath
1886 gcpy
1887 gcpy     def cutarcNWCWdxf(self, ex, ey, ez, xcenter, ycenter, radius):
1888 gcpy #         global toolpath
1889 gcpy #         toolpath = self.currenttool()
1890 gcpy #         toolpath = toolpath.translate([self.xpos(), self.ypos(),
          self.zpos()])
1891 gcpy         self.dxfarc(xcenter, ycenter, radius, 90, 180)
1892 gcpy         if (self.zpos == ez):
1893 gcpy             self.settzpos(0)
1894 gcpy         else:
1895 gcpy             self.settzpos((self.zpos()-ez)/90)
1896 gcpy #         self.setxpos(ex)
1897 gcpy #         self.setypos(ey)
1898 gcpy #         self.setzpos(ez)
1899 gcpy         if self.generatepaths == True:
1900 gcpy             self.narcloop(179, 90, xcenter, ycenter, radius)
1901 gcpy #             self.toolpaths = self.toolpaths.union(toolpath)
1902 gcpy         else:
1903 gcpy             toolpath = self.narcloop(179, 90, xcenter, ycenter,
                     radius)
1904 gcpy             return toolpath
```

Using such commands to create a circle is quite straight-forward:

```
cutarcNECCdxf(-stockXwidth/4, stockYheight/4+stockYheight/16, -stockZthickness, -stockXwidth/4, stockYh
cutarcNWCCdxf(-(stockXwidth/4+stockYheight/16), stockYheight/4, -stockZthickness, -stockXwidth/4, stock
cutarcSWCCdxf(-stockXwidth/4, stockYheight/4-stockYheight/16, -stockZthickness, -stockXwidth/4, stockYh
cutarcSECCdxf(-(stockXwidth/4-stockYheight/16), stockYheight/4, -stockZthickness, -stockXwidth/4, stock
```

```
1906 gcpy     def arcCCgc(self, ex, ey, ez, xcenter, ycenter, radius):
1907 gcpy         self.writegc("G03␣X", str(ex), "␣Y", str(ey), "␣Z", str(ez)
                      , "␣R", str(radius))
1908 gcpy
1909 gcpy     def arcCWgc(self, ex, ey, ez, xcenter, ycenter, radius):
1910 gcpy         self.writegc("G02␣X", str(ex), "␣Y", str(ey), "␣Z", str(ez)
                      , "␣R", str(radius))
```

The above commands may be called if G-code is also wanted with writing out G-code added:

```
1912 gcpy     def cutarcNECCdxfgc(self, ex, ey, ez, xcenter, ycenter, radius)
                  :
1913 gcpy         self.arcCCgc(ex, ey, ez, xcenter, ycenter, radius)
1914 gcpy         if self.generatepaths == True:
1915 gcpy             self.cutarcNECCdxf(ex, ey, ez, xcenter, ycenter, radius
                        )
1916 gcpy         else:
1917 gcpy             return self.cutarcNECCdxf(ex, ey, ez, xcenter, ycenter,
                         radius)
1918 gcpy
1919 gcpy     def cutarcNWCCdxfgc(self, ex, ey, ez, xcenter, ycenter, radius)
                  :
1920 gcpy         self.arcCCgc(ex, ey, ez, xcenter, ycenter, radius)
1921 gcpy         if self.generatepaths == False:
1922 gcpy             return self.cutarcNWCCdxf(ex, ey, ez, xcenter, ycenter,
                         radius)
1923 gcpy
1924 gcpy     def cutarcSWCCdxfgc(self, ex, ey, ez, xcenter, ycenter, radius)
                  :
1925 gcpy         self.arcCCgc(ex, ey, ez, xcenter, ycenter, radius)
1926 gcpy         if self.generatepaths == False:
1927 gcpy             return self.cutarcSWCCdxf(ex, ey, ez, xcenter, ycenter,
                         radius)
1928 gcpy
1929 gcpy     def cutarcSECCdxfgc(self, ex, ey, ez, xcenter, ycenter, radius)
                  :
1930 gcpy         self.arcCCgc(ex, ey, ez, xcenter, ycenter, radius)
1931 gcpy         if self.generatepaths == False:
1932 gcpy             return self.cutarcSECCdxf(ex, ey, ez, xcenter, ycenter,
                         radius)
1933 gcpy
1934 gcpy     def cutarcNECWdxfgc(self, ex, ey, ez, xcenter, ycenter, radius)
                  :
1935 gcpy         self.arcCWgc(ex, ey, ez, xcenter, ycenter, radius)
1936 gcpy         if self.generatepaths == False:
1937 gcpy             return self.cutarcNECWdxf(ex, ey, ez, xcenter, ycenter,
                         radius)
1938 gcpy
1939 gcpy     def cutarcSECWdxfgc(self, ex, ey, ez, xcenter, ycenter, radius)
                  :
1940 gcpy         self.arcCWgc(ex, ey, ez, xcenter, ycenter, radius)
1941 gcpy         if self.generatepaths == False:
1942 gcpy             return self.cutarcSECWdxf(ex, ey, ez, xcenter, ycenter,
                         radius)
1943 gcpy
1944 gcpy     def cutarcSWCWdxfgc(self, ex, ey, ez, xcenter, ycenter, radius)
                  :
1945 gcpy         self.arcCWgc(ex, ey, ez, xcenter, ycenter, radius)
1946 gcpy         if self.generatepaths == False:
1947 gcpy             return self.cutarcSWCWdxf(ex, ey, ez, xcenter, ycenter,
                         radius)
1948 gcpy
1949 gcpy     def cutarcNWCWdxfgc(self, ex, ey, ez, xcenter, ycenter, radius)
                  :
1950 gcpy         self.arcCWgc(ex, ey, ez, xcenter, ycenter, radius)
1951 gcpy         if self.generatepaths == False:
1952 gcpy             return self.cutarcNWCWdxf(ex, ey, ez, xcenter, ycenter,
                         radius)
```

```
166 gcpscad module cutarcNECCdxfgc(ex, ey, ez, xcenter, ycenter, radius){
167 gcpscad     gcp.cutarcNECCdxfgc(ex, ey, ez, xcenter, ycenter, radius);
168 gcpscad }
169 gcpscad
170 gcpscad module cutarcNWCCdxfgc(ex, ey, ez, xcenter, ycenter, radius){
171 gcpscad     gcp.cutarcNWCCdxfgc(ex, ey, ez, xcenter, ycenter, radius);
172 gcpscad }
173 gcpscad
174 gcpscad module cutarcSWCCdxfgc(ex, ey, ez, xcenter, ycenter, radius){
175 gcpscad     gcp.cutarcSWCCdxfgc(ex, ey, ez, xcenter, ycenter, radius);
176 gcpscad }
177 gcpscad
178 gcpscad module cutarcSECCdxfgc(ex, ey, ez, xcenter, ycenter, radius){
179 gcpscad     gcp.cutarcSECCdxfgc(ex, ey, ez, xcenter, ycenter, radius);
180 gcpscad }
```

3.7.3 G-code Overview

The G-code commands and their matching modules may include (but are not limited to):

Command/Module	G-code

`opengcodefile(s)(...); setupstock(...)`	`(export.nc)`
	`(stockMin: -109.5, -75mm, -8.35mm)`
	`(stockMax:109.5mm, 75mm, 0.00mm)`
	`(STOCK/BLOCK, 219, 150, 8.35, 109.5, 75, 8.35)`
	`G90`
	`G21`

`movetosafez()`	`(Move to safe Z to avoid workholding)` `G53G0Z-5.000`

`toolchange(...);`	`(TOOL/MILL, 3.17, 0.00, 0.00, 0.00)`
	`M6T102`
	`M03S16000`

`cutoneaxis_setfeed(...);`
 	
`cutwithfeed(...);`
	`(PREPOSITION FOR RAPID PLUNGE)`
	`G0X0Y0`
	`Z0.25`
	`G1Z0F100`
	`G1 X109.5 Y75 Z-8.35F400`
	`Z9`

`closegcodefile();`	`M05` `M02`

Conversely, the G-code commands which are supported are generated by the following modules:

G-code	Command/Module

`(Design File: )`
`(stockMin:0.00mm, -152.40mm, -34.92mm)`
`(stockMax:109.50mm, -77.40mm, 0.00mm)`
`(STOCK/BLOCK, 109.50, 75.00, 34.92, 0.00, 152.40, 34.92)`
`G90`
`G21`
	`opengcodefile(s)(...); setupstock(.`

`(Move to safe Z to avoid workholding)` `G53G0Z-5.000`	`movetosafez()`

`(Toolpath: Contour Toolpath 1)`
`M05`
`(TOOL/MILL, 3.17, 0.00, 0.00, 0.00)`
`M6T102`
`M03S10000`
	`toolchange(...);`

`(PREPOSITION FOR RAPID PLUNGE)`	`writecomment(...)`
`G0X0.000Y-152.400` `Z0.250`	`rapid(...)` `rapid(...)`

`G1Z-1.000F203.2`
`X109.500Y-77.400F508.0`
`X57.918Y16.302Z-0.726`
`Y22.023Z-1.023`
`X61.190Z-0.681`
`Y21.643`
`X57.681`
`Z12.700`
	`cutwithfeed(...);`
	`cutwithfeed(...);`

`M05` `M02`	`closegcodefile();`

The implication here is that it should be possible to read in a G-code file, and for each line/command instantiate a matching command so as to create a 3D model/preview of the file. This is addressed by making specialized commands for movement which correspond to the various axis combinations (XYZ, XY, XZ, YZ, X, Y, Z).

A further consideration is that rather than hard-coding all possibilities or any changes, having an option for a "post-processor" will be far more flexible.

Described at: https://carbide3d.com/hub/faq/create-pro-custom-post-processor/ the necessary hooks would be:

- onOpen

- onClose
- onSection (which is where tool changes are defined, since "section" in this case is segmented per tool)

closegcodefile closedxffile

3.7.3.1 Closings At the end of the program it will be necessary to close each file using the commands: `closegcodefile`, and `closedxffile`. In some instances it may be necessary to write additional information, depending on the file format. Note that these commands will need to be within the `gcodepreview class`.

```
1954 gcpy     def dxfpostamble(self):
1955 gcpy #         self.writedxf(str(tn))
1956 gcpy         self.writedxf("0")
1957 gcpy         self.writedxf("ENDSEC")
1958 gcpy         self.writedxf("0")
1959 gcpy         self.writedxf("EOF")
```

```
1961 gcpy     def gcodepostamble(self):
1962 gcpy         if self.generatecut == True:
1963 gcpy             self.writegc("Z12.700")
1964 gcpy             self.writegc("M05")
1965 gcpy             self.writegc("M02")
1966 gcpy         if self.generateprint == True:
1967 gcpy             self.writegc("G92␣E0")
1968 gcpy             self.writegc("M107␣␣␣␣;␣turn␣off␣cooling␣fans")
1969 gcpy             self.writegc("M104␣S0␣;␣turn␣off␣temperature")
1970 gcpy             self.writegc("G28␣X0␣␣;␣home␣X␣axis")
1971 gcpy             self.writegc("M84␣␣␣␣␣;␣disable␣motors")
```

```
1973 gcpy     def closegcodefile(self):
1974 gcpy         if self.generategcode == True:
1975 gcpy             self.gcodepostamble()
1976 gcpy             self.gc.close()
1977 gcpy
1978 gcpy     def closedxffile(self):
1979 gcpy         if self.generatedxf == True:
1980 gcpy #             global dxfclosed
1981 gcpy             self.dxfpostamble()
1982 gcpy #             self.dxfclosed = True
1983 gcpy             self.dxf.close()
1984 gcpy             print("Closing␣opened␣file.")
```

closegcodefile closedxffile dxfpostamble

The commands: `closegcodefile`, and `closedxffile` are used to close the files at the end of a program. For efficiency, each references the command: `dxfpostamble` which when called provides the boilerplate needed at the end of their respective files.

```
182 gcpscad module closegcodefile(){
183 gcpscad     gcp.closegcodefile();
184 gcpscad }
185 gcpscad
186 gcpscad module closedxffile(){
187 gcpscad     gcp.closedxffile();
188 gcpscad }
```

3.8 Cutting shapes and expansion

Certain basic shapes (arcs, circles, rectangles), will be incorporated in the main code. Other shapes will be added as they are developed, and of course the user is free to develop their own systems.

It is most expedient to test out new features in a new/separate file insofar as the file structures will allow (tool definitions for example will need to consolidated in 3.4.1.1) which will need to be included in the projects which will make use of said features until such time as they are added into the main `gcodepreview.scad` file.

A basic requirement for two-dimensional regions will be to define them so as to cut them out. Two different geometric treatments will be necessary: modeling the geometry which defines the region to be cut out (output as a DXF); and modeling the movement of the tool, the toolpath which will be used in creating the 3D model and outputting the G-code.

3.8.1 Building blocks

The outlines of shapes will be defined using:

- lines — `dxfline`

- arcs — dxfarc

It may be that splines or Bézier curves will be added as well.

3.8.1.1 List of shapes In the TUG presentation/paper: http://tug.org/TUGboat/tb40-2/tb125adams-3d.pdf a list of 2D shapes was put forward — which of these will need to be created, or if some more general solution will be put forward is uncertain. For the time being, shapes will be implemented on an as-needed basis, as modified by the interaction with the requirements of toolpaths. Shapes for which code exists (or is trivially coded) are indicated by Forest Green — for those which have sub-classes, if all are feasible only the higher level is so called out.

- 0
  - circle — dxfcircle
  - ellipse (oval) (requires some sort of non-arc curve)
    - egg-shaped
  - annulus (one circle within another, forming a ring) — handled by nested circles
  - superellipse (see astroid below)
- 1
  - cone with rounded end (arc)—see also "sector" under 3 below
- 2 (digons and similar shapes)
  - semicircle/circular/half-circle segment (arc and a straight line); see also sector below
  - arch—curve possibly smoothly joining a pair of straight lines with a flat bottom
  - lens/vesica piscis (two convex curves)
  - lune/crescent (one convex, one concave curve)
  - heart (two curves)
  - tomoe (comma shape)—non-arc curves
- 3 (triangles and other 3 pointed figures)
  - triangle
    - equilateral
    - isosceles
    - right triangle
    - scalene
  - (circular) sector (two straight edges, one convex arc)
    - quadrant (90°)
    - sextants (60°)
    - octants (45°)
  - deltoid curve (three concave arcs)
  - Reuleaux triangle (three convex arcs)
  - arbelos (one convex, two concave arcs)
  - two straight edges, one concave arc—an example is the hyperbolic sector[7]
  - two convex, one concave arc
- 4 (quadrilaterals and so forth)
  - rectangle (including square) — dxfrectangle, dxfrectangleround
  - parallelogram
  - rhombus
  - trapezoid/trapezium
  - kite
  - ring/annulus segment (straight line, concave arc, straight line, convex arc)
  - astroid (four concave arcs)
  - salinon (four semicircles)
  - three straight lines and one concave arc

Note that most shapes will also exist in a rounded form where sharp angles/points are replaced by arcs/portions of circles, with the most typical being dxfrectangleround.

Is the list of shapes for which there are not widely known names interesting for its lack of notoriety?

[7] en.wikipedia.org/wiki/Hyperbolic_sector and www.reddit.com/r/Geometry/comments/bkbzgh/is_there_a_name_for_a_3_pointed_figure_with_two

- two straight edges, one concave arc—oddly, an asymmetric form (hyperbolic sector) has a name, but not the symmetrical—while the colloquial/prosaic "arrowhead" was considered, it was rejected as being better applied to the shape below. (It's also the shape used for the spaceship in the game Asteroids (or Hyperspace), but that is potentially confusing with astroid.) At the conference, Dr. Knuth suggested "dart" as a suitable term.
- two convex, one concave arc—with the above named, the term "arrowhead" is freed up to use as the name for this shape.
- three straight lines and one concave arc.

The first in particular is sorely needed for this project (it's the result of inscribing a circle in a square or other regular geometric shape). Do these shapes have names in any other languages which might be used instead?

These shapes will then be used in constructing toolpaths. The program Carbide Create has toolpath types and options which are as follows:

- Contour — No Offset — the default, this is already supported in the existing code
- Contour — Outside Offset
- Contour — Inside Offset
- Pocket — such toolpaths/geometry should include the rounding of the tool at the corners, c.f., `dxfrectangleround`
- Drill — note that this is implemented as the plunging of a tool centered on a circle and normally that circle is the same diameter as the tool which is used.
- Keyhole — also beginning from a circle, the command for this also models the areas which should be cleared for the sake of reducing wear on the tool and ensuring chip clearance

Some further considerations:

- relationship of geometry to toolpath — arguably there should be an option for each toolpath (we will use Carbide Create as a reference implementation) which is to be supported. Note that there are several possibilities: modeling the tool movement, describing the outline which the tool will cut, modeling a reference shape for the toolpath
- tool geometry — support is included for specialty tooling such as dovetail cutters allowing one to to get an accurate 3D model, including for tooling which undercuts since they cannot be modeled in Carbide Create.
- Starting and Max Depth — are there CAD programs which will make use of Z-axis information in a DXF? — would it be possible/necessary to further differentiate the DXF geometry? (currently written out separately for each toolpath in addition to one combined file) — would supporting layers be an option?

3.8.1.1.1 circles Circles are made up of a series of arcs:

```
1986 gcpy     def dxfcircle(self, xcenter, ycenter, radius):
1987 gcpy         self.dxfarc(xcenter, ycenter, radius,  0, 90)
1988 gcpy         self.dxfarc(xcenter, ycenter, radius, 90, 180)
1989 gcpy         self.dxfarc(xcenter, ycenter, radius, 180, 270)
1990 gcpy         self.dxfarc(xcenter, ycenter, radius, 270, 360)
```

Actually cutting the circle is much the same, with the added consideration of entry point if Z height is not above the surface of the stock/already removed material, directionality (counter-clockwise vs. clockwise), and depth (beginning and end depths must be specified which should allow usage of this for thread-cutting and similar purposes).

Center is specified, but the actual entry point is the right-most edge.

```
1992 gcpy     def cutcircleCC(self, xcenter, ycenter, bz, ez, radius):
1993 gcpy         self.setzpos(bz)
1994 gcpy         self.cutquarterCCNE(xcenter, ycenter + radius, self.zpos()
                      + ez/4, radius)
1995 gcpy         self.cutquarterCCNW(xcenter - radius, ycenter, self.zpos()
                      + ez/4, radius)
1996 gcpy         self.cutquarterCCSW(xcenter, ycenter - radius, self.zpos()
                      + ez/4, radius)
1997 gcpy         self.cutquarterCCSE(xcenter + radius, ycenter, self.zpos()
                      + ez/4, radius)
1998 gcpy
1999 gcpy     def cutcircleCCdxf(self, xcenter, ycenter, bz, ez, radius):
2000 gcpy         self.cutcircleCC(self, xcenter, ycenter, bz, ez, radius)
2001 gcpy         self.dxfcircle(self, xcenter, ycenter, radius)
```

A Drill toolpath is a simple plunge operation which will have a matching circle to define it.

3.8.1.1.2 rectangles There are two obvious forms for rectangles, square cornered and rounded:

```
2003 gcpy     def dxfrectangle(self, xorigin, yorigin, xwidth, yheight,
                  corners = "Square", radius = 6):
2004 gcpy         if corners == "Square":
2005 gcpy             self.dxfline(xorigin, yorigin, xorigin + xwidth,
                          yorigin)
2006 gcpy             self.dxfline(xorigin + xwidth, yorigin, xorigin +
                          xwidth, yorigin + yheight)
2007 gcpy             self.dxfline(xorigin + xwidth, yorigin + yheight,
                          xorigin, yorigin + yheight)
2008 gcpy             self.dxfline(xorigin, yorigin + yheight, xorigin,
                          yorigin)
2009 gcpy         elif corners == "Fillet":
2010 gcpy             self.dxfrectangleround(xorigin, yorigin, xwidth,
                          yheight, radius)
2011 gcpy         elif corners == "Chamfer":
2012 gcpy             self.dxfrectanglechamfer(xorigin, yorigin, xwidth,
                          yheight, radius)
2013 gcpy         elif corners == "Flipped␣Fillet":
2014 gcpy             self.dxfrectangleflippedfillet(xorigin, yorigin, xwidth
                          , yheight, radius)
```

Note that the rounded shape below would be described as a rectangle with the "Fillet" corner treatment in Carbide Create.

```
2016 gcpy     def dxfrectangleround(self, xorigin, yorigin, xwidth, yheight,
                  radius):
2017 gcpy         self.dxfarc(xorigin + radius, yorigin + radius, radius,
                      180, 270)
2018 gcpy         self.dxfarc(xorigin + xwidth - radius, yorigin + radius,
                      radius, 270, 360)
2019 gcpy         self.dxfarc(xorigin + xwidth - radius, yorigin + yheight -
                      radius, radius, 0, 90)
2020 gcpy         self.dxfarc(xorigin + radius, yorigin + yheight - radius,
                      radius, 90, 180)
2021 gcpy         self.dxfline(xorigin + radius, yorigin, xorigin + xwidth -
                      radius, yorigin)
2022 gcpy         self.dxfline(xorigin + xwidth, yorigin + radius, xorigin +
                      xwidth, yorigin + yheight - radius)
2023 gcpy         self.dxfline(xorigin + xwidth - radius, yorigin + yheight,
                      xorigin + radius, yorigin + yheight)
2024 gcpy         self.dxfline(xorigin, yorigin + yheight - radius, xorigin,
                      yorigin + radius)
```

```
2026 gcpy     def dxfrectanglelwp(self, bx, by, xwidth, yheight):
2027 gcpy         self.writedxf("0")
2028 gcpy         self.writedxf("LWPOLYLINE")
2029 gcpy         self.writedxf("8")
2030 gcpy         self.writedxf(self.dxflayer)
2031 gcpy         self.writedxf("90")
2032 gcpy         self.writedxf("4")
2033 gcpy         self.writedxf("70")
2034 gcpy         self.writedxf("1")
2035 gcpy         self.writedxf("10")
2036 gcpy         self.writedxf(str(bx))#lower-left X
2037 gcpy         self.writedxf("20")
2038 gcpy         self.writedxf(str(by))#lower-left Y
2039 gcpy         self.writedxf("10")
2040 gcpy         self.writedxf(str(bx+xwidth))#lower-right X
2041 gcpy         self.writedxf("20")
2042 gcpy         self.writedxf(str(by))
2043 gcpy         self.writedxf("10")
2044 gcpy         self.writedxf(str(bx+xwidth))
2045 gcpy         self.writedxf("20")
2046 gcpy         self.writedxf(str(by+yheight))
2047 gcpy         self.writedxf("10")
2048 gcpy         self.writedxf(str(bx))
2049 gcpy         self.writedxf("20")
2050 gcpy         self.writedxf(str(by+yheight))
```

So we add the balance of the corner treatments which are decorative (and easily implemented).

Chamfer:

```
2052 gcpy     def dxfrectanglechamfer(self, tool_num, xorigin, yorigin,
                  xwidth, yheight, radius):
2053 gcpy         self.dxfline(tool_num, xorigin + radius, yorigin, xorigin,
                      yorigin + radius)
2054 gcpy         self.dxfline(tool_num, xorigin, yorigin + yheight - radius,
                       xorigin + radius, yorigin + yheight)
2055 gcpy         self.dxfline(tool_num, xorigin + xwidth - radius, yorigin +
                       yheight, xorigin + xwidth, yorigin + yheight - radius)
2056 gcpy         self.dxfline(tool_num, xorigin + xwidth - radius, yorigin,
                      xorigin + xwidth, yorigin + radius)
2057 gcpy
2058 gcpy         self.dxfline(tool_num, xorigin + radius, yorigin, xorigin +
                       xwidth - radius, yorigin)
2059 gcpy         self.dxfline(tool_num, xorigin + xwidth, yorigin + radius,
                      xorigin + xwidth, yorigin + yheight - radius)
2060 gcpy         self.dxfline(tool_num, xorigin + xwidth - radius, yorigin +
                       yheight, xorigin + radius, yorigin + yheight)
2061 gcpy         self.dxfline(tool_num, xorigin, yorigin + yheight - radius,
                       xorigin, yorigin + radius)
```

Flipped Fillet:

```
2063 gcpy     def dxfrectangleflippedfillet(self, xorigin, yorigin, xwidth,
                  yheight, radius):
2064 gcpy         self.dxfarc(xorigin, yorigin, radius,  0, 90)
2065 gcpy         self.dxfarc(xorigin + xwidth, yorigin, radius, 90, 180)
2066 gcpy         self.dxfarc(xorigin + xwidth, yorigin + yheight, radius,
                      180, 270)
2067 gcpy         self.dxfarc(xorigin, yorigin + yheight, radius, 270, 360)
2068 gcpy
2069 gcpy         self.dxfline(xorigin + radius, yorigin, xorigin + xwidth -
                      radius, yorigin)
2070 gcpy         self.dxfline(xorigin + xwidth, yorigin + radius, xorigin +
                      xwidth, yorigin + yheight - radius)
2071 gcpy         self.dxfline(xorigin + xwidth - radius, yorigin + yheight,
                      xorigin + radius, yorigin + yheight)
2072 gcpy         self.dxfline(xorigin, yorigin + yheight - radius, xorigin,
                      yorigin + radius)
```

Cutting rectangles while writing out their perimeter in the DXF files (so that they may be assigned a matching toolpath in a traditional CAM program upon import) will require the origin coordinates, height and width and depth of the pocket, and the tool # so that the corners may have a radius equal to the tool which is used. Whether a given module is an interior pocket or an outline (interior or exterior) will be determined by the specifics of the module and its usage/positioning, with `outline` being added to those modules which cut perimeter.

A further consideration is that cut orientation as an option should be accounted for if writing out G-code, as well as stepover, and the nature of initial entry (whether ramping in would be implemented, and if so, at what angle). Advanced toolpath strategies such as trochoidal milling could also be implemented.

`cutrectangle` The routine `cutrectangle` cuts the outline of a rectangle creating rounded corners.

```
2074 gcpy     def cutrectangle(self, bx, by, bz, xwidth, yheight, zdepth):
2075 gcpy         self.cutline(bx, by, bz)
2076 gcpy         self.cutline(bx, by, bz - zdepth)
2077 gcpy         self.cutline(bx + xwidth, by, bz - zdepth)
2078 gcpy         self.cutline(bx + xwidth, by + yheight, bz - zdepth)
2079 gcpy         self.cutline(bx, by + yheight, bz - zdepth)
2080 gcpy         self.cutline(bx, by, bz - zdepth)
2081 gcpy
2082 gcpy     def cutrectangledxf(self, bx, by, bz, xwidth, yheight, zdepth):
2083 gcpy         self.cutrectangle(bx, by, bz, xwidth, yheight, zdepth)
2084 gcpy         self.dxfrectangle(bx, by, xwidth, yheight, "Square")
2085 gcpy
2086 gcpy     def cutnarrowrectangledxf(self, bx, by, bz, xwidth, yheight,
                  zdepth, radius):
2087 gcpy         self.cutpocketrectangleroundnarrow(bx, by, bz, xwidth,
                      yheight, zdepth, radius)
2088 gcpy         self.dxfrectanglelwp(bx, by, xwidth, yheight)
2089 gcpy #         self.dxfrectangle(bx, by, xwidth, yheight, "Square")
```

The rounded forms instantiate a radius:

```
2091 gcpy     def cutrectangleround(self, bx, by, bz, xwidth, yheight, zdepth
                  , radius):
2092 gcpy #         self.rapid(bx + radius, by, bz)
2093 gcpy         self.cutline(bx + radius, by, bz - zdepth)
```

```
2094 gcpy          self.cutline(bx + xwidth - radius, by, bz - zdepth)
2095 gcpy          self.cutquarterCCSE(bx + xwidth, by + radius, bz - zdepth,
                       radius)
2096 gcpy          self.cutline(bx + xwidth, by + yheight - radius, bz -
                       zdepth)
2097 gcpy          self.cutquarterCCNE(bx + xwidth - radius, by + yheight, bz
                       - zdepth, radius)
2098 gcpy          self.cutline(bx + radius, by + yheight, bz - zdepth)
2099 gcpy          self.cutquarterCCNW(bx, by + yheight - radius, bz - zdepth,
                        radius)
2100 gcpy          self.cutline(bx, by + radius, bz - zdepth)
2101 gcpy          self.cutquarterCCSW(bx + radius, by, bz - zdepth, radius)
2102 gcpy
2103 gcpy      def cutrectanglerounddxf(self, bx, by, bz, xwidth, yheight,
                   zdepth, radius):
2104 gcpy          self.cutrectangleround(bx, by, bz, xwidth, yheight, zdepth,
                        radius)
2105 gcpy          self.dxfrectangleround(bx, by, xwidth, yheight, radius)
2106 gcpy
2107 gcpy      def cutrectangledxf(self, bx, by, bz, xwidth, yheight, zdepth,
                   radius):
2108 gcpy          self.cutrectangleround(bx, by, bz, xwidth, yheight, zdepth,
                        radius)
2109 gcpy          self.dxfrectangle(bx, by, xwidth, yheight)
2110 gcpy
2111 gcpy      def cutpocketrectanglerounddxf(self, bx, by, bz, xwidth,
                   yheight, zdepth, radius):
2112 gcpy          self.cutpocketrectangleround(bx, by, bz, xwidth, yheight,
                       zdepth, radius)
2113 gcpy          self.dxfrectanglelwp(bx, by, xwidth, yheight)
2114 gcpy      # half-depth cut around perimeter for checking
2115 gcpy      #    self.cutrectangleround(bx, by, bz, xwidth, yheight, zdepth
                   /2, radius)
```

Two options to cut inside the perimeter and clear the field:

```
2117 gcpy      def cutpocketrectangleroundnarrow(self, bx, by, bz, xwidth,
                   yheight, zdepth, radius):
2118 gcpy      #first, cut perimeter
2119 gcpy          self.movetosafeZ()
2120 gcpy          self.rapid(bx + radius, by+radius, 0)
2121 gcpy          self.cutline(bx + xwidth - radius, by+radius, bz - zdepth)
2122 gcpy          self.cutline(bx + xwidth - radius, by+yheight - radius, bz
                       - zdepth)
2123 gcpy          self.cutline(bx + radius, by+yheight - radius, bz - zdepth)
2124 gcpy          self.cutline(bx + radius, by+radius, bz - zdepth)
2125 gcpy          self.cutline(bx + xwidth - radius, by+radius, bz - zdepth)
2126 gcpy          if yheight > xwidth:
2127 gcpy              self.cutline(bx + xwidth/2, by+radius, bz - zdepth)
2128 gcpy              self.cutline(bx + xwidth/2, by+yheight -radius, bz -
                           zdepth)
2129 gcpy          else:
2130 gcpy              self.cutline(bx + radius, by+yheight/2, bz - zdepth)
2131 gcpy              self.cutline(bx + xwidth-radius, by+yheight/2, bz -
                           zdepth)
2132 gcpy          self.movetosafeZ()
2133 gcpy
2134 gcpy      def cutpocketrectangleround(self, bx, by, bz, xwidth, yheight,
                   zdepth, radius):
2135 gcpy          if yheight > xwidth:
2136 gcpy              self.movetosafeZ()
2137 gcpy              self.rapid(bx + radius*2.5, by+radius, 0)
2138 gcpy              self.cutline(bx + radius*2.5, by+radius, bz - zdepth)
2139 gcpy              self.cutline(bx + xwidth - radius*2, by+radius, bz -
                           zdepth)
2140 gcpy              self.cutquarterCCSE(bx + xwidth - radius, by + radius
                           *1.5, bz - zdepth, radius/2)
2141 gcpy              self.cutline(bx + xwidth-radius, by + yheight - radius
                           *2, bz - zdepth)
2142 gcpy              self.cutquarterCCNE(bx + xwidth - radius*1.5, by +
                           yheight-radius, bz - zdepth, radius/2)
2143 gcpy              self.cutline(bx + radius*1.5, by + yheight-radius, bz -
                            zdepth)
2144 gcpy              self.cutquarterCCNW(bx+radius, by + yheight - radius
                           *1.5, bz - zdepth, radius/2)
2145 gcpy              self.cutline(bx+radius*2, by + radius*2, bz - zdepth)
2146 gcpy              self.cutquarterCCSW(bx + radius*2.5, by+radius, bz -
                           zdepth, radius/2)
```

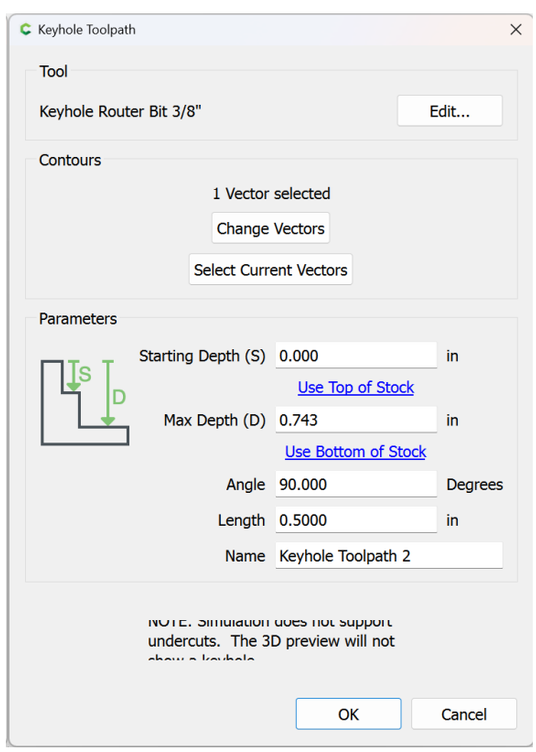
```
2147 gcpy                number_of_passes = int((xwidth)/(radius*1.999))
2148 gcpy                so = xwidth/number_of_passes/2
2149 gcpy                cw = 0
2150 gcpy                while cw < xwidth-radius*1.5:
2151 gcpy                    self.cutline(bx+radius+cw, by+radius, bz - zdepth)
2152 gcpy                    self.cutline(bx+radius+cw, by+yheight-radius, bz -
                                zdepth)
2153 gcpy                    cw += so
2154 gcpy            else:
2155 gcpy                self.movetosafeZ()
2156 gcpy                self.rapid(bx + radius*1.5, by+radius, 0)
2157 gcpy                self.cutline(bx + radius*2.5, by+radius, bz - zdepth)
2158 gcpy                self.cutline(bx + xwidth - radius*2, by+radius, bz -
                            zdepth)
2159 gcpy                self.cutquarterCCSE(bx + xwidth - radius, by + radius
                            *1.5, bz - zdepth, radius/2)
2160 gcpy                self.cutline(bx + xwidth-radius, by + yheight - radius
                            *2, bz - zdepth)
2161 gcpy                self.cutquarterCCNE(bx + xwidth - radius*1.5, by +
                            yheight-radius, bz - zdepth, radius/2)
2162 gcpy                self.cutline(bx + radius*1.5, by + yheight-radius, bz -
                             zdepth)
2163 gcpy                self.cutquarterCCNW(bx+radius, by + yheight - radius
                            *1.5, bz - zdepth, radius/2)
2164 gcpy                self.cutline(bx+radius*1, by + radius*2, bz - zdepth)
2165 gcpy                self.cutquarterCCSW(bx + radius*1.5, by+radius, bz -
                            zdepth, radius/2)
2166 gcpy                number_of_passes = int((yheight)/(radius*1.999))
2167 gcpy                so = yheight/number_of_passes/2
2168 gcpy                cw = 0
2169 gcpy                self.cutline(bx+radius, by+radius, bz - zdepth)
2170 gcpy                while cw < yheight-radius*1.5:
2171 gcpy                    self.cutline(bx+radius, by+radius+cw, bz - zdepth)
2172 gcpy                    self.cutline(bx+xwidth-radius, by+radius+cw, bz -
                                zdepth)
2173 gcpy                    cw += so
2174 gcpy            self.movetosafeZ()
2175 gcpy            self.rapid(bx+radius, by, 0)
```

3.8.1.1.3 Keyhole toolpath and undercut tooling

The first topologically unusual toolpath is `cutkeyhole toolpath` `cutkeyhole toolpath` — where other toolpaths have a direct correspondence between the associated geometry and the area cut, that Keyhole toolpaths may be used with tooling which undercuts and which will result in the creation of two different physical physical regions: the visible surface matching the union of the tool perimeter at the entry point and the linear movement of the shaft and the larger region of the tool perimeter at the depth which the tool is plunged to and moved along.

Tooling for such toolpaths is defined at paragraph 3.5.1

The interface which is being modeled is that of Carbide Create:



Hence the parameters:

- Starting Depth == `kh_start_depth`
- Max Depth == `kh_max_depth`
- Angle == `kht_direction`
- Length == `kh_distance`
- Tool == `kh_tool_num`

Due to the possibility of rotation, for the in-between positions there are more cases than one would think — for each quadrant there are the following possibilities:

- one node on the clockwise side is outside of the quadrant
- two nodes on the clockwise side are outside of the quadrant
- all nodes are w/in the quadrant
- one node on the counter-clockwise side is outside of the quadrant
- two nodes on the counter-clockwise side are outside of the quadrant

Supporting all of these would require trigonometric comparisons in the `if...else` blocks, so only the 4 quadrants, N, S, E, and W will be supported in the initial version. This will be done by wrapping the command with a version which only accepts those options:

```
2177 gcpy      def cutkeyholegcdxf(self, kh_tool_num, kh_start_depth,
                   kh_max_depth, kht_direction, kh_distance):
2178 gcpy          if (kht_direction == "N"):
2179 gcpy              toolpath = self.cutKHgcdxf(kh_tool_num, kh_start_depth,
                           kh_max_depth, 90, kh_distance)
2180 gcpy          elif (kht_direction == "S"):
2181 gcpy              toolpath = self.cutKHgcdxf(kh_tool_num, kh_start_depth,
                           kh_max_depth, 270, kh_distance)
2182 gcpy          elif (kht_direction == "E"):
2183 gcpy              toolpath = self.cutKHgcdxf(kh_tool_num, kh_start_depth,
                           kh_max_depth, 0, kh_distance)
2184 gcpy          elif (kht_direction == "W"):
2185 gcpy              toolpath = self.cutKHgcdxf(kh_tool_num, kh_start_depth,
                           kh_max_depth, 180, kh_distance)
2186 gcpy #         if self.generatepaths == True:
2187 gcpy #             self.toolpaths = union([self.toolpaths, toolpath])
2188 gcpy          return toolpath
2189 gcpy #         else:
2190 gcpy #             return cube([0.01, 0.01, 0.01])
```

```
190 gcpscad module cutkeyholegcdxf(kh_tool_num, kh_start_depth, kh_max_depth,
        kht_direction, kh_distance){
191 gcpscad     gcp.cutkeyholegcdxf(kh_tool_num, kh_start_depth, kh_max_depth,
            kht_direction, kh_distance);
192 gcpscad }
```

`cutKHgcdxf` The original version of the command, `cutKHgcdxf` retains an interface which allows calling it for arbitrary beginning and ending points of an arc. Note that code is still present for the partial calculation of one quadrant (for the case of all nodes within the quadrant).

This command makes use of `self.currenttool()` and will require that the appropriate tool is selected.

The first task is to place a circle at the origin which is invariant of angle:

```
2192 gcpy      def cutKHgcdxf(self, kh_tool_num, kh_start_depth, kh_max_depth,
                   kh_angle, kh_distance):
2193 gcpy          oXpos = self.xpos()
2194 gcpy          oYpos = self.ypos()
2195 gcpy          self.dxfKH(kh_tool_num, self.xpos(), self.ypos(),
                       kh_start_depth, kh_max_depth, kh_angle, kh_distance)
2196 gcpy          toolpath = self.cutline(self.xpos(), self.ypos(), -
                       kh_max_depth)
2197 gcpy          self.setxpos(oXpos)
2198 gcpy          self.setypos(oYpos)
2199 gcpy #         if self.generatepaths == False:
2200 gcpy          return toolpath
2201 gcpy #         else:
2202 gcpy #             return cube([0.001, 0.001, 0.001])
```

```
2204 gcpy      def dxfKH(self, oXpos, oYpos, kh_start_depth, kh_max_depth,
                   kh_angle, kh_distance):
2205 gcpy #         oXpos = self.xpos()
2206 gcpy #         oYpos = self.ypos()
2207 gcpy #Circle at entry hole
2208 gcpy          self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                       currenttoolnumber(), 7), 0, 90)
2209 gcpy          self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                       currenttoolnumber(), 7), 90, 180)
2210 gcpy          self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                       currenttoolnumber(), 7), 180, 270)
2211 gcpy          self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                       currenttoolnumber(), 7), 270, 360)
```

Then it will be necessary to test for each possible case in a series of If Else blocks:

```
2212 gcpy #pre-calculate needed values
2213 gcpy          r = self.tool_radius(self.currenttoolnumber(), 7)
2214 gcpy #         print(r)
2215 gcpy          rt = self.tool_radius(self.currenttoolnumber(), 1)
2216 gcpy #         print(rt)
2217 gcpy          ro = math.sqrt((self.tool_radius(self.currenttoolnumber(),
                       1))**2-(self.tool_radius(self.currenttoolnumber(), 7))
                       **2)
2218 gcpy #         print(ro)
2219 gcpy          angle = math.degrees(math.acos(ro/rt))
2220 gcpy #Outlines of entry hole and slot
2221 gcpy          if (kh_angle == 0):
2222 gcpy #Lower left of entry hole
2223 gcpy              self.dxfarc(self.xpos(), self.ypos(), self.tool_radius(
                           self.currenttoolnumber(), 1), 180, 270)
2224 gcpy #Upper left of entry hole
2225 gcpy              self.dxfarc(self.xpos(), self.ypos(), self.tool_radius(
                           self.currenttoolnumber(), 1), 90, 180)
2226 gcpy #Upper right of entry hole
2227 gcpy #             self.dxfarc(self.xpos(), self.ypos(), rt, 41.810, 90)
2228 gcpy              self.dxfarc(self.xpos(), self.ypos(), rt, angle, 90)
2229 gcpy #Lower right of entry hole
2230 gcpy              self.dxfarc(self.xpos(), self.ypos(), rt, 270, 360-
                           angle)
2231 gcpy #             self.dxfarc(self.xpos(), self.ypos(), self.tool_radius
          (self.currenttoolnumber(), 1), 270, 270+math.acos(self.
          tool_diameter(self.currenttoolnumber(), 5)/self.tool_diameter(
          self.currenttoolnumber(), 1)))
2232 gcpy #Actual line of cut
2233 gcpy #             self.dxfline(self.xpos(), self.ypos(), self.xpos()+
          kh_distance, self.ypos())
2234 gcpy #upper right of end of slot (kh_max_depth+4.36))/2
2235 gcpy              self.dxfarc(self.xpos()+kh_distance, self.ypos(), self.
                           tool_diameter(self.currenttoolnumber(), (
                           kh_max_depth+4.36))/2, 0, 90)
2236 gcpy #lower right of end of slot
2237 gcpy              self.dxfarc(self.xpos()+kh_distance, self.ypos(), self.
                           tool_diameter(self.currenttoolnumber(), (
                           kh_max_depth+4.36))/2, 270, 360)
2238 gcpy #upper right slot
2239 gcpy              self.dxfline(self.xpos()+ro, self.ypos()-(self.
                           tool_diameter(self.currenttoolnumber(), 7)/2), self.
                           xpos()+kh_distance, self.ypos()-(self.tool_diameter(
                           self.currenttoolnumber(), 7)/2))
2240 gcpy #             self.dxfline(self.xpos()+(math.sqrt((self.
          tool_diameter(self.currenttoolnumber(), 1)^2)-(self.
          tool_diameter(self.currenttoolnumber(), 5)^2))/2), self.ypos()+
          self.tool_diameter(self.currenttoolnumber(), (kh_max_depth))/2,
          ( (kh_max_depth-6.34))/2)^2-(self.tool_diameter(self.
          currenttoolnumber(), (kh_max_depth-6.34))/2)^2, self.xpos()+
          kh_distance, self.ypos()+self.tool_diameter(self.
          currenttoolnumber(), (kh_max_depth))/2, self.currenttoolnumber()
          )
2241 gcpy #end position at top of slot
2242 gcpy #lower right slot
2243 gcpy              self.dxfline(self.xpos()+ro, self.ypos()+(self.
                           tool_diameter(self.currenttoolnumber(), 7)/2), self.
                           xpos()+kh_distance, self.ypos()+(self.tool_diameter(
                           self.currenttoolnumber(), 7)/2))
2244 gcpy #         dxfline(self.xpos()+(math.sqrt((self.tool_diameter(self.
          currenttoolnumber(), 1)^2)-(self.tool_diameter(self.
```

```
            currenttoolnumber(), 5)^2))/2), self.ypos()-self.tool_diameter(
            self.currenttoolnumber(), (kh_max_depth))/2, ( (kh_max_depth
            -6.34))/2)^2-(self.tool_diameter(self.currenttoolnumber(), (
            kh_max_depth-6.34))/2)^2, self.xpos()+kh_distance, self.ypos()-
            self.tool_diameter(self.currenttoolnumber(), (kh_max_depth))/2,
            self.currenttoolnumber())
2245 gcpy #end position at top of slot
2246 gcpy #    hull(){
2247 gcpy #      translate([xpos(), ypos(), zpos()]){
2248 gcpy #        keyhole_shaft(6.35, 9.525);
2249 gcpy #      }
2250 gcpy #      translate([xpos(), ypos(), zpos()-kh_max_depth]){
2251 gcpy #        keyhole_shaft(6.35, 9.525);
2252 gcpy #      }
2253 gcpy #    }
2254 gcpy #    hull(){
2255 gcpy #      translate([xpos(), ypos(), zpos()-kh_max_depth]){
2256 gcpy #        keyhole_shaft(6.35, 9.525);
2257 gcpy #      }
2258 gcpy #      translate([xpos()+kh_distance, ypos(), zpos()-kh_max_depth])
            {
2259 gcpy #        keyhole_shaft(6.35, 9.525);
2260 gcpy #      }
2261 gcpy #    }
2262 gcpy #    cutwithfeed(getxpos(), getypos(), -kh_max_depth, feed);
2263 gcpy #    cutwithfeed(getxpos()+kh_distance, getypos(), -kh_max_depth,
            feed);
2264 gcpy #    setxpos(getxpos()-kh_distance);
2265 gcpy #  } else if (kh_angle > 0 && kh_angle < 90) {
2266 gcpy #//echo(kh_angle);
2267 gcpy #  dxfarc(getxpos(), getypos(), tool_diameter(self.
            currenttoolnumber(), (kh_max_depth))/2, 90+kh_angle, 180+
            kh_angle, self.currenttoolnumber());
2268 gcpy #  dxfarc(getxpos(), getypos(), tool_diameter(self.
            currenttoolnumber(), (kh_max_depth))/2, 180+kh_angle, 270+
            kh_angle, self.currenttoolnumber());
2269 gcpy #dxfarc(getxpos(), getypos(), tool_diameter(self.currenttoolnumber
            (), (kh_max_depth))/2, kh_angle+asin((tool_diameter(self.
            currenttoolnumber(), (kh_max_depth+4.36))/2)/(tool_diameter(self
            .currenttoolnumber(), (kh_max_depth))/2)), 90+kh_angle, self.
            currenttoolnumber());
2270 gcpy #dxfarc(getxpos(), getypos(), tool_diameter(self.currenttoolnumber
            (), (kh_max_depth))/2, 270+kh_angle, 360+kh_angle-asin((
            tool_diameter(self.currenttoolnumber(), (kh_max_depth+4.36))/2)
            /(tool_diameter(self.currenttoolnumber(), (kh_max_depth))/2)),
            self.currenttoolnumber());
2271 gcpy #dxfarc(getxpos()+(kh_distance*cos(kh_angle)),
2272 gcpy #  getypos()+(kh_distance*sin(kh_angle)), tool_diameter(self.
            currenttoolnumber(), (kh_max_depth+4.36))/2, 0+kh_angle, 90+
            kh_angle, self.currenttoolnumber());
2273 gcpy #dxfarc(getxpos()+(kh_distance*cos(kh_angle)), getypos()+(
            kh_distance*sin(kh_angle)), tool_diameter(self.currenttoolnumber
            (), (kh_max_depth+4.36))/2, 270+kh_angle, 360+kh_angle, self.
            currenttoolnumber());
2274 gcpy #dxfline( getxpos()+tool_diameter(self.currenttoolnumber(), (
            kh_max_depth))/2*cos(kh_angle+asin((tool_diameter(self.
            currenttoolnumber(), (kh_max_depth+4.36))/2)/(tool_diameter(self
            .currenttoolnumber(), (kh_max_depth))/2))),
2275 gcpy # getypos()+tool_diameter(self.currenttoolnumber(), (kh_max_depth))
            /2*sin(kh_angle+asin((tool_diameter(self.currenttoolnumber(), (
            kh_max_depth+4.36))/2)/(tool_diameter(self.currenttoolnumber(),
            (kh_max_depth))/2))),
2276 gcpy # getxpos()+(kh_distance*cos(kh_angle))-((tool_diameter(self.
            currenttoolnumber(), (kh_max_depth+4.36))/2)*sin(kh_angle)),
2277 gcpy # getypos()+(kh_distance*sin(kh_angle))+((tool_diameter(self.
            currenttoolnumber(), (kh_max_depth+4.36))/2)*cos(kh_angle)),
            self.currenttoolnumber());
2278 gcpy #//echo("a", tool_diameter(self.currenttoolnumber(), (kh_max_depth
            +4.36))/2);
2279 gcpy #//echo("c", tool_diameter(self.currenttoolnumber(), (kh_max_depth)
            )/2);
2280 gcpy #echo("Aangle", asin((tool_diameter(self.currenttoolnumber(), (
            kh_max_depth+4.36))/2)/(tool_diameter(self.currenttoolnumber(),
            (kh_max_depth))/2)));
2281 gcpy #//echo(kh_angle);
2282 gcpy # cutwithfeed(getxpos()+(kh_distance*cos(kh_angle)), getypos()+(
            kh_distance*sin(kh_angle)), -kh_max_depth, feed);
2283 gcpy #            toolpath = toolpath.union(self.cutline(self.xpos()+
```

```
            kh_distance, self.ypos(), -kh_max_depth))
2284 gcpy          elif (kh_angle == 90):
2285 gcpy #Lower left of entry hole
2286 gcpy              self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                           currenttoolnumber(), 1), 180, 270)
2287 gcpy #Lower right of entry hole
2288 gcpy              self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                           currenttoolnumber(), 1), 270, 360)
2289 gcpy #left slot
2290 gcpy              self.dxfline(oXpos-r, oYpos+ro, oXpos-r, oYpos+
                           kh_distance)
2291 gcpy #right slot
2292 gcpy              self.dxfline(oXpos+r, oYpos+ro, oXpos+r, oYpos+
                           kh_distance)
2293 gcpy #upper left of end of slot
2294 gcpy              self.dxfarc(oXpos, oYpos+kh_distance, r, 90, 180)
2295 gcpy #upper right of end of slot
2296 gcpy              self.dxfarc(oXpos, oYpos+kh_distance, r, 0, 90)
2297 gcpy #Upper right of entry hole
2298 gcpy              self.dxfarc(oXpos, oYpos, rt, 0, 90-angle)
2299 gcpy #Upper left of entry hole
2300 gcpy              self.dxfarc(oXpos, oYpos, rt, 90+angle, 180)
2301 gcpy #             toolpath = toolpath.union(self.cutline(oXpos, oYpos+
            kh_distance, -kh_max_depth))
2302 gcpy          elif (kh_angle == 180):
2303 gcpy #Lower right of entry hole
2304 gcpy              self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                           currenttoolnumber(), 1), 270, 360)
2305 gcpy #Upper right of entry hole
2306 gcpy              self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                           currenttoolnumber(), 1), 0, 90)
2307 gcpy #Upper left of entry hole
2308 gcpy              self.dxfarc(oXpos, oYpos, rt, 90, 180-angle)
2309 gcpy #Lower left of entry hole
2310 gcpy              self.dxfarc(oXpos, oYpos, rt, 180+angle, 270)
2311 gcpy #upper slot
2312 gcpy              self.dxfline(oXpos-ro, oYpos-r, oXpos-kh_distance,
                           oYpos-r)
2313 gcpy #lower slot
2314 gcpy              self.dxfline(oXpos-ro, oYpos+r, oXpos-kh_distance,
                           oYpos+r)
2315 gcpy #upper left of end of slot
2316 gcpy              self.dxfarc(oXpos-kh_distance, oYpos, r, 90, 180)
2317 gcpy #lower left of end of slot
2318 gcpy              self.dxfarc(oXpos-kh_distance, oYpos, r, 180, 270)
2319 gcpy #             toolpath = toolpath.union(self.cutline(oXpos-
            kh_distance, oYpos, -kh_max_depth))
2320 gcpy          elif (kh_angle == 270):
2321 gcpy #Upper left of entry hole
2322 gcpy              self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                           currenttoolnumber(), 1), 90, 180)
2323 gcpy #Upper right of entry hole
2324 gcpy              self.dxfarc(oXpos, oYpos, self.tool_radius(self.
                           currenttoolnumber(), 1), 0, 90)
2325 gcpy #left slot
2326 gcpy              self.dxfline(oXpos-r, oYpos-ro, oXpos-r, oYpos-
                           kh_distance)
2327 gcpy #right slot
2328 gcpy              self.dxfline(oXpos+r, oYpos-ro, oXpos+r, oYpos-
                           kh_distance)
2329 gcpy #lower left of end of slot
2330 gcpy              self.dxfarc(oXpos, oYpos-kh_distance, r, 180, 270)
2331 gcpy #lower right of end of slot
2332 gcpy              self.dxfarc(oXpos, oYpos-kh_distance, r, 270, 360)
2333 gcpy #lower right of entry hole
2334 gcpy              self.dxfarc(oXpos, oYpos, rt, 180, 270-angle)
2335 gcpy #lower left of entry hole
2336 gcpy              self.dxfarc(oXpos, oYpos, rt, 270+angle, 360)
2337 gcpy #             toolpath = toolpath.union(self.cutline(oXpos, oYpos-
            kh_distance, -kh_max_depth))
2338 gcpy #         print(self.zpos())
2339 gcpy #         self.setxpos(oXpos)
2340 gcpy #         self.setypos(oYpos)
2341 gcpy #         if self.generatepaths == False:
2342 gcpy #             return toolpath
2343 gcpy
2344 gcpy #  } else if (kh_angle == 90) {
2345 gcpy #    //Lower left of entry hole
```

```
2346 gcpy #    dxfarc(getxpos(), getypos(), 9.525/2, 180, 270, self.
             currenttoolnumber());
2347 gcpy #    //Lower right of entry hole
2348 gcpy #    dxfarc(getxpos(), getypos(), 9.525/2, 270, 360, self.
             currenttoolnumber());
2349 gcpy #    //Upper right of entry hole
2350 gcpy #    dxfarc(getxpos(), getypos(), 9.525/2, 0, acos(tool_diameter(
             self.currenttoolnumber(), 5)/tool_diameter(self.
             currenttoolnumber(), 1)), self.currenttoolnumber());
2351 gcpy #    //Upper left of entry hole
2352 gcpy #    dxfarc(getxpos(), getypos(), 9.525/2, 180-acos(tool_diameter(
             self.currenttoolnumber(), 5)/tool_diameter(self.
             currenttoolnumber(), 1)), 180, self.currenttoolnumber());
2353 gcpy #    //Actual line of cut
2354 gcpy #    dxfline(getxpos(), getypos(), getxpos(), getypos()+kh_distance
             );
2355 gcpy #    //upper right of slot
2356 gcpy #    dxfarc(getxpos(), getypos()+kh_distance, tool_diameter(self.
             currenttoolnumber(), (kh_max_depth+4.36))/2, 0, 90, self.
             currenttoolnumber());
2357 gcpy #    //upper left of slot
2358 gcpy #    dxfarc(getxpos(), getypos()+kh_distance, tool_diameter(self.
             currenttoolnumber(), (kh_max_depth+6.35))/2, 90, 180, self.
             currenttoolnumber());
2359 gcpy #    //right of slot
2360 gcpy #    dxfline(
2361 gcpy #         getxpos()+tool_diameter(self.currenttoolnumber(), (
             kh_max_depth))/2,
2362 gcpy #         getypos()+(math.sqrt((tool_diameter(self.currenttoolnumber
             (), 1)^2)-(tool_diameter(self.currenttoolnumber(), 5)^2))/2),
             //( (kh_max_depth-6.34))/2)^2-(tool_diameter(self.
             currenttoolnumber(), (kh_max_depth-6.34))/2)^2,
2363 gcpy #         getxpos()+tool_diameter(self.currenttoolnumber(), (
             kh_max_depth))/2,
2364 gcpy #    //end position at top of slot
2365 gcpy #         getypos()+kh_distance,
2366 gcpy #         self.currenttoolnumber());
2367 gcpy #    dxfline(getxpos()-tool_diameter(self.currenttoolnumber(), (
             kh_max_depth))/2, getypos()+(math.sqrt((tool_diameter(self.
             currenttoolnumber(), 1)^2)-(tool_diameter(self.currenttoolnumber
             (), 5)^2))/2), getxpos()-tool_diameter(self.currenttoolnumber(),
              (kh_max_depth+6.35))/2, getypos()+kh_distance, self.
             currenttoolnumber());
2368 gcpy #    hull(){
2369 gcpy #      translate([xpos(), ypos(), zpos()]){
2370 gcpy #        keyhole_shaft(6.35, 9.525);
2371 gcpy #      }
2372 gcpy #      translate([xpos(), ypos(), zpos()-kh_max_depth]){
2373 gcpy #        keyhole_shaft(6.35, 9.525);
2374 gcpy #      }
2375 gcpy #    }
2376 gcpy #    hull(){
2377 gcpy #      translate([xpos(), ypos(), zpos()-kh_max_depth]){
2378 gcpy #        keyhole_shaft(6.35, 9.525);
2379 gcpy #      }
2380 gcpy #      translate([xpos(), ypos()+kh_distance, zpos()-kh_max_depth])
             {
2381 gcpy #        keyhole_shaft(6.35, 9.525);
2382 gcpy #      }
2383 gcpy #    }
2384 gcpy #    cutwithfeed(getxpos(), getypos(), -kh_max_depth, feed);
2385 gcpy #    cutwithfeed(getxpos(), getypos()+kh_distance, -kh_max_depth,
             feed);
2386 gcpy #    setypos(getypos()-kh_distance);
2387 gcpy #  } else if (kh_angle == 180) {
2388 gcpy #    //Lower right of entry hole
2389 gcpy #    dxfarc(getxpos(), getypos(), 9.525/2, 270, 360, self.
             currenttoolnumber());
2390 gcpy #    //Upper right of entry hole
2391 gcpy #    dxfarc(getxpos(), getypos(), 9.525/2, 0, 90, self.
             currenttoolnumber());
2392 gcpy #    //Upper left of entry hole
2393 gcpy #    dxfarc(getxpos(), getypos(), 9.525/2, 90, 90+acos(
             tool_diameter(self.currenttoolnumber(), 5)/tool_diameter(self.
             currenttoolnumber(), 1)), self.currenttoolnumber());
2394 gcpy #    //Lower left of entry hole
2395 gcpy #    dxfarc(getxpos(), getypos(), 9.525/2, 270-acos(tool_diameter(
             self.currenttoolnumber(), 5)/tool_diameter(self.
```

```
            currenttoolnumber(), 1)), 270, self.currenttoolnumber());
2396 gcpy #     //upper left of slot
2397 gcpy #     dxfarc(getxpos()-kh_distance, getypos(), tool_diameter(self.
            currenttoolnumber(), (kh_max_depth+6.35))/2, 90, 180, self.
            currenttoolnumber());
2398 gcpy #     //lower left of slot
2399 gcpy #     dxfarc(getxpos()-kh_distance, getypos(), tool_diameter(self.
            currenttoolnumber(), (kh_max_depth+6.35))/2, 180, 270, self.
            currenttoolnumber());
2400 gcpy #     //Actual line of cut
2401 gcpy #     dxfline(getxpos(), getypos(), getxpos()-kh_distance, getypos()
            );
2402 gcpy #     //upper left slot
2403 gcpy #     dxfline(
2404 gcpy #         getxpos()-(math.sqrt((tool_diameter(self.currenttoolnumber
            (), 1)^2)-(tool_diameter(self.currenttoolnumber(), 5)^2))/2),
2405 gcpy #         getypos()+tool_diameter(self.currenttoolnumber(), (
            kh_max_depth))/2, //( (kh_max_depth-6.34))/2)^2-(tool_diameter(
            self.currenttoolnumber(), (kh_max_depth-6.34))/2)^2,
2406 gcpy #         getxpos()-kh_distance,
2407 gcpy #     //end position at top of slot
2408 gcpy #         getypos()+tool_diameter(self.currenttoolnumber(), (
            kh_max_depth))/2,
2409 gcpy #         self.currenttoolnumber());
2410 gcpy #     //lower right slot
2411 gcpy #     dxfline(
2412 gcpy #         getxpos()-(math.sqrt((tool_diameter(self.currenttoolnumber
            (), 1)^2)-(tool_diameter(self.currenttoolnumber(), 5)^2))/2),
2413 gcpy #         getypos()-tool_diameter(self.currenttoolnumber(), (
            kh_max_depth))/2, //( (kh_max_depth-6.34))/2)^2-(tool_diameter(
            self.currenttoolnumber(), (kh_max_depth-6.34))/2)^2,
2414 gcpy #         getxpos()-kh_distance,
2415 gcpy #     //end position at top of slot
2416 gcpy #         getypos()-tool_diameter(self.currenttoolnumber(), (
            kh_max_depth))/2,
2417 gcpy #         self.currenttoolnumber());
2418 gcpy #     hull(){
2419 gcpy #       translate([xpos(), ypos(), zpos()]){
2420 gcpy #         keyhole_shaft(6.35, 9.525);
2421 gcpy #       }
2422 gcpy #       translate([xpos(), ypos(), zpos()-kh_max_depth]){
2423 gcpy #         keyhole_shaft(6.35, 9.525);
2424 gcpy #       }
2425 gcpy #     }
2426 gcpy #     hull(){
2427 gcpy #       translate([xpos(), ypos(), zpos()-kh_max_depth]){
2428 gcpy #         keyhole_shaft(6.35, 9.525);
2429 gcpy #       }
2430 gcpy #       translate([xpos()-kh_distance, ypos(), zpos()-kh_max_depth])
            {
2431 gcpy #         keyhole_shaft(6.35, 9.525);
2432 gcpy #       }
2433 gcpy #     }
2434 gcpy #     cutwithfeed(getxpos(), getypos(), -kh_max_depth, feed);
2435 gcpy #     cutwithfeed(getxpos()-kh_distance, getypos(), -kh_max_depth,
            feed);
2436 gcpy #     setxpos(getxpos()+kh_distance);
2437 gcpy #   } else if (kh_angle == 270) {
2438 gcpy #     //Upper right of entry hole
2439 gcpy #     dxfarc(getxpos(), getypos(), 9.525/2, 0, 90, self.
            currenttoolnumber());
2440 gcpy #     //Upper left of entry hole
2441 gcpy #     dxfarc(getxpos(), getypos(), 9.525/2, 90, 180, self.
            currenttoolnumber());
2442 gcpy #     //lower right of slot
2443 gcpy #     dxfarc(getxpos(), getypos()-kh_distance, tool_diameter(self.
            currenttoolnumber(), (kh_max_depth+4.36))/2, 270, 360, self.
            currenttoolnumber());
2444 gcpy #     //lower left of slot
2445 gcpy #     dxfarc(getxpos(), getypos()-kh_distance, tool_diameter(self.
            currenttoolnumber(), (kh_max_depth+4.36))/2, 180, 270, self.
            currenttoolnumber());
2446 gcpy #     //Actual line of cut
2447 gcpy #     dxfline(getxpos(), getypos(), getxpos(), getypos()-kh_distance
            );
2448 gcpy #     //right of slot
2449 gcpy #     dxfline(
2450 gcpy #         getxpos()+tool_diameter(self.currenttoolnumber(), (
```

```
            kh_max_depth))/2,
2451 gcpy #         getypos()-(math.sqrt((tool_diameter(self.currenttoolnumber
            (), 1)^2)-(tool_diameter(self.currenttoolnumber(), 5)^2))/2),
            //( (kh_max_depth-6.34))/2)^2-(tool_diameter(self.
            currenttoolnumber(), (kh_max_depth-6.34))/2)^2,
2452 gcpy #         getxpos()+tool_diameter(self.currenttoolnumber(), (
            kh_max_depth))/2,
2453 gcpy #       //end position at top of slot
2454 gcpy #         getypos()-kh_distance,
2455 gcpy #         self.currenttoolnumber());
2456 gcpy #       //left of slot
2457 gcpy #       dxfline(
2458 gcpy #         getxpos()-tool_diameter(self.currenttoolnumber(), (
            kh_max_depth))/2,
2459 gcpy #         getypos()-(math.sqrt((tool_diameter(self.currenttoolnumber
            (), 1)^2)-(tool_diameter(self.currenttoolnumber(), 5)^2))/2),
            //( (kh_max_depth-6.34))/2)^2-(tool_diameter(self.
            currenttoolnumber(), (kh_max_depth-6.34))/2)^2,
2460 gcpy #         getxpos()-tool_diameter(self.currenttoolnumber(), (
            kh_max_depth))/2,
2461 gcpy #       //end position at top of slot
2462 gcpy #         getypos()-kh_distance,
2463 gcpy #         self.currenttoolnumber());
2464 gcpy #       //Lower right of entry hole
2465 gcpy #       dxfarc(getxpos(), getypos(), 9.525/2, 360-acos(tool_diameter(
            self.currenttoolnumber(), 5)/tool_diameter(self.
            currenttoolnumber(), 1)), 360, self.currenttoolnumber());
2466 gcpy #       //Lower left of entry hole
2467 gcpy #       dxfarc(getxpos(), getypos(), 9.525/2, 180, 180+acos(
            tool_diameter(self.currenttoolnumber(), 5)/tool_diameter(self.
            currenttoolnumber(), 1)), self.currenttoolnumber());
2468 gcpy #       hull(){
2469 gcpy #         translate([xpos(), ypos(), zpos()]){
2470 gcpy #           keyhole_shaft(6.35, 9.525);
2471 gcpy #         }
2472 gcpy #         translate([xpos(), ypos(), zpos()-kh_max_depth]){
2473 gcpy #           keyhole_shaft(6.35, 9.525);
2474 gcpy #         }
2475 gcpy #       }
2476 gcpy #       hull(){
2477 gcpy #         translate([xpos(), ypos(), zpos()-kh_max_depth]){
2478 gcpy #           keyhole_shaft(6.35, 9.525);
2479 gcpy #         }
2480 gcpy #         translate([xpos(), ypos()-kh_distance, zpos()-kh_max_depth])
            {
2481 gcpy #           keyhole_shaft(6.35, 9.525);
2482 gcpy #         }
2483 gcpy #       }
2484 gcpy #       cutwithfeed(getxpos(), getypos(), -kh_max_depth, feed);
2485 gcpy #       cutwithfeed(getxpos(), getypos()-kh_distance, -kh_max_depth,
            feed);
2486 gcpy #       setypos(getypos()+kh_distance);
2487 gcpy #     }
2488 gcpy #}
```

3.8.1.2 Joinery

3.8.1.2.1 Dovetail joinery and tooling One focus of this project from the beginning has been cutting joinery. The first such toolpath to be developed is half-blind dovetails, since they are intrinsically simple to calculate since their geometry is dictated by the geometry of the tool.

BlocksCAD project page at: https://www.blockscad3d.com/community/projects/1941456 and discussion at: https://community.carbide3d.com/t/tool-paths-for-different-sized-dovetail-bit 89098

Making such cuts will require dovetail tooling such as:

- 808079 https://www.amanatool.com/45828-carbide-tipped-dovetail-8-deg-x-1-2-dia-x-825-x-1 html
- 814 https://www.leevalley.com/en-us/shop/tools/power-tool-accessories/router-bits/ 30172-dovetail-bits?item=18J1607

Two commands are required:

```
2490 gcpy     def cut_pins(self, Joint_Width, stockZthickness,
                  Number_of_Dovetails, Spacing, Proportion, DTT_diameter,
                  DTT_angle):
```

```
2491 gcpy          DTO = Tan(DTT_angle) * (stockZthickness * Proportion)
2492 gcpy          DTR = DTT_diameter/2 - DTO
2493 gcpy          cpr = self.rapidXY(0, stockZthickness + Spacing/2)
2494 gcpy          ctp = self.cutlinedxfgc(self.xpos(), self.ypos(), -
                       stockZthickness * Proportion)
2495 gcpy #         ctp = ctp.union(self.cutlinedxfgc(Joint_Width / (
                   Number_of_Dovetails * 2), self.ypos(), -stockZthickness *
                   Proportion))
2496 gcpy          i = 1
2497 gcpy          while i < Number_of_Dovetails * 2:
2498 gcpy #             print(i)
2499 gcpy              ctp = ctp.union(self.cutlinedxfgc(i * (Joint_Width / (
                           Number_of_Dovetails * 2)), self.ypos(), -
                           stockZthickness * Proportion))
2500 gcpy              ctp = ctp.union(self.cutlinedxfgc(i * (Joint_Width / (
                           Number_of_Dovetails * 2)), (stockZthickness +
                           Spacing) + (stockZthickness * Proportion) - (
                           DTT_diameter/2), -(stockZthickness * Proportion)))
2501 gcpy              ctp = ctp.union(self.cutlinedxfgc(i * (Joint_Width / (
                           Number_of_Dovetails * 2)), stockZthickness + Spacing
                           /2, -(stockZthickness * Proportion)))
2502 gcpy              ctp = ctp.union(self.cutlinedxfgc((i + 1) * (
                           Joint_Width / (Number_of_Dovetails * 2)),
                           stockZthickness + Spacing/2,-(stockZthickness *
                           Proportion)))
2503 gcpy              self.dxfrectangleround(self.currenttoolnumber(),
2504 gcpy                  i * (Joint_Width / (Number_of_Dovetails * 2))-DTR,
2505 gcpy                  stockZthickness + (Spacing/2) - DTR,
2506 gcpy                  DTR * 2,
2507 gcpy                  (stockZthickness * Proportion) + Spacing/2 + DTR *
                               2 - (DTT_diameter/2),
2508 gcpy                  DTR)
2509 gcpy              i += 2
2510 gcpy          self.rapidZ(0)
2511 gcpy          return ctp
```

and

```
2513 gcpy      def cut_tails(self, Joint_Width, stockZthickness,
                   Number_of_Dovetails, Spacing, Proportion, DTT_diameter,
                   DTT_angle):
2514 gcpy          DTO = Tan(DTT_angle) * (stockZthickness * Proportion)
2515 gcpy          DTR = DTT_diameter/2 - DTO
2516 gcpy          cpr = self.rapidXY(0, 0)
2517 gcpy          ctp = self.cutlinedxfgc(self.xpos(), self.ypos(), -
                       stockZthickness * Proportion)
2518 gcpy          ctp = ctp.union(self.cutlinedxfgc(
2519 gcpy              Joint_Width / (Number_of_Dovetails * 2) - (DTT_diameter
                           - DTO),
2520 gcpy              self.ypos(),
2521 gcpy              -stockZthickness * Proportion))
2522 gcpy          i = 1
2523 gcpy          while i < Number_of_Dovetails * 2:
2524 gcpy              ctp = ctp.union(self.cutlinedxfgc(
2525 gcpy                  i * (Joint_Width / (Number_of_Dovetails * 2)) - (
                               DTT_diameter - DTO),
2526 gcpy                  stockZthickness * Proportion - DTT_diameter / 2,
2527 gcpy                  -(stockZthickness * Proportion)))
2528 gcpy              ctp = ctp.union(self.cutarcCWdxf(180, 90,
2529 gcpy                  i * (Joint_Width / (Number_of_Dovetails * 2)),
2530 gcpy                  stockZthickness * Proportion - DTT_diameter / 2,
2531 gcpy #                 self.ypos(),
2532 gcpy                  DTT_diameter - DTO,  0, 1))
2533 gcpy              ctp = ctp.union(self.cutarcCWdxf(90, 0,
2534 gcpy                  i * (Joint_Width / (Number_of_Dovetails * 2)),
2535 gcpy                  stockZthickness * Proportion - DTT_diameter / 2,
2536 gcpy                  DTT_diameter - DTO,  0, 1))
2537 gcpy              ctp = ctp.union(self.cutlinedxfgc(
2538 gcpy                  i * (Joint_Width / (Number_of_Dovetails * 2)) + (
                               DTT_diameter - DTO),
2539 gcpy                  0,
2540 gcpy                  -(stockZthickness * Proportion)))
2541 gcpy              ctp = ctp.union(self.cutlinedxfgc(
2542 gcpy                  (i + 2) * (Joint_Width / (Number_of_Dovetails * 2))
                               - (DTT_diameter - DTO),
2543 gcpy                  0,
2544 gcpy                  -(stockZthickness * Proportion)))
2545 gcpy              i += 2
```

```
2546 gcpy           self.rapidZ(0)
2547 gcpy           self.rapidXY(0, 0)
2548 gcpy           ctp = ctp.union(self.cutlinedxfgc(self.xpos(), self.ypos(),
                         -stockZthickness * Proportion))
2549 gcpy           self.dxfarc(0, 0, DTR, 180, 270)
2550 gcpy           self.dxfline(-DTR, 0, -DTR, stockZthickness + DTR)
2551 gcpy           self.dxfarc(0, stockZthickness + DTR, DTR, 90, 180)
2552 gcpy           self.dxfline(0, stockZthickness + DTR * 2, Joint_Width,
                       stockZthickness + DTR * 2)
2553 gcpy           i = 0
2554 gcpy           while i < Number_of_Dovetails * 2:
2555 gcpy               ctp = ctp.union(self.cutline(i * (Joint_Width / (
                           Number_of_Dovetails * 2)), stockZthickness + DTO, -(
                           stockZthickness * Proportion)))
2556 gcpy               ctp = ctp.union(self.cutline((i+2) * (Joint_Width / (
                           Number_of_Dovetails * 2)), stockZthickness + DTO, -(
                           stockZthickness * Proportion)))
2557 gcpy               ctp = ctp.union(self.cutline((i+2) * (Joint_Width / (
                           Number_of_Dovetails * 2)), 0, -(stockZthickness *
                           Proportion)))
2558 gcpy               self.dxfarc(i * (Joint_Width / (Number_of_Dovetails *
                           2)), 0, DTR, 270, 360)
2559 gcpy               self.dxfline(
2560 gcpy                   i * (Joint_Width / (Number_of_Dovetails * 2)) + DTR
                               ,
2561 gcpy                   0,
2562 gcpy                   i * (Joint_Width / (Number_of_Dovetails * 2)) + DTR
                               , stockZthickness * Proportion - DTT_diameter /
                               2)
2563 gcpy               self.dxfarc((i + 1) * (Joint_Width / (
                           Number_of_Dovetails * 2)), stockZthickness *
                           Proportion - DTT_diameter / 2, (Joint_Width / (
                           Number_of_Dovetails * 2)) - DTR, 90, 180)
2564 gcpy               self.dxfarc((i + 1) * (Joint_Width / (
                           Number_of_Dovetails * 2)), stockZthickness *
                           Proportion - DTT_diameter / 2, (Joint_Width / (
                           Number_of_Dovetails * 2)) - DTR, 0, 90)
2565 gcpy               self.dxfline(
2566 gcpy                   (i + 2) * (Joint_Width / (Number_of_Dovetails * 2))
                               - DTR,
2567 gcpy                   0,
2568 gcpy                   (i + 2) * (Joint_Width / (Number_of_Dovetails * 2))
                               - DTR, stockZthickness * Proportion -
                              DTT_diameter / 2)
2569 gcpy               self.dxfarc((i + 2) * (Joint_Width / (
                           Number_of_Dovetails * 2)), 0, DTR, 180, 270)
2570 gcpy               i += 2
2571 gcpy           self.dxfarc(Joint_Width, stockZthickness + DTR, DTR, 0, 90)
2572 gcpy           self.dxfline(Joint_Width + DTR, stockZthickness + DTR,
                       Joint_Width + DTR, 0)
2573 gcpy           self.dxfarc(Joint_Width, 0, DTR, 270, 360)
2574 gcpy           return ctp
```

which are used as:

```
toolpaths = gcp.cut_pins(stockXwidth, stockZthickness, Number_of_Dovetails, Spacing, Proportion, DTT_dia

toolpaths.append(gcp.cut_tails(stockXwidth, stockZthickness, Number_of_Dovetails, Spacing, Proportion,
```

Future versions may adjust the parameters passed in, having them calculate from the specifications for the currently active dovetail tool.

3.8.1.2.2 Full-blind box joints BlocksCAD project page at: https://www.blockscad3d.com/community/projects/1943966 and discussion at: https://community.carbide3d.com/t/full-blind-box-joints-in-carbide-create/53329

Full-blind box joints will require 3 separate tools:

- small V tool — this will be needed to make a cut along the edge of the joint
- small square tool — this should be the same diameter as the small V tool
- large V tool — this will facilitate the stock being of a greater thickness and avoid the need to make multiple cuts to cut the blind miters at the ends of the joint

`Full Blind Box Joint` There is one command `Full Blind Box Joint` which may be called for either orientation:

- horizontal
- vertical

```
2576 gcpy     def Full_Blind_Box_Joint(self, bx, by, orientation, side, width
                  , thickness, largeVdiameter, smallDiameter, squaretool =
                  102, smallV = 390, largeV = 301):
2577 gcpy         """
2578 gcpy ␣␣␣␣Cuts␣a␣full␣blind␣box␣joint.
2579 gcpy ␣␣␣␣Parameters:
2580 gcpy ␣␣␣␣bx␣(float):␣X-origin
2581 gcpy ␣␣␣␣by␣(float):␣Y-origin
2582 gcpy ␣␣␣␣orientation␣(string):␣"Vertical"␣or␣"Horizontal"
2583 gcpy ␣␣␣␣side␣(string):␣"Right"␣or␣"Left"␣(for␣Vertical),␣"Upper"␣or␣"
         Lower"␣(for␣Horizontal)␣---␣append␣"Mirror"␣to␣toggle␣number␣of␣
         pins
2584 gcpy ␣␣␣␣width␣(float):␣length␣of␣the␣joint
2585 gcpy ␣␣␣␣thickness␣(float):␣thickness␣of␣the␣stock
2586 gcpy ␣␣␣␣largeVdiameter␣(float):␣diameter␣of␣large␣V␣tool
2587 gcpy ␣␣␣␣smallDiameter␣(float):␣diameter␣of␣small␣V␣tool
2588 gcpy ␣␣␣␣squaretool
2589 gcpy ␣␣␣␣smallV
2590 gcpy ␣␣␣␣largeV
2591 gcpy ␣␣␣␣"""
2592 gcpy     # initial calculations
2593 gcpy         Number_of_Pins = int(((width - thickness * 2) / (
                     smallDiameter * 2.2) / 2) + 0.0) * 2 + 1
2594 gcpy         Finger_Width = ((Number_of_Pins * 2) - 1) * smallDiameter *
                      1.1
2595 gcpy         Finger_Origin = width/2 - Finger_Width/2
2596 gcpy     # change tool and set up for cut
2597 gcpy         self.toolchange(squaretool, 17000)
2598 gcpy         self.setdxfcolor("Cyan")
2599 gcpy         self.setdxflayer("Square␣-␣top␣depth")
2600 gcpy         self.movetosafeZ()
2601 gcpy         self.rapidXY(bx, by)
2602 gcpy         self.rapidZ(0)
2603 gcpy         if (orientation == "Vertical"):
2604 gcpy             if (side == "Both"):
2605 gcpy                     self.cutpocketrectanglerounddxf(
2606 gcpy                     bx - (thickness - smallDiameter/2),
2607 gcpy                     by-smallDiameter/2-smallDiameter/4,
2608 gcpy                     0,
2609 gcpy                     (thickness * 2) - smallDiameter,
2610 gcpy                     width+smallDiameter+smallDiameter/2,
2611 gcpy                     (smallDiameter / 2) / Tan(45),
2612 gcpy                     smallDiameter/2)
2613 gcpy             if (side == "Right" or side =="RightMirror"):
2614 gcpy                     self.cutpocketrectanglerounddxf(
2615 gcpy                     bx - (smallDiameter/2),
2616 gcpy                     by-smallDiameter/2-smallDiameter/4,
2617 gcpy                     0,
2618 gcpy                     thickness,
2619 gcpy                     width+smallDiameter+smallDiameter/2,
2620 gcpy                     ((smallDiameter / 2) / Tan(45)),
2621 gcpy                     smallDiameter/2)
2622 gcpy             if (side == "Left" or side == "LeftMirror"):
2623 gcpy                     self.cutpocketrectanglerounddxf(
2624 gcpy                     bx - (thickness - smallDiameter/2),
2625 gcpy                     by-smallDiameter/2-smallDiameter/4,
2626 gcpy                     0,
2627 gcpy                     thickness,
2628 gcpy                     width+smallDiameter+smallDiameter/2,
2629 gcpy                     ((smallDiameter / 2) / Tan(45)),
2630 gcpy                     smallDiameter/2)
2631 gcpy         if (orientation == "Horizontal"):
2632 gcpy             if (side == "Both"):
2633 gcpy                 self.cutpocketrectanglerounddxf(
2634 gcpy                     bx-smallDiameter/2-smallDiameter/4,
2635 gcpy                     by - (thickness - smallDiameter/2),
2636 gcpy                     0,
2637 gcpy                     width+smallDiameter+smallDiameter/2,
2638 gcpy                     (thickness * 2) - smallDiameter,
2639 gcpy                     (smallDiameter / 2) / Tan(45),
2640 gcpy                     smallDiameter/2)
2641 gcpy             if (side == "Upper" or side =="UpperMirror"):
2642 gcpy                 self.cutpocketrectanglerounddxf(
2643 gcpy                     bx-smallDiameter/2-smallDiameter/4,
2644 gcpy                     by - smallDiameter/2,
2645 gcpy                     0,
2646 gcpy                     width+smallDiameter+smallDiameter/2,
2647 gcpy                     thickness,
```

```
2648 gcpy                     ((smallDiameter / 2) / Tan(45)),
2649 gcpy                     smallDiameter/2)
2650 gcpy             if (side == "Lower" or side == "LowerMirror"):
2651 gcpy                 self.cutpocketrectanglerounddxf(
2652 gcpy                     bx-smallDiameter/2-smallDiameter/4,
2653 gcpy                     by - (thickness - smallDiameter/2),
2654 gcpy                     0,
2655 gcpy                     width+smallDiameter+smallDiameter/2,
2656 gcpy                     thickness,
2657 gcpy                     ((smallDiameter / 2) / Tan(45)),
2658 gcpy                     smallDiameter/2)
2659 gcpy     # continue with small square tool to cut fingers and central
                 channel for small V tool
2660 gcpy         self.setdxfcolor("Blue")
2661 gcpy         self.setdxflayer("Square␣-␣pocket␣depth")
2662 gcpy     # Joint_Orientation = "Horizontal" "Even" == "Upper", "Odd" ==
                 "Lower"
2663 gcpy     # Joint_Orientation = "Vertical" "Even" == "Right", "Odd" == "
                 Left"
2664 gcpy         jointdepth = -(thickness - (smallDiameter / 2) / Tan(45))
2665 gcpy         radius = smallDiameter/2
2666 gcpy         jointwidth = thickness - smallDiameter
2667 gcpy     #    toolpath = self.currenttool()
2668 gcpy         self.rapidXY(bx, by)
2669 gcpy         self.rapidZ(0)
2670 gcpy         if (orientation == "Vertical"):
2671 gcpy             self.cutlineZgcfeed(jointdepth,1000)
2672 gcpy             self.beginopenpolyline(bx, by)
2673 gcpy             self.cutpolylinedxf(bx, by + Finger_Origin, jointdepth)
2674 gcpy             i = 0
2675 gcpy             while i <= Number_of_Pins - 1:
2676 gcpy                 if (side == "Left"):
2677 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 smallDiameter + radius/5, jointdepth)
2678 gcpy                 if (side == "RightMirror"):
2679 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 smallDiameter + radius/5, jointdepth)
2680 gcpy                 if (side == "Right" or side == "Both"):
2681 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 radius, jointdepth)
2682 gcpy                     self.cutpolylinedxf(self.xpos() + jointwidth,
                                 self.ypos(), jointdepth)
2683 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 radius/5, jointdepth)
2684 gcpy                     self.cutpolylinedxf(self.xpos() - jointwidth,
                                 self.ypos(), jointdepth)
2685 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 radius, jointdepth)
2686 gcpy                 if (side == "Right"):
2687 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 smallDiameter + radius/5, jointdepth)
2688 gcpy                 if (side == "LeftMirror"):
2689 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 radius, jointdepth)
2690 gcpy                     self.cutpolylinedxf(self.xpos() - jointwidth,
                                 self.ypos(), jointdepth)
2691 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 radius/5, jointdepth)
2692 gcpy                     self.cutpolylinedxf(self.xpos() + jointwidth,
                                 self.ypos(), jointdepth)
2693 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 radius, jointdepth)
2694 gcpy                     self.cutpolylinedxf(self.xpos(), self.ypos() +
                                 smallDiameter + radius/5, jointdepth)
2695 gcpy                 if (i < (Number_of_Pins - 1)):
2696 gcpy                     if (side == "Left" or side == "Both"):
2697 gcpy                         self.cutpolylinedxf(self.xpos(), self.ypos
                                     () + radius, jointdepth)
2698 gcpy                         self.cutpolylinedxf(self.xpos() -
                                     jointwidth, self.ypos(), jointdepth)
2699 gcpy                         self.cutpolylinedxf(self.xpos(), self.ypos
                                     () + radius/5, jointdepth)
2700 gcpy                         self.cutpolylinedxf(self.xpos() +
                                     jointwidth, self.ypos(), jointdepth)
2701 gcpy                         self.cutpolylinedxf(self.xpos(), self.ypos
                                     () + radius, jointdepth)
2702 gcpy                     if (side == "RightMirror"):
2703 gcpy                         self.cutpolylinedxf(self.xpos(), self.ypos
```

```
                                () + radius, jointdepth)
2704 gcpy                       self.cutpolylinedxf(self.xpos() +
                                    jointwidth, self.ypos(), jointdepth)
2705 gcpy                       self.cutpolylinedxf(self.xpos(), self.ypos
                                    () + radius/5, jointdepth)
2706 gcpy                       self.cutpolylinedxf(self.xpos() -
                                    jointwidth, self.ypos(), jointdepth)
2707 gcpy                       self.cutpolylinedxf(self.xpos(), self.ypos
                                    () + radius, jointdepth)
2708 gcpy                   i += 1
2709 gcpy               self.cutpolylinedxf(bx, by + width, jointdepth)
2710 gcpy           if (orientation == "Horizontal"):
2711 gcpy               self.rapidXY(bx, by)
2712 gcpy               self.beginopenpolyline(bx, by)
2713 gcpy               self.cutpolylinedxf(bx + Finger_Origin, by, jointdepth)
2714 gcpy               self.cutlineZgcfeed(jointdepth,1000)
2715 gcpy               i = 0
2716 gcpy               while i <= Number_of_Pins - 1:
2717 gcpy                   if (side == "Lower"):
2718 gcpy                       self.cutpolylinedxf(self.xpos() + smallDiameter
                                    + radius/5, self.ypos(), jointdepth)
2719 gcpy                   if (side == "UpperMirror"):
2720 gcpy                       self.cutpolylinedxf(self.xpos() + smallDiameter
                                    + radius/5, self.ypos(), jointdepth)
2721 gcpy                   if (side == "Upper" or side == "Both"):
2722 gcpy                       self.cutpolylinedxf(self.xpos() + radius, self.
                                   ypos(), jointdepth)
2723 gcpy                       self.cutpolylinedxf(self.xpos(), self.ypos() +
                                    jointwidth, jointdepth)
2724 gcpy                       self.cutpolylinedxf(self.xpos() + radius/5,
                                    self.ypos(), jointdepth)
2725 gcpy                       self.cutpolylinedxf(self.xpos(), self.ypos() -
                                    jointwidth, jointdepth)
2726 gcpy                       self.cutpolylinedxf(self.xpos() + radius, self.
                                    ypos(), jointdepth)
2727 gcpy                   if (side == "LowerMirror"):
2728 gcpy                       self.cutpolylinedxf(self.xpos() + radius, self.
                                    ypos(), jointdepth)
2729 gcpy                       self.cutpolylinedxf(self.xpos(), self.ypos() -
                                    jointwidth, jointdepth)
2730 gcpy                       self.cutpolylinedxf(self.xpos() + radius/5,
                                    self.ypos(), jointdepth)
2731 gcpy                       self.cutpolylinedxf(self.xpos(), self.ypos() +
                                    jointwidth, jointdepth)
2732 gcpy                       self.cutpolylinedxf(self.xpos() + radius, self.
                                    ypos(), jointdepth)
2733 gcpy                   if (side == "Upper"):
2734 gcpy                       self.cutpolylinedxf(self.xpos() + smallDiameter
                                    + radius/5, self.ypos(), jointdepth)
2735 gcpy                   if (side == "LowerMirror"):
2736 gcpy                       self.cutpolylinedxf(self.xpos() + smallDiameter
                                    + radius/5, self.ypos(), jointdepth)
2737 gcpy                   if (i < (Number_of_Pins - 1)):
2738 gcpy                       if (side == "Lower" or side == "Both"):
2739 gcpy                           self.cutpolylinedxf(self.xpos() + radius,
                                       self.ypos(), jointdepth)
2740 gcpy                           self.cutpolylinedxf(self.xpos(), self.ypos
                                       () - jointwidth, jointdepth)
2741 gcpy                           self.cutpolylinedxf(self.xpos() + radius/5,
                                        self.ypos(), jointdepth)
2742 gcpy                           self.cutpolylinedxf(self.xpos(), self.ypos
                                       () + jointwidth, jointdepth)
2743 gcpy                           self.cutpolylinedxf(self.xpos() + radius,
                                       self.ypos(), jointdepth)
2744 gcpy                       if (side == "UpperMirror"):
2745 gcpy                           self.cutpolylinedxf(self.xpos() + radius,
                                       self.ypos(), jointdepth)
2746 gcpy                           self.cutpolylinedxf(self.xpos(), self.ypos
                                       () + jointwidth, jointdepth)
2747 gcpy                           self.cutpolylinedxf(self.xpos() + radius/5,
                                        self.ypos(), jointdepth)
2748 gcpy                           self.cutpolylinedxf(self.xpos(), self.ypos
                                       () - jointwidth, jointdepth)
2749 gcpy                           self.cutpolylinedxf(self.xpos() + radius,
                                       self.ypos(), jointdepth)
2750 gcpy                   i += 1
2751 gcpy               self.cutpolylinedxf(bx + width, by, jointdepth)
2752 gcpy           self.closepolyline()
```

```
2753 gcpy          self.movetosafeZ()
2754 gcpy      # switch to Large V to cut along interior edge and miters at
                   begin/end of joint area
2755 gcpy          self.toolchange(largeV, 17000)
2756 gcpy          self.rapidXY(bx, by)
2757 gcpy          self.rapidZ(0)
2758 gcpy          self.setdxfcolor("Dark␣Gray")
2759 gcpy          self.setdxflayer("Large␣V␣-␣top␣depth")
2760 gcpy          if (orientation == "Vertical"):
2761 gcpy              if (side == "Right" or side == "Both" or side == "
                       RightMirror"):
2762 gcpy                  self.rapidZ(0)
2763 gcpy                  self.rapidXY(bx+thickness-(smallDiameter / 2) / Tan
                           (45), by - radius/2)
2764 gcpy                  self.cutlineZgcfeed(-(smallDiameter / 2) / Tan(45)
                           ,10000)
2765 gcpy                  self.cutlinedxfgc(bx+thickness-(smallDiameter / 2)
                           / Tan(45), by + width + radius/2, -(
                           smallDiameter / 2) / Tan(45))
2766 gcpy                  self.rapidZ(0)
2767 gcpy              if (side == "Left" or side == "Both"):
2768 gcpy                  self.rapidZ(0)
2769 gcpy                  self.rapidXY(bx-(thickness-(smallDiameter / 2) /
                           Tan(45)), by - radius/2)
2770 gcpy                  self.cutlineZgcfeed(-(smallDiameter / 2) / Tan(45)
                           ,10000)
2771 gcpy                  self.cutlinedxfgc(bx-(thickness-(smallDiameter / 2)
                            / Tan(45)), by + width + radius/2, -(
                           smallDiameter / 2) / Tan(45))
2772 gcpy                  self.rapidZ(0)
2773 gcpy          if (orientation == "Horizontal"):
2774 gcpy              if (side == "Upper" or side == "Both" or side == "
                       UpperMirror"):
2775 gcpy                  self.rapidZ(0)
2776 gcpy                  self.rapidXY(bx - radius, by+thickness-(
                           smallDiameter / 2) / Tan(45))
2777 gcpy                  self.cutlineZgcfeed(-(smallDiameter / 2) / Tan(45)
                           ,10000)
2778 gcpy                  self.cutlinedxfgc(bx + width + radius, by+thickness
                           -(smallDiameter / 2) / Tan(45), -(smallDiameter
                           / 2) / Tan(45))
2779 gcpy                  self.rapidZ(0)
2780 gcpy              if (side == "Lower" or side == "Both" or side == "
                       LowerMirror"):
2781 gcpy                  self.rapidZ(0)
2782 gcpy                  self.rapidXY(bx - radius, by-(thickness-(
                           smallDiameter / 2) / Tan(45)))
2783 gcpy                  self.cutlineZgcfeed(-(smallDiameter / 2) / Tan(45)
                           ,10000)
2784 gcpy                  self.cutlinedxfgc(bx + width + radius, by-(
                           thickness-(smallDiameter / 2) / Tan(45)), -(
                           smallDiameter / 2) / Tan(45))
2785 gcpy                  self.rapidZ(0)
2786 gcpy          self.setdxfcolor("Magenta")
2787 gcpy          self.setdxflayer("Large␣V␣-␣half␣thickness")
2788 gcpy          if (orientation == "Vertical"):
2789 gcpy              if (side == "Right" or side == "Both" or side == "
                       RightMirror"):
2790 gcpy                  self.rapidZ(0)
2791 gcpy                  self.rapidXY(bx+thickness/2, by+width)
2792 gcpy                  self.cutlineZgcfeed(-thickness/2,1000)
2793 gcpy                  self.cutlinedxfgc(bx+thickness/2, by + width -
                           thickness, -thickness/2)
2794 gcpy                  self.rapidZ(0)
2795 gcpy                  self.rapidXY(bx+thickness/2, by)
2796 gcpy                  self.cutlineZgcfeed(-thickness/2,1000)
2797 gcpy                  self.cutlinedxfgc(bx+thickness/2, by +thickness, -
                           thickness/2)
2798 gcpy                  self.rapidZ(0)
2799 gcpy              if (side == "Left" or side == "Both" or side == "
                       LeftMirror"):
2800 gcpy                  self.rapidZ(0)
2801 gcpy                  self.rapidXY(bx-thickness/2, by+width)
2802 gcpy                  self.cutlineZgcfeed(-thickness/2,1000)
2803 gcpy                  self.cutlinedxfgc(bx-thickness/2, by + width -
                           thickness, -thickness/2)
2804 gcpy                  self.rapidZ(0)
2805 gcpy                  self.rapidXY(bx-thickness/2, by)
```

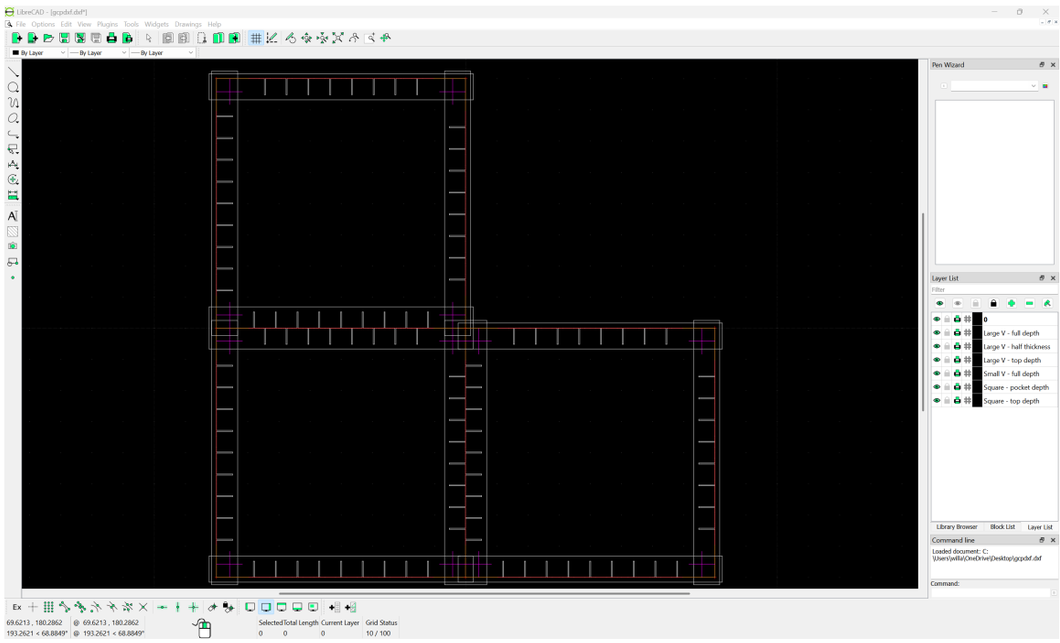
```
2806 gcpy                 self.cutlineZgcfeed(-thickness/2,1000)
2807 gcpy                 self.cutlinedxfgc(bx-thickness/2, by +thickness, -
                              thickness/2)
2808 gcpy                 self.rapidZ(0)
2809 gcpy             if (orientation == "Horizontal"):
2810 gcpy                 if (side == "Lower" or side == "Both" or side == "
                              LowerMirror"):
2811 gcpy                     self.rapidZ(0)
2812 gcpy                     self.rapidXY(bx+width, by-thickness/2)
2813 gcpy                     self.cutlineZgcfeed(-thickness/2,1000)
2814 gcpy                     self.cutlinedxfgc(bx + width -thickness, by-
                                  thickness/2, -thickness/2)
2815 gcpy                     self.rapidZ(0)
2816 gcpy                     self.rapidXY(bx, by-thickness/2)
2817 gcpy                     self.cutlineZgcfeed(-thickness/2,1000)
2818 gcpy                     self.cutlinedxfgc(bx +thickness, by-thickness/2, -
                                  thickness/2)
2819 gcpy                     self.rapidZ(0)
2820 gcpy                 if (side == "Upper" or side == "Both" or side == "
                              UpperMirror"):
2821 gcpy                     self.rapidZ(0)
2822 gcpy                     self.rapidXY(bx+width, by+thickness/2)
2823 gcpy                     self.cutlineZgcfeed(-thickness/2,1000)
2824 gcpy                     self.cutlinedxfgc(bx + width -thickness, by+
                                  thickness/2, -thickness/2)
2825 gcpy                     self.rapidZ(0)
2826 gcpy                     self.rapidXY(bx, by+thickness/2)
2827 gcpy                     self.cutlineZgcfeed(-thickness/2,1000)
2828 gcpy                     self.cutlinedxfgc(bx +thickness, by+thickness/2, -
                                  thickness/2)
2829 gcpy                     self.rapidZ(0)
2830 gcpy         ## Joint_Orientation = "Horizontal" "Even" == "Upper", "Odd" ==
                      "Lower"
2831 gcpy             self.setdxfcolor("Green")
2832 gcpy             self.setdxflayer("Large␣V␣-␣full␣depth")
2833 gcpy             if (orientation == "Horizontal"):
2834 gcpy                 self.rapidZ(0)
2835 gcpy                 self.rapidXY(bx, by)
2836 gcpy                 self.cutlineZgcfeed(-thickness,1000)
2837 gcpy                 self.cutlinedxfgc(bx + Finger_Origin, by, -thickness)
2838 gcpy                 self.rapidZ(0)
2839 gcpy                 self.rapidXY(bx + width - Finger_Origin, by)
2840 gcpy                 self.cutlineZgcfeed(-thickness,1000)
2841 gcpy                 self.cutlinedxfgc(bx + width, by, -thickness)
2842 gcpy                 self.rapidZ(0)
2843 gcpy             if (orientation == "Vertical"):
2844 gcpy                 self.rapidZ(0)
2845 gcpy                 self.rapidXY(bx, by)
2846 gcpy                 self.cutlineZgcfeed(-thickness,1000)
2847 gcpy                 self.cutlinedxfgc(bx, by + Finger_Origin, -thickness)
2848 gcpy                 self.rapidZ(0)
2849 gcpy                 self.rapidXY(bx, by + width - Finger_Origin)
2850 gcpy                 self.cutlineZgcfeed(-thickness,1000)
2851 gcpy                 self.cutlinedxfgc(bx, by + width, -thickness)
2852 gcpy                 self.rapidZ(0)
2853 gcpy             self.movetosafeZ()
2854 gcpy         # switch to smallV tool to cut V channel along joint edge at
                      bottom of stock
2855 gcpy             self.toolchange(smallV)
2856 gcpy             self.rapidXY(bx, by)
2857 gcpy             self.rapidZ(0)
2858 gcpy             self.setdxfcolor("Red")
2859 gcpy             self.setdxflayer("Small␣V␣-␣full␣depth")
2860 gcpy             if (orientation == "Vertical"):
2861 gcpy                 self.rapidXY(bx, by - smallDiameter/6)
2862 gcpy                 self.cutlineZgcfeed(-thickness,1000)
2863 gcpy                 self.cutlinedxfgc(bx, by + width + smallDiameter/6, -
                              thickness)
2864 gcpy             if (orientation == "Horizontal"):
2865 gcpy                 self.rapidXY(bx - smallDiameter/6, by)
2866 gcpy                 self.cutlineZgcfeed(-thickness,1000)
2867 gcpy                 self.cutlinedxfgc(bx + width + smallDiameter/6, by, -
                              thickness)
2868 gcpy             self.movetosafeZ()
```

Examples of usage:

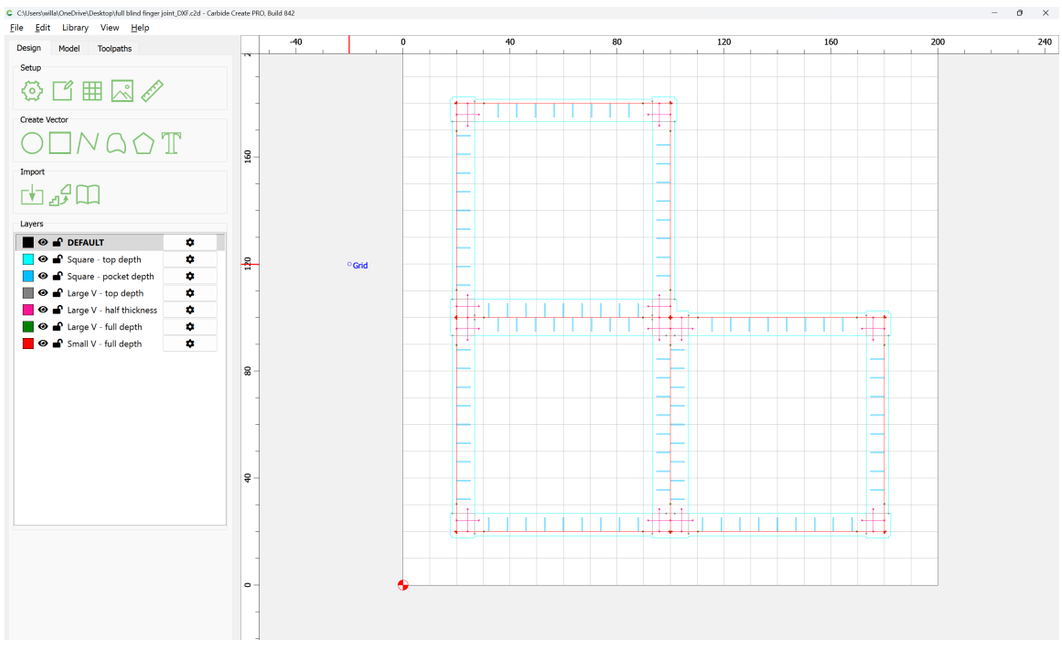
```
1 gcptmpl gcp.Full_Blind_Box_Joint(100, 20, "Vertical", "Both", Joint_Width,
```

```
            stockZthickness, largeVdiameter, smallDiameter)
gcptmpl
gcptmpl gcp.Full_Blind_Box_Joint(20, 20, "Vertical", "Left", Joint_Width,
            stockZthickness, largeVdiameter, smallDiameter)
gcptmpl
gcptmpl gcp.Full_Blind_Box_Joint(20, 100, "Vertical", "Left", Joint_Width,
            stockZthickness, largeVdiameter, smallDiameter)
gcptmpl
gcptmpl gcp.Full_Blind_Box_Joint(180, 20, "Vertical", "Right", Joint_Width,
             stockZthickness, largeVdiameter, smallDiameter)
gcptmpl
gcptmpl gcp.Full_Blind_Box_Joint(100, 100, "Vertical", "Right", Joint_Width
            , stockZthickness, largeVdiameter, smallDiameter)
gcptmpl
gcptmpl # Joint_Orientation = "Horizontal" "Even" == "Lower", "Odd" == "
            Upper"
gcptmpl
gcptmpl gcp.Full_Blind_Box_Joint(20, 20, "Horizontal", "Lower", Joint_Width
            , stockZthickness, largeVdiameter, smallDiameter)
gcptmpl
gcptmpl gcp.Full_Blind_Box_Joint(20, 100, "Horizontal", "Both", Joint_Width
            , stockZthickness, largeVdiameter, smallDiameter)
gcptmpl
gcptmpl gcp.Full_Blind_Box_Joint(20, 180, "Horizontal", "Upper",
            Joint_Width, stockZthickness, largeVdiameter, smallDiameter)
gcptmpl
gcptmpl gcp.Full_Blind_Box_Joint(100, 100, "Horizontal", "Upper",
            Joint_Width, stockZthickness, largeVdiameter, smallDiameter)
gcptmpl
gcptmpl gcp.Full_Blind_Box_Joint(100, 20, "Horizontal", "Lower",
            Joint_Width, stockZthickness, largeVdiameter, smallDiameter)
```

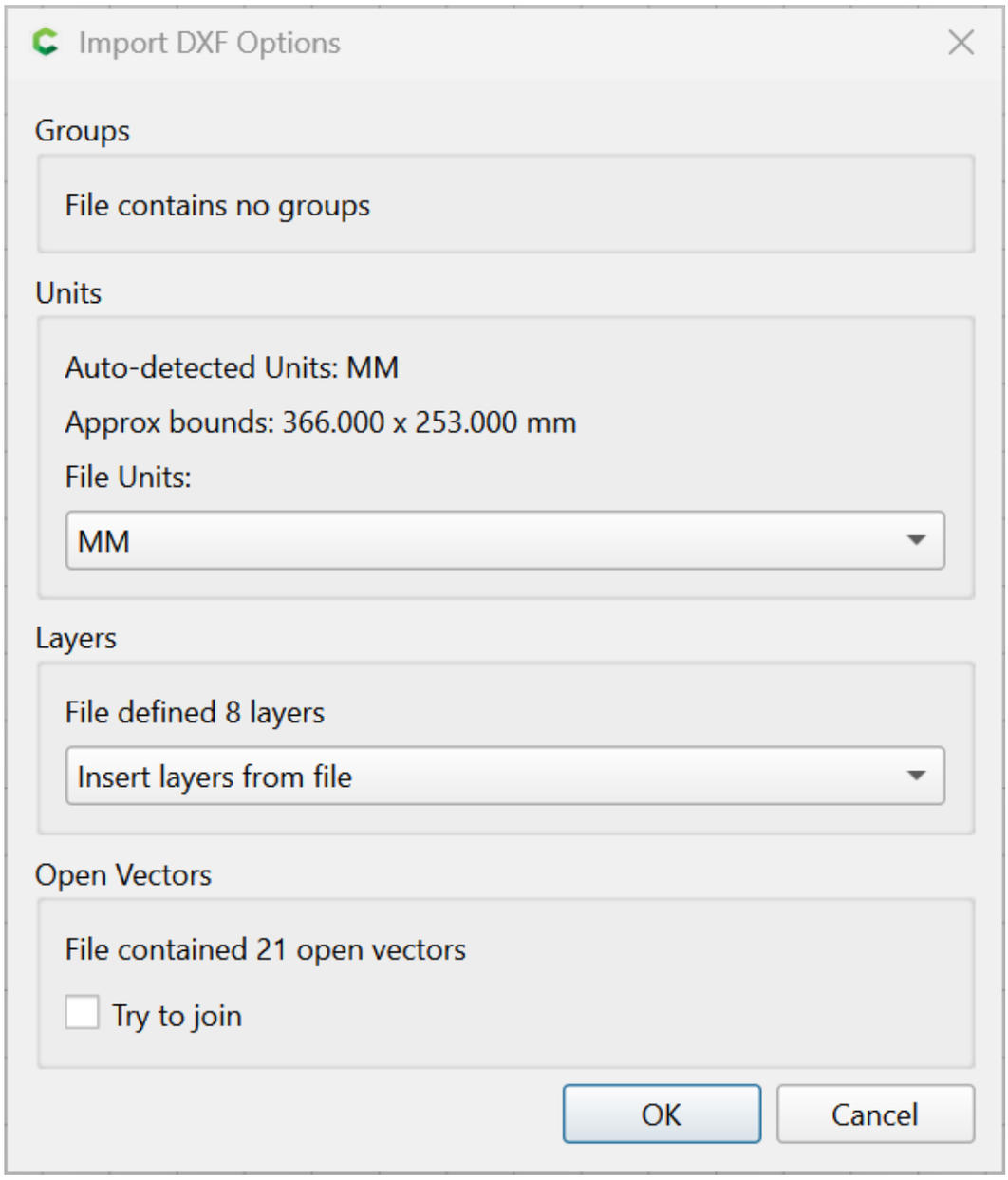
It is intended that it be used to generate a DXF:



which may then be imported into a Carbide Create template file which has matching layers and toolpaths:



Note that when importing the DXF it will be necessary to clear the checkbox for



Further note, it will be necessary to double-click on layer “Square - top depth” to make it active, use the Layer menu to select everything on that layer, then use Boolean Union to create geometry on it which will work as expected for the associated toolpath.

3.8.1.3 Fonts While OpenSCAD, and by extension PythonSCAD support the use of fonts, the requirements of typography and OpenType/TrueType fonts do not align with the limitations of CNC, making the development of a mechanism for directly creating the stroke geometry of single line fonts desirable.

An initial version of only capital letters and numerals comprised of lines and arcs which a circle (representing the tool or extruded plastic) is either `hull()`ed or directly set and `difference()`d (making the same weight stroke) is a suitable trial run for later more complex designs using curves. Such a font would have the following parameters:

- Size — the height of glyphs
- Width — this would adjust from the narrowest possible design through a "normal" version where the *O* was a circle terminating in an expanded version with glyphs extended horizontally
- Weight — the radius/diameter of the circle used for `hull()` operations, or the dimension arrived at when `difference()`ing circles
- Spacing — adjustment for the space between glyphs
- overshoot — optical adjustment for narrow strokes/elements
- roundover — further adjustment for applying overshoot to larger rounded forms

Rather than directly program these, drawing them on a grid in a vector design program which permits changing stroke weight globally is the most expedient option, then the dimensions may be copied to create the letterforms.

3.9 (Reading) G-code Files

With all other features in place, it becomes possible to read in a G-code file and then create a 3D preview of how it will cut.

First, a template file will be necessary:

```
1 gcpgcpy #Requires OpenPythonSCAD, so load support for 3D modeling in that
          tool:
2 gcpgcpy from openscad import *
3 gcpgcpy
4 gcpgcpy #The gcodepreview library must be loaded, either from github (first
           line below) or from a local library (second line below),
          uncomment one and comment out the other, depending on where one
          wishes to load from
5 gcpgcpy #nimport("https://raw.githubusercontent.com/WillAdams/gcodepreview/
          refs/heads/main/gcodepreview.py")
6 gcpgcpy from gcodepreview import *
7 gcpgcpy
8 gcpgcpy #The file to be loaded must be specified:
9 gcpgcpy #gc_file = "filename_of_G-code_file_to_process.gcodefilext"
10 gcpgcpy #
11 gcpgcpy #if using windows the full filepath should be provided with
          backslashes replaced with double backslashes and wrapped in
          quotes since it is provided as a string:
12 gcpgcpy gc_file = "C:\\Users\\willa\\OneDrive\\Desktop\\19mm_1_32_depth.nc"
13 gcpgcpy
14 gcpgcpy #Create the gcodepreview object:
15 gcpgcpy gcp = gcodepreview("cut", False, False)
16 gcpgcpy
17 gcpgcpy #Process the file
18 gcpgcpy gcp.previewgcodefile(gc_file)
```

previewgcodefile Which simply needs to call the `previewgcodefile` command:

```
2870 gcpy     def previewgcodefile(self, gc_file):
2871 gcpy         gc_file = open(gc_file, 'r')
2872 gcpy         gcfilecontents = []
2873 gcpy         with gc_file as file:
2874 gcpy             for line in file:
2875 gcpy                 command = line
2876 gcpy                 gcfilecontents.append(line)
2877 gcpy
2878 gcpy         numlinesfound = 0
2879 gcpy         for line in gcfilecontents:
2880 gcpy #            print(line)
2881 gcpy             if line[:10] == "(stockMin:":
2882 gcpy                 subdivisions = line.split()
2883 gcpy                 extentleft = float(subdivisions[0][10:-3])
2884 gcpy                 extentfb = float(subdivisions[1][:-3])
```

```
2885 gcpy                 extentd = float(subdivisions[2][:-3])
2886 gcpy                 numlinesfound = numlinesfound + 1
2887 gcpy             if line[:13] == "(STOCK/BLOCK,":
2888 gcpy                 subdivisions = line.split()
2889 gcpy                 sizeX = float(subdivisions[0][13:-1])
2890 gcpy                 sizeY = float(subdivisions[1][:-1])
2891 gcpy                 sizeZ = float(subdivisions[4][:-1])
2892 gcpy                 numlinesfound = numlinesfound + 1
2893 gcpy             if line[:3] == "G21":
2894 gcpy                 units = "mm"
2895 gcpy                 numlinesfound = numlinesfound + 1
2896 gcpy             if numlinesfound >=3:
2897 gcpy                 break
2898 gcpy # print(numlinesfound)
```

Once the initial parameters are parsed, the stock may be set up:

```
2900 gcpy         self.setupcuttingarea(sizeX, sizeY, sizeZ, extentleft,
                     extentfb, extentd)
2901 gcpy
2902 gcpy         commands = []
2903 gcpy         for line in gcfilecontents:
2904 gcpy             Xc = 0
2905 gcpy             Yc = 0
2906 gcpy             Zc = 0
2907 gcpy             Fc = 0
2908 gcpy             Xp = 0.0
2909 gcpy             Yp = 0.0
2910 gcpy             Zp = 0.0
2911 gcpy             if line == "G53G0Z-5.000\n":
2912 gcpy                 self.movetosafeZ()
2913 gcpy             if line[:3] == "M6T":
2914 gcpy                 tool = int(line[3:])
2915 gcpy                 self.toolchange(tool)
```

Processing tool changes will require examining lines such as:

```
;TOOL/MILL, Diameter, Corner radius, Height, Taper Angle

;TOOL/CRMILL, Diameter1, Diameter2, Radius, Height, Length

;TOOL/CHAMFER, Diameter, Point Angle, Height
```

which once parsed will be passed to a command which uses them to set the variables necessary to effect the toolchange:

```
if line[:11] == "(TOOL/MILL,"
    subdivisions = line.split()
    diameter = float(subdivisions[1][:-3])
    cornerradius = float(subdivisions[2][:-3])
    height = float(subdivisions[3][:-3])
    taperangle = float(subdivisions[4][:-3])
    self.settoolparameters("mill", diameter, cornerradius, height, taperangle)

if line[:14] == "(TOOL/CHAMFER,"
    subdivisions = line.split()
    tipdiameter = float(subdivisions[1][:-3])
    diameter = float(subdivisions[2][:-3])
    radius = float(subdivisions[3][:-3])
    height = float(subdivisions[4][:-3])
    length = float(subdivisions[4][:-3])
    self.settoolparameters("chamfer", tipdiameter, diameter, radius, height, length)
```

```
2916 gcpy             if line[:2] == "G0":
2917 gcpy                 machinestate = "rapid"
2918 gcpy             if line[:2] == "G1":
2919 gcpy                 machinestate = "cutline"
2920 gcpy             if line[:2] == "G0" or line[:2] == "G1" or line[:1] ==
                         "X" or line[:1] == "Y" or line[:1] == "Z":
2921 gcpy                 if "F" in line:
2922 gcpy                     Fplus = line.split("F")
2923 gcpy                     Fc = 1
2924 gcpy                     fr = float(Fplus[1])
2925 gcpy                     line = Fplus[0]
2926 gcpy                 if "Z" in line:
2927 gcpy                     Zplus = line.split("Z")
```

```
2928 gcpy                    Zc = 1
2929 gcpy                    Zp = float(Zplus[1])
2930 gcpy                    line = Zplus[0]
2931 gcpy                if "Y" in line:
2932 gcpy                    Yplus = line.split("Y")
2933 gcpy                    Yc = 1
2934 gcpy                    Yp = float(Yplus[1])
2935 gcpy                    line = Yplus[0]
2936 gcpy                if "X" in line:
2937 gcpy                    Xplus = line.split("X")
2938 gcpy                    Xc = 1
2939 gcpy                    Xp = float(Xplus[1])
2940 gcpy                if Zc == 1:
2941 gcpy                    if Yc == 1:
2942 gcpy                        if Xc == 1:
2943 gcpy                            if machinestate == "rapid":
2944 gcpy                                command = "rapidXYZ(" + str(Xp) + "
                                        ,␣" + str(Yp) + ",␣" + str(Zp) +
                                         ")"
2945 gcpy                                self.rapidXYZ(Xp, Yp, Zp)
2946 gcpy                            else:
2947 gcpy                                command = "cutlineXYZ(" + str(Xp) +
                                         ",␣" + str(Yp) + ",␣" + str(Zp)
                                         + ")"
2948 gcpy                                self.cutlineXYZ(Xp, Yp, Zp)
2949 gcpy                        else:
2950 gcpy                            if machinestate == "rapid":
2951 gcpy                                command = "rapidYZ(" + str(Yp) + ",
                                        ␣" + str(Zp) + ")"
2952 gcpy                                self.rapidYZ(Yp, Zp)
2953 gcpy                            else:
2954 gcpy                                command = "cutlineYZ(" + str(Yp) +
                                        ",␣" + str(Zp) + ")"
2955 gcpy                                self.cutlineYZ(Yp, Zp)
2956 gcpy                    else:
2957 gcpy                        if Xc == 1:
2958 gcpy                            if machinestate == "rapid":
2959 gcpy                                command = "rapidXZ(" + str(Xp) + ",
                                        ␣" + str(Zp) + ")"
2960 gcpy                                self.rapidXZ(Xp, Zp)
2961 gcpy                            else:
2962 gcpy                                command = "cutlineXZ(" + str(Xp) +
                                        ",␣" + str(Zp) + ")"
2963 gcpy                                self.cutlineXZ(Xp, Zp)
2964 gcpy                        else:
2965 gcpy                            if machinestate == "rapid":
2966 gcpy                                command = "rapidZ(" + str(Zp) + ")"
2967 gcpy                                self.rapidZ(Zp)
2968 gcpy                            else:
2969 gcpy                                command = "cutlineZ(" + str(Zp) + "
                                        )"
2970 gcpy                                self.cutlineZ(Zp)
2971 gcpy                else:
2972 gcpy                    if Yc == 1:
2973 gcpy                        if Xc == 1:
2974 gcpy                            if machinestate == "rapid":
2975 gcpy                                command = "rapidXY(" + str(Xp) + ",
                                        ␣" + str(Yp) + ")"
2976 gcpy                                self.rapidXY(Xp, Yp)
2977 gcpy                            else:
2978 gcpy                                command = "cutlineXY(" + str(Xp) +
                                        ",␣" + str(Yp) + ")"
2979 gcpy                                self.cutlineXY(Xp, Yp)
2980 gcpy                        else:
2981 gcpy                            if machinestate == "rapid":
2982 gcpy                                command = "rapidY(" + str(Yp) + ")"
2983 gcpy                                self.rapidY(Yp)
2984 gcpy                            else:
2985 gcpy                                command = "cutlineY(" + str(Yp) + "
                                        )"
2986 gcpy                                self.cutlineY(Yp)
2987 gcpy                    else:
2988 gcpy                        if Xc == 1:
2989 gcpy                            if machinestate == "rapid":
2990 gcpy                                command = "rapidX(" + str(Xp) + ")"
2991 gcpy                                self.rapidX(Xp)
2992 gcpy                            else:
2993 gcpy                                command = "cutlineX(" + str(Xp) + "
```

```
                                                )"
2994 gcpy                                     self.cutlineX(Xp)
2995 gcpy                    commands.append(command)
2996 gcpy #                  print(line)
2997 gcpy #                  print(command)
2998 gcpy #                  print(machinestate, Xc, Yc, Zc)
2999 gcpy #                  print(Xp, Yp, Zp)
3000 gcpy #                  print("/n")
3001 gcpy
3002 gcpy #          for command in commands:
3003 gcpy #              print(command)
3004 gcpy
3005 gcpy #          show(self.stockandtoolpaths())
3006 gcpy           self.stockandtoolpaths()
```

3.10 Parts, Assemblies, Cut-Lists, and Views

With all the digital tooling in place to make a given part, extending the tooling to projects which involve multiple parts becomes the next obvious step. For this, it is necessary to have a facility to manage the parts individually and in concert and to manipulate them so as to assemble and view them in ways appropriate to the project. Furniture has a long history in woodworking for this, and a number of standardized views, the arrangement and usage of which will require certain basic capabilities and parameters:

- Part
  - Orientation (cut, right/left, front/back) relative to the compleated project — note that parts such as lids may require a sub-rotation as defined by State (see below)
  - Component Level (0.000) where this could be iterated in a loop, allowing for animations
  - Displacement (X, Y, Z) relative to the origin, lower-left for a cut-list (and with an added need for an identifier for a given board/sheet) or from the center for an assembled view
  - State (open/closed, 00.00%) expressing this as a numerical value will allow stepping through this for animations
- Assembly — this will require the parts be arranged relative to the center origin (Displacement) and rotated (Orientation) and further adjusted per settings of the project such as lids or drawers being opened or closed (State)
- Cut-list

With those parameters for a part in place, it is then possible to implement the different types of images and diagrams used for a woodworking plan. Typically, a woodworking plan uses a coordinated set of image and diagram types, each serving a distinct purpose in communicating dimensions, joinery, structure, and assembly. Typically orthographic views were the primary "working drawings" which communicate dimensions and relationships between parts.

- Front View (Elevation) Shows height and width
- Top View (Plan View) Shows width and depth
- Side/End View Shows depth and height from the left or right side
- Back View Used when rear details differ from the front

with the addition of views such as Section Views (cutaway diagrams) which would reveal interior structure, joinery, and hidden features and may be of the full/compleat project, or partial/detail views of specific sections/features/aspects of the design, as well as Detail Drawings (zoomed-in technical illustrations) which would isolate small but critical features such as:

- Joinery Details — Mortise-and-tenon, dovetails, dados, rabbets, etc.
- Hardware Details — Hinges, drawer slides, fasteners
- Profile Details — Moldings, edge treatments, decorative elements

Detail drawings ensure precise fabrication of holes or recesses for hardware or other complex or ornamental portions of a design part.

Additionally, in the past, there might have been exploded Views (assembly diagrams) which would show how parts would fit together by separating components spatially while maintaining alignment and orientation/rotation and Shop Drawings or Cut-lists which would facilitate production, or even full-size Patterns or Templates.

The interactivity of a computer system affords two notable views:

- 2D used for Cut-lists and CNC-ready DXF or G-code files

- 3D which would want controls for showing an individual part or assembly or sub-assembly, and the spacing of the various components

Note that it might be desirable to have hybrid views, such as a 2D arrangement of parts for cutting which are shown as 3D previews of how they would be cut, or a 3D view in wireframe which allows one to slice and focus on a specific 2D plane.

The above then suggests a pattern for a given part which will set parameters controlling:

- Position of part in 2D space (as when cut)
- Orientation/rotation of part (front/back, left/right, top/bottom)
- Displacement of part in 3D space when assembled/shown in an exploded-view
- Component Level — from center/unmoving (0) through outermost part (99), note that this has an implicit decimal component where the decimal portion would be used to control positioning of hardware used to secure a given Component Level
- State — open (100)/closed (0) which allows a percentage based animation
- View — a description of the camera position relative to the part which could be used in an animation to allow a rotating or other shifting viewpoint

The most prosaic version would be to include all of these in the module definition....

```
def examplepart (pxpos, pypos, pzpos, # position in 2D space when cutting
                 porr, # orientation/rotation front/back, left/right, top/bottom
                 pdisx, pdisy, pdisz, # view displacement of part in 3D space
                 pcl, # component level 0.0 where decimal is for hardware
                 pstate, # range from open (100) closed (0)
                 pview): # camera position
```

with the implementation of the various views left to the user at this time/version.

A workable option might be to make that as a wrapper module which takes a previously defined cut part and outputs that part based on the parameters, or make it a general module which includes a parameter for enumerating a specific part or range of parts or the entirety of the design.

A further potential complication is the possibility of nested parts/assemblies, such as a writing desk with a drawer and a locking mechanism (or hidden compartment).

The command `returnstockandtoolpaths` allows one to access the 3D model which may then be `intersection`ed with a cube of the part dimensions and shifted in space as desired. This will require that the entirety of the design be laid out in a cut view:

```
#BOTTOM
btXorigin = 10
btYorigin = 10

gcp.Full_Blind_Box_Joint(btXorigin, btYorigin, "Vertical", "Left", Box_Width, stockZthickness, largeVdia

gcp.Full_Blind_Box_Joint(btXorigin, btYorigin, "Horizontal", "Lower", Box_Depth, stockZthickness, large

gcp.Full_Blind_Box_Joint(btXorigin, btYorigin + Box_Width, "Horizontal", "Upper", Box_Depth, stockZthick

gcp.movetosafeZ()

gcp.toolchange(102, 17000)

gcp.setdxflayer("Square - full depth")

gcp.cutrectanglerounddxf(btXorigin + Box_Depth, btYorigin-smallDiameter/2, 0, smallDiameter*1.1, Box_Wi
```

with the positions of each cut part noted so that it may then be re-used:

```
bt = gcp.returnstockandtoolpaths()

cb = cube([Box_Depth, Box_Width, stockZthickness])

cu = cb.translate([btXorigin,btYorigin,-stockZthickness])

bottom = bt.intersection(cu)

#show(bottom)
```

The optimal setup for such modules is still a matter for further research and testing.

4 Notes

4.1 Other Resources

4.1.1 Mathematics

https://www.2dcurves.com/
https://elementsofprogramming.com/
Note that Bézier curves are listed below.

4.1.2 Coding Style

A notable influence on the coding style in this project is John Ousterhout's *A Philosophy of Software Design*[SoftwareDesign]. Complexity is managed by the overall design and structure of the code, structuring it so that each component may be worked with on an individual basis, hiding the maximum information, and exposing the maximum functionality, with names selected so as to express their functionality/usage.

Red Flags to avoid include:

- Shallow Module
- Information Leakage
- Temporal Decomposition
- Overexposure
- Pass-Through Method
- Repetition
- Special-General Mixture
- Conjoined Methods
- Comment Repeats Code
- Implementation Documentation Contaminates Interface
- Vague Name
- Hard to Pick Name
- Hard to Describe
- Nonobvious Code

4.1.3 Coding References

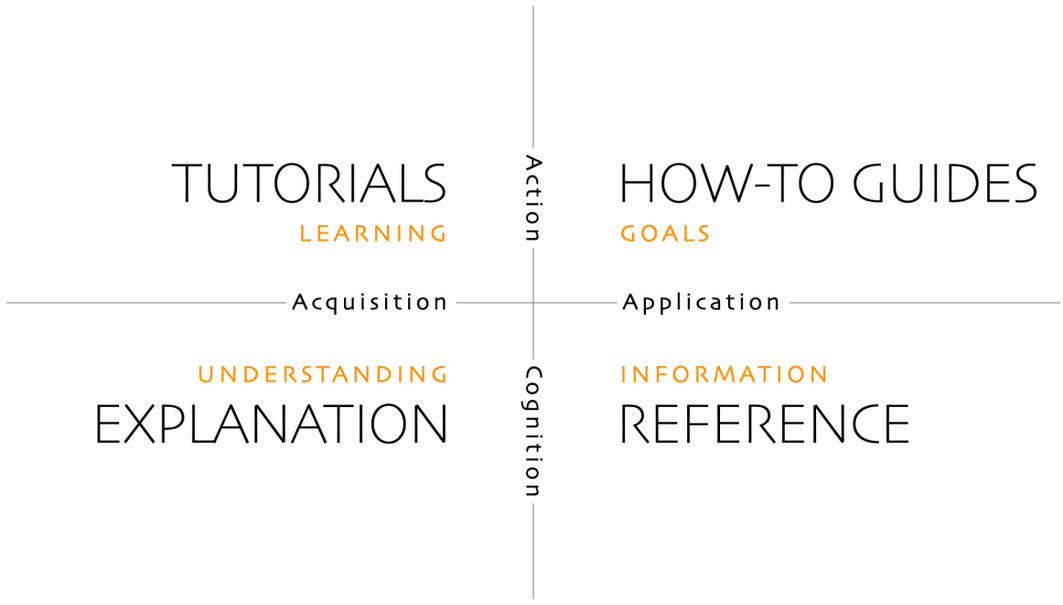
https://thewhitetulip.gitbook.io/py/06-file-handling

4.1.4 Documentation Style

https://diataxis.fr/ (originally developed at: https://docs.divio.com/documentation-system/) — divides documentation along two axes:

- Action (Practical) vs. Cognition (Theoretical)
- Acquisition (Studying) vs. Application (Working)

resulting in a matrix of:



where:

1. readme.md — (Overview) Explanation (understanding-oriented)
2. Templates (`.py`, `.scad`) — Tutorials (learning-oriented)
3. gcodepreview (PDF) — How-to Guides (problem-oriented)
4. Index (of the above PDF)— Reference (information-oriented)

Straddling the boundary between coding and documenation are `docstrings` and general coding style with the latter discussed at: https://peps.python.org/pep-0008/

Another approach, *The Seven-action Documentation Model* https://passo.uno/seven-action-model/ affords seven action points in sequence:

- Appraise (Discern)
- Understand (Learn)
- Explore (Discover)
- Practice (Train)
- Remember (Recall)
- Develop (Integrate)
- Troubleshoot (Solve)

Holidays

Holidays are from https://nationaltoday.com/

DXFs

http://www.paulbourke.net/dataformats/dxf/
https://paulbourke.net/dataformats/dxf/min3d.html

Historical Insights

"On CAD" https://1299651405.com/cad/

> Parametric CAD, in my view, is a perfect example of "visual programming", you have variables, iteration/patterning reducing repetition, composability of objects/sketches again reducing repetition, modularity of design though a hierarchy of assemblies. The alignment between programming principles and CAD modelling principles, while not immediately obvious, are very much there. An elegantly designed CAD model is just as beautiful (in its construction) as elegantly written code.
> https://news.ycombinator.com/item?id=31471109

> ... there are four big CAD companies: Dassault, Autodesk, Siemens, PTC, and once a company commits to one ecosystem, it's very hard to change.
> https://news.ycombinator.com/item?id=27518732

4.2 Future

Future considerations include: https://en.wikipedia.org/wiki/Elastica_theory, mentioned at: https://linebender.org/blog/hyperbezier/ which is notable for being an expression of how a physical object will bend under tension.

4.2.1 Images

Would it be helpful to re-create code algorithms/sections using OpenSCAD Graph Editor so as to represent/illustrate the program?

4.2.2 Bézier curves in 2 dimensions

Take a Bézier curve definition and approximate it as arcs and write them into a DXF?
https://pomax.github.io/bezierinfo/
https://ciechanow.ski/curves-and-surfaces/
https://www.youtube.com/watch?v=aVwxzDHniEw
c.f., https://linuxcnc.org/docs/html/gcode/g-code.html#gcode:g5

4.2.3 Bézier curves in 3 dimensions

One question is how many Bézier curves would it be necessary to have to define a surface in 3 dimensions. Attributes for this which are desirable/necessary:

- concise — a given Bézier curve should be represented by just the point coordinates, so two on-curve points, two off-curve points, each with a pair of coordinates
- For a given shape/region it will need to be possible to have a matching definition exactly match up with it so that one could piece together a larger more complex shape from smaller/simpler regions
- similarly it will be necessary for it to be possible to sub-divide a defined region — for example it should be possible if one had 4 adjacent regions, then the four quadrants at the intersection of the four regions could be used to construct a new region — is it possible to derive a new Bézier curve from half of two other curves?

For the three planes:

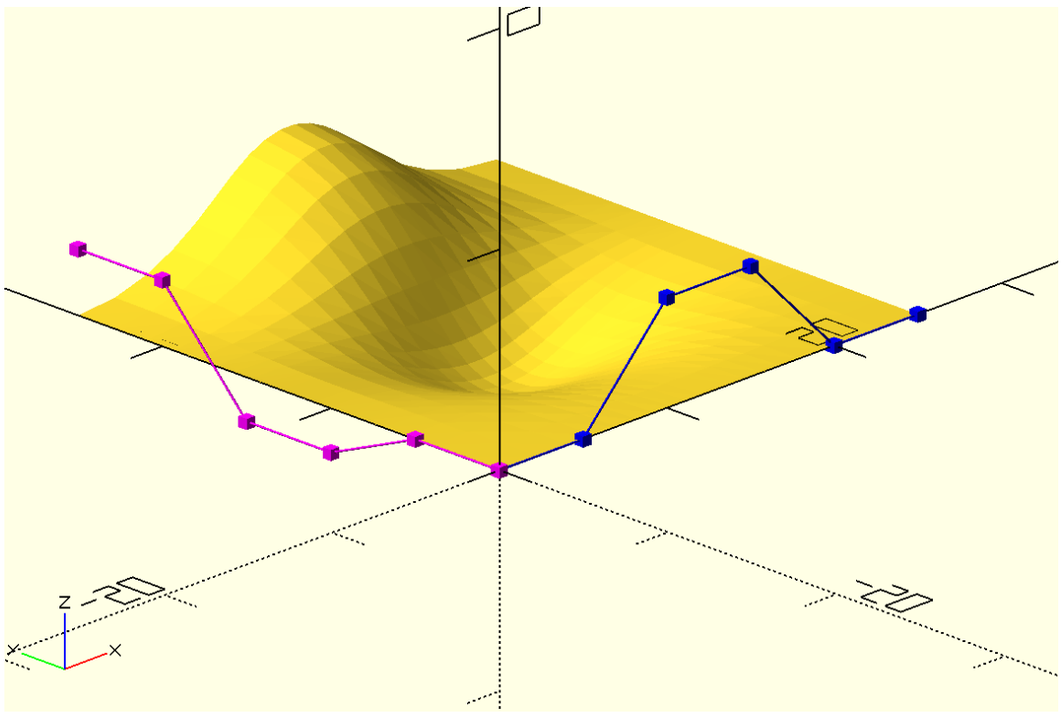
- XY
- XZ
- ZY

it should be possible to have three Bézier curves (left-most/right-most or front-back or top/bottom for two, and a mid-line for the third), so a region which can be so represented would be definable by:

```
3 planes * 3 Béziers * (2 on-curve + 2 off-curve points) == 36 coordinate pairs
```

which is a marked contrast to representations such as:
https://github.com/DavidPhillipOster/Teapot
and regions which could not be so represented could be sub-divided until the representation is workable.

Or, it may be that fewer (only two?) curves are needed:



https://pages.mtu.edu/~shene/COURSES/cs3621/NOTES/notes.html
c.f., https://github.com/BelfrySCAD/BOSL2/wiki/nurbs.scad and https://old.reddit.com/r/OpenPythonSCAD/comments/1gjcz4z/pythonscad_will_get_a_new_spline_function/

References

[ConstGeom] Walmsley, Brian. *Construction Geometry*. 2d ed., Centennial College Press, 1981.

[MkCalc] Horvath, Joan, and Rich Cameron. *Make: Calculus: Build models to learn, visualize, and explore*. First edition., Make: Community LLC, 2022.

[MkGeom] Horvath, Joan, and Rich Cameron. *Make: Geometry: Learn by* 3D *Printing, Coding and Exploring*. First edition., Make: Community LLC, 2021.

[MkTrig] Horvath, Joan, and Rich Cameron. *Make: Trigonometry: Build your way from triangles to analytic geometry*. First edition., Make: Community LLC, 2023.

[PractShopMath] Begnal, Tom. *Practical Shop Math: Simple Solutions to Workshop Fractions, Formulas + Geometric Shapes*. Updated edition, Spring House Press, 2018.

[RS274] Thomas R. Kramer, Frederick M. Proctor, Elena R. Messina.
https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=823374
https://www.nist.gov/publications/nist-rs274ngc-interpreter-version-3

[SoftwareDesign] Ousterhout, John K. *A Philosophy of Software Design*. First Edition., Yaknyam Press, Palo Alto, Ca., 2018

Command Glossary

. 25

setupstock setupstock(200, 100, 8.35, "Top", "Lower-left", 8.35). 23

Index

addopline, 89
addvertex, 89

ballnose, 56
beginopenpolyline, 89
beginpolyline, 89

closedxffile, 98
closegcodefile, 98
closepolyline, 89
currenttool, 33
currenttoolnum, 33
cut..., 53, 64
cutarcCC, 66
cutarcCW, 66
cutkeyhole toolpath, 104
cutKHgcdxf, 105
cutline, 64
cutrectangle, 102

diameter, 34
dovetail, 58
dxfarc, 89
dxfcircle, 89
dxfcolor, 89
dxflayer, 89
dxfline, 88
dxfpostamble, 98
dxfpreamble, 88

endmill square, 56
endmill v, 57
endmilltype, 34

feed, 72
finishopenpolyline, 89
flute, 34
Full Blind Box Joint, 113

gcodepreview, 31
    writeln, 85
gcp.setupstock, 34
generatecut, 28
generatedxf, 28
generategcode, 28

init, 31

mpx, 33
mpy, 33
mpz, 33

opendxffile, 86
opengcodefile, 86

plunge, 72
previewgcodefile, 121

ra, 34
rapid, 62
rapid..., 53
rapids, 34, 38
roundover, 58

settoolparameters, 42
setupstock, 34
    gcodepreview, 34
setxpos, 34
setypos, 34
setzpos, 34
shaftmovement, 54, 59
speed, 72
stockzero, 35
subroutine
    gcodepreview, 34
    writeln, 85

tip, 34
tool diameter, 70, 71
tool number, 42
tool radius, 72
toolchange, 41–43
toolmovement, 33, 34, 38, 43, 54
toolpaths, 34, 38
tpzinc, 33

xpos, 33

ypos, 33

zeroheight, 35
zpos, 33

Routines

addopline, 89
addvertex, 89

ballnose, 56
beginopenpolyline, 89
beginpolyline, 89

closedxffile, 98
closegcodefile, 98
closepolyline, 89
cut..., 53, 64
cutarcCC, 66
cutarcCW, 66
cutkeyhole toolpath, 104
cutKHgcdxf, 105
cutline, 64
cutrectangle, 102

dovetail, 58
dxfarc, 89
dxfcircle, 89
dxfline, 88
dxfpostamble, 98
dxfpreamble, 88

endmill square, 56
endmill v, 57

finishopenpolyline, 89
Full Blind Box Joint, 113

gcodepreview, 31, 34
gcp.setupstock, 34

init, 31

opendxffile, 86
opengcodefile, 86

previewgcodefile, 121

rapid, 62
rapid..., 53
roundover, 58

setupstock, 34
setxpos, 34
setypos, 34
setzpos, 34
shaftmovement, 54, 59

tool diameter, 70, 71
tool radius, 72
toolchange, 41–43
toolmovement, 33, 34, 38, 43, 54

writeln, 85

xpos, 33

ypos, 33

zpos, 33

Variables

currenttool, 33
currenttoolnum, 33

diameter, 34
dxfcolor, 89
dxflayer, 89

endmilltype, 34

feed, 72
flute, 34

generatecut, 28
generatedxf, 28
generategcode, 28

mpx, 33
mpy, 33
mpz, 33

plunge, 72

ra, 34
rapids, 34, 38

settoolparameters, 42
speed, 72
stockzero, 35

tip, 34
tool number, 42
toolchange, 42
toolpaths, 34, 38
tpzinc, 33

zeroheight, 35